HSBI: IFM 3.2: Grundlagen der KI (Winter 2026/27)

Canan Yıldız (TDU) & Carsten Gips (HSBI)

17. Oktober 2026

Inhaltsverzeichnis

I. Syllabus 1

1. Syllabus HSBI 2
- 1.1. Kursbeschreibung 2
- 1.2. Überblick Modulinhalte 2
- 1.3. Team 3
- 1.4. Kursformat (HSBI) 3
- 1.5. Fahrplan (HSBI) 3
- 1.6. Prüfungsform, Note und Credits (HSBI) 4
  - 1.6.1. **Studienleistung**: “Portfolio 4
  - 1.6.2. **Gesamtnote**: (Digitale) Klausur im B40 (90 Minuten) 5
  - 1.6.3. Hinweise 5
- 1.7. Materialien 5

II. Vorlesungsunterlagen 6

2. Einführung KI 7
- 2.1. Intro1: Was ist Künstliche Intelligenz? 7
  - 2.1.1. Was ist (künstliche) Intelligenz? 8
  - 2.1.2. Definition Intelligenz 8
  - 2.1.3. Versuch einer Definition für “KI” 9
  - 2.1.4. Alan Turing 1950: Turing-Test (begründet Zweige der KI) 10
  - 2.1.5. Starke vs. schwache KI 11
  - 2.1.6. Typische Ansätze in der KI 12
  - 2.1.7. Modelle, Lernen und Vorhersagen 14
  - 2.1.8. Modellkomplexität 15
  - 2.1.9. Kurzer Geschichtsüberblick – Wichtigste Meilensteine 16
  - 2.1.10. Wrap-Up 16
- 2.2. Intro: Einführung in Jupyter-Notebooks 17
- 2.3. Intro2: Problemlösen 17
  - 2.3.1. Motivation: Roboter in einer Bibliothek 18
  - 2.3.2. Zustände der Bibliotheks-Welt 19
  - 2.3.3. Suche im Problemgraphen 20
  - 2.3.4. Definition Zustand und Aktion 20
  - 2.3.5. Definition Problem 20

2.3.6. Hinweis: Unterschied Zustand und Knoten bzw. Zustandsraum und Problemgraph . . . 21
2.3.7. Definition Problemlösen 21
2.3.8. Suche: Einfache Basisvariante 22
2.3.9. Erweiterung der Suche: Vermeiden von Wiederholungen 22
2.3.10. Bewertung von Suchalgorithmen 23
2.3.11. Wrap-Up 23

3. Suche **25**
3.1. Search1: Suche mit Tiefensuche 25
3.1.1. Hole das Buch 25
3.1.2. Problemgraph zum Kopiererbeispiel 26
3.1.3. Anmerkungen Wegesuche (Landkarte) 26
3.1.4. Tiefensuche (*TS*, *DFS*) 27
3.1.5. Bemerkungen 27
3.1.6. Konventionen für diese Lehrveranstaltung 28
3.1.7. Weitere Hinweise 28
3.1.8. Tiefensuche (rekursive Variante) 28
3.1.9. Eigenschaften der Tiefensuche 29
3.1.10. Wrap-Up 29
3.2. Search2: Suche mit Breitensuche 29
3.2.1. Hole das Buch 30
3.2.2. Breitensuche (*BS*, *BFS*) 30
3.2.3. Bemerkungen 31
3.2.4. Konventionen für diese Lehrveranstaltung 31
3.2.5. Weitere Hinweise 32
3.2.6. Eigenschaften Breitensuche vs. Tiefensuche 32
3.2.7. Praxisvergleich Breitensuche vs. Tiefensuche 33
3.2.8. Wrap-Up 33
3.3. Search3: Suche mit Branch-and-Bound 34
3.3.1. Hole das Buch 34
3.3.2. Branch-and-Bound (*BnB*) 35
3.3.3. BnB: Finde einen Weg von A nach H 35
3.3.4. Bemerkungen zu BnB mit Graph-Search 35
3.3.5. Konventionen für diese Lehrveranstaltung 36
3.3.6. Eigenschaften von BnB 36
3.3.7. Wrap-Up 36
3.4. Search4: Suche mit Best First 37
3.4.1. Hole das Buch 37
3.4.2. Best-First (*BF*, *BFS*) 38
3.4.3. Konventionen BF 38
3.4.4. Eigenschaften von BF 38
3.4.5. Wrap-Up 39
3.5. Search5: Suche mit A* 40
3.5.1. Hole das Buch 41

3.5.2. A*-Suche 41
3.5.3. Konventionen für diese Lehrveranstaltung 42
3.5.4. A*-Suche – Anforderungen an Heuristik (Tree-Search) 42
3.5.5. A* ist optimal 43
3.5.6. Einfache Verbesserungen A* (Tree-Search) 43
3.5.7. A*-Suche – Anforderungen an Heuristik (Graph-Search) 43
3.5.8. Eigenschaften Branch-and-Bound, Best-First, A* 44
3.5.9. Wrap-Up 44
3.6. Search6: Lokale Suche - Gradientensuche 45
3.6.1. Unterschiede in den Suchproblemen? 46
3.6.2. Analogie: Bergsteigen ohne Karte und Pfade 47
3.6.3. Pseudoalgorithmus Gradientensuche 47
3.6.4. Beispiel Gradientensuche: n-Damen 48
3.6.5. Eigenschaften Gradientensuche 48
3.6.6. Wrap-Up 49
3.7. Search7: Lokale Suche - Simulated Annealing 51
3.7.1. Motivation 51
3.7.2. Gedankenexperiment: Ausweg aus lokalen Minima 52
3.7.3. Analogie Härten von Metall 52
3.7.4. Übertragen der Idee 52
3.7.5. Pseudocode Simulated Annealing (Minimierungsproblem) 52
3.7.6. Detail: Akzeptieren von Verschlechterungen 53
3.7.7. Abkühlungsplan problemabhängig wählen 54
3.7.8. Eigenschaften Simulated Annealing 54
3.7.9. Anwendungen von Simulated Annealing 54
3.7.10. Wrap-Up 54

4. Spiele **56**
4.1. Games1: Einführung Optimale Spiele 56
4.1.1. Backgammon: Zwei Spieler, was ist der beste Zug? 56
4.1.2. Motivation: Unterschied zu Suche?! 57
4.1.3. Spiele und Umgebungen 57
4.1.4. Brettspiele sind interessant für KI 57
4.1.5. Eigenschaften guter Spielalgorithmen 58
4.1.6. Wrap-Up 58
4.2. Games2: Minimax 59
4.2.1. Spiele als Suchproblem: Minimax 59
4.2.2. Minimax-Algorithmus: Funktionen für MAX- und MIN-Knoten 60
4.2.3. Minimax-Algorithmus: Sonderbehandlung Startknoten 61
4.2.4. Minimax Beispiel 62
4.2.5. Wrap-Up 62
4.3. Games3: Heuristiken 63
4.3.1. Wenn die Zeit nicht reicht: Suchtiefe begrenzen 64
4.3.2. Beispiel Schach 64

4.3.3. Minimax mit mehreren Spielern 66
4.3.4. Zufallsspiele 66
4.3.5. Minimax mit Zufallsspielen: ZUFALLS-Knoten 67
4.3.6. Minimax mit Zufall: Expectimax 67
4.3.7. Wrap-Up 68
4.4. Games4: Alpha-Beta-Pruning 68
4.4.1. Verbesserung Minimax-Algorithmus 69
4.4.2. Alpha-beta-Pruning 69
4.4.3. Pruning-Regeln 69
4.4.4. Alpha-beta-Pruning – Der Algorithmus (Skizze) 70
4.4.5. Alpha-beta-Pruning – Eigenschaften 70
4.4.6. Verbesserungen für Alpha-beta-Pruning 71
4.4.7. Beispiel DeepBlue (IBM, 1997) 71
4.4.8. Beispiel AlphaGo (Google, 2016) 71
4.4.9. Wrap-Up 72

5. Entscheidungsbäume (Decision Tree Learner - DTL) 74
5.1. DTL1: Machine Learning 101 74
5.1.1. Was ist Lernen? 75
5.1.2. Warum Lernen? 75
5.1.3. Learning Agent 75
5.1.4. Feedback während des Lernens 75
5.1.5. Beispiel: Kreditrisiko 76
5.1.6. Beispiel: Autoreparatur 76
5.1.7. Lernen durch Beobachten: Lernen einer Funktion f 77
5.1.8. Konstruieren einer konsistenten Hypothese 77
5.1.9. Konstruieren einer konsistenten Hypothese (cnt.) 78
5.1.10. Konstruieren einer konsistenten Hypothese (cnt.) 78
5.1.11. Konstruieren einer konsistenten Hypothese (cnt.) 79
5.1.12. Konstruieren einer konsistenten Hypothese (cnt.) 79
5.1.13. Occam’s Razor 80
5.1.14. Trainingsdaten und Merkmalsvektoren 80
5.1.15. Beispiel: Schwimmen im See 80
5.1.16. Trainingsdaten – Merkmalsvektoren 81
5.1.17. Wrap-Up 81
5.2. DTL2: CAL2 82
5.2.1. Entscheidungsbäume: Klassifikation 83
5.2.2. Definition Entscheidungsbaum 84
5.2.3. Induktion von Entscheidungsbäumen: CAL2 84
5.2.4. Beispiel mit CAL2 85
5.2.5. CAL2: Bemerkungen 85
5.2.6. Wrap-Up 85
5.3. DTL5: Entropie 87
5.3.1. Wie Attribute wählen? 87

5.3.2. Definition der Entropie $H(V)$ für Zufallsvariable V 88
5.3.3. Beispiele Entropie: faire Münze 88
5.3.4. Beispiele Entropie: unfaire Münze 89
5.3.5. Beispiele Entropie: 4-seitiger Würfel 89
5.3.6. Entropie der Trainingsmenge: Häufigkeit der Klassen zählen 89
5.3.7. Mittlere Entropie nach Betrachtung von Attribut A 90
5.3.8. Entropie der Trainingsmenge nach Attributwahl 90
5.3.9. Ausblick: Gini Impurity 91
5.3.10. Wrap-Up 92
5.4. DTL6: ID3 und C4.5 92
5.4.1. Wie Attribute wählen? 93
5.4.2. Erinnerung Entropie: Maß für die Unsicherheit 93
5.4.3. Informationsgewinn: Kriterium zur Auswahl von Attributen 94
5.4.4. Entscheidungsbaumlerner ID3 (Quinlan, 1986) 94
5.4.5. Beobachtung: Gain ist bei mehrwertigen Attributen höher 95
5.4.6. C4.5 als Verbesserung zu ID3 96
5.4.7. Beispiele zur Normierung bei C4.5 96
5.4.8. Wrap-Up 97
5.5. DTL7: Random Forest 98
5.5.1. Vom einzelnen Baum zum Wald 98
5.5.2. Wie funktioniert ein Random Forest? 99
5.5.3. Beispiel: Random Forest in Aktion (Klassifikation) 100
5.5.4. Pseudocode: Random Forest 101
5.5.5. Bias und Varianz und Grenzen von Entscheidungsbäumen / Random Forest 102
5.5.6. Wrap-Up 103

6. NN: Einführung in Neuronale Netze **105**
6.1. NN01: Das Perzeptron 105
6.1.1. Kurze Übersicht 105
6.2. NN02: Lineare Regression und Gradientenabstieg 109
6.2.1. Kurze Übersicht 109
6.3. NN03: Logistische Regression 112
6.3.1. Kurze Übersicht 113
6.4. NN05: Multilayer Perzeptron 115
6.4.1. Kurze Übersicht 115
6.5. NN06: Backpropagation 116
6.5.1. Kurze Übersicht 117
6.6. NN04: Overfitting und Regularisierung 118
6.6.1. Kurze Übersicht 118
6.7. NN07: Training & Testing 119
6.7.1. Kurze Übersicht 120
6.8. NN08: Performanzanalyse 125
6.8.1. Kurze Übersicht 125

6.9. NN11: RNN . . . 128
  6.9.1. Kurze Übersicht . . . 128
6.10. NN12: Transformer . . . 128
  6.10.1. Kurze Übersicht . . . 128

7. Constraintsolving **130**
7.1. CSP1: Einführung Constraints . . . 130
  7.1.1. Motivation: Einfärben von Landkarten . . . 131
  7.1.2. Einfärben von Landkarten: Formalisierung . . . 131
  7.1.3. Definition: Constraint Satisfaction Problem (CSP) . . . 131
  7.1.4. Constraint-Graph . . . 132
  7.1.5. Constraints – Arität . . . 132
  7.1.6. Constraints – Wertebereiche . . . 133
  7.1.7. CSP sind überall 133
  7.1.8. Wrap-Up . . . 133
7.2. CSP2: Lösen von diskreten CSP (Backtracking-Search) . . . 134
  7.2.1. Einfärben von Landkarten als CSP . . . 135
  7.2.2. Endliche Domänen: Formulierung als Suchproblem . . . 135
  7.2.3. BT-Suche für CSP am Beispiel Landkartenfärbeproblem . . . 136
  7.2.4. Wrap-Up . . . 136
7.3. CSP3: Heuristiken . . . 137
  7.3.1. VARIABLES: Variablen-Sortierung, Welche Variable soll betrachtet werden? . . . 137
  7.3.2. VARIABLES: Gleichstand bei MRV . . . 138
  7.3.3. VALUES: Werte-Sortierung, Welchen Wert soll ich ausprobieren? . . . 139
  7.3.4. Wrap-Up . . . 139
7.4. CSP4: Kantenkonsistenz und AC-3 . . . 141
  7.4.1. Problem bei BT-Suche . . . 141
  7.4.2. INFERENCE: Vorab-Prüfung (Forward Checking) . . . 142
  7.4.3. Forward Checking findet nicht alle Inkonsistenzen! . . . 142
  7.4.4. Übergang von Forward Checking zu Kantenkonsistenz . . . 143
  7.4.5. Definition Kantenkonsistenz (Arc Consistency) . . . 143
  7.4.6. Beispiel Kantenkonsistenz . . . 143
  7.4.7. AC-3 Algorithmus: Herstellen von Kantenkonsistenz . . . 143
  7.4.8. Einsatz des AC-3 Algorithmus . . . 144
  7.4.9. Wrap-Up . . . 144
7.5. CSP5: Min-Conflicts Heuristik . . . 146
  7.5.1. Problem bei BT-Suche . . . 146
  7.5.2. Idee: Würfeln und Schütteln . . . 147
  7.5.3. Beispiel: Einfärben von Landkarten . . . 147
  7.5.4. Min-Conflicts Heuristik . . . 148
  7.5.5. Diskussion . . . 149
  7.5.6. Wrap-Up . . . 150

8. Naive Bayes **151**
8.1. NB1: Wiederholung Wahrscheinlichkeitstheorie . . . 151
8.1.1. Ereignisse und Wahrscheinlichkeit . . . 152
8.1.2. Rechenregeln: Kolmogorov Axiome . . . 153
8.1.3. Verbundwahrscheinlichkeiten . . . 153
8.1.4. Bedingte Wahrscheinlichkeit . . . 154
8.1.5. Marginalisierung . . . 154
8.1.6. Kettenregel . . . 155
8.1.7. Bayes-Regel . . . 155
8.1.8. Beispiel Bayes . . . 156
8.1.9. Unabhängige Ereignisse . . . 156
8.1.10. Wrap-Up . . . 157
8.2. NB2: Klassifikation mit Naive Bayes . . . 158
8.2.1. Medizinische Diagnostik mit NB . . . 159
8.2.2. Textklassifikation mit NB . . . 159
8.2.3. Naive Bayes . . . 159
8.2.4. Bayes'sches Lernen . . . 160
8.2.5. Beispiel Klassifikation mit NB . . . 160
8.2.6. Textklassifikation mit NB . . . 161
8.2.7. Naivität im Naive Bayes . . . 162
8.2.8. Laplace-Schätzer . . . 164
8.2.9. Probleme mit Floating Point Underflow . . . 164
8.2.10. Maximum Likelihood . . . 165
8.2.11. Ausblick: Kontinuierliche Attribute . . . 165
8.2.12. Hinweis zum Sprachgebrauch . . . 165
8.2.13. Wrap-Up . . . 166
8.3. NB3: Text-Klassifikation mit Naive Bayes . . . 167
8.3.1. Textklassifikation mit NB . . . 167
8.3.2. Vorverarbeitung . . . 168
8.3.3. Merkmalsvektor: Bag of Words . . . 168
8.3.4. Bag of Words für Trainingsdaten . . . 169
8.3.5. Naive Bayes Training (Multinomial NB) . . . 169
8.3.6. Naive Bayes Klassifikation (Multinomial NB) . . . 170
8.3.7. Naive Bayes mit Laplace-Glättung (Multinomial NB) . . . 171
8.3.8. Naive Bayes mit Out-of-Vocabulary-Termen (OOV, Multinomial NB) . . . 172
8.3.9. Naive Bayes mit UNK-Term (OOV, Multinomial NB) . . . 173
8.3.10. Naive Bayes Training (Bernoulli NB, mit Laplace) . . . 174
8.3.11. Naive Bayes Klassifikation (Bernoulli NB) . . . 175
8.3.12. Wrap-Up . . . 176

9. Genetische Algorithmen **178**
9.1. EA1: Einführung Evolutionäre Algorithmen . . . 178
9.1.1. Evolution sehr erfolgreich bei Anpassung . . . 178
9.1.2. EA – Zutaten und Mechanismen . . . 179

9.1.3. EA – Allgemeiner Ablauf . . . 179
9.1.4. EA – Beispiel . . . 180
9.1.5. EA – Strömungen . . . 180
9.1.6. Anwendungsbeispiele für Evolutionäre Algorithmen . . . 181
9.1.7. Wrap-Up . . . 181
9.2. EA2: Modellierung mit Genetischen Algorithmen . . . 181
9.2.1. EA – Allgemeiner Ablauf . . . 182
9.2.2. Kodierung Individuen . . . 182
9.2.3. Selektion: Erstelle Matingpool mit μ Individuen . . . 183
9.2.4. Crossover: Erzeuge zwei Nachkommen aus zwei Eltern . . . 183
9.2.5. Mutation . . . 184
9.2.6. Bewertungskriterien . . . 185
9.2.7. Typische Läufe . . . 185
9.2.8. Wrap-Up . . . 186

III. Praktikum/Übung **188**

10. Übungsblatt: Problemlösen, Suche **190**
10.1. Bonus: Möglichkeiten und Grenzen sowie Auswirkungen der KI (2P) . . . 190
10.2. Search.01: Problemformalisierung, Zustandsraum (3P) . . . 190
10.3. Search.02: Suchverfahren (5P) . . . 191
10.4. Search.03: Dominanz (1P) . . . 192
10.5. Search.04: Beweis der Optimalität von A* (1P) . . . 192

11. Übungsblatt: Spiele **193**
11.1. Bonus: Verlässlichkeit von ChatGPT (2P) . . . 193
11.2. Games.01: Handsimulation: Minimax und alpha-beta-Pruning (3P) . . . 193
11.3. Games.02: Optimale Spiele: Minimax und alpha-beta-Pruning (4P) . . . 194
11.4. Games.03: Minimax vereinfachen (1P) . . . 194
11.5. Games.04: Suchtiefe begrenzen (1P) . . . 194
11.6. Games.05: Minimax generalisiert (1P) . . . 194

12. Übungsblatt: Entscheidungsbäume (Decision Tree Learner DTL) **196**
12.1. Starke vs. Schwache KI (1P) . . . 196
12.2. DTL.01: Entscheidungsbäume mit ID3 (6P) . . . 196
12.3. DTL.03: Machine Learning mit Weka (3P) . . . 197

13. Übungsblatt: Perzeptron **198**
13.1. NN.Perzeptron.01: Entscheidungsgrenze (2P) . . . 198
13.2. NN.Perzeptron.02: Logische Funktionen als Perzeptron (2P) . . . 198
13.3. NN.Perzeptron.03: Perzeptron Lernalgorithmus (6P) . . . 198
13.3.1. Datensatz (1P) . . . 198
13.3.2. Training (3P) . . . 199
13.3.3. Experimente (2P) . . . 199

13.3.4. Visualisierung (optional) 199

14. Übungsblatt: Lineare / Logistische Regression & Gradientenabstieg 200
14.1. NN.Regression.01: Lineare Regression & Gradientenabstieg (3P) 200
14.2. NN.Regression.02: Logistische Regression & Gradientenabstieg (7P) 201
14.2.1. Datensatz (1P) 201
14.2.2. Training (3P) 201
14.2.3. Experimente (3P) 201

15. Übungsblatt: Overfitting & MLP 202
15.1. NN.MLP.01: Perzeptron-Netze (2P) 202
15.2. NN.MLP.02: Vorwärtslauf im MLP (2P) 202
15.3. NN.MLP.03: Tensorflow Playground (6P) 203

16. Übungsblatt: Backpropagation 204
16.1. NN.Backprop.01: Gewichtsupdates für versteckte Schichten (2P) 204
16.2. NN.Backprop.02: Forward- und Backpropagation (2P) 204
16.3. NN.Backprop.03: MLP und Backpropagation (6P) 204

17. Übungsblatt: Constraints 206
17.1. CSP.01: Logikrätsel (2P) 206
17.2. CSP.02: Framework für Constraint Satisfaction (4P) 206
17.3. CSP.03: Kantenkonsistenz mit AC-3 (2P) 206
17.4. CSP.04: Forward Checking und Kantenkonsistenz (2P) 207
17.5. Bonus: Planung von Indoor-Spielplätzen (3P) 207
17.6. Bonus: DSL für Constraint Solving: MiniZinc (2P) 208

18. Übungsblatt: Naive Bayes 209
18.1. NB.01: Wahlkampf mit Naive Bayes (4P) 209
18.2. NB.02: Textklassifikation mit Naive Bayes: Spam-Erkennung (6P) 209

19. Übungsblatt: Lokale Suche, GA 210
19.1. Bonus: ChatGPT Taboo (2P) 210
19.2. EA.01: Modellierung von GA (2P) 210
19.3. EA.02: Implementierung (5P) 210
19.4. EA.03: Anwendungen (3P) 211

IV. Organisatorisches 212

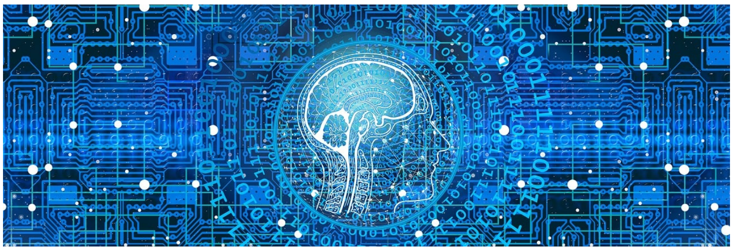
20. Prüfungsvorbereitung (HSBI) 213
20.1. Elektronische Klausur: Termin, Materialien 213
20.1.1. Termin 213
20.1.2. Zugelassene Hilfsmittel: DIN-A4-Spickzettel (beidseitig) 214
20.1.3. Einsicht 214
20.2. Technische Vorbereitungen 214

20.3. Bearbeitung des E-Assessment . . . 214
20.4. Fragetypen-Demo . . . 215
20.5. Hinweise zu den Inhalten . . . 215
20.6. Beispiele für mögliche Fragen . . . 215
    20.6.1. Breitensuche . . . 215
    20.6.2. Suche und Constraints . . . 216
    20.6.3. Handsimulation Perzeptron . . . 216
    20.6.4. Entscheidungsbäume mit ID3 . . . 216
    20.6.5. CSP . . . 217
    20.6.6. NB . . . 217

Teil I.

Syllabus

1. Syllabus HSBI



Quelle: “künstliche intelligenz” by Gerd Altmann (geralt) on Pixabay.com (Pixabay License)

1.1. Kursbeschreibung

Ausgehend von den Fragen “Was ist *Intelligenz*?” und “Was ist *künstliche* Intelligenz?” werden wir uns in diesem Modul mit **verschiedenen Teilgebieten der KI** beschäftigen und uns anschauen, welche **Methoden und Algorithmen** es gibt und wie diese funktionieren. Dabei werden wir auch das Gebiet *Machine Learning* berühren, aber auch andere wichtige Gebiete betrachten. Sie erarbeiten sich im Laufe der Veranstaltung einen **Methoden-Baukasten** zur Lösung unterschiedlichster Probleme und erwerben ein grundlegendes Verständnis für die Anwendung in Spielen, Navigation, Planung, smarten Assistenten, autonomen Fahrzeugen, …

1.2. Überblick Modulinhalte

1. Problemlösen
    - Zustände, Aktionen, Problemraum
    - Suche (blind, informiert): Breiten-, Tiefensuche, Best-First, Branch-and-Bound, A-Stern
    - Lokale Suche: Gradientenabstieg, Genetische/Evolutionäre Algorithmen (GA/EA)
    - Spiele: Minimax, Alpha-Beta-Pruning, Heuristiken
    - Constraints: Backtracking, Heuristiken, Propagation, AC-3
2. Maschinelles Lernen
    - Merkmalsvektor, Trainingsmenge, Trainingsfehler, Generalisierung
    - Entscheidungsbäume: CAL2, ID3/C4.5, Random Forest
    - Neuronale Netze
        - Perzeptron, Lernregel

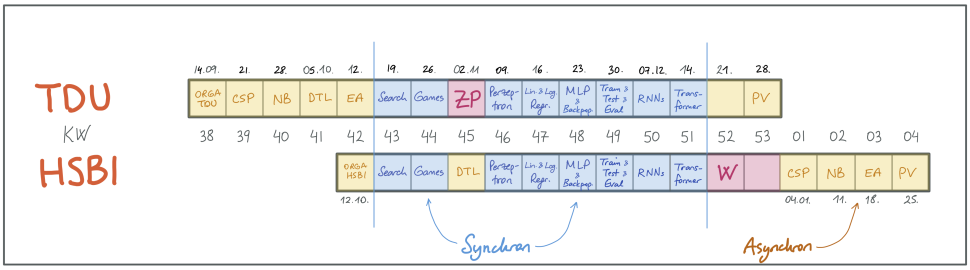
    - Feedforward Multilayer Perzeptron (MLP), Backpropagation, Trainings- vs. Generalisierungsfehler
    - Steuerung des Trainings: Kreuzvalidierung, Regularisierung
    - Ausblick: Support-Vektor-Maschinen
  - Naive Bayes Klassifikator

3. ~~Inferenz, Logik~~ (**entfällt im W26**)
   - ~~Prädikatenlogik: Modellierung, semantische und formale Beweise, Unifikation, Resolution~~
   - ~~Ausblick: Anwendung in Prolog~~

1.3. Team

- Carsten Gips (HSBI, Sprechstunde nach Vereinbarung)
- Canan Yıldız (TDU)

1.4. Kursformat (HSBI)



KW	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	01	02	03	04
	14.09.	21.	28.	05.10.	12.	19.	26.	02.11	09.	16.	23.	30.	07.12.	14.	21.	28.				
TDU	ORGA TDU	CSP	NB	DTL	EA	Search	Games	ZP	Perzeptron	Lin. & Log. Regr.	MLP & Backprop.	Train & Test & Eval	RNNs	Transformer		PV				
HSBI					ORGA HSBI	Search	Games	DTL	Perzeptron	Lin. & Log. Regr.	MLP & Backprop.	Train & Test & Eval	RNNs	Transformer	W		CSP	NB	EA	PV
					12.10.												04.01.	11.	18.	25.

Vorlesung (2 SWS)	Praktikum (2 SWS)
Mo, 09:00 - 10:30 Uhr (DE)	G1: Mo, 10:45 - 12:15 Uhr (DE)
(*Flipped Classroom*)	G2: Mi, 15:45 - 17:15 Uhr (DE)
	G3: Mo, 14:00 - 15:30 Uhr (DE)
	G4: Do, 14:00 - 15:30 Uhr (DE)

Alle Sitzungen online per Zoom (**Zugangsdaten siehe ILIAS**).

1.5. Fahrplan (HSBI)

Monat	Woche vom	Thema	Vorlesung (Mo)	Praktikum (Mo/Mi/Do)
Oktober	12.10.	Orga	Orga HSBI || Einführung KI | Einführung Jupyter Notebook	-
	19.10.	Search	Problemlösen | Tiefensuche | Breitensuche | Branch-and-Bound | Best First | A-Stern	Suche
	26.10.	Games	Optimale Spiele | Games mit Minimax | Minimax und Heuristiken | Alpha-Beta-Pruning	Games
November	02.11.	DTL	Machine Learning 101 | CAL2 | Entropie | ID3 und C4.5 | Random Forest	DTL
	09.11.	Perzeptron	Perzeptron	Perzeptron
	16.11.	Regression	Lineare Regression und Gradientenabstieg | Logistische Regression	Regression
	23.11.	MLP	Multilayer Perceptron (MLP) | Backpropagation	MLP
Dezember	30.11.	Train&Test	Overfitting und Regularisierung | Training & Testing | Performanzanalyse	Backpropagation
	07.12.	RNN	RNN	-
	14.12.	Transformer	Transformer	-
	21.12.	-	***Weihnachtspause***	-
	28.12.	-	***Weihnachtspause***	-
Januar	04.01.	CSP	Einführung Constraints | Lösen von diskreten CSP | CSP und Heuristiken | Kantenkonsistenz und AC-3 | Min-Conflicts Heuristik	CSP
	11.01.	NB	Wahrscheinlichkeitstheorie | Naive Bayes | Textklassifikation mit NB	Naive Bayes
	18.01.	EA	Gradientensuche | Simulated Annealing || Intro EA/GA | Genetische Algorithmen	EA/GA
	25.01.	PV	Rückblick | Prüfungsvorbereitung HSBI	-

1.6. Prüfungsform, Note und Credits (HSBI)

(Digitale) Klausur plus Studienleistung (Portfolio), 5 ECTS

1.6.1. Studienleistung: “Portfolio

Die Studienleistung ist eine unbenotete Leistung und setzt sich aus mehreren Komponenten zusammen:

1. Erfolgreiche Bearbeitung von mind. **sechs Übungsblättern** inkl. fristgerechter Abgabe des zugehörigen **Post Mortems** (s.u.).

Abgabe der Post Mortems: Spätestens bis Montag der Folgewoche 09:00 Uhr im ILIAS.

1.6.2. Gesamtnote: (Digitale) Klausur im B40 (90 Minuten)

Die Modul-Note ergibt sich aus der Leistung in der Klausur.

Sie können die Prüfung in der ersten oder in der zweiten Prüfungsphase ablegen. In beiden Prüfungszeiträumen wird je eine digitale Klausur im B40 mit 90 Minuten Dauer angeboten. Die Note ergibt sich aus der Leistung in der Klausur.

1.6.3. Hinweise

- Die Klausur und die Studienleistung sind zwei unabhängige Prüfungsleistungen und werden separat im CAT angemeldet. Die Studienleistung schaltet die Credits für das Modul frei, die Klausur ergibt die Modulnote. Die Studienleistung ist **keine** Vorleistung für die Klausur.
- Die Bearbeitung der Aufgaben erfolgt individuell.
- Im Praktikum beginnen wir gemeinsam mit der Bearbeitung der Übungsblätter und diskutieren über Lösungsansätze. Die Lösung soll anschließend individuell fertiggestellt werden und kann auf Wunsch im nächsten Praktikum von Ihnen vorgestellt werden.
- "Erfolgreiche Bearbeitung" eines Blattes umfasst die Bearbeitung aller Aufgaben des Blattes und die fristgerechte Abgabe des ausreichenden Post Mortems im ILIAS. Die intensive Beschäftigung mit den Aufgaben muss erkennbar sein.
- Die Teilnahme am Praktikum ist freiwillig, wird aber deutlich empfohlen.
- Eine Bewertung einzelner Übungsblätter findet nicht statt.
- Die Post Mortems sind individuell zu erstellen und abzugeben.
- "Aktive Beteiligung" umfasst Anwesenheit und sachbezogene Beiträge; Anwesenheit/Beteiligung werden dokumentiert.

- **Post Mortem**: Jede Person beschreibt individuell(!) die Bearbeitung des jeweiligen Blattes zurückblickend mit mind. 150 bis max. 400 Wörtern (Nutzlast! Überschriften und Links zählen nicht mit). Gehen Sie dabei aussagekräftig und nachvollziehbar auf folgende Punkte ein:

  1. **Zusammenfassung**: Was wurde gemacht?
  2. **Details**: Kurze Beschreibung besonders interessanter Aspekte.
  3. **Reflexion**: Was war der schwierigste Teil? Wie haben Sie dieses Problem gelöst?
  4. **Reflexion**: Was haben Sie gelernt oder (besser) verstanden?

  Die Post Mortems geben Sie bitte pro Person bis spätestens zur jeweiligen Deadline im ILIAS ab.

1.7. Materialien

1. **"Artificial Intelligence: A Modern Approach"** (*AIMA*). Russell, S. und Norvig, P., Pearson, 2021. ISBN 978-0134610993.
2. "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow". Géron, A., O'Reilly, 2023. ISBN 978-1-098-12597-4. Online über die O'Reilly-Lernplattform.

Teil II.

Vorlesungsunterlagen

2. Einführung KI

Was ist Intelligenz? Was ist künstliche Intelligenz? Woran kann man das erkennen? Wie kann man eine Welt (ein Problem) modellieren, um es dann anschließend lösen zu können?

2.1. Intro1: Was ist Künstliche Intelligenz?

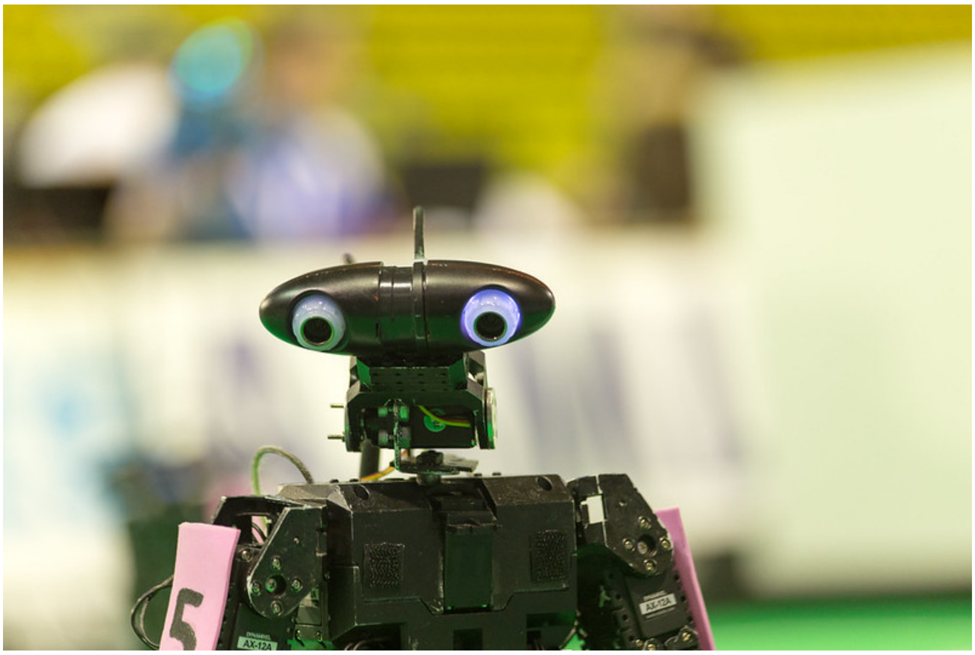
TL;DR

KI ist ein altes und modernes Forschungsgebiet, welches periodisch Hype-Zeiten erlebt hat und sich aktuell wieder in einer Hoch-Phase befindet. Wer heute "KI" sagt, meint meist Maschinelles Lernen oder (noch genauer) eine Form von *Deep Learning*. Dabei gibt es in der KI viele weitere Gebiete: Suche (Problemlösen), Spiele, Constraintprobleme, Entscheidungsbäume, …, um nur einige zu nennen.
Der *Turing-Test* (Alan Turin, 1950) hat gewissermaßen die modernen Zweige der KI begründet, u.a. *Wissensrepräsentation*, *Logisches Schließen*, *Maschinelles Lernen*, *Verarbeitung natürlicher Sprache*, *Computer Vision* und *Robotik*. Dabei ist zwischen *starker KI* und *schwacher KI* zu unterscheiden.
Häufig werden die Gebiete in einem Diagramm eingeordnet, wobei die x-Achse Verhalten vs. Denken und die y-Achse Rational vs. Menschlich aufspannen. So kann man beispielsweise Logik dem rationalen Denken zuordnen oder die Erforschung kognitiver Prozesse dem Quadranten menschliches Denken.
Wenn man sich die Geschichte der KI anschaut, beobachtet man bei fast allen Themen, dass sie in der Vergangenheit eine Hype-Phase erlebt haben und dabei die oft stark überzogenen Erwartungen enttäuscht haben und danach meist nur wenig Beachtung erfuhren. Nach einer Weile kamen die Themen wieder "auf die Tagesordnung", diesmal mit vernünftig(er)en Erwartungen.

Videos

Vorlesung [YT], [HSBI]

2.1.1. Was ist (künstliche) Intelligenz?



Quelle: AvB - RoboCup 2013 - Eindhoven by RoboCup2013 on Flickr.com (CC BY 2.0)

- Was ist (künstliche) Intelligenz?
- Ist Commander Data intelligent?
- Woran erkennen Sie das?

2.1.2. Definition Intelligenz

> Intelligenz (von lat. *intellegere* "verstehen", wörtlich "wählen zwischen …" von lat. *inter* "zwischen" und *legere* "lesen, wählen") ist in der Psychologie ein Sammelbegriff für die **kognitive Leistungsfähigkeit** des Menschen. Da einzelne kognitive Fähigkeiten unterschiedlich stark ausgeprägt sein können und keine Einigkeit besteht, wie diese zu bestimmen und zu unterscheiden sind, gibt es **keine allgemeingültige Definition der Intelligenz**.
>
> Quelle: "Intelligenz" by Cumtempore and others on Wikipedia (CC BY-SA 3.0)

Das ist spannend: Es gibt keine allgemeingültige Definition für den Begriff "Intelligenz". Damit wird es schwierig, auch "Künstliche Intelligenz" zu definieren …

Aber wir können aus dieser Definition auf Wikipedia mitnehmen, dass es um kognitive Fähigkeiten geht. Verstehen und im weiteren Sinne Denken sind bereits im Begriff enthalten und damit auch Teil der kognitiven Fähigkeiten.

Schauen wir uns nun noch die Definition von "kognitiven Fähigkeiten" genauer an.

> Zu den **kognitiven Fähigkeiten** eines Menschen zählen die **Wahrnehmung**, die Aufmerksamkeit, die Erinnerung, das **Lernen**, das **Problemlösen**, die Kreativität, das **Planen**, die Orientierung, die Imagination, die **Argumentation**, die Introspektion, der Wille, das Glauben und einige mehr. Auch Emotionen haben einen wesentlichen kognitiven Anteil.
>
> Quelle: "Kognition" by Arbraxan and others on Wikipedia (CC BY-SA 3.0)

Zu den kognitiven Fähigkeiten und damit zur Intelligenz zählen also eine Reihe von Fähigkeiten, die man Menschen im allgemeinen zuschreibt. Lernen und Problemlösen und Planen sind Dinge, die vermutlich jeder direkt mit dem Begriff Intelligenz verbindet. Interessanterweise gehören auf auch Aufmerksamkeit und Wahrnehmung und Orientierung mit dazu – Fähigkeiten, die beispielsweise in der Robotik sehr wichtig sind. Kreativität und Vorstellung zählen aber auch mit in den Bereich der kognitiven Fähigkeiten und damit zum Begriff Intelligenz. In der KI werden diese Gebiete zunehmend interessant, etwa beim Komponieren von Musik und beim Erzeugen von Bildern oder Texten. Mit Emotionen beschäftigt sich die KI-Forschung aktuell nur am Rande, etwa bei der Erkennung von Emotionen in Texten. Andere Gebiete der kognitiven Fähigkeiten wie Glaube und Wille spielen derzeit keine Rolle in der KI.

2.1.3. Versuch einer Definition für "KI"

> Ziel der KI ist es, Maschinen zu entwickeln, die sich verhalten, als verfügten sie über Intelligenz.
>
> – John McCarthy (1955)

Einwand: Braitenberg-Vehikel zeigen so etwas wie "intelligentes" Verhalten, sind aber noch lange nicht intelligent! Bezieht sich nur auf *Verhalten*!

> KI ist die Fähigkeit digitaler Computer oder computergesteuerter Roboter, Aufgaben zu lösen, die normalerweise mit den höheren intellektuellen Verarbeitungsfähigkeiten von Menschen in Verbindung gebracht werden …
>
> – Encyclopedia Britannica

Einwand: Dann wäre aber auch Auswendig-Lernen oder das Multiplizieren langer Zahlen als intelligent zu betrachten! Bezieht sich vor allem auf "*Denken*"!

> Artificial Intelligence is the study of how to make computers do things at which, at the moment, people are better.
>
> – Elaine Rich ("Artificial Intelligence", McGraw-Hill, 1983)

Quelle: nach (Ertel 2025, pp. 1-3)

Dazu gehört auch

- Anpassung an sich verändernde Situationen
- Erkennung von Bildern und Gesichtern und Emotionen

- Erkennung von Sprache
- Umgang mit kontextbehafteten, unvollständigen Informationen
- Ausräumen von Geschirrspülern ;-)
- Über Emotionen und Empathie verfügen

Tip

KI-Grundverordnung der EU

Die EU hat am 13. Juni 2024 die sog. “KI-Verordnung” verabschiedet (“VERORDNUNG (EU) 2024/1689 DES EUROPÄISCHEN PARLAMENTS UND DES RATES”, Document 32024R1689: Verordnung (EU) 2024/1689 des Europäischen Parlaments und des Rates vom 13. Juni 2024 zur Festlegung harmonisierter Vorschriften für künstliche Intelligenz und zur Änderung der Verordnungen (EG) Nr. 300/2008, (EU) Nr. 167/2013, (EU) Nr. 168/2013, (EU) 2018/858, (EU) 2018/1139 und (EU) 2019/2144 sowie der Richtlinien 2014/90/EU, (EU) 2016/797 und (EU) 2020/1828 (Verordnung über künstliche Intelligenz) (Text von Bedeutung für den EWR)). Dort finden Sie unter Artikel 3 “Begriffsbestimmungen” unter Absatz 1 eine Begriffsdefinition. Ein “KI-System” wird darin als ein maschinengestütztes System definiert, in irgendeiner Art für einen autonomen Betrieb ausgelegt ist und eine gewisse Anpassungsfähigkeit haben kann. Zusätzlich soll das “KI-System” aus den Eingaben Vorhersagen und Entscheidungen zu treffen oder auch Inhalte erzeugen und mit der physischen oder digitalen Umwelt interagieren können.

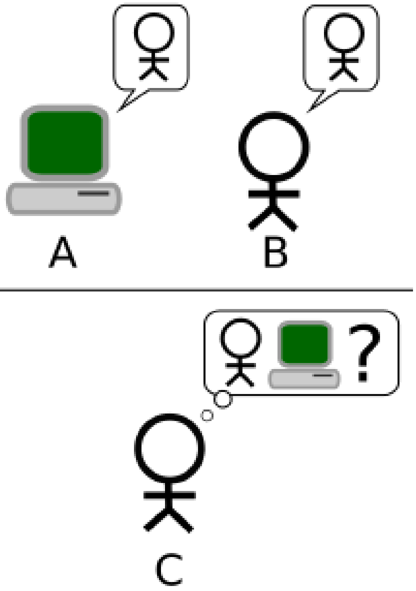
Quelle: VERORDNUNG (EU) 2024/1689 DES EUROPÄISCHEN PARLAMENTS UND DES RATES, Art. 3 Abs. 1

Interessant ist, dass dabei nicht explizit auf Softwaresysteme eingegangen wird. Später im Text finden sich Hinweise, dass sich ein KI-System vermutlich als Software repräsentiert, auch kommen Modelle und Daten vor. Bei den Modellen kommt der Begriff des Lernens vor, in allen derzeit üblichen Varianten (überwachtes Lernen, unüberwachtes Lernen, Reinforcement Learning). Große Teile des Dokuments beschäftigen sich mit weitreichenden Bestimmungen für Akteure, die ein KI-System zur Verfügung stellen wollen.

Lesen Sie selbst: VERORDNUNG (EU) 2024/1689 DES EUROPÄISCHEN PARLAMENTS UND DES RATES.

2.1.4. Alan Turing 1950: Turing-Test (begründet Zweige der KI)

- Wann verhält sich eine Maschine intelligent?



Quelle: Turing Test version 3.png by Bilby on Wikimedia Commons (Public Domain)

Zum Bestehen des Turing-Tests ist (u.a.) erforderlich:

- *Wissensrepräsentation*: Speichern des gesammelten Wissens → **Wissensbasierte Systeme**
- *Logisches Schließen*: Beantworten von Fragen mit Hilfe des vorhandenen Wissens → **Logik, Resolution**
- *Maschinelles Lernen*: Anpassung an veränderliches Umfeld → **Musteranalyse und Mustererkennung und Mustervorhersage**
- *Verarbeitung natürlicher Sprache*: Erfolgreiche Kommunikation, beispielsweise in Englisch → **NLP**

"Totaler Turing-Test": zusätzlich **Computer Vision** (Erkennen von Objekten) und **Robotik** (Manipulation von Objekten)

Damit begründet der Turing-Test die Gebiete der KI.

Problem: Der Turing-Test ist nicht reproduzierbar, nicht konstruktiv und nicht mathematisch analysierbar … Außerdem wird durch den Turing-Test im Wesentlichen nur Funktionalität geprüft, nicht ob Intention oder Bewusstsein vorhanden ist.

2.1.5. Starke vs. schwache KI

2.1.5.1. "Schwache KI"

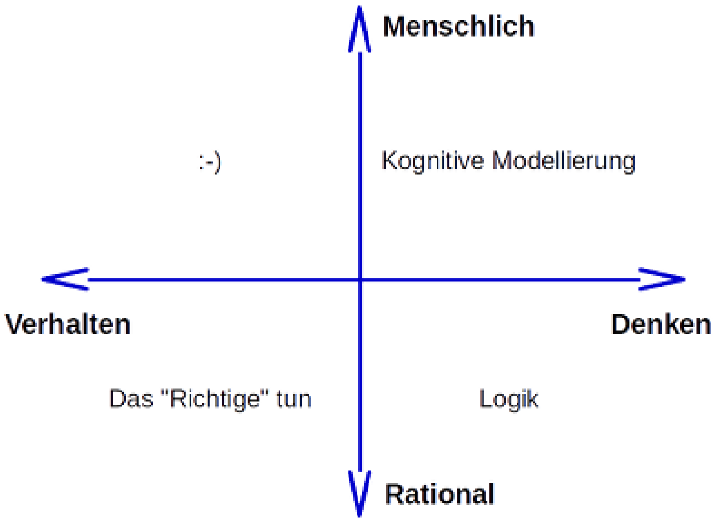
- Simulation intelligenten Verhaltens
- Lösung konkreter Probleme
- Adaptives Verhalten ("Lernen")
- Umgang mit Unsicherheit und unvollständigen Informationen

2.1.5.2. "Starke KI"

- Eigenschaften der "schwachen KI"
- Intelligenz nach menschlichen Maßstäben (auf "Augenhöhe")
- Bewusstsein
- Emotionen (?)
- Empathie

> **Tip**
>
> **Frage**
> Wie würden Sie Systeme wie ChatGPT einordnen? Woran machen Sie das fest?

2.1.6. Typische Ansätze in der KI


Menschlich
:-)
Kognitive Modellierung
Verhalten
Denken
Das "Richtige" tun
Logik
Rational

Untersuchung von

- Verhalten vs. Denken
- Rational vs. menschlich

Motivation dabei

- Menschliche Intelligenz verstehen
- Intelligente/intelligent wirkende Systeme bauen

Damit erhält man vier Kombinationen:

- Menschliches Denken
- Rationales Denken
- Rationales Verhalten
- Menschliches Verhalten

2.1.6.1. Menschliches Denken: Kognitive Modellierung

- Welche kognitiven Fähigkeiten sind für intelligentes Verhalten nötig? Wie laufen Denkprozesse im Gehirn ab?
- Erfordert Theorien über interne Aktivitäten des Gehirns
- Ansätze:
  - top-down: Vorhersage und Test von menschlichem Verhalten
  - bottom-up: Auswertung neurobiologischer Daten
- Wissenschaftszweige: Kognitionswissenschaft (Verbindung der Computermodelle der KI mit den Experimenten und Theorien der Psychologie), Neurobiologie/-informatik

> **Tip**
>
> **Neuronale bzw. Konnektionistische KI**
> Die Schule der sogenannten "**Neuronalen bzw. Konnektionistischen KI**" verfolgt den Ansatz, die biologischen Prozesse im Gehirn zu verstehen und nachzubauen (bottom-up Ansatz) und auf reale Probleme anzuwenden.
> Dank massiver Rechenleistung, riesigen Datenmengen und geeigneten Modellen (Deep Learning) kann diese Tradition aktuell große Erfolge vorzeigen.

2.1.6.2. Rationales Denken: Aristoteles: Was sind korrekte Argumente und Denkprozesse? → Wie sollen wir denken?

Beispiel:

```
Fakt: Sokrates ist ein Mensch.
Fakt: Alle Menschen sind sterblich.
Folgerung: Sokrates ist sterblich.
```

- Formalisierte Problembeschreibung
- Problemlösung durch *logische Prozesse*
- Verbindung von moderner KI zur Mathematik und Philosophie

> **Tip**
>
> **Symbolische KI**
> Die Schule der sogenannten "**Symbolische KI**" verfolgt den top-down-Ansatz, mit Hilfe formaler Beweise Schlüsse zu ziehen und damit Fragen zu beantworten bzw. Probleme zu lösen. Dabei wird die betrachtete "Welt", also Gedanken, Konzepte und Beziehungen zwischen Objekten durch Symbole und Formeln repräsentiert, ähnlich der Art und Weise, wie Menschen logisch denken und kommunizieren.
> Das Hauptproblem dieser Tradition liegt im Aufwand bei der geeigneten Formalisierung der realen Welt.

2.1.6.3. Rationales Verhalten: Das “Richtige” tun

- Das “Richtige”: Verhalten zum Erzielen des besten (erwarteten) Ergebnisses (unter Berücksichtigung der verfügbaren Informationen)

  Ein System ist rational, wenn es das seinen Kenntnissen nach “Richtige” macht.
- Denken ist nicht unbedingt notwendig (zb. Reflexe können auch rationales Verhalten sein)
- Interessant: “richtige” Handlung unter unvollständigen/unsicheren Informationen

> **Tip**
>
> **Statistische KI**
> Die Schule der sogenannten “**Statistischen KI**” verfolgt einen Ansatz, der sich stark auf statistische Methoden und Modelle stützt, um Muster in Daten zu erkennen und Entscheidungen zu treffen, also um aus großen Datenmengen Erkenntnisse zu gewinnen und Vorhersagen zu treffen.
> Aus der Analyse von Datenpunkten und deren Merkmalen werden Wahrscheinlichkeiten für bestimmte Ereignisse berechnet, beispielsweise in Regressionsanalysen, Klassifizierungsverfahren oder Zeitreihenanalysen.
> Diese Tradition spielt eine zentrale Rolle in zahlreichen Anwendungsbereichen wie Gesundheitswesen, Finanzen, Marketing und vielen weiteren.

2.1.6.4. Menschliches Verhalten: Na ja, Sie wissen schon ;-)

2.1.7. Modelle, Lernen und Vorhersagen

In der Informatik allgemein und auch in der KI versuchen wir, Probleme der realen Welt mit Hilfe von künstlichen Systemen (Algorithmen, Software) zu lösen. Dafür brauchen wir zunächst ein abstraktes mathematisches **Modell** der Welt, in der wir uns bewegen. Das Modell sollte alle für das zu lösende Problem relevanten Aspekte der Welt repräsentieren - und möglichst nicht mehr als diese, um unnötige Komplexität zu vermeiden. Es kommt häufig vor, dass selbst die relevanten Aspekte zu umfangreich oder teilweise sogar unbekannt sind und nicht vollständig dargestellt werden können. Modelle stellen also eine Abstraktion der echten Welt dar und sind *verlustbehaftet*. Es gibt viele verschiedene Modelle.

Beispiel: Wir möchten von Bielefeld nach Minden fahren. Neben den offensichtlichen Parametern (Womit wollen wir fahren? Wo genau ist der Startpunkt, wo genau der Zielpunkt? Wann wollen wir fahren?) spielen in der realen Welt unendlich viele Aspekte eine Rolle: Farben, Gerüche, Licht, Beschaffenheit der einzelnen Straßen, exakte Positionen auf der Straße/im Ort … Sind diese wirklich relevant für dieses Problem? Am Ende wird es wichtig sein, eine abstrakte Darstellung zu finden, die irgendwie die Städte und Dörfer repräsentiert und die Verbindungen dazwischen. Und vermutlich muss ich wissen, wie lang die Strecken jeweils sind (oder wie lange ich brauche oder wieviel Geld mich das Abfahren kostet). Es scheint also so zu sein, dass eine Abstraktion des Problems als *Graph* sinnvoll ist: Die Knoten entsprechen den Orten, die Kanten den Straßen (oder Bahnlinien o.ä.). An den Kanten sind Kosten annotiert (Kilometer, Zeit, …). Damit ignorieren wir die Komplexität der realen Welt und fokussieren uns auf die Aspekte, die zur Lösung des Problems wichtig sind. Behalten Sie im Gedächtnis, dass unser Modell verlustbehaftet ist und wir damit tatsächlich nur das Wegeproblem lösen können! Wenn wir Wege vergessen haben oder falsch bewertet haben, wird unser Algorithmus später möglicherweise nicht die gewünschte Lösung finden! Wir schauen uns das Thema Modellierung am Beispiel des Problemlösens und insbesondere für Suchprobleme in der Lektion Problemlösen noch genauer an.

Ein Modell kann Parameter haben. Im obigen Beispiel wären dies die Werte an den Kanten. Es kann sein, dass diese Werte nicht im Vorfeld bekannt sind, sondern aus einem Datensatz extrahiert werden müssen. Dies nennt man **Lernen**: Das Modell wird (besser gesagt: die Parameter des Modells werden) an das Problem angepasst. Dafür gibt es unterschiedliche Algorithmen. In der Regel benötigt man ein Ziel für den Adaptionsprozess: eine sogenannte Ziel- oder Kostenfunktion. Anpassen der Modellparameter mit Hilfe von Daten und einer Zielfunktion bedeutet auch, dass man das Ziel möglicherweise nie zu 100% erreicht, sondern nur in die Nähe kommt. Je nach Problem kann man auch nur eine Modellfamilie vorgeben und den konkreten Aufbau des Modells im Trainingsprozess erarbeiten lassen.

Wichtig: Lernen bietet sich immer dann an, wenn eine analytische Lösung nicht möglich ist (fehlende Informationen, Komplexität des Problems). Das bedeutet im Umkehrschluss aber auch: Wenn eine analytische Lösung bekannt ist (oder zu finden ist), dann gibt es keinen Grund für den Einsatz von adaptiven Systemen!

Mit dem Modell der Welt kann nun das Problem gelöst werden. Dazu wird das Modell mit Daten versorgt (im obigen Beispiel: Start und Ziel) und ein passender Algorithmus kann auf dem Modell die Lösung berechnen. Dies kann eine **Vorhersage** sein, welchen Weg ich nehmen soll, wie lange es dauern wird, welchen Spielzug ich als nächstes machen sollte, ob in einem Bild eine Katze zu sehen ist, … Es könnte aber auch im Fall von sogenannten *generativen Modellen* ein erzeugter Text oder ein erzeugtes Bild sein.

Hinweis: In manchen Quellen wird dieser Vorgang auch “Inferenz” genannt. Da dieser Begriff aus der Logik stammt und mit bestimmten Prozessen zur Schlussfolgerung verbunden ist, möchte ich in diesem Skript diesen Begriff nicht für das Generieren einer Vorhersage nutzen.

2.1.8. Modellkomplexität

In der KI werden sehr unterschiedliche Modelle betrachtet, die auch eine sehr unterschiedliche Komplexität aufweisen.

Besonders einfache Modelle sind Modelle, die für einen Input direkt einen Output berechnen. Für jeden Input wird eine feste Berechnung durchgeführt, es erfolgt kein Backtracking und es gibt keine (inneren) Zustände. Dies ähnelt dem reflexartigen Verhalten in biologischen Vorbildern, weshalb diese Modell oft auch “*reflex-based models*” genannt werden. In diese Kategorie fallen beispielsweise lineare Regression und das Perzeptron, aber auch Modelle mit vielen Parametern wie ein Multilagen-Perzeptron (MLP) oder die daraus abgeleiteten Deep Neural Networks.

In der nächst komplexeren Stufe haben die Modelle einen internen Zustand (“*state-based models*”). Darüber wird ein Zustand der betrachteten Welt modelliert. Zwischen den Zuständen gibt es Übergänge (sogenannte Aktionen), so dass sich hier ein Graph aufspannt (der sogenannte Problemgraph, vgl. Problemlösen). In diese Klasse fallen die typischen Suchprobleme (wie Breitensuche, Tiefensuche, A*), aber auch Spiele mit Zufallskomponente oder mit gegnerischen Mitspielern.

Noch eine Stufe komplexer sind Modelle mit Variablen (“*variable-based models*”). Während es bei den zustandsbasierten Modellen immer (auch) um den Weg zwischen den Zuständen geht und damit um eine prozedurale

Beschreibung, wie von einem Startzustand zu einem Zielzustand zu gelangen ist, steht bei Modellen mit Variablen nur die Lösung im Vordergrund: Das Modell enthält verschiedene Variablen, denen ein passender Wert aus einem Wertebereich zugeordnet werden muss. Wie diese Belegung entsteht, ist am Ende nicht mehr so interessant. Denken Sie beispielsweise an ein Sudoku oder die Erstellung eines Stundenplans. Die Variablen sind entsprechend die einzelnen Felder, gesucht ist eine insgesamt korrekte Belegung aller Felder. In diese Klasse fallen Constraint Satisfaction Probleme (CSP), aber auch Bayes'sche Netze und die sogenannte "lokale Suche".

Auf der höchsten Komplexitätsstufe stehen logische Modelle. Hier wird Wissen über die Welt in Form von Fakten und Regeln modelliert, und über eine entsprechende Anfrage wird daraus mit Hilfe von formal definierten Beweisen eine korrekte Antwort generiert. Dies nennt man auch "*Inferenz*". Hier kommt beispielsweise das Prädikatenkalkül zum Einsatz oder die Programmiersprache Prolog.

2.1.9. Kurzer Geschichtsüberblick – Wichtigste Meilensteine

- 1943: McCulloch/Pitts: binäres Modell eines Neurons
- 1950: Turing-Test
- 1956: Dartmouth Workshop (Minsky, McCarthy, …) – McCarthy schlägt den Begriff "Artificial Intelligence" vor
- 1960er: Symbolische KI (Theorembeweiser), Blockwelt, LISP
- 1970er: Wissensbasierte System (Expertensysteme)
- 1980er: zunächst kommerzieller Erfolg der Expertensysteme, später Niedergang ("KI-Eiszeit"); Zuwendung zu Neuronalen Netzen
- 1990er: Formalisierung der KI-Methoden, Einführung probabilistischer Methoden (Bayes'sches Schließen), verstärkte mathematische Fundierung
- heute: friedliche Koexistenz verschiedener Paradigmen und Methoden; Rückkehr zu "Human-Level AI" (McCarthy, Minsky, Nilsson, Winston); Rückkehr zu Neuronalen Netzen (CNN/RNN)

Siehe auch Abbildung 1.3 in (Ertel 2025, 8) …

2.1.10. Wrap-Up

- Definition von "Intelligenz" nicht ganz einfach
- Dimensionen: Denken vs. Verhalten, menschlich vs. rational
- Ziele der KI: Verständnis menschlicher Fähigkeiten, Übertragen auf künstliche Systeme

Schauen Sie sich zur Einführung auch gern das YouTube-Video Overview Artificial Intelligence Course | Stanford CS221 an. (Vorsicht: Das ist recht lang.)

Zum Nachlesen

- Russell und Norvig (2021): Kapitel 1: Introduction
- Ertel (2025): Kapitel 1: Introduction

Lernziele

- k1: Ich kenne verschiedene Aspekte, die zur Intelligenz gerechnet werden
- k1: Ich kenne den Turing-Test und kann Aufbau, Gebiete, Kritik wiedergeben
- k1: Ich kenne verschiedene Gebiete der KI sowie deren Zielsetzung
- k1: Ich kann starke vs. schwache KI unterscheiden
- k1: Ich kenne wichtige Strömungen in der KI und ihre zeitliche Einordnung

Quizzes

- Selbsttest KI Einführung (ILIAS)

2.2. Intro: Einführung in Jupyter-Notebooks

Videos

Video-Mitschnitt “Einführung in Jupyter-Notebooks

Weitere Unterlagen

starter.ipynb

2.3. Intro2: Problemlösen

TL;DR

Um ein Problem lösen zu können, muss es zunächst geeignet dargestellt werden. In der KI betrachten wir Zustände einer Welt, auf die Aktionen angewendet werden können und die die betrachtete Welt in den/einen Folgezustand bringen. Hier muss unterschieden werden zwischen deterministischen und stochastischen Welten, ebenso spielt die Beobachtbarkeit durch den Agenten (die die Welt betrachtende und durch die Aktionen auf die Welt einwirkende Instanz) eine Rolle: Kann der Agent die Welt komplett beobachten, nur einen Teil davon oder gar nichts? Es spielt auch eine Rolle, ob es diskrete Zustände gibt, oder ob man mit kontinuierlichen Variablen zu tun hat. Gibt es nur einen Agenten oder können mehrere Agenten beteiligt sein … (In dieser Veranstaltung werden wir uns auf deterministische und voll beobachtbare Welten mit diskreten Zuständen konzentrieren.)

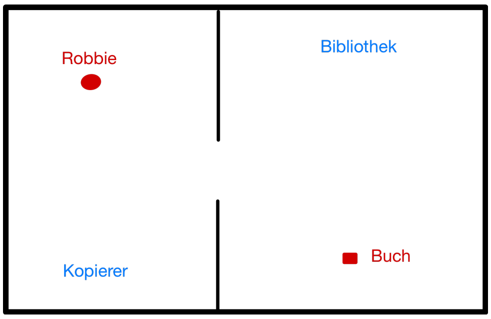
Dies alles muss bei der Modellierung betrachtet werden. Es empfiehlt sich, die Zustände und die Aktionen so abstrakt wie möglich zu beschreiben. Anderenfalls kann später die Lösung des Problems zumindest stark erschwert werden.

Durch das wiederholte Anwenden der Aktionen auf den Startzustand bzw. auf die sich daraus ergebenden Folgezustände wird der Zustandsraum aufgebaut. Dabei ist zu beachten, dass Aktionen Vorbedingungen haben können, d.h. unter Umständen nicht auf alle Zustände angewendet werden können. Die entstehende Struktur (Zustandsraum) kann man formal als Graph repräsentieren: Die Zustände werden durch die Knoten und die Aktionen als (gerichtete) Kanten im Graph dargestellt ($\rightarrow$ Problemgraph).

> Das Problemlösen entspricht nun einer Suche im Problemgraphen: Man sucht einen Weg von einem Startknoten zu einem Zielknoten, d.h. eine Folge von Aktionen, die den Start- in den Zielzustand überführen. Der Weg entspricht dann der Lösung des Problems. Normalerweise will man eine bestimmte Qualität der Lösungen haben: Es sollen die kürzesten Wege (also die mit den wenigsten Zwischenstationen/Knoten) oder die billigsten Wege (die Summe der an den Kanten annotierten Gewichte soll minimal sein) gefunden werden.
> Zur Suche kann man bei den in dieser Veranstaltung betrachteten deterministischen Problemen mit diskreten Zuständen den einfachen "Tree-Search"-Algorithmus (Benennung in Anlehnung an (Russell und Norvig 2021)) einsetzen, der allerdings Wiederholungen und Schleifen zulässt. Mit zwei Erweiterungen wird daraus der "Graph-Search"-Algorithmus (Benennung in Anlehnung an (Russell und Norvig 2021)), der die wiederholte Untersuchung von bereits besuchten Knoten vermeidet. In beiden Algorithmen wird eine zentrale Datenstruktur eingesetzt (im (Russell und Norvig 2021) auch "Frontier" genannt), die die als Nächstes zu untersuchenden Knoten hält und die damit die Grenze zwischen dem bereits untersuchten Teil des Graphen und dem unbekannten Teil des Graphen bildet. Je nach Art der Datenstruktur und je nach den betrachteten Kosten ergeben sich eine Reihe unterschiedlicher Suchalgorithmen, die wir in einer späteren Sitzung betrachten.
> Die Suchverfahren können im Hinblick auf Optimalität, Vollständigkeit und Komplexität beurteilt werden.

> **Videos**
>
> Vorlesung [YT], [HSBI]

2.3.1. Motivation: Roboter in einer Bibliothek



Aktionen:

- Right (R)
- Left (L)
- Take (T)
- Drop (D)

Wahrnehmungen:

- In welchem Raum bin ich?

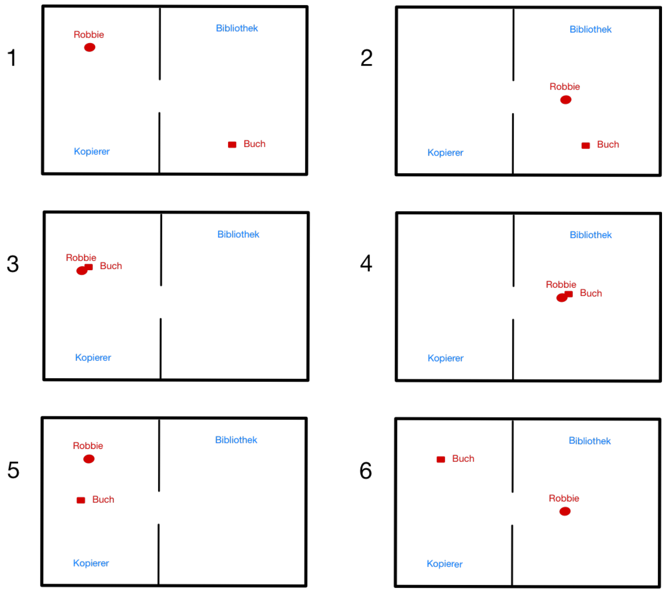
- Habe ich das Buch?

Aufgabe: Das Buch aus der Bibliothek holen und in den Kopiererraum bringen.

Bemerkungen zur Umwelt:

- Beobachtbarkeit der Umwelt kann variieren: “voll beobachtbar” bis zu “unbeobachtbar”
- Umwelt kann “deterministisch” oder “stochastisch” sein: Führt eine Aktion in einem Zustand immer zum selben Folgezustand?
- Wann erfolgt die Rückmeldung an den Agenten über die Auswirkung der Aktionen? Sofort (“sequentiell”) oder erst am Ende einer Aktionsfolge (“episodisch”)?
- Wird die Umwelt nur durch die Aktionen des Agenten verändert (“statisch”)? Oder verändert sich die Umwelt zwischen den Aktionen eines Agenten, beispielsweise durch andere Agenten (“dynamisch”)?
- Gibt es diskrete Zustände (wie im Beispiel)?

2.3.2. Zustände der Bibliotheks-Welt


1
Robbie
Bibliothek
Kopierer
Buch
2
Bibliothek
Robbie
Kopierer
Buch
3
Robbie
Buch
Bibliothek
Kopierer
4
Bibliothek
Robbie
Buch
Kopierer
5
Robbie
Bibliothek
Buch
Kopierer
6
Buch
Bibliothek
Robbie
Kopierer

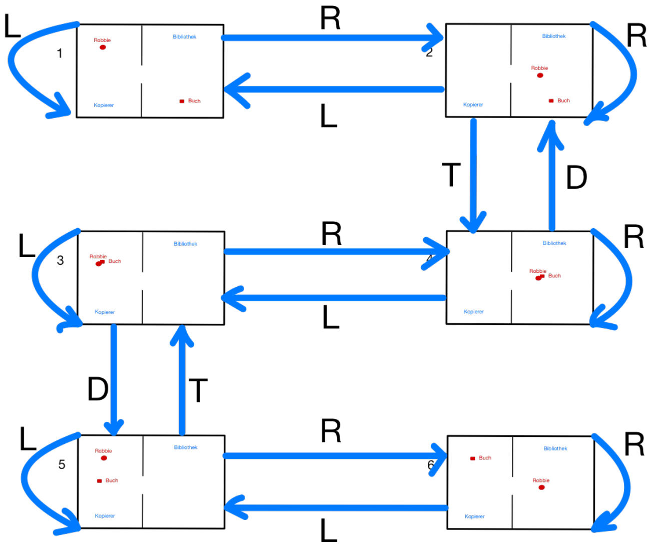
Problem: Gegeben einen Startzustand, wie komme ich zum Ziel?

- Welche Aktionen können in einem Zustand (zb. Nr. 4) angewendet werden?
- Welche Aktionen können in den Folgezuständen angewendet werden?

Ergebnis:

- Zustandsraum: Menge aller von den Startzuständen aus erreichbaren Zustände
- Problemgraph: Repräsentation der Zustände und Aktionen als Knoten und (gerichtete) Kanten

2.3.3. Suche im Problemgraphen



- Durch die Suche im Problemgraphen wird ein Suchbaum aufgespannt
- Varianten: Zustände können in einem Pfad wiederholt vorkommen vs. Wiederholungen werden ausgeschlossen

2.3.4. Definition Zustand und Aktion

- **Zustand**: (Formale) Beschreibung eines Zustandes der Welt
- **Aktion**: (Formale) Beschreibung einer durch Agenten ausführbaren Aktion
  - Anwendbar auf bestimmte Zustände
  - Überführt Welt in neuen Zustand ("Nachfolge-Zustand")

Geeignete Abstraktionen wählen für Zustände und Aktionen!

Anmerkung: (Russell und Norvig 2021) unterscheidet zw. Aktionen und Transitionsmodell; hier nur Aktionen! D.h. die Aktionen und das Übergangsmodell aus dem (Russell und Norvig 2021) werden direkt zusammen betrachtet. Bei den hier diskutierten Problemen ist das ohne Nachteile möglich, es wird lediglich etwas Flexibilität genommen bzw. Komplexität vermieden (je nach Sichtweise :-) …

2.3.5. Definition Problem

Ein Problem besteht aus:

- **Startzustände**: Menge $S_A \subset S$

- **Aktionen**: Menge von Funktionen $\text{op} : S \to S$
- **Zustandsraum**: Menge aller Zustände S, die durch (wiederholte) Anwendung von Aktionen von den Startzuständen aus erreichbar sind

- **Zieltest**: Funktion $\text{goal} : S \to \{0, 1\}$
- **Zielzustände**: Menge $S_E \subseteq S$ mit $\forall x \in S_E : \text{goal}(x) = 1$

- **Kosten**: **Gesamtkosten**: $f(n) = g(n) + h(n)$
  - $n \in S$ auf dem aktuellen Weg erreichter Knoten
  - $g(n)$ tatsächliche Kosten für den Weg vom Start bis zu Knoten n
  - $h(n)$ geschätzte Restkosten für den Weg von Knoten n zum Ziel

2.3.6. Hinweis: Unterschied Zustand und Knoten bzw. Zustandsraum und Problemgraph

- Zustände und Aktionen kann man als einen Graph darstellen: **Problemgraph**
  - Zustände werden als Knoten im Graphen abgebildet
  - Aktionen werden als (gerichtete) Kanten im Graphen abgebildet
- Unterscheidung "Zustand" und "Knoten":
  - Zustand: Beschreibung/Modellierung eines Zustandes der Welt
  - Knoten: Datenstruktur, Bestandteil des Graphen, *symbolisiert* einen Zustand

Das bedeutet, dass der Problemgraph eine Repräsentation des Zustandsraumes ist.

Die beiden Begriffe werden normalerweise synonym verwendet, sofern eindeutig ist, was gemeint ist.

2.3.7. Definition Problemlösen

- Problemlösen: Wegesuche im Graph vom Startknoten zu einem Zielknoten
  - Spannt den **Suchbaum** auf

- **Lösung**: Folge von Aktionen, die Start- in Zielzustand überführen
  - Ergebnis des Problemlösens

2.3.8. Suche: Einfache Basisvariante

1. Füge Startknoten in leere Datenstruktur (Stack, Queue, …) ein
2. Entnehme Knoten aus der Datenstruktur:
    - Knoten ist gesuchtes Element: Abbruch, melde "*gefunden*"
    - Expandiere alle Nachfolger des Knotens und füge diese in die Datenstruktur ein
3. Falls die Datenstruktur leer ist: Abbruch, melde "*nicht gefunden*"
4. Gehe zu Schritt 2

Für die in dieser Veranstaltung betrachteten deterministischen Probleme mit diskreten Zuständen ist diese Basisvariante der Suche eine Art generischer Suchalgorithmus: Durch die Variation der eingesetzten Datenstruktur und durch die Betrachtung unterschiedlicher Kosten erhält man die in den nächsten Sitzungen betrachteten verschiedenen klassischen Suchalgorithmen.

Anmerkung: Für Handsimulation besserer Überblick, wenn statt der Knoten immer **partielle Wege** in Datenstruktur gespeichert werden!

Anmerkung: Im (Russell und Norvig 2021, Abschn. 3.3.3, S.92) wird ein Algorithmus mit den vorgestellten Eigenschaften als "**tree-like search**" bezeichnet. In Anlehnung an (Russell und Norvig 2021) wird diese Basisvariante der Suche in dieser Lehrveranstaltung kurz als "Tree-Search"-Algorithmus bezeichnet.

Anmerkung: Im (Russell und Norvig 2021) wird für die Datenstruktur, mit der die Suche arbeitet, auch "*Frontier*" genannt. Hier werden alle Knoten gehalten, die in einem der nächsten Schritte betrachtet werden sollen, d.h. diese Knoten bilden die Grenze zwischen dem bereits untersuchten Teil des Graphen und dem noch unbekannten Teil des Graphen (deshalb auch "Frontier").

2.3.9. Erweiterung der Suche: Vermeiden von Wiederholungen

1. Füge Startknoten in leere Datenstruktur (Stack, Queue, …) ein
2. Entnehme Knoten aus der Datenstruktur:
    - Knoten ist gesuchtes Element: Abbruch, melde "*gefunden*"
    - Markiere aktuellen Knoten, und
    - Expandiere alle Nachfolger des Knotens und füge alle unmarkierten Nachfolger, die noch nicht in der Datenstruktur sind, in die Datenstruktur ein
3. Falls die Datenstruktur leer ist: Abbruch, melde "*nicht gefunden*"
4. Gehe zu Schritt 2

Dieser Algorithmus ist eine Erweiterung der einfachen Basisvariante der Suche:

1. Man markiert bereits besuchte (expandierte) Knoten und besucht diese nie wieder (man würde diese bei einer Expansion nicht wieder in die Datenstruktur aufnehmen).
2. Außerdem vermeidet man, dass ein Knoten mehrfach in der Datenstruktur vorkommt: Dies würde bedeuten, dass man hier verschiedene Wege vom Start zu diesem Knoten in der Datenstruktur hat, die dann auch alle weiter untersucht werden müssten. In der Regel reicht aber ein Weg vom Start zu einem Zwischenknoten (meist wird der kürzeste genommen, dazu in einer späteren Sitzung mehr).

Anmerkung: Für Handsimulation besserer Überblick, wenn statt der Knoten immer **partielle Wege** in Datenstruktur gespeichert werden!

Anmerkung: Im (Russell und Norvig 2021, Abschn. 3.3.3, S.92) wird ein Algorithmus mit den vorgestellten Eigenschaften als "**graph search**" bezeichnet. In Anlehnung an (Russell und Norvig 2021) wird diese erweiterter Variante der Suche in dieser Lehrveranstaltung kurz als "Graph-Search"-Algorithmus bezeichnet.

2.3.10. Bewertung von Suchalgorithmen

- **Vollständigkeit**: Findet der Algorithmus eine Lösung, wenn es eine gibt?
- **Optimalität**: Findet der Algorithmus die beste Lösung?
- **Zeitkomplexität**: Wie lange dauert es eine Lösung zu finden?
- **Speicherkomplexität**: Wieviel Speicher benötigt die Suche?

Größen zur Bewertung:

- **b**: Verzweigungsfaktor
- **d**: Ebene (Tiefe) des höchsten Lösungsknotens
- **m**: Länge des längsten Pfades

2.3.11. Wrap-Up

- Begriffe "Problem", "Zustand", "Aktion", "Zustandsraum", "Problemgraph", "Suchbaum"

- Problemlösen: Suche in Graphen nach Weg vom Start zum Ziel
  - Suche spannt einen Suchbaum auf
  - Unterschiedliche Kostenfunktionen möglich
  - Suchalgorithmen: Einfache Basisvariante, Erweiterung mit Vermeidung von Redundanzen
  - Beurteilung der Suchverfahren: Optimalität, Vollständigkeit, Komplexität

Zum Nachlesen

- Russell und Norvig (2021): Problemlösen: Kapitel 3.1 - 3.3

Lernziele

- k2: Ich kann die Definition "Problem" erklären und dabei die Begriffe Zustand, Aktion, Zustandsraum, Problemgraph und Suchbaum erläutern
- k2: Ich kann Problemlösen als Suche nach Wegen im Problemgraph erklären
- k2: Ich kann den Unterschied zw. Tree-Search und Graph-Search an einem Beispiel erklären

Quizzes

- Selbsttest Problemlösen (ILIAS)

Challenges

Sie stehen in einem Dungeon, der in `N x M` Felder unterteilt ist. Die Felder können entweder aus festem Boden oder einem Abgrund bestehen, dies ist aber zunächst nicht sichtbar und muss durch schrittweises Erkunden festgestellt werden. Zur Sicherheit wollen Sie alle Felder mit festem Boden mit einer Leuchtfarbe markieren.

1. Formalisieren Sie das Problem (Zustände, Aktionen, Start- und Endzustand).
2. Skizzieren Sie den Problemgraph.

3. Suche

Problemlösen durch Suche im Problemgraphen. Aus den Basisalgorithmen Tree-Search und Graph-Search entstehen je nach verwendeter Datenstruktur und nach betrachteten Kosten unterschiedliche Suchalgorithmen.

- Uninformierte Suche: … jeder Schritt “kostet” gleich viel: nur die Anzahl der Schritte zählt …
- Informierte Suche: … Einsatz einer Kostenfunktion …
- Lokale Suche: … das Ziel ist im Weg …

3.1. Search1: Suche mit Tiefensuche

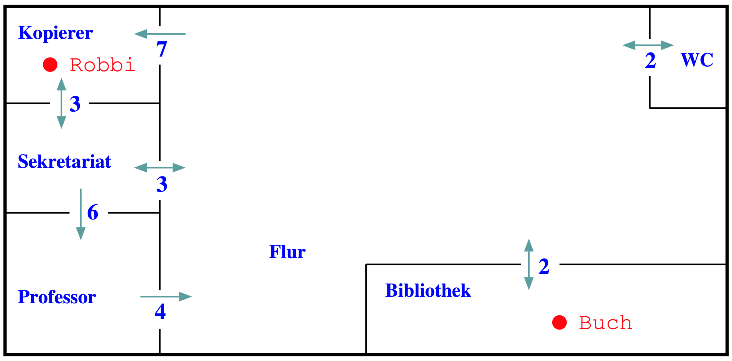
TL;DR

Die Tiefensuche gehört zu den “Uninformierten Suchverfahren”: Es werden keine weiteren Pfadkosten, sondern nur die Anzahl der Schritte berücksichtigt.
Die Tiefensuche entsteht, wenn man bei der Tree-Search oder der Graph-Search für die Datenstruktur einen **Stack** benutzt: Expandierte Nachfolger werden immer **oben** auf den Stack gelegt, und der nächste zu expandierende Knoten wird **oben** vom Stack genommen. Dadurch verfolgt die Tiefensuche einen Pfad immer erst in die Tiefe.
Bei Sackgassen erfolgt automatisch ein Backtracking, d.h. es wird zum letzten Knoten mit einer Alternative zurückgegangen. Dies liegt daran, dass bei einer Sackgasse keine Nachfolger expandiert und oben auf den Stack gelegt werden.

Videos

Vorlesung [YT], [HSBI]

3.1.1. Hole das Buch



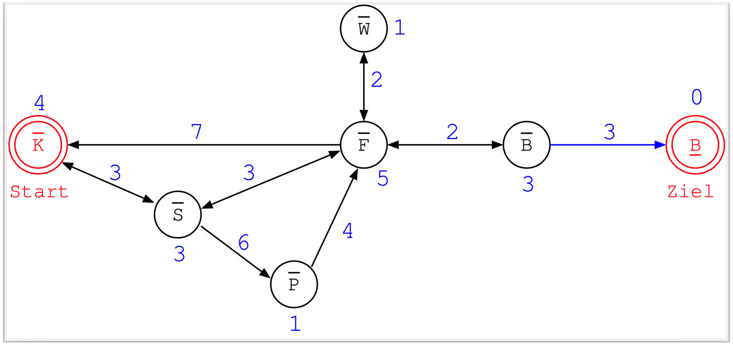
Das Beispiel ist ein Büroflur in der Uni. Neben den Büros gibt es eine Bibliothek und einen Kopiererraum, wo auch der Roboter sich gerade aufhält. Die Aufgabe für den Roboter lautet: Hole das Buch aus der Bibliothek (und bringe es zum Kopier). (Damit das Beispiel und der sich daraus ergebende Problemgraph nicht zu groß und zu unübersichtlich werden, soll das Ziel hier darin liegen, dass der Roboter das Buch in der Bibliothek aufnimmt.)

Es stehen zwei Aktionen zur Verfügung:

1. Der Roboter kann von einem in den nächsten Raum wechseln (Kosten siehe Pfeile)
2. Der Roboter kann das Buch aufnehmen (Kosten: 3)

Dabei sind die Durchgänge teilweise nur in einer Richtung zu benutzen (Pfeilrichtung).

3.1.2. Problemgraph zum Kopiererbeispiel



→ **Problemlösen == Suche im Graphen**

Uninformierte ("blinde") Suche:

Keine Informationen über die Kosten eines Pfades: Nur die Pfadlänge (Anzahl der Schritte) zählt.

Varianten:

- **Tiefensuche**
- Breitensuche

3.1.3. Anmerkungen Wegesuche (Landkarte)

Bei der Wegesuche hat man den Problemgraphen bereits durch die Orte und die Verbindungen (Straßen) zwischen ihnen gegeben. Es gibt nur eine ausführbare Aktion: "*fahre nach*".

Dabei können nur die Anzahl der Zwischenstationen auf dem Weg gezählt werden ("uninformierte Suche"), oder man ordnet den Kanten Kosten zu (bei der Wegesuche wären dies die Entfernungen zwischen den Orten oder die Zeit, die man von A nach B braucht) und landet damit bei der "informierten Suche".

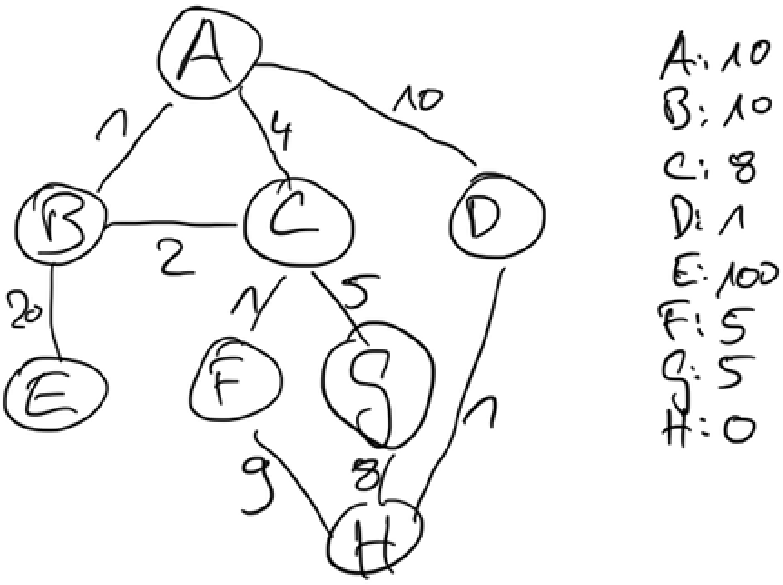
Normalerweise hat man eine Ordnung auf den Aktionen, d.h. für einen Knoten ergibt sich daraus eine Reihenfolge, in der die Aktionen angewendet werden und die Nachfolger expandiert werden. Bei der Wegesuche hat man dies nicht, insofern muss man willkürlich eine Ordnung festlegen. In dieser Veranstaltung ist dies die alphabetische Reihenfolge der Knoten (Orte).

3.1.4. Tiefensuche (*TS*, *DFS*)

Erinnerung Tree-Search

1. Füge Startknoten in leere Datenstruktur (Stack, Queue, …) ein
2. Entnehme Knoten aus der Datenstruktur:
    - Knoten ist gesuchtes Element: Abbruch, melde "*gefunden*"
    - Expandiere alle Nachfolger des Knotens und füge diese in die Datenstruktur ein
3. Falls die Datenstruktur leer ist: Abbruch, melde "*nicht gefunden*"
4. Gehe zu Schritt 2

→ Was passiert, wenn wir einen **Stack** einsetzen?



3.1.5. Bemerkungen

- Nachfolger eines Knotens: Alle von diesem Zustand durch Aktionen erreichbare Zustände
- Suchalgorithmus mit **Stack** als Datenstruktur → **Tiefensuche**
    - Zu betrachtender Knoten in Schritt 2 wird *oben* vom Stack genommen
    - Expandierte Knoten werden in Schritt 2.a *oben* auf den Stack gelegt Dabei i.A. die vorgegebene Reihenfolge der Nachfolgeknoten beachten!

Auswirkung: Weg wird in die **Tiefe** verfolgt (deshalb "Tiefensuche")

- Im (Russell und Norvig 2021) wird die Datenstruktur zum Halten der zu expandierenden Knoten (also hier im Fall der Tiefensuche der Stack) auch "**Frontier**" genannt.
- **Backtracking**: Wenn der Weg in eine Sackgasse führt, d.h. ein Knoten keine Nachfolger hat, werden bei der Expansion des Knotens keine Nachfolger auf den Stack gelegt. Die Evaluation des nächsten Knotens auf dem Stack bewirkt deshalb ein *Zurückspringen* im Suchbaum zum letzten Knoten auf dem aktuellen Weg mit noch offenen Alternativen …

3.1.6. Konventionen für diese Lehrveranstaltung

In der Beschreibung der Algorithmen werden häufig nur die letzten Knoten der partiellen Wege in den Datenstrukturen mitgeführt (das gilt auch für die Beschreibung im (Russell und Norvig 2021)). Dies erschwert die Nachvollziehbarkeit, wenn man die Queue oder den Stack schrittweise aufschreibt. Deshalb wird für diese Veranstaltung die Konvention eingeführt, immer die **partiellen Wege** aufzuschreiben.

Nicht Bestandteil der Algorithmen, dient aber der Nachvollziehbarkeit: Expandierte Knoten sollen alphabetisch sortiert an der korrekten Stelle in der Datenstruktur auftauchen, dabei soll aber natürlich die Reihenfolge der ursprünglich in der Datenstruktur enthaltenen Knoten nicht modifiziert werden. (Bei "echten" Problemen wird die Reihenfolge der expandierten Nachfolger in der Regel durch eine Reihenfolge der anwendbaren Operationen bestimmt.)

3.1.7. Weitere Hinweise

- Die Tiefensuche wurde zufällig am Beispiel Tree-Search eingeführt. Man kann auch Graph-Search einsetzen. Wichtig ist nur, dass als Datenstruktur ein **Stack** genutzt wird.
- Bei Tree-Search werden bereits besuchte Knoten u.U. immer wieder besucht. Zyklen im aktuell entwickelten Pfad sind also möglich! Außerdem sind mehrere Wege zum selben (Zwischen-/End-) Knoten in der Datenstruktur möglich!
- Im (Russell und Norvig 2021) wird der Begriff "Backtracking" für den rekursiven Tiefensuche-Algorithmus verwendet. Dies steht im Gegensatz zum üblichen Sprachgebrauch in der KI!

3.1.8. Tiefensuche (rekursive Variante)

1. Startknoten ist gesuchtes Element: Abbruch, melde "*gefunden*"
2. Für jeden Nachfolger des Startknotens:
    - Rufe Tiefensuche für aktuellen (Nachfolger-) Knoten auf
    - Ergebnis "*gefunden*": Abbruch, melde "*gefunden*"
3. Abbruch, melde "*nicht gefunden*"

3.1.8.1. Bemerkungen

- Eigenschaften wie "normale" Tiefensuche
- Einfacher zu implementieren: Nutzung des Stacks wird auf den Compiler verlagert (Funktionsaufruf, Stack des Prozesses …)
- Speicherbedarf: Für jeden Knoten wird nur der nächste Knoten expandiert, plus Speicher für die Funktion

3.1.9. Eigenschaften der Tiefensuche

Siehe Breitensuche

3.1.10. Wrap-Up

- Uninformierte Suchverfahren
  - Keine weiteren Pfadkosten (nur Anzahl der Schritte)
  - Tiefensuche: Verfolge einen Pfad zuerst in die Tiefe
  - Backtracking bei Sackgassen (automatisch durch den Stack)

Zum Nachlesen

- Russell und Norvig (2021): Tiefensuche: Abschnitt 3.4.3

Lernziele

- k2: Ich kann erklären, welche Datenstrukturen in der Tiefensuche verwendet werden und wie diese sich auswirken
- k2: Ich kann die algorithmische Abläufe und die Terminierung der Tiefensuche erklären
- k2: Ich kann Optimalität, Vollständigkeit und Komplexität für die Tiefensuche erklären
- k3: Ich kann die Tiefensuche implementieren und auf ein konkretes Beispiel anwenden

Quizzes

- Selbsttest Tiefensuche (ILIAS)

3.2. Search2: Suche mit Breitensuche

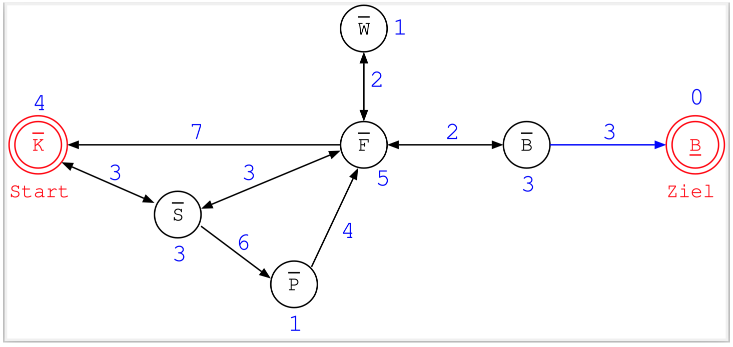
TL;DR

Die Breitensuche gehört zu den "Uninformierten Suchverfahren": Es werden keine weiteren Pfadkosten, sondern nur die Anzahl der Schritte berücksichtigt.
Die Breitensuche entsteht, wenn man bei der Tree-Search oder der Graph-Search für die Datenstruktur eine **Queue** benutzt: Expandierte Nachfolger werden immer **hinten** in die Queue eingefügt, und der nächste zu expandierende Knoten wird **vorn** aus der Queue genommen. Dadurch wird bei der Breitensuche der

> Suchbaum ebenenweise entwickelt.

> **Videos**
>
> Vorlesung [YT], [HSBI]

3.2.1. Hole das Buch



→ **Problemlösen == Suche im Graphen**

Uninformierte ("blinde") Suche:

Keine Informationen über die Kosten eines Pfades: Nur die Pfadlänge (Anzahl der Schritte) zählt.

Varianten:

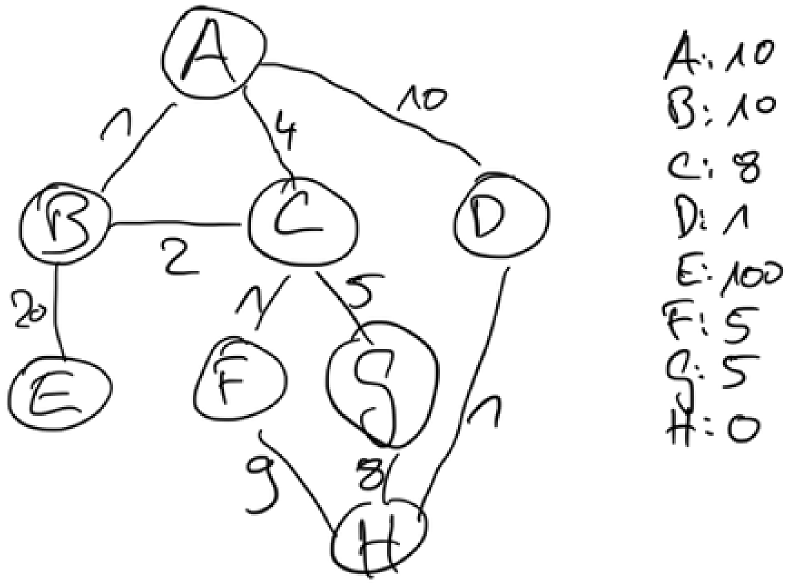
- Tiefensuche
- **Breitensuche**

3.2.2. Breitensuche (*BS*, *BFS*)

Erinnerung Graph-Search

1. Füge Startknoten in leere Datenstruktur (Stack, Queue, …) ein
2. Entnehme Knoten aus der Datenstruktur:
    - Knoten ist gesuchtes Element: Abbruch, melde "*gefunden*"
    - Markiere aktuellen Knoten, und
    - Expandiere alle Nachfolger des Knotens und füge alle unmarkierten Nachfolger, die noch nicht in der Datenstruktur sind, in die Datenstruktur ein
3. Falls die Datenstruktur leer ist: Abbruch, melde "*nicht gefunden*"
4. Gehe zu Schritt 2

→ Was passiert, wenn wir eine **Queue** einsetzen?



3.2.3. Bemerkungen

- Nachfolger eines Knotens: Alle von diesem Zustand durch Aktionen erreichbare Zustände
- Suchalgorithmus mit **Queue** als Datenstruktur → **Breitensuche**
  - Zu betrachtender Knoten in Schritt 2 wird *vorn* aus der Queue genommen
  - Expandierte Knoten werden in Schritt 2.a *hinten* in die Queue eingefügt Dabei i.A. die vorgegebene Reihenfolge der Nachfolgeknoten beachten!

  Auswirkung: Suchbaum wird **ebenenweise** aufgebaut (deshalb "Breitensuche")
- Graph-Search: Markierte Knoten müssen geeignet gespeichert werden: separate Datenstruktur → Aufwand!

3.2.4. Konventionen für diese Lehrveranstaltung

In der Beschreibung der Algorithmen werden häufig nur die letzten Knoten der partiellen Wege in den Datenstrukturen mitgeführt (das gilt auch für die Beschreibung im (Russell und Norvig 2021)). Dies erschwert die Nachvollziehbarkeit, wenn man die Queue oder den Stack schrittweise aufschreibt. Deshalb wird für diese Veranstaltung die Konvention eingeführt, immer die **partiellen Wege** aufzuschreiben.

Nicht Bestandteil der Algorithmen, dient aber der Nachvollziehbarkeit: Expandierte Knoten sollen alphabetisch sortiert an der korrekten Stelle in der Datenstruktur auftauchen, dabei soll aber natürlich die Reihenfolge der ursprünglich in der Datenstruktur enthaltenen Knoten nicht modifiziert werden. (Bei "echten" Problemen wird die Reihenfolge der expandierten Nachfolger in der Regel durch eine Reihenfolge der anwendbaren Operationen bestimmt.)

3.2.5. Weitere Hinweise

- Die Breitensuche wurde zufällig am Beispiel Graph-Search eingeführt. Man kann auch die Tree-Search-Variante einsetzen. Wichtig ist nur, dass als Datenstruktur eine Queue genutzt wird.
- Im (Russell und Norvig 2021) wird die Breitensuche ebenfalls auf der Basis des Graph-Search-Algorithmus eingeführt. Allerdings wird die Abbruchbedingung modifiziert: Die Zielbedingung wird nicht erst (wie bei Graph-Search eigentlich definiert) geprüft, wenn ein Knoten aus der Queue entnommen wird, sondern bereits bei der Erzeugung der Nachfolgerknoten (vor dem Einfügen in die Queue). Dadurch spart man sich die Expansion einer zusätzlichen Ebene: Die Komplexität wäre in diesem Fall "nur" $O(b^d)$.

3.2.6. Eigenschaften Breitensuche vs. Tiefensuche

	Tiefensuche	**Breitensuche**
Vollständigkeit	nein[1]	ja[2]
Optimalität	nein	ja
Zeitkomplexität	$O(b^m)$	$O(b^{d+1})$
Speicherkomplexität	$O(bm)$	$O(b^{d+1})$

- Zeitkomplexität: maximal zu expandierende Knotenzahl
- Speicher:
  - TS: in jeder Tiefe weitere b Knoten speichern
  - BS: alle Knoten einer Ebene im Speicher halten[3]

b: Verzweigungsfaktor, **d**: Ebene d. höchsten Lösungsknotens, **m**: Länge d. längsten Pfades

---

[1] gilt für Tree-Search-Variante; vollständig in Graph-Search-Variante bei endlichem Suchraum

[2] falls *b* endlich

[3] $O(b^d)$ mit vorgezogener Zielprüfung (vgl. (Russell und Norvig 2021))

Tiefe	exp. Knoten	Zeit	Speicher

3.2.7. Praxisvergleich Breitensuche vs. Tiefensuche

Breitensuche: Annahme: $b = 10$, 10.000 Knoten/s, 1.000 Byte/Knoten

Tiefe	exp. Knoten	Zeit	Speicher
2	10^3	0.1 s	1 MB
4	10^5	10 s	100 MB
6	10^7	20 min	10 GB
8	10^9	30 h	1 TB
10	10^{11}	130 d	100 TB

Tiefensuche: Annahme: längster Pfad (Tiefe) $m = 1000$

$\rightarrow$ Speicherbedarf **ca. 10 MB**

3.2.8. Wrap-Up

- Uninformierte Suchverfahren
  - Keine weiteren Pfadkosten (nur Anzahl der Schritte)
  - Breitensuche: Verfolge alle Pfade (baue den Suchbaum ebenenweise auf)

Zum Nachlesen

- Russell und Norvig (2021): Breitensuche: Abschnitt 3.4.1

Lernziele

- k2: Ich kann erklären, welche Datenstrukturen in der Breitensuche verwendet werden und wie diese sich auswirken
- k2: Ich kann die algorithmische Abläufe und die Terminierung der Breitensuche erklären
- k2: Ich kann Optimalität, Vollständigkeit und Komplexität für die Breitensuche erklären
- k3: Ich kann die Breitensuche implementieren und auf ein konkretes Beispiel anwenden

Quizzes

- Selbsttest Breitensuche (ILIAS)

3.3. Search3: Suche mit Branch-and-Bound

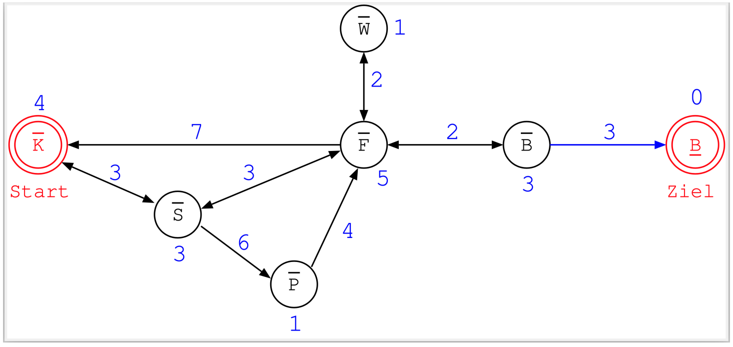
TL;DR

Branch-and-Bound gehört zu den "Informierten Suchverfahren": Es werden (reale) Pfadkosten statt der Anzahl der Schritte berücksichtigt.
Branch-and-Bound entsteht, wenn man bei der Tree-Search oder der Graph-Search für die Datenstruktur eine sortierte **Queue** (Prioritätsqueue) benutzt: Expandierte Nachfolger werden immer **hinten** in die Queue eingefügt, diese wird nach den **Kosten der partiellen Pfade sortiert** und der nächste zu expandierende Knoten (d.h. der bisher günstigste partielle Weg) wird **vorn** aus der Queue genommen. Branch-and-Bound arbeitet mit den bisher entstandenen (realen) Kosten der partiellen Wege.

Videos

Vorlesung [YT], [HSBI]

3.3.1. Hole das Buch



$\rightarrow$ **Problemlösen == Suche im Graphen**

Informierte Suche: Nutzung der Kostenfunktion:

Gesamtkosten: $f(n) = g(n) + h(n)$

- $n \in S$ auf aktuellem Weg erreichter Knoten
- $g(n)$ tatsächliche Kosten für Weg vom Start bis Knoten n
- $h(n)$ geschätzte Restkosten für Weg von Knoten n zum Ziel $\rightarrow$ $h(n)$ wird auch "heuristische Funktion" oder "Heuristik" genannt

Varianten:

- **Branch-and-Bound**
- Best First
- A*

3.3.2. Branch-and-Bound (*BnB*)

Variante der Breitensuche mit Kosten

- Idee: Expandiere den *bisher* günstigsten partiellen Weg

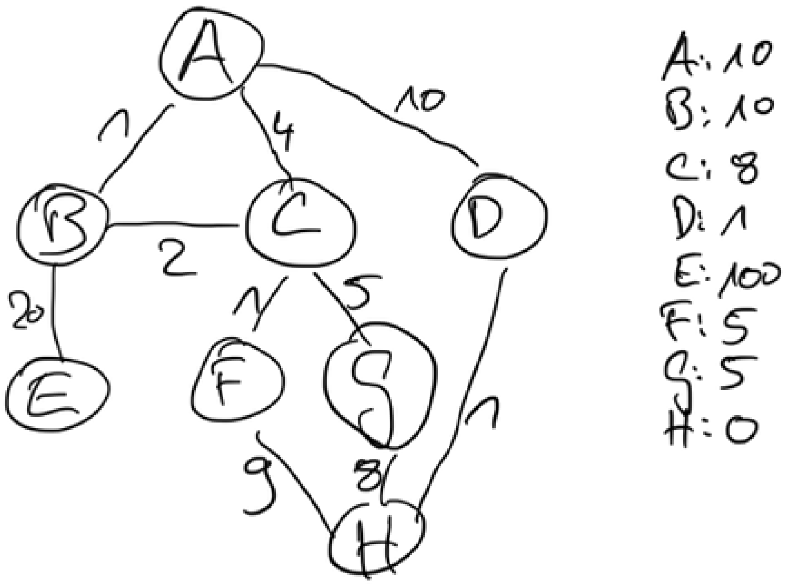
- Kostenfunktion: $f(n) = g(n)$
- Datenstruktur: **sortierte Queue** (Prioritätsqueue)

- Voraussetzung: alle Aktionen haben positive Kosten (jeden Knoten n gilt: $g(n) > 0$)

Hinweis: Die Branch-and-Bound-Suche taucht im (Russell und Norvig 2021) als Erweiterung der "Uniformen Suche" auf …

Hinweis: Es gibt auch den Dijkstra-Algorithmus. Im Unterschied zu Branch-and-Bound berechnet der Dijkstra-Algorithmus die kürzesten Wege von einem Startknoten zu allen anderen Knoten, der restliche algorithmische Ablauf ist aber identisch (Nutzung von $g(n)$ und einer sortierten Queue). (Man könnte also sagen, Branch-and-Bound ist ein Spezialfall vom Dijkstra, wenn wir nämlich die Berechnung auf den kürzesten Pfad vom Start zum Ziel beschränken.)

3.3.3. BnB: Finde einen Weg von A nach H



3.3.4. Bemerkungen zu BnB mit Graph-Search

Graph-Search fordert: Expandierte Nachfolgerknoten, die schon in der Queue sind, sollen nicht (erneut) in die Queue aufgenommen werden.

- Problem dabei: Was ist mit den Kosten?! Der neu expandierte Weg könnte günstiger sein als der schon in der Queue enthaltene.
- Lösung (vgl. Optimierungsmöglichkeiten für A*):

  Füge zunächst alle neu expandierten partiellen Pfade (mit unmarkierten Endknoten) in die Queue ein, sortiere diese und behalte von mehreren Pfaden zum gleichen Knoten nur den jeweils günstigsten in der Queue

Pfade, deren Endknoten bereits früher im Pfad vorkommt (Schleifen), werden bei Graph-Search in Schritt 2 nicht in die Queue aufgenommen (der Endknoten wäre bei einer Schleife ja bereits markiert und der neue Pfad würde bei Graph-Search nicht weiter beachtet).

Das Einfärben ist kein Problem, da nur der jeweils günstigste Knoten aus der Queue genommen, gefärbt und expandiert wird. D.h. alle anderen Wege sind zu diesem Zeitpunkt bereits teurer. Wenn man nun (später) über einen anderen Weg zu einem bereits gefärbten Knoten kommt, kann der neue Weg nicht günstiger sein (positive Kosten vorausgesetzt).

3.3.5. Konventionen für diese Lehrveranstaltung

In der Beschreibung der Algorithmen werden häufig nur die letzten Knoten der partiellen Wege in den Datenstrukturen mitgeführt (das gilt auch für die Beschreibung im (Russell und Norvig 2021)). Dies erschwert die Nachvollziehbarkeit, wenn man die Queue oder den Stack schrittweise aufschreibt. Deshalb wird für diese Veranstaltung die Konvention eingeführt, immer die **partiellen Wege** aufzuschreiben.

Auf dem Papier sortiert sich die Queue schlecht, deshalb können Sie darauf verzichten, wenn Sie den im nächsten Schritt zu expandierenden Weg unterstreichen. Wer nicht mit Unterstreichen arbeiten will, muss eben dann manuell sortieren …

Wenn bei der Graph-Search-Variante ein Weg nicht in die Queue aufgenommen wird, weil bereits ein anderer (günstigerer) Weg zum selben (Zwischen-/End-) Knoten bereits in der Queue enthalten ist, schreiben Sie dies geeignet auf. Dies gilt auch für den analogen Fall, wenn ein Weg aus der Queue entfernt wird, weil ein günstigerer Weg zum selben (Zwischen-/End-) Knoten eingefügt werden soll.

3.3.6. Eigenschaften von BnB

Siehe A*

3.3.7. Wrap-Up

- Informierte Suchverfahren
  - Nutzen reale Pfadkosten und/oder Schätzungen der Restkosten
  - Branch-and-Bound: nur reale Pfadkosten $g(n)$

Zum Nachlesen

- Russell und Norvig (2021): Branch-and-Bound (Uniforme Suche): Abschnitt 3.4.2

Lernziele

- k2: Ich kann erklären, welche Datenstrukturen in Branch-and-Bound verwendet werden und wie diese sich auswirken
- k2: Ich kann die algorithmische Abläufe und die Terminierung von Branch-and-Bound erklären
- k2: Ich kann Optimalität, Vollständigkeit und Komplexität für Branch-and-Bound erklären
- k2: Ich kann die Bedeutung nicht-negativer Kosten für Branch-and-Bound erklären
- k3: Ich kann Branch-and-Bound implementieren und auf ein konkretes Beispiel anwenden

Quizzes

- Selbsttest Branch-and-Bound (ILIAS)

3.4. Search4: Suche mit Best First

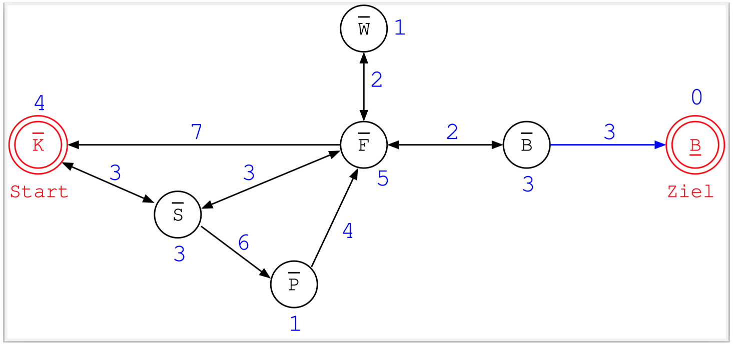
TL;DR

Best First gehört wie Branch-and-Bound zu den "Informierten Suchverfahren": Es werden Pfadkosten (in diesem Fall Schätzungen) statt der Anzahl der Schritte berücksichtigt.
Best First arbeitet algorithmisch wie Branch-and-Bound, allerdings werden immer nur die geschätzten Restkosten eines Knotens zum Ziel berücksichtigt.

Videos

Vorlesung [YT], [HSBI]

3.4.1. Hole das Buch



→ **Problemlösen == Suche im Graphen**

Informierte Suche: Nutzung der Kostenfunktion:

Gesamtkosten: $f(n) = g(n) + h(n)$

- $n \in S$ auf aktuellem Weg erreichter Knoten
- $g(n)$ tatsächliche Kosten für Weg vom Start bis Knoten n
- $h(n)$ geschätzte Restkosten für Weg von Knoten n zum Ziel → $h(n)$ wird auch "heuristische Funktion" oder "Heuristik" genannt

Varianten:

- Branch-and-Bound
- **Best First**
- A*

3.4.2. Best-First (*BF*, *BFS*)

- Idee: Expandiere den partiellen Weg, der verspricht, dem Ziel am nächsten zu sein (**Heuristik**)

- Kostenfunktion: $f(n) = h(n)$
- Datenstruktur: **sortierte Queue** (Prioritätsqueue)

- Voraussetzungen: $h(n)$ positiv, $h(n) = 0$ für den Zielknoten

3.4.3. Konventionen BF

In der Beschreibung der Algorithmen werden häufig nur die letzten Knoten der partiellen Wege in den Datenstrukturen mitgeführt (das gilt auch für die Beschreibung im (Russell und Norvig 2021)). Dies erschwert die Nachvollziehbarkeit, wenn man die Queue oder den Stack schrittweise aufschreibt. Deshalb wird für diese Veranstaltung die Konvention eingeführt, immer die **partiellen Wege** aufzuschreiben.

Auf dem Papier sortiert sich die Queue schlecht, deshalb können Sie darauf verzichten, wenn Sie den im nächsten Schritt zu expandierenden Weg unterstreichen. Wer nicht mit Unterstreichen arbeiten will, muss eben dann manuell sortieren …

Wenn bei der Graph-Search-Variante ein Weg nicht in die Queue aufgenommen wird, weil bereits ein anderer (günstigerer) Weg zum selben (Zwischen-/End-) Knoten bereits in der Queue enthalten ist, schreiben Sie dies geeignet auf. Dies gilt auch für den analogen Fall, wenn ein Weg aus der Queue entfernt wird, weil ein günstigerer Weg zum selben (Zwischen-/End-) Knoten eingefügt werden soll.

3.4.4. Eigenschaften von BF

Siehe A*

3.4.5. Wrap-Up

- Informierte Suchverfahren
  - Nutzen reale Pfadkosten und/oder Schätzungen der Restkosten
  - Best-First: nur Schätzungen $h(n)$

Zum Nachlesen

- Russell und Norvig (2021): Best First: Abschnitt 3.5.1, Heuristiken: Kapitel 3.6

Lernziele

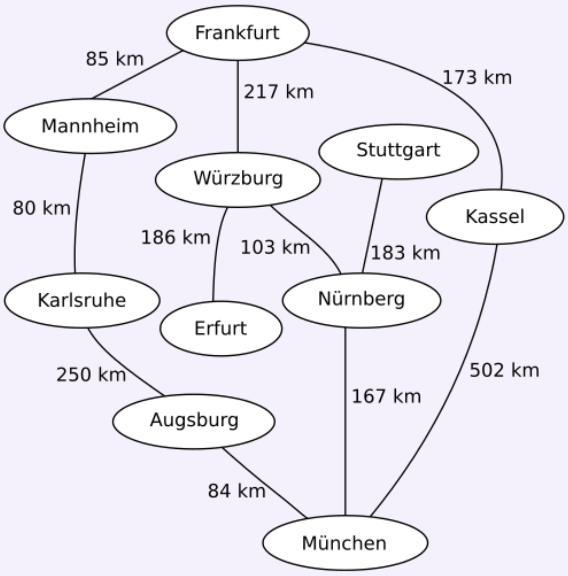
- k2: Ich kann erklären, welche Datenstrukturen in Best First verwendet werden und wie diese sich auswirken
- k2: Ich kann die algorithmische Abläufe und die Terminierung von Best First erklären
- k2: Ich kann Optimalität, Vollständigkeit und Komplexität für Best First erklären
- k3: Ich kann Best First implementieren und auf ein konkretes Beispiel anwenden

Quizzes

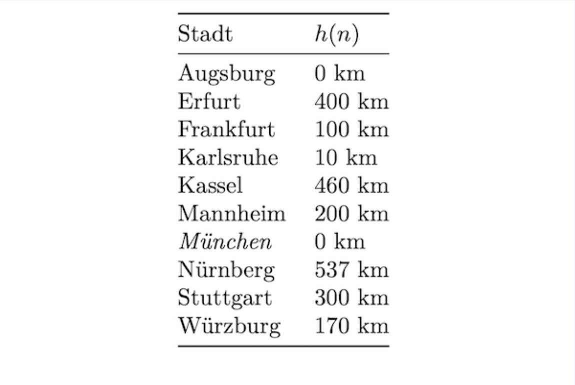
- Selbsttest Best First (ILIAS)

Challenges

Betrachten Sie folgende Landkarte und Restwegschätzungen:


Frankfurt
85 km
217 km
173 km
Mannheim
Stuttgart
Würzburg
Kassel
80 km
186 km
103 km
183 km
Karlsruhe
Erfurt
Nürnberg
250 km
502 km
167 km
Augsburg
84 km
München

Quelle: MapGermanyGraph.svg by Regnaron and Jahobr on Wikimedia Commons (Public Domain)



Stadt	$h(n)$
Augsburg	0 km
Erfurt	400 km
Frankfurt	100 km
Karlsruhe	10 km
Kassel	460 km
Mannheim	200 km
München	0 km
Nürnberg	537 km
Stuttgart	300 km
Würzburg	170 km

Schätzungen der Restwegkosten für das Ziel *München*.

Finden Sie mit der **Best-First-Suche** jeweils einen Weg von Würzburg nach München. Vergleichen Sie das Ergebnis mit der Gradienten-Suche.

3.5. Search5: Suche mit A*

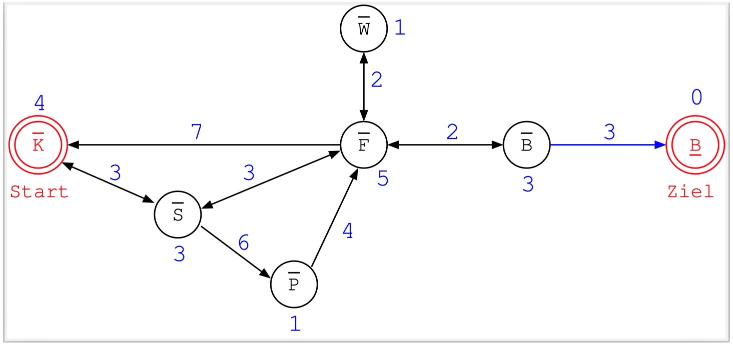
TL;DR

A* zählt zu den Verfahren der informierten Suche. Dabei verwendet A* sowohl die realen Pfadkosten als auch die Schätzungen der Restkosten, d.h. die Kostenfunktion für A* ist $f(n) = g(n) + h(n)$.
A* ist vollständig und optimal, allerdings muss die Heuristik bei der Tree-Search-Variante **zulässig** sein (d.h. sie muss *unterschätzen*, beispielsweise die Luft-Linie) und bei der Graph-Search-Variante muss sie **konsistent** sein (d.h. für jeden Knoten die Dreiecks-Ungleichung erfüllen).
A* hat wie BnB exponentiellen Aufwand. Durch die zusätzliche Verwendung der Heuristik werden die partiellen Pfade in der Queue aber geschickter sortiert, so dass A* in der Regel mit weniger Suchschritten als BnB auskommt.

Videos

Vorlesung [YT], [HSBI]

3.5.1. Hole das Buch



→ **Problemlösen == Suche im Graphen**

Informierte Suche: Nutzung der Kostenfunktion:

Gesamtkosten: $f(n) = g(n) + h(n)$

- $n \in S$ auf aktuellem Weg erreichter Knoten
- $g(n)$ tatsächliche Kosten für Weg vom Start bis Knoten n
- $h(n)$ geschätzte Restkosten für Weg von Knoten n zum Ziel → $h(n)$ wird auch "heuristische Funktion" oder "Heuristik" genannt

Varianten:

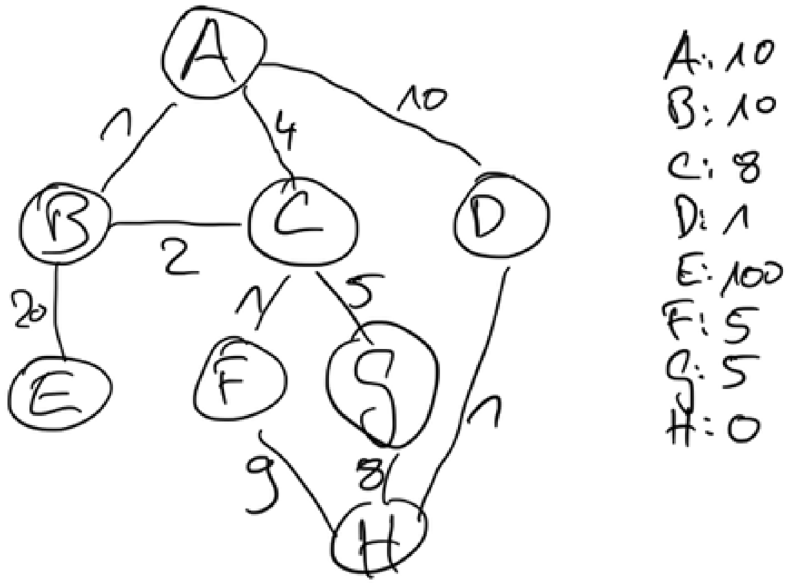
- Branch-and-Bound
- Best First
- **A***

3.5.2. A*-Suche

- Kombination aus Branch-and-Bound und Best-First-Suche

- Kostenfunktion: $f(n) = g(n) + h(n)$
- Datenstruktur: **sortierte Queue** (Prioritätsqueue)

- Voraussetzung:
    1. Alle Aktionen haben positive Kosten ($g(n) \geq \epsilon$)
    2. Heuristik $h(n)$ muss *zulässig/konsistent* sein



3.5.3. Konventionen für diese Lehrveranstaltung

In der Beschreibung der Algorithmen werden häufig nur die letzten Knoten der partiellen Wege in den Datenstrukturen mitgeführt (das gilt auch für die Beschreibung im (Russell und Norvig 2021)). Dies erschwert die Nachvollziehbarkeit, wenn man die Queue oder den Stack schrittweise aufschreibt. Deshalb wird für diese Veranstaltung die Konvention eingeführt, immer die **partiellen Wege** aufzuschreiben.

Notieren Sie die Bestandteile der Kosten für jeden partiellen Weg in der Queue einzeln: “$g(n)+h(n) = f(n)$”. Das erleichtert Ihnen die weiteren Schritte, da Sie dort ja nur mit $g(n)$ weiter rechnen dürfen. Gleichzeitig erleichtert es die Nachvollziehbarkeit.

Auf dem Papier sortiert sich die Queue schlecht, deshalb können Sie darauf verzichten, wenn Sie den im nächsten Schritt zu expandierenden Weg unterstreichen. Wer nicht mit Unterstreichen arbeiten will, muss eben dann manuell sortieren …

Wenn bei der Graph-Search-Variante ein Weg nicht in die Queue aufgenommen wird, weil bereits ein anderer (günstigerer) Weg zum selben (Zwischen-/End-) Knoten bereits in der Queue enthalten ist, schreiben Sie dies geeignet auf. Dies gilt auch für den analogen Fall, wenn ein Weg aus der Queue entfernt wird, weil ein günstigerer Weg zum selben (Zwischen-/End-) Knoten eingefügt werden soll.

3.5.4. A*-Suche – Anforderungen an Heuristik (Tree-Search)

Tree-Search-Variante: Die Heuristik muss **zulässig** sein:

- Seien $h^\star(n)$ die tatsächlichen optimalen Restkosten von einem Knoten n zum nächsten Ziel.
- Dann muss für jeden beliebigen Knoten n gelten:

$$h(n) \leq h^\star(n)$$

- Außerdem muss gelten:
  - $h(n) \geq 0$ für jeden Knoten n
  - $h(n) = 0$ für jeden Zielknoten n

$\rightarrow$ Beispiel: Luftlinie als Abschätzung

Hinweis: Im der englischen Ausgabe des (Russell und Norvig 2021) wird die **zulässige Heuristik** auch "**admissible heuristic**" genannt.

3.5.5. A* ist optimal

A* (Tree-Search-Variante) mit zulässiger Heuristik ist optimal.

Beweis siehe Übung :-)

3.5.6. Einfache Verbesserungen A* (Tree-Search)

- Dynamische Programmierung: Behalte von mehreren Pfaden zum gleichen Knoten nur den günstigsten in der Queue
- Pfade, deren Endknoten bereits früher im Pfad vorkommt (Schleifen), werden in Schritt 2 nicht in die Queue aufgenommen

- Übergang zur Graph-Search-Variante und Markierung von Knoten

  $\rightarrow$ Achtung: Dann schärfere Anforderungen an Heuristik (Konsistenz)

3.5.7. A*-Suche – Anforderungen an Heuristik (Graph-Search)

Graph-Search-Variante: Die Heuristik muss **konsistent** sein:

Für jeden Knoten n und jeden durch eine Aktion a erreichten Nachfolger m gilt:

$$h(n) \leq c(n, a, m) + h(m)$$

mit $c(n, a, m)$ Schrittkosten für den Weg von n nach m mit Aktion a.

Außerdem muss gelten:

- $h(n) \geq 0$ für jeden Knoten n
- $h(n) = 0$ für jeden Zielknoten n

$\rightarrow$ Eine konsistente Heuristik ist gleichzeitig zulässig.

Hinweis: Im der englischen Ausgabe des (Russell und Norvig 2021) wird die **konsistente Heuristik** auch "**consistent heuristic**" genannt.

3.5.8. Eigenschaften Branch-and-Bound, Best-First, A*

	Branch-and-Bound	**Best-First**	**A***
Kosten	$f(n) = g(n)$	$f(n) = h(n)$	$f(n) = g(n) + h(n)$
Vollständigkeit	ja[4]	nein[5]	ja
Optimalität	ja	nein	ja
Aufwand	exponentiell	exponentiell	exponentiell
Bemerkung	Probiert erst alle "kleinen" Pfade	Suchverlauf stark abh. v. Heuristik	Heuristik: zulässig bzw. konsistent

3.5.9. Wrap-Up

- Informierte Suchverfahren
  - Nutzen reale Pfadkosten und/oder Schätzungen der Restkosten
  - A*: komplette Kostenfunktion $f(n) = g(n) + h(n)$
    $\rightarrow$ besondere Anforderungen an die Heuristik! (Tree-Search: *zulässige* Heuristik; Graph-Search: *konsistente* Heuristik)

- Ausblick auf Verbesserungen der vorgestellten Suchverfahren:
  - Beschränkung der Suchtiefe ("Depth-Limited-Search")
  - Iterative Vertiefung der Suchtiefe ("Iterative-Deepening-Search"), beispielsweise IDA* ("Iterative-Deepening A*")
  - Beschränkung des verwendeten Speichers, beispielsweise SMA *("Simplified Memory-Bounded A*")

Zum Nachlesen

- Russell und Norvig (2021): A*: Abschnitt 3.5.2, Heuristiken: Kapitel 3.6

Lernziele

- k2: Ich kann erklären, welche Datenstrukturen in A* verwendet werden und wie diese sich auswirken
- k2: Ich kann die algorithmische Abläufe und die Terminierung von A* erklären
- k2: Ich kann Optimalität, Vollständigkeit und Komplexität für A* erklären
- k2: Ich kann die Bedingung(en) an Restkostenabschätzung bei A* erklären
- k3: Ich kann A* implementieren und auf ein konkretes Beispiel anwenden

---

[4] BnB vollständig: Kosten größer Epsilon (positiv)
[5] gilt für Tree-Search-Variante; vollständig bei Graph-Search und endlichen Problemräumen

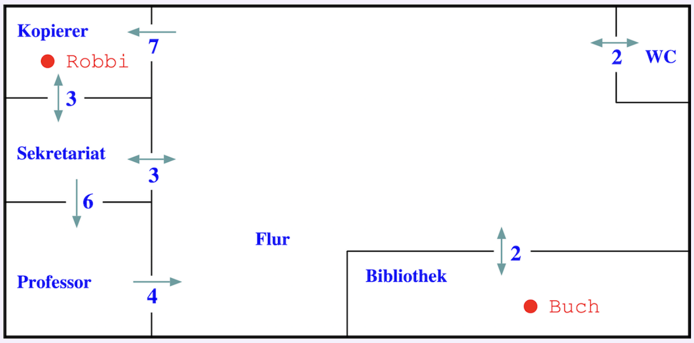
Quizzes

- Selbsttest A* (ILIAS)

Challenges

Informierte und uninformierte Suche

Betrachten Sie folgendes Problem:



Dargestellt ist eine typische Büroumgebung mit verschiedenen Räumen und einem Flur. Die Pfeile in den Durchgängen geben an, in welche Richtung der jeweilige Durchgang durchschritten werden darf. Die Werte an den Pfeilen geben die Kosten für den Übergang von einem Raum in den anderen an.

Ein Roboter `Robbi`, der sich zunächst im Kopierer-Raum aufhält, soll den Weg zur Bibliothek finden und dort das Buch aufnehmen. Der Roboter kann sich immer nur entlang den Pfeilen in einen Nachbarraum bewegen (Aktion `move`). Die Kosten für das Aufnehmen des Buches betragen 3 Einheiten (Aktion `take`). Weitere Aktionen gibt es nicht.

1. Zeichnen Sie den Problemgraphen. Markieren Sie Start- und Zielknoten.
2. Finden Sie den Weg mit Tiefensuche und mit Breitensuche (Tree-Search). Welche Unterschiede stellen Sie fest?
3. Welche Wege würden mit der jeweiligen Graph-Search-Variante nicht weiter untersucht?
4. Suchen Sie nun einen Weg mit A* (Tree-Search). Definieren Sie zunächst Restkostenschätzungen. Was müssen Sie dabei beachten?

3.6. Search6: Lokale Suche - Gradientensuche

TL;DR

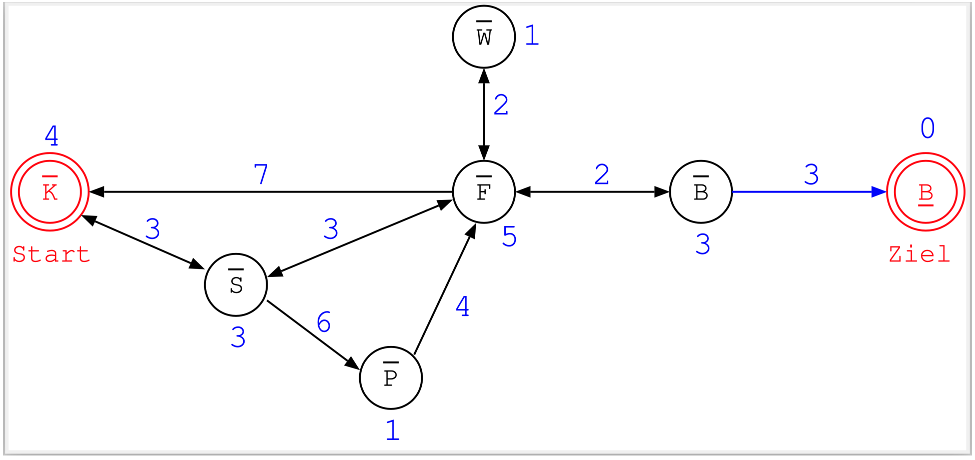
Lokale Suchverfahren: Nur das Ergebnis zählt! Nicht der Weg ist das Ziel, sondern nur das Erreichen des Ziels.

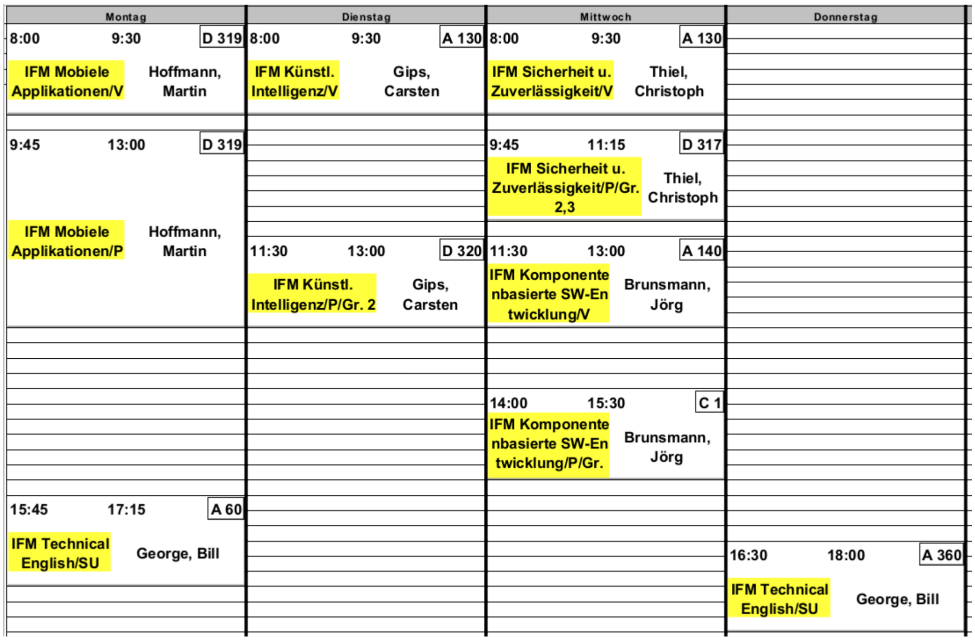
In Analogie zum Bergsteigen: Gehe in Richtung des stärksten Anstiegs kann man die Suche so formulieren, dass man in jedem Suchschritt den Nachfolgeknoten nach dem stärksten Anstieg der Kostenfunktion auswählen. Dieses Verfahren nennt sich auch **Hill-Climbing** (bzw. Gradientensuche).

Videos

Vorlesung [YT], [HSBI]

3.6.1. Unterschiede in den Suchproblemen?





Montag	Dienstag	Mittwoch	Donnerstag
8:00 9:30 D 319 IFM Mobiele Applikationen/V Hoffmann, Martin	8:00 9:30 A 130 IFM Künstl. Intelligenz/V Gips, Carsten	8:00 9:30 A 130 IFM Sicherheit u. Zuverlässigkeit/V Thiel, Christoph	
9:45 13:00 D 319 IFM Mobiele Applikationen/P Hoffmann, Martin		9:45 11:15 D 317 IFM Sicherheit u. Zuverlässigkeit/P/Gr. 2,3 Thiel, Christoph	
	11:30 13:00 D 320 IFM Künstl. Intelligenz/P/Gr. 2 Gips, Carsten	11:30 13:00 A 140 IFM Komponentenbasierte SW-Entwicklung/V Brunsmann, Jörg	
		14:00 15:30 C 1 IFM Komponentenbasierte SW-Entwicklung/P/Gr. Brunsmann, Jörg	
15:45 17:15 A 60 IFM Technical English/SU George, Bill			
			16:30 18:00 A 360 IFM Technical English/SU George, Bill

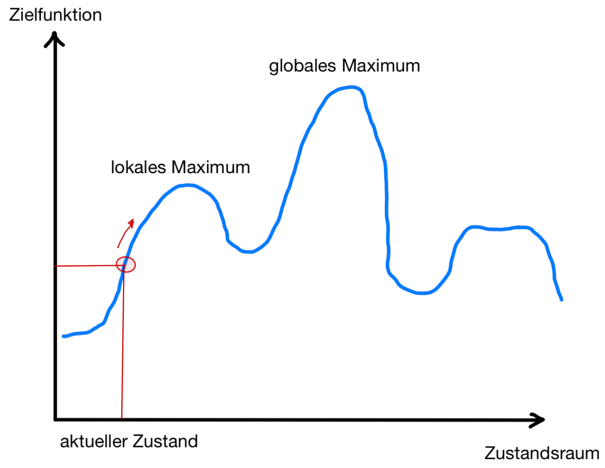
Bisher betrachtete Suchverfahren:

- Systematische Erkundung des Suchraums
- **Weg** zur Lösung wichtig

→ Oft aber nur das **Ziel an sich** interessant!
(Und nicht, wie man dort hin gelangt.)

Beispiel: Stundenplan

3.6.2. Analogie: Bergsteigen ohne Karte und Pfade



Gradienten-Suche: "Gehe in Richtung des steilsten Anstiegs der Zielfunktion."

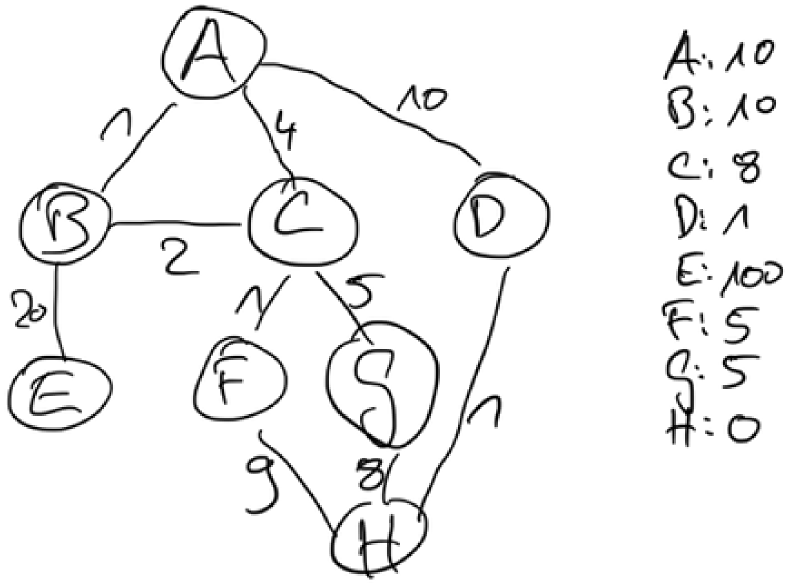
→ Schrittweise Verbesserung des aktuellen Zustands (Lokale Suche)

- Verschiedene Namen: "Hill-climbing", "Greedy local search"
- Kann auch als Minimierung angewendet werden

3.6.3. Pseudoalgorithmus Gradientensuche

> "*Wie Bergsteigen am Mount Everest in dickem Nebel mit Gedächtnisverlust*"

1. Setze `currNode` auf den Startknoten
2. `currNode` ist gesuchtes Element: Abbruch, melde "*gefunden*"
    - Expandiere alle Nachfolger von `currNode`
    - Setze `nextNode` auf Nachfolger mit höchster Bewertung
    - Falls Bewertung von `nextNode` $\leq$ Bewertung von `currNode`:
      Abbruch, melde "*nicht gefunden*"
    - Setze `currNode` auf `nextNode`
3. Gehe zu Schritt 2



3.6.4. Beispiel Gradientensuche: n-Damen

- **Ziel**: Setze n Damen auf ein $n \times n$-Spielfeld ohne Konflikte
- **Start**: Setze n Damen auf ein $n \times n$-Spielfeld (mit Konflikten)
- **Suche**: Bewege jeweils eine Dame so, daß die Anzahl der Konflikte reduziert wird

Schauen Sie sich auch Abb. 4.3 auf Seite 130 im Russell und Norvig (2021) an!

Hinweis: Alle Damen stehen von Anfang an auf dem Brett und werden nur verschoben → "vollständige Zustandsformulierung"

3.6.4.1. Eigenschaften 8-Damen-Problem ($n = 8$)

- Zustandsraum: $8^8 \approx 17$ Millionen Zustände!
- Beginnend mit zufällig erzeugtem Startzustand:
    - bleibt in 86% der Fälle stecken, d.h.
    - findet Lösung nur in 14% der Fälle.
- Beobachtung: Lösung nach durchschnittlich 4 Schritten, oder Verfahren bleibt nach durchschnittlich 3 Schritten stecken.

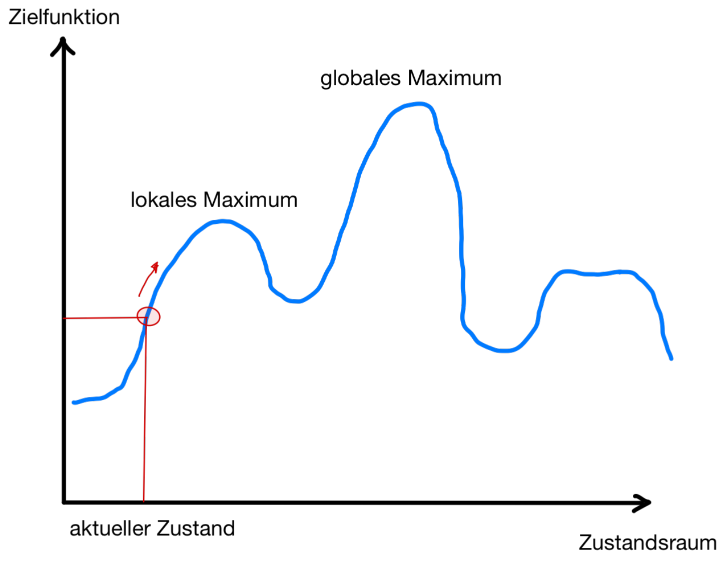
Quelle: nach (Russell und Norvig 2021, p. 131)

3.6.5. Eigenschaften Gradientensuche

- Vollständigkeit: nein

- Optimalität: nein
- Komplexität: linear in der Anzahl der zu expandierenden Knoten

Zielfunktion (Bewertung) nötig!



Problem: lokale Maxima und Plateaus

- Lokale Maxima/Minima: Algorithmus findet nur eine suboptimale Lösung
- Plateaus: Hier muss der Algorithmus mit zufälligen Zügen explorieren

3.6.6. Wrap-Up

Lokale Suchverfahren: Nur das Ergebnis zählt!

- Gradientenverfahren: Gehe in Richtung des stärksten Anstiegs der Kostenfunktion

Zum Nachlesen

- Russell und Norvig (2021): Gradientenabstieg: Abschnitt 4.1.1

Lernziele

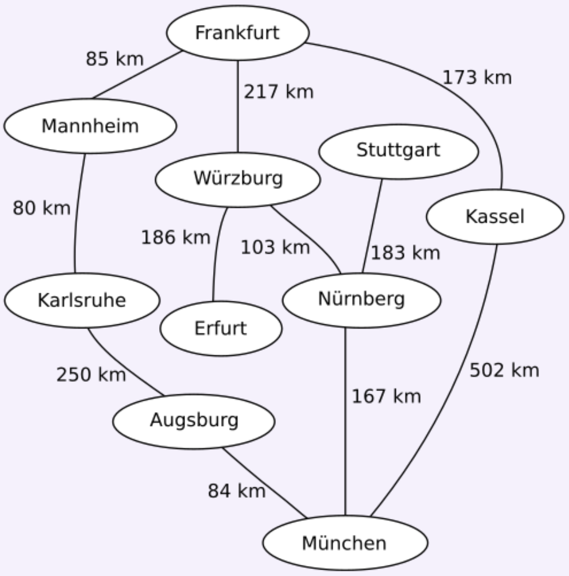
- k2: Ich kann die Problematik der lokalen Minima bei Gradientenverfahren erklären
- k3: Ich kann die lokale Suche (Gradientenabstieg) auf ein konkretes Problem anwenden

Quizzes

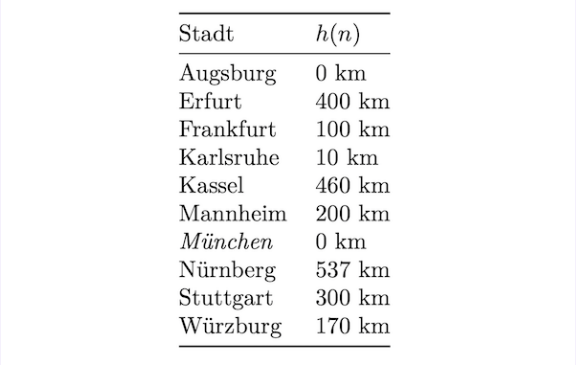
- Selbsttest Gradientensuche (ILIAS)

Challenges

Betrachten Sie folgende Landkarte und Restwegschätzungen:


Frankfurt
85 km
217 km
173 km
Mannheim
Stuttgart
Würzburg
Kassel
80 km
186 km
103 km
183 km
Karlsruhe
Erfurt
Nürnberg
250 km
502 km
167 km
Augsburg
84 km
München

Quelle: MapGermanyGraph.svg by Regnaron and Jahobr on Wikimedia Commons (Public Domain)



Stadt	$h(n)$
Augsburg	0 km
Erfurt	400 km
Frankfurt	100 km
Karlsruhe	10 km
Kassel	460 km
Mannheim	200 km
München	0 km
Nürnberg	537 km
Stuttgart	300 km
Würzburg	170 km

Schätzungen der Restwegkosten für das Ziel *München*.

Finden Sie mit der **Gradienten-Suche** jeweils einen Weg von Würzburg nach München. Vergleichen Sie das Ergebnis mit der Best-First-Suche.

3.7. Search7: Lokale Suche - Simulated Annealing

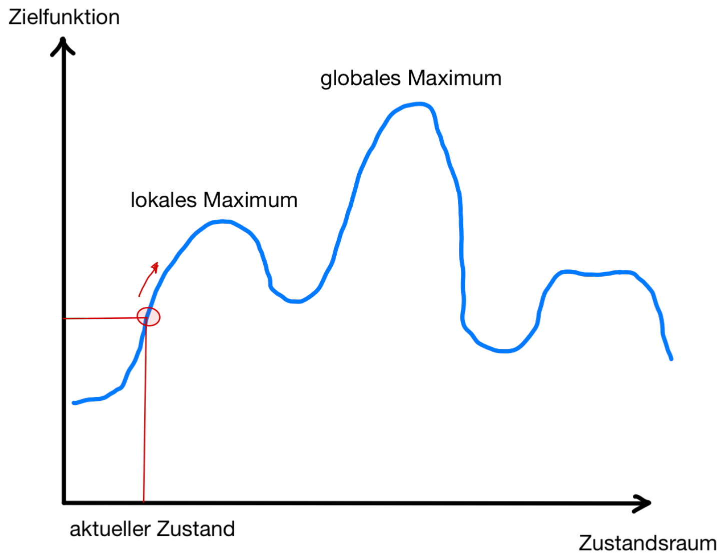
TL;DR

Lokale Suchverfahren: Nur das Ergebnis zählt! Nicht der Weg ist das Ziel, sondern nur das Erreichen des Ziels.
Das Problem bei der Gradientensuche ist, dass man eine Kostenfunktion benötigt und diese auch **lokale Minima** enthalten kann. Mit der reinen Gradientensuche würde man bei Erreichen lokaler Minima die Suche abbrechen (müssen), da es keine weitere Verbesserung unter den Nachfolgern mehr geben kann.
In Anlehnung an das Abkühlen von Metall kann hier eine Variante der lokalen Suche helfen: **Simulated Annealing**. Man führt einen "Temperatur"-Parameter ein, der im Laufe der Suche immer kleiner wird und schließlich gegen Null geht. In Abhängigkeit von dieser "Temperatur" wird mit einer bestimmten Wahrscheinlichkeit eine Verschlechterung akzeptiert: Bei einer hohen Temperatur ist diese Wahrscheinlichkeit höher, bei einer niedrigen Temperatur niedriger, so dass das Verfahren in ein normales Hill-Climbing übergeht. Damit kann man ein Festfressen in lokalen Minima vermeiden bzw. überwinden.

Videos

Vorlesung [YT], [HSBI]

3.7.1. Motivation



Problem: lokale Maxima und Plateaus

- Lokale Maxima/Minima: Algorithmus findet nur eine suboptimale Lösung
- Plateaus: Hier muss der Algorithmus mit zufälligen Zügen explorieren

Mögliche Lösungen:

- Neustart des Algorithmus, wenn kein Fortschritt erzielt wird
- Rauschen "injizieren"

3.7.2. Gedankenexperiment: Ausweg aus lokalen Minima

- "Drehen der Landschaft": Minimieren statt Maximieren
- Ball wird in Zustandsraum-Landschaft gesetzt.
- Folge:
  - rollt steilsten Abstieg hinunter
  - rollt evtl. in Tal auf halber Höhe (lokales Minimum) → bleibt dort gefangen

→ "**Schütteln** der Landschaft" – Ball springt aus dem Tal und rollt in anderes Tal

Nicht zu stark schütteln – sonst wird u.U. globales Minimum verlassen!

3.7.3. Analogie Härten von Metall

- Metall erhitzen bis Atome frei beweglich
- Langsam abkühlen

→ stabiles Atomgitter mit minimalem Energiezustand

3.7.4. Übertragen der Idee

- Starkes "Schütteln" (hohe "Temperatur") am Anfang
- Schrittweises "Abkühlen" → "Schütteln" im Laufe der Zeit verringern

→ **Simulated Annealing**

3.7.5. Pseudocode Simulated Annealing (Minimierungsproblem)

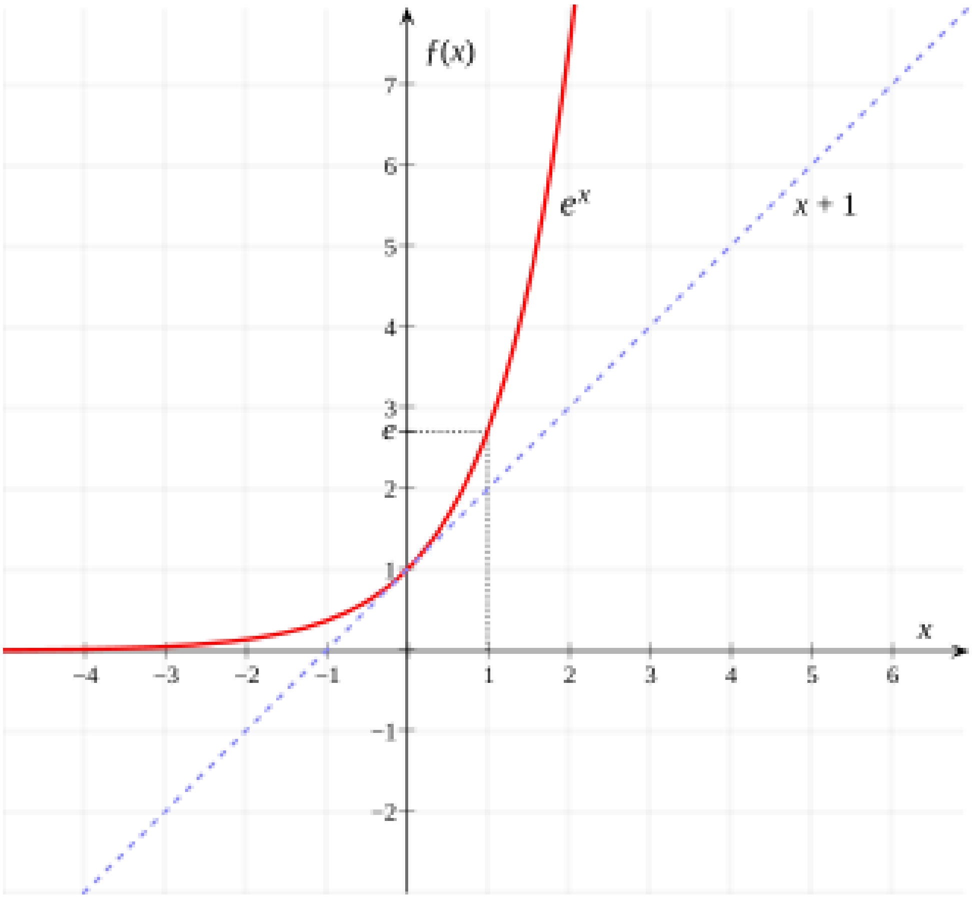
```python
def simulated_annealing(problem):
    current = problem.startNode
    t = 0;  temp = schedule(t)

    while temp>0:
        temp = schedule(++t)
        neighbors = current.expandSuccessors()
        if not neighbors: return current
        working = random.choice(neighbors)
        dE = problem.value(current) - problem.value(working)
        if dE > 0 or probability(math.exp(dE/temp)):
            current = working

    return current
```

Wenn `dE` positiv ist, dann ist der Nachfolger “besser” (hier: kleiner bewertet) als der aktuelle Knoten und wird immer als nächster Knoten übernommen.

Wenn `dE` negativ ist, dann ist der betrachtete Nachfolger “schlechter” (hier: größer bewertet) als der aktuelle Knoten. Dann wird er mit einer Wahrscheinlichkeit `math.exp(dE/temp)` als nächster Knoten übernommen. Diese Wahrscheinlichkeit ist bei hohen Temperaturen `temp` eher hoch, und sinkt, je niedriger die Temperatur `temp` wird.

Die Temperatur `temp` bewegt sich dabei von hohen positiven Werten auf den Wert Null (wird also nicht negativ).

3.7.6. Detail: Akzeptieren von Verschlechterungen



Quelle: “Exp e.svg” by Marcel Marnitz, reworked by Georg-Johann on Wikimedia Commons (Public Domain)

- Wahrscheinlichkeit zum Akzeptieren einer Verschlechterung: `math.exp(dE/temp)`
- Wenn dE negativ ist, wird `math.exp(dE/temp)` ausgewertet
- Wenn dE negativ, dann gilt (Umformung):

$$\exp\left(\mathsf{dE}/\mathsf{temp}\right) = \exp\left(-\frac{|\mathsf{dE}|}{\mathsf{temp}}\right) = \frac{1}{\exp\left(\frac{|\mathsf{dE}|}{\mathsf{temp}}\right)}$$

- Betrachtung von $\exp(a)$ bzw. e^a:
  - $a < 0$: geht gegen 0
  - $a = 0$: 1
  - $a > 0$: steil (exponentiell) gegen Unendlich …
- Damit ergibt sich für dE (nur negativer Fall!) und temp:
  - Temperatur temp hoch: $a = \frac{|\text{dE}|}{\text{temp}}$ ist positiv und klein (nahe Null), d.h. $\exp(a)$ nahe 1 (oder größer), d.h. die Wahrscheinlichkeit $1/\exp(a)$ ist nahe 1 (oder kleiner)
  - Temperatur temp wird kleiner und geht gegen Null: $a = \frac{|\text{dE}|}{\text{temp}}$ ist positiv und wird größer, d.h. $\exp(a)$ geht schnell gegen Unendlich, d.h. die Wahrscheinlichkeit $1/\exp(a)$ geht gegen 0

3.7.7. Abkühlungsplan problemabhängig wählen

- Initiale Temperatur: So hoch, daß praktisch jede Änderung akzeptiert wird
- Abkühlen: $T_k = \alpha T_{k-1}$ mit $0.8 \leq \alpha \leq 0.99$
  - Ausreichend langsam abkühlen!
  - Typisch: jede Stufe so lange halten, daß etwa 10 Änderungen akzeptiert wurden
- Stop: Keine Verbesserungen in 3 aufeinander folgenden Temperaturen

Der Abkühlungsplan muss problemabhängig gewählt werden. Das Beispiel zeigt typische Elementes eines solchen Abkühlungsplans.

3.7.8. Eigenschaften Simulated Annealing

- Vollständigkeit: ja (mit gewisser Wahrscheinlichkeit)
- Optimalität: ja (mit gewisser Wahrscheinlichkeit)

Voraussetzung: geeigneter Abkühlungsplan

3.7.9. Anwendungen von Simulated Annealing

- Flugplan-Scheduling
- Layout-Probleme (Chipentwurf, Leiterplatten)
- Produktionsplanung

3.7.10. Wrap-Up

Lokale Suchverfahren: Nur das Ergebnis zählt!

- Gradientenverfahren
  - Analogie Bergsteigen: Gehe in Richtung des stärksten Anstiegs der Kostenfunktion → **Hill-Climbing**
  - Achtung: Probleme mit lokalen Minima → **Simulated Annealing**

Zum Nachlesen

- Russell und Norvig (2021): Simulated Annealing: Abschnitt 4.1.2

Lernziele

- k3: Ich kann die Funktionsweise von Simulated Annealing erklären und den Algorithmus auf ein konkretes Problem anwenden

Quizzes

- Selbsttest Simulated Annealing (ILIAS)

Challenges

Team-Planung

Sie haben 12 Personen mit Rollen/Skills und paarweisen Synergiewerten (positiv = gut, negativ = Konflikt). Es gibt die Rollen/Skills "Frontend" (FE), "Backend" (BE), "Fullstack" (FS) sowie "User Experience" (UX). Für die Synergie-Werte überlegen Sie sich einige passende Beispiele, etwa "Person A und B kommen gut miteinander aus" o.ä.
Bilden Sie 3 Teams zu je 4 Personen, so dass

- die Summe der Synergien innerhalb der Teams maximiert wird,
- pro Team mindestens je 1x FE, 1x BE, 1x FS vorhanden ist (harte oder weiche Nebenbedingung),
- optional: UX in einem Team bringt Bonus.

Es gibt exakt 12 Personen, darunter jeweils 3 FE, 3 BE, 3 FS, 3 UX.
Modellieren Sie dieses Problem geeignet, damit Sie es mit Simulated Annealing lösen können.

4. Spiele

Man kann Spiele auch als Suchproblem betrachten und als Ziel die Suche nach dem optimalen Zug definieren.

4.1. Games1: Einführung Optimale Spiele

TL;DR

Spiele können als Suchproblem betrachtet werden. Dabei sind in der Regel mehrere Spieler ("Agenten") beteiligt. Bei manchen Spielen ist die Umgebung (der Spielzustand) vollständig einsehbar, bei anderen nur teilweise (Kartenspiele). Bei manchen Spielen kommt eine Zufallskomponente zum Wirken.
Spiele sind in der KI deshalb so interessant, weil bei der Suche riesige Suchbäume entstehen (bzw. durchsucht werden müssten). Da die Ressourcen normalerweise begrenzt sind (denken Sie an die Reaktionszeit auf einen Zug des Gegners), muss man hier intelligente Lösungen finden. (Einige davon werden wir in den folgenden Sitzungen anschauen).

Videos

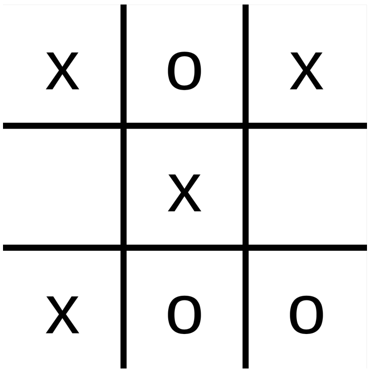
Vorlesung [YT], [HSBI]

4.1.1. Backgammon: Zwei Spieler, was ist der beste Zug?

Quelle: Backgammon lg.png by Ptkfgs on Wikimedia Commons (Public Domain)

Zwei Spieler, ein Spielstand und ein Würfelergebnis: **Was ist jetzt der beste Zug?!**

4.1.2. Motivation: Unterschied zu Suche?!



→ Mehrere **konkurrierende** Agenten an Suche beteiligt!

→ (Re-) Aktion des Gegners unbekannt/nicht vorhersehbar.

4.1.3. Spiele und Umgebungen

	Deterministisch	Zufallskomponente
Voll beobachtbar	Schach, Go, …	Backgammon, Monopoly
Partiell beobachtbar	Schiffe-versenken	Bridge, Poker, Skat, …

→ Bis auf Roboterfußball in KI traditionell keine physischen Spiele!

4.1.4. Brettspiele sind interessant für KI

- Brettspiele gut abstrakt darstellbar:
    - Zustände einfach repräsentierbar
    - Aktionen wohldefiniert (und i.d.R. sehr einfach)
    - Realisierung als Suchproblem möglich
- **Problem**: Suchbäume werden in Praxis riesig

  Beispiel **Schach**:

    - Im Mittel 35 Aktionen (*branching factor*) von jeder Position

  - Oft mehr als 40 Züge pro Spieler → Suchbäume mit mehr als 80 Ebenen
  - $35^{80} \approx 10^{123}$ mögliche Knoten!
  - (Aber “nur” rund 10^{40} *verschiedene* Zustände)

Quelle: (Russell und Norvig 2021, pp. 193-196)

4.1.5. Eigenschaften guter Spielalgorithmen

- Zeit begrenzt
  - Irgendeine gute Entscheidung treffen! → Bewertungsfunktion (auch für Zwischenzustände)
- Speicher begrenzt
  - Evaluierungsfunktion für Zwischenzustände
  - Löschen von irrelevanten Zweigen
- Strategien nötig
  - Vorausschauend spielen (Züge “vorhersehen”)

4.1.6. Wrap-Up

- Spiele kann man als Suchproblem betrachten
- Merkmale:
  - Mehrere Agenten beteiligt
  - Beobachtbarkeit der Umgebung
  - Zufallskomponente
  - Spielstrategie
- Problem: Riesige Spielbäume
- Umgang mit begrenzten Ressourcen (Zeit, Speicher)

Zum Nachlesen

- Russell und Norvig (2021): Einführung Spiele: Abschnitt 6.1
- Ertel (2025): Kapitel 6.4 “Spiele mit Gegner”

Lernziele

- k2: Ich kann Spiele als Suchproblem formulieren
- k2: Ich kann Eigenschaften guter Spielalgorithmen erklären

4.2. Games2: Minimax

TL;DR

Mit dem Minimax-Algorithmus können optimale Züge berechnet werden. Dabei wird von zwei Spielern `Max` und `Min` ausgegangen, die abwechselnd ziehen und beide optimal spielen. Wenn `Max` gewonnen hat, wird der Spielausgang mit +1 bewertet, wenn `Min` gewonnen hat mit -1, und mit 0 sonst. Damit hat man ein sogenanntes "Nullsummenspiel" (der Gewinn des einen Spielers ist der Verlust des anderen) und kann den Algorithmus so gestalten, dass `Max` stets den Zug wählt, der das Spielergebnis maximiert und `Min` entsprechend den Zug wählt, der das Spielergebnis minimiert (daher auch die Namen der Spieler). Minimax baut den gesamten Spielbaum bis zu den Blättern auf. Die Blätter (Spielausgang) werden mit einer `Utility`-Funktion bewertet, und diese Bewertung wird dann im Spielbaum nach oben gereicht.

Videos

Vorlesung [YT], [HSBI]

4.2.1. Spiele als Suchproblem: Minimax

4.2.1.1. Terminologie

- Zwei abwechselnd spielende Spieler: `MAX` und `MIN`, wobei `MAX` beginnt
    - Beide Spieler spielen in jedem Zug optimal
    - Spielergebnis wird aus Sicht von `MAX` bewertet:
        - $+1$, wenn Spieler `MAX` gewinnt
        - -1, wenn Spieler `MIN` gewinnt
        - 0, wenn unentschieden
    - Spieler `MAX` versucht, das Spielergebnis zu **maximieren**
    - Spieler `MIN` versucht, das Spielergebnis zu **minimieren**
- Startzustand: Initialer Zustand des Spielbrettes
- Aktionen: Legale Züge, abhängig vom Spielzustand
- Zieltest: Ist das Spiel vorbei?

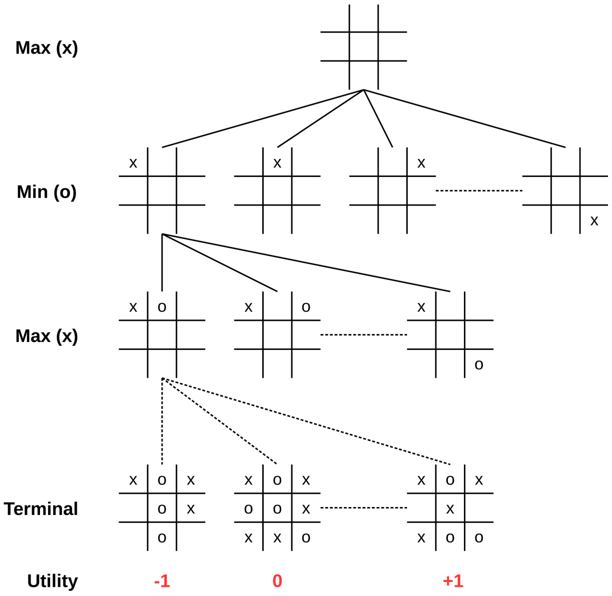
  $\rightarrow$ Startzustand und anwendbare Aktionen definieren den Zustandsraum.
- Nutzenfunktion: UTILITY(s, p): Wert des Spiels für Spieler p im Spielzustand s
- Strategie: Spieler benötigen **Strategie**, um zu gewünschtem Endzustand zu kommen *(unabhängig von den Entscheidungen des Gegenspielers)* $\rightarrow$ einfacher Pfad von Start zu Ziel reicht nicht

Hinweis: Nullsummenspiel! (Der Gewinn des einen Spielers ist der Verlust des anderen Spielers.)

Eine mit dem Minimax-Algorithmus berechnete Strategie wird auch *Minimax-Strategie* genannt. Sie sichert dem betreffenden Spieler den höchstmöglichen Gewinn, der **unabhängig** von der Spielweise des Gegners erreichbar ist.

Bei Nicht-Nullsummenspielen, bei denen die Niederlage des Gegners nicht zwangsläufig mit dem eigenen Gewinn zusammenfällt (d.h. Gewinn des einen Spielers $\neq$ Verlust des anderen Spielers), liefert der Minimax-Algorithmus nicht unbedingt eine optimale Strategie.

4.2.1.2. Spielbaum TTT



4.2.1.3. Minimax (Idee)

1) Erzeuge kompletten Suchbaum mit Tiefensuche
2) Wende Nutzenfunktion (*Utility*) auf jeden Endzustand an
3) Ausgehend von Endzuständen → Bewerte Vorgängerknoten:
    - Knoten ist `Min`-Knoten:
      Nutzen ist das **Minimum** der Kindknoten
    - Knoten ist `Max`-Knoten:
      Nutzen ist das **Maximum** der Kindknoten
4) Startknoten: `Max` wählt den Zug, der zum Nachfolger mit der höchsten Bewertung führt

Annahme: Beide spielen perfekt. Fehler verbessern das Resultat für den Gegner.

4.2.2. Minimax-Algorithmus: Funktionen für MAX- und MIN-Knoten

```
def Max-Value(state):
    if Terminal-Test(state): return Utility(state)

    v = -INF
    for (a, s) in Successors(state):
        v = MAX(v, Min-Value(s))
    return v
```

```
def Min-Value(state):
    if Terminal-Test(state): return Utility(state)

    v = +INF
    for (a, s) in Successors(state):
        v = MIN(v, Max-Value(s))
    return v
```

Hinweis I: Auf wikipedia.org/wiki/Minimax finden Sie eine Variante mit einem zusätzlichen Tiefenparameter, um bei einer bestimmten Suchtiefe abbrechen zu können. Dies ist bereits eine erweiterte Version, wo man beim Abbruch durch das Erreichen der Suchtiefe statt `Utility()` eine `Eval()`-Funktion braucht (vgl. Minimax: Heuristiken).

Wenn man ohne Suchtiefenbeschränkung arbeiten will, braucht man diesen Parameter nicht! Der Algorithmus terminiert auch ohne Suchtiefenbeschränkung!

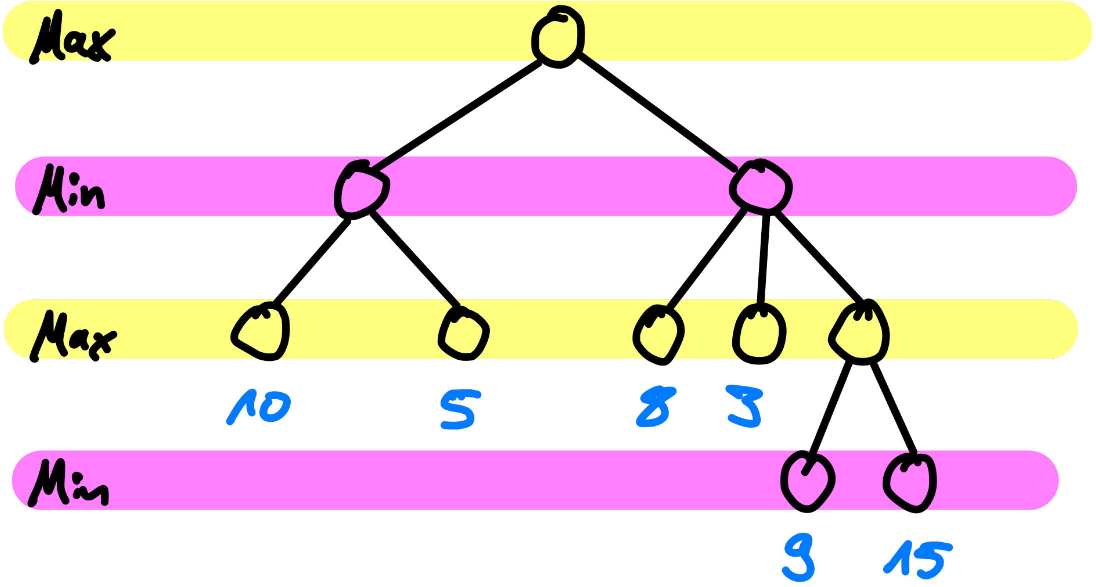
Hinweis II: Im (Russell und Norvig 2021, 196, Abb. 6.3) findet sich eine Variante, die die auf der nächsten Folien gezeigte Startfunktion mit den hier gezeigten `Min-Value()`- und `Max-Value()`-Funktionen verschmilzt. Dabei wird in den beiden Hilfsfunktionen nicht nur der `min`- bzw. `max`-Wert optimiert, sondern auch der jeweils beste Zug gespeichert und als Rückgabe zurückgeliefert.

4.2.3. Minimax-Algorithmus: Sonderbehandlung Startknoten

```
def Minimax(state):
    (val, action) = (-INF, null)
    for (a, s) in Successors(state):
        v = Min-Value(s)
        if (val <= v):
            (val, action) = (v, a)
    return action
```

- Startknoten ist ein MAX-Knoten
- Nicht nur Kosten, sondern auch die zugehörige Aktion speichern

4.2.4. Minimax Beispiel



4.2.4.1. Aufwand Minimax

- maximale Tiefe des Spielbaums: m
- in jedem Zustand b gültige Züge
- $\rightarrow$ Zeitkomplexität $O(b^m)$

Gedankenexperiment:

- erster Zug: b Möglichkeiten,
- zweiter Zug: jeweils wieder b Möglichkeiten $\rightarrow b \star b = b^2$,
- dritter Zug: jeweils wieder b Möglichkeiten $\rightarrow b \star (b \star b) = b^3$,
- $\ldots$,
- m. Zug: jeweils wieder b Möglichkeiten $\rightarrow b^m$

4.2.5. Wrap-Up

- Minimax: Entwickelt Spielbaum, bewertet Zustände entsprechend `Max` und `Min`
  - Gewinn von `Max`: +1, Gewinn von `Min`: -1
  - `Max` wählt das Maximum der möglichen Züge von `Min`
  - `Min` wählt das Minimum der möglichen Züge von `Max`
  - Spielbaum wird bis zu den Blättern entwickelt, Bewertung mit `Utility`

Zum Nachlesen

- Russell und Norvig (2021): Minimax: Abschnitt 6.2
- Ertel (2025): Kapitel 6.4.1 "Minimax-Suche"

Lernziele

- k3: Ich kann den Minimax-Algorithmus auf ein Beispiel anwenden

Quizzes

- Selbsttest Minimax (ILIAS)

Challenges

Optimale Spiele und MiniMax
Auf einem Tisch liegen nebeneinander 5 Streichhölzer. Es gibt zwei Spieler - Weiß und Schwarz - die abwechselnd ein oder zwei Streichhölzer wegnehmen dürfen (es muss mind. ein Streichholz genommen werden). Wer das letzte Streichholz nehmen muss, hat verloren. Zu Beginn ist Weiß am Zug.

1. Spielbaum

   Zeichnen Sie den **kompletten** Spielbaum auf. Geben Sie an den Kanten jeweils die Zahl der genommenen und der verbleibenden Hölzer an.

   Beispiel: Wenn in einem Zug ein Holz genommen wird und 3 Hölzer verbleiben, steht an der entsprechenden Kante "1/3". Geben Sie jeweils an, welcher Spieler am Zug ist.

2. Minimax

   Geben Sie die Bewertung aller Spielzustände mit Hilfe des Minimax-Algorithmus an. Bewerten Sie die Endzustände mit +1, wenn Spieler Weiß gewonnen hat, mit -1, falls Schwarz gewonnen hat.

3. Optimaler Zug

   Mit welchem Zug muss Weiß beginnen, um das Spiel garantiert zu gewinnen (beide Spieler verhalten sich stets optimal)? Argumentieren Sie mit der Minimax-Bewertung.

4.3. Games3: Heuristiken

TL;DR

Minimax entwickelt den gesamten Spielbaum. Wenn nicht genug Zeit dafür zur Verfügung steht, kann man die Suchtiefe begrenzen. Für die Bewertung der Zustände benötigt man eine `Eval`-Funktion, die die Knoten in der selben Reihenfolge sortieren sollte wie es in der vollständigen Version über die `Utility`-Funktion geschieht. Die `Eval`-Funktion sollte zudem schnell zu berechnen sein. Typische Varianten für die `Eval`-Funktion sind gewichtete Features oder ein Nachschlagen in Spieldatenbanken (Spielzustand plus Bewertung).

> Minimax kann auf Spiele mit mehr als zwei Spielern erweitert werden. Dabei versucht dann jeder Spieler für sich, das Ergebnis des Spiels (aus seiner Sicht) zu maximieren.
> Bei Spielen mit Zufall (Würfelereignisse) kann man jedem Würfelereignis eine Wahrscheinlichkeit zuordnen und damit den jeweils erreichbaren `Max`- oder `Min`-Wert gewichten. Die Summe dieser gewichteten Bewertungen ist die Bewertung des entsprechenden "Chance"-Knotens, der dann in der darüberliegenden Ebene nach dem Minimax-Prinzip ausgewertet wird (→ *Expectimax*).

> **Videos**
>
> Vorlesung [YT], [HSBI]

4.3.1. Wenn die Zeit nicht reicht: Suchtiefe begrenzen

- Einführung neuer Funktionen:

  1. `Cutoff-Test` statt `Terminal-Test`

     Beispielsweise bei erreichter Tiefe oder Zeitüberschreitung

  2. `Eval` statt `Utility`

```
Bewertung der erreichten Position (statt nur Bewertung des Endzustandes)
```

- Bedingungen an `Eval`:

  1. Endknoten in selber Reihenfolge wie bei `Utility`
  2. Schnell zu berechnen (!)

4.3.2. Beispiel Schach

- Mögliche Evaluierungskriterien:

  - Materialwert: Bauer 1, Läufer/Springer 3, Turm 5, Dame 9
  - Stellungsbewertung: Sicherheit des Königs, Stellung der Bauern
  - Daumenregeln: 3 Punkte Vorteil → sicherer Sieg

- Nutzung gewichteter Features f_i: $\text{Eval}(s) = w_1 f_1(s) + w_2 f_2(s) + \ldots$

  - Beispiel: $w_1 = 9$ und $f_1(s)$ = (# weiße Königinnen) - (# schwarze Königinnen)

- **Alternativ**:

  - Speicherung von Positionen plus Bewertung in **Datenbanken**
    → Lookup mit Eval(s) (statt Berechnung zur Laufzeit)

  - Training von **ML-Modellen** (Eingabe: Position, Ausgabe: Bewertung)
    $\rightarrow$ Lookup mit Eval(s) (statt Berechnung zur Laufzeit also eine Art Klassifikation der aktuellen Position durch das MLP, welches die gelernte Bewertung ausgibt)

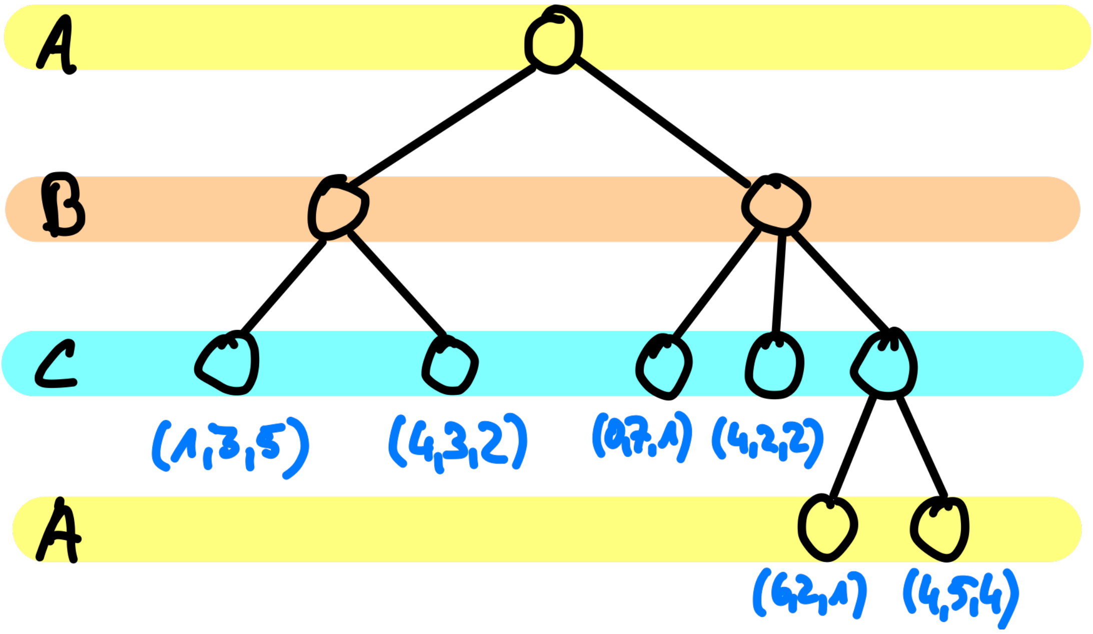
Oft ist das vollständige Berechnen eines Zweiges im Suchbaum sehr zeitaufwändig und kostet viele Ressourcen (Speicher, Rechenkapazität). In einem laufenden Spiel hat man aber nur begrenzt Zeit, und oft laufen Spiele auf dedizierten Geräten mit eher beschränkter Hardware.

Wenn die einzelnen Zweige nicht mehr bis zu den Blättern berechnet werden (können), muss man den aktuellen Zustand aber dennoch bewerten können. Dies wird im Algorithmus durch die Funktion Eval(s) erledigt (für den Zustand s).

Für diese Funktion kann man unterschiedliche Strategien anwenden:

- Oft gibt es Heuristiken, mit denen eine Stellung im Spiel ungefähr bewertet werden kann (obiges Beispiel: Schach mit der Materialbewertung). Damit müssen nicht alle Züge im Vorfeld durchgerechnet werden.
- Eine andere häufig genutzte Strategie ist das Berechnen von möglichst vielen Positionen und der jeweiligen Bewertung vor dem Spiel und das Abspeichern der Tupel (Position, Bewertung) in einer Datenbank. Im Spiel selbst kann man dann relativ schnell die Bewertung der aktuellen Position aus der Datenbank holen.
- Eine weitere häufig angewendete Strategie ist das Trainieren eines neuronalen Netzes (etwa Multilagen-Perzeptron, MLP) mit vorab berechneten Positionen und Bewertungen. Im Spiel wird dann die aktuelle Position in das Netz gegeben und der Output als Bewertung genutzt.
- Eine weitere Strategie ist die Monte Carlo Tree Search. In einer Position hat man i.d.R. relativ viele Möglichkeiten, d.h. der Knoten im Suchbaum hat entsprechend viele Ausgänge. Statt nun zeitaufwändig die vollständige Suche über alle Ausgänge durchzuführen, wird die Berechnung nur für zufällig gewählte Zweige durchgeführt und das Ergebnis gemittelt und als Schätzung für die Bewertung des Knotens genutzt.
- Zum Berechnen der Bewertungen kann auch Reinforcement Learning herangezogen werden. Hier machen die Agenten nacheinander ihre Züge und erst am Ende erfolgt eine Bewertung durch den Trainer. Diese Bewertung wird dann auf die einzelnen Züge zurückgerechnet, und durch mehrfaches Durchspielen immer weiter verbessert.

4.3.3. Minimax mit mehreren Spielern



Hier maximiert jeder Spieler sein eigenes Ergebnis. Im Grunde müsste diese Variante dann besser "Maximax" heissen …

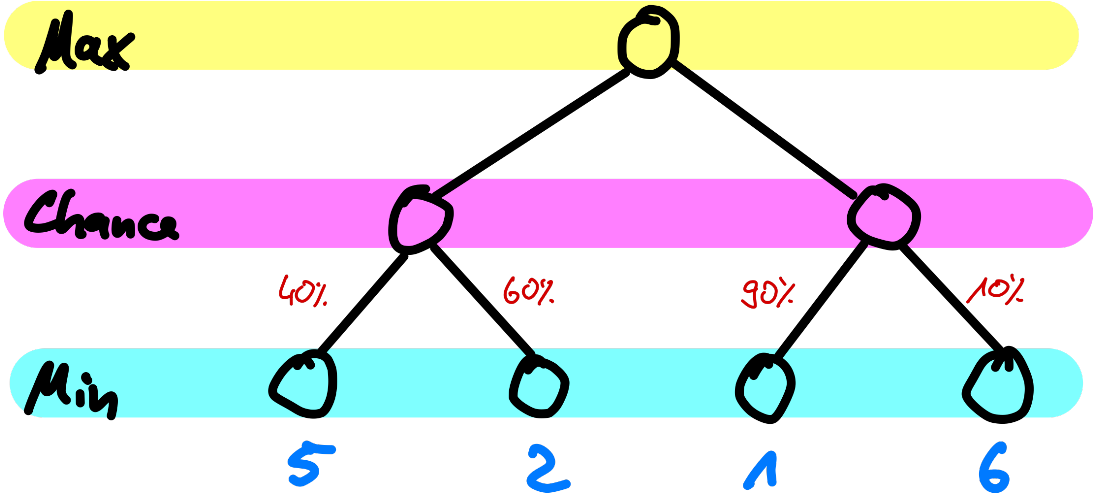
Wenn es an einer Stelle im Suchbaum mehrere gleich gute (beste) Züge geben sollte, kann der Spieler Allianzen bilden: Er könnte dann einen Zug auswählen, der für einen der Mitspieler günstiger ist.

4.3.4. Zufallsspiele

Quelle: Backgammon lg.png by Ptkfgs on Wikimedia Commons (Public Domain)

Backgammon: Was ist in dieser Situation der optimale Zug?

4.3.5. Minimax mit Zufallsspielen: ZUFALLS-Knoten



Zusätzlich zu den MIN- und MAX-Knoten führt man noch Zufalls-Knoten ein, um das Würfelergebnis repräsentieren zu können. Je möglichem Würfelergebnis i gibt es einen Ausgang, an dem die Wahrscheinlichkeit $P(i)$ dieses Ausgangs annotiert wird.

$\rightarrow$ Für Zufallsknoten **erwarteten** Minimax-Wert (*Expectimax*) nutzen

4.3.6. Minimax mit Zufall: Expectimax

Expectimax-Wert für Zufallsknoten C:

$$\text{Expectimax}(C) = \sum_i P(i)\,\text{Expectimax}(s_i)$$

- i mögliches Würfelergebnis
- $P(i)$ Wahrscheinlichkeit für Würfelergebnis
- s_i Nachfolgezustand von C gegeben Würfelergebnis i

Für die normalen Min- und Max-Knoten liefert `Expectimax()` die üblichen Aufrufe von `Min-Value()` bzw. `Max-Value()`.

Auf wikipedia.org/wiki/Expectiminimax finden Sie eine Variante mit einem zusätzlichen Tiefenparameter, um bei einer bestimmten Suchtiefe abbrechen zu können. Dies ist bereits eine erweiterte Version, wo man beim Abbruch durch das Erreichen der Suchtiefe statt `Utility()` eine `Eval()`-Funktion braucht. Zusätzlich kombiniert der dort gezeigte Algorithmus die Funktionen `Expectimax()`, `Min-Value()` und `Max-Value()` in eine einzige Funktion.

Eine ähnliche geschlossene Darstellung finden Sie im (Russell und Norvig 2021, 212).

Hinweis: Üblicherweise sind die Nachfolger der Zufallsknoten gleich wahrscheinlich. Dann kann man einfach mit dem Mittelwert der Bewertung der Nachfolger arbeiten.

4.3.7. Wrap-Up

- Minimax:
  - Kriterien zur Begrenzung der Suchtiefe, Bewertung `Eval` statt `Utility`
  - Erweiterung auf > 2 Spieler
  - Erweiterung auf Spiele mit Zufall: *Expectimax*

Zum Nachlesen

- Russell und Norvig (2021): Erweiterungen und Heuristiken: Abschnitte 6.2.2, 6.3, 6.5
- Ertel (2025): Kapitel 6.5 "Heuristische Bewertungsfunktionen"

Lernziele

- k2: Ich kann die Erweiterung von Minimax für mehr als zwei Spieler erklären
- k2: Ich kann die Erweiterung von Minimax mit Zufallskomponenten erklären
- k2: Ich kann die Heuristik erklären: Sortierung der Nachfolger
- k2: Ich kann erklären, wie die Suchtiefe beschränkt werden kann durch den Übergang zu Bewertungsfunktion
- k2: Ich kann erklären, wie eine Stellungsbewertung über Spieldatenbanken funktioniert
- k3: Ich kann den Minimax-Algorithmus implementieren
- k3: Ich kann Tiefenbeschränkung und Bewertungsfunktion bei Minimax einsetzen

4.4. Games4: Alpha-Beta-Pruning

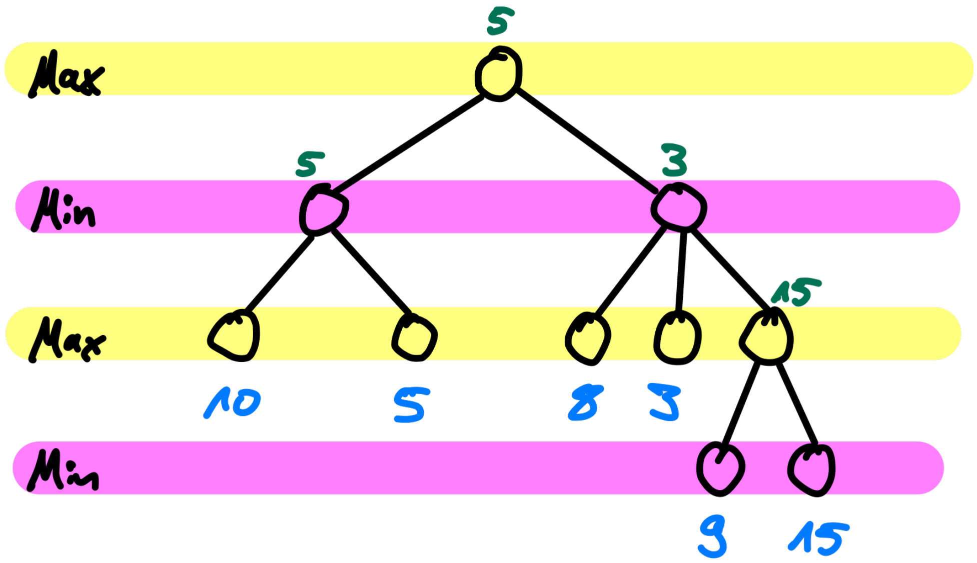
TL;DR

Minimax entwickelt den gesamten Spielbaum. Wenn man dabei die bisher besten Werte für MAX und MIN als α und β mitführt, beobachtet man, dass ein α-Wert nie kleiner wird und ein β-Wert nie größer wird. Dies kann man ausnutzen und das Entwickeln von Pfaden abbrechen, wenn in einem MIN-Knoten der β-Wert kleiner wird als der α-Wert des MAX-Vorgängers: (a) kann der β-Wert bei der weiteren Untersuchung der verbleibenden Nachfolger im MIN-Knoten nur noch kleiner werden, und (b) würde der MAX-Vorgänger diesen MIN-Knoten nie als Nachfolger in Betracht ziehen, da er bereits einen besseren Zug gesehen hat (da sein α größer ist als das β im Nachfolger). Deshalb kann man hier sofort die Untersuchung der verbleibenden Nachfolger im MIN-Knoten abbrechen ("Pruning"). Eine analoge Überlegung gilt für einen MAX-Nachfolger unter einem MIN-Knoten.
Dabei bleibt das Endergebnis erhalten. Man schneidet nur Suchpfade weg, die das Ergebnis von Minimax nicht verändern.
Die mögliche Effizienzsteigerung ist sehr stark abhängig von Sortierung der Nachfolger! Deshalb stellt man häufig Heuristiken zur "richtigen" Sortierung der Nachfolger auf ("Killer-Moves").
Zusätzlich kann man wie bei Minimax auch die Suchtiefe begrenzen und eine Bewertungsfunktion (statt der Nutzenfunktion) einsetzen. Auch hier kann die Bewertungsfunktion wieder gewichtete Features nutzen und/oder Positionen mit in Datenbanken gespeicherten Positionen und Bewertungen abgleichen.

Videos

Vorlesung [YT], [HSBI]

4.4.1. Verbesserung Minimax-Algorithmus



→ **Minimax-Baum**: Verbesserungen möglich?

4.4.2. Alpha-beta-Pruning

Minimax-Algorithmus mit zusätzlichen Informationen:

- α: bisher bester Wert für MAX (höchster Wert)
- β: bisher bester Wert für MIN (kleinster Wert)

→ Beobachtungen:

1. α für MAX-Knoten wird nie kleiner
2. β für MIN-Knoten wird nie größer

4.4.3. Pruning-Regeln

1. Schneide (unter) MIN-Knoten ab, deren $\beta \leq$ dem α des MAX-Vorgängers ist.

2. Schneide (unter) MAX-Knoten ab, deren $\alpha \geq$ dem β des MIN-Vorgängers ist.

Abbruch, wenn kein Platz mehr zwischen Alpha und Beta

4.4.4. Alpha-beta-Pruning – Der Algorithmus (Skizze)

```
def Max-Value(state, alpha, beta):
    if Terminal-Test(state): return Utility(state)

    v = -INF
    for (a, s) in Successors(state):
        v = MAX(v, Min-Value(s, alpha, beta))
        if (v >= beta): return v
        alpha = MAX(alpha, v)
    return v
```

```
def Min-Value(state, alpha, beta):
    if Terminal-Test(state): return Utility(state)

    v = +INF
    for (a, s) in Successors(state):
        v = MIN(v, Max-Value(s, alpha, beta))
        if (v <= alpha): return v
        beta = MIN(beta, v)
    return v
```

Initialer Aufruf von `Max-Value()` mit $\alpha = -\infty$ und $\beta = +\infty$

Achtung: Es kursieren Varianten von diesem Algorithmus, bei denen auf die Hilfsvariable `v` verzichtet wird und stattdessen `alpha` bzw. `beta` direkt modifiziert werden und als Rückgabewert dienen. Das *kann* zu anderen oder falschen Ergebnissen führen! Sie können das in der Aufgabe auf Blatt 03 gut beobachten.

Hier noch zwei Links zu Online-Simulatoren für Minimax und Alpha-Beta-Pruning:

homepage.ufp.pt/jtorres/ensino/ia/alfabeta.html

raphsilva.github.io/utilities/minimax_simulator/ (englische Variante von https://github.com/lerneumann/minimax)

Vergleichen Sie die Ergebnisse im Simulator mit den selbst berechneten Werten.

4.4.5. Alpha-beta-Pruning – Eigenschaften

1. Pruning beeinflusst nicht das Endergebnis!
2. Sortierung der Nachfolger spielt große Rolle
3. Perfekte Sortierung: $O(b^{d/2}) \rightarrow$ Verdopplung der Suchtiefe möglich

Für Schach immer noch zu aufwändig …

4.4.6. Verbesserungen für Alpha-beta-Pruning

- "Killer-Move": Maximale Effizienz nur wenn **optimaler Zug immer zuerst** untersucht
  → Zu untersuchende Züge **sortieren/priorisieren**, zb. Schach:

  a) Figuren schlagen
  b) Drohen
  c) Vorwärts ziehen
  d) Rückwärts ziehen

- Verändern der Suchtiefe nach Spielsituation

- Bewertungsfunktion `Eval`:
  - Datenbanken mit Spielsituationen und Expertenbewertung:
    - Eröffnungsspiele (besonders viele Verzweigungen)
    - Endspiele
  - Lernen der optimalen Gewichte für `Eval`-Funktion
  - Berücksichtigung von Symmetrien

4.4.7. Beispiel DeepBlue (IBM, 1997)

- Alpha-beta-Pruning mit Tiefenbeschränkung: ca. 14 Halbzüge
- Dynamische Tiefenbeschränkung (stellungsabhängig, max. ca. 40 Züge)
- Heuristische Stellungsbewertung `Eval`:
  - mehr als 8.000 Features
  - ca. 4.000 Eröffnungsstellungen
  - ca. 700.000 Spielsituationen (von Experten bewertet)
  - Endspiel-Datenbank: alle Spiele mit 5 Steinen, viele mit 6 Steinen

Quelle: (Russell und Norvig 2014, p. 185)

4.4.8. Beispiel AlphaGo (Google, 2016)

- Beschränkung der Suchtiefe: Bewertung der Stellung durch *"Value Network"*
- Beschränkung der Verzweigungsbreite: Bestimmung von Zugkandidaten durch *"Policy Network"*

- Training dieser *"Deep Neural Networks"*:
  - Überwachtes Lernen: "Analyse" von Spiel-Datenbanken
  - Reinforcement-Lernen: Self-Play, Bewertung am Ende
    - Züge mit Monte-Carlo-Baumsuche ausgewählt

Quelle: (Silver u. a. 2016), siehe auch deepmind.com/research/alphago/

4.4.9. Wrap-Up

- Alpha-beta-Pruning:
  - Mitführen der bisher besten Werte für MAX und MIN: α und β
  - Abschneiden von Pfaden, die Verschlechterung bewirken würden
  - Endergebnis bleibt erhalten
  - Effizienzsteigerung abhängig von Sortierung der Nachfolger
- Viele Verbesserungen denkbar:
  - Zu untersuchende Züge "richtig" sortieren (Heuristik)
  - Suchtiefe begrenzen und Bewertungsfunktion (statt Nutzenfunktion)
  - Positionen mit Datenbank abgleichen

Zum Nachlesen

- Russell und Norvig (2021): Alpha-beta-Pruning: Abschnitt 6.2.3, Erweiterungen: Abschnitt 6.3
- Ertel (2025): Kapitel 6.4.2 "Alpha-Beta-Pruning"

Lernziele

- k3: Ich kann Alpha-Beta-Pruning implementieren und anwenden
- k2: Ich kann die Anwendung verschiedener von Minimax her bekannter Optimierungsmöglichkeiten auf Alpha-Beta-Pruning erklären: Sortierung der Nachfolger, Suchtiefe beschränken (Übergang zu Bewertungsfunktion), Spielzug-Bewertung über Spieldatenbanken

Quizzes

- Selbsttest Alpha-Beta-Pruning (ILIAS)

Challenges

Optimale Spiele und MiniMax

Auf einem Tisch liegen nebeneinander 5 Streichhölzer. Es gibt zwei Spieler - Weiß und Schwarz - die abwechselnd ein oder zwei Streichhölzer wegnehmen dürfen (es muss mind. ein Streichholz genommen werden). Wer das letzte Streichholz nehmen muss, hat verloren. Zu Beginn ist Weiß am Zug.

1. Spielbaum

   Zeichnen Sie den **kompletten** Spielbaum auf. Geben Sie an den Kanten jeweils die Zahl der genommenen und der verbleibenden Hölzer an.

   Beispiel: Wenn in einem Zug ein Holz genommen wird und 3 Hölzer verbleiben, steht an der entsprechenden Kante "1/3". Geben Sie jeweils an, welcher Spieler am Zug ist.

2. Minimax

   Geben Sie die Bewertung aller Spielzustände mit Hilfe des Minimax-Algorithmus an. Bewerten Sie die Endzustände mit +1, wenn Spieler Weiß gewonnen hat, mit -1, falls Schwarz gewonnen hat.

3. Alpha-Beta-Pruning

   Wenden Sie Alpha-Beta-Pruning auf den Spielbaum an. Werden damit mehr oder weniger oder gleich viele Spielzüge wie mit Minimax entwickelt? Begründen Sie Ihre Antwort.

5. Entscheidungsbäume (Decision Tree Learner - DTL)

Beim überwachten Lernen soll eine Hypothese aufgebaut werden, die der echten (zu lernenden) Funktion möglichst nahe kommt. Eine Hypothese kann im einfachsten Fall als Entscheidungsbaum dargestellt werden. Die Merkmale bilden dabei die Knoten im Baum, und je Ausprägung gibt es eine Kante zu einem Nachfolgerknoten. Ein Merkmal bildet die Wurzel des Baums, an den Blättern sind die Klassen zugeordnet.

Einen Entscheidungsbaum kann man zur Klassifikation eines Objekts schrittweise durchlaufen: Für jeden Knoten fragt man die Ausprägung des Merkmals im Objekt ab und wählt den passenden Ausgang aus dem Knoten. Wenn man am Blatt angekommen ist, hat man die Antwort des Baumes auf das Objekt, d.h. üblicherweise die Klasse.

5.1. DTL1: Machine Learning 101

TL;DR

Lernen wird in der KI oft als Verhaltensänderung (eines Systems) aufgefasst. Dabei soll eine Gütefunktion optimiert werden.
Je nach verfügbarem Feedback eines “Lehrers” werden typischerweise drei Arten von Lernen unterschieden: Überwachtes Lernen, Unüberwachtes Lernen, Reinforcement Lernen. Dabei stellt der Lehrer beim überwachten Lernen Trainingsbeispiele plus eine Vorgabe (Klasse, Funktionswert) zur Verfügung, während beim unüberwachten Lernen nur die Trainingsbeispiele bereitgestellt werden und der Algorithmus selbst Zusammenhänge in den Daten erkennen soll. Beim Reinforcement Learning erfolgt das Feedback am Ende einer Kette von Aktionen, d.h. der Algorithmus muss diese Bewertung auf die einzelnen Aktionen zurückrechnen.
Beim überwachten Lernen soll eine Hypothese aufgebaut werden, die der echten (zu lernenden) Funktion möglichst nahe kommt. Eine konsistente Hypothese erklärt die Trainingsdaten, eine generalisierende Hypothese kann auch unbekannte Daten (die aus der selben Quelle stammen, also zum selben Problem gehören) korrekt bewerten. Es wird unterschieden zwischen Klassifikation (einige wenige diskrete Label/Klassen, die den Trainingsbeispielen zugeordnet sind) und Regression (Lernen eines Funktionsverlaufs).
Merkmalsvektoren gruppieren Eigenschaften des Problems bzw. der Objekte, d.h. jedes Objekt kann über einen Merkmalsvektor beschrieben werden. Trainingsdaten sind ausgewählte Beispielobjekte (durch Merkmalsvektoren beschrieben) plus die Vorgabe (Klasse oder Funktionswert) vom Lehrer.

Videos

Vorlesung [YT], [HSBI]

5.1.1. Was ist Lernen?

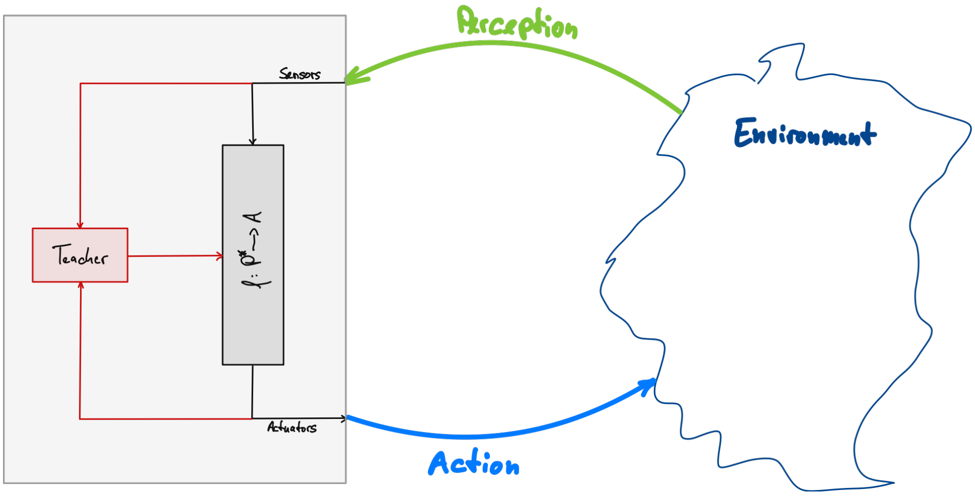
> Verhaltensänderung eines Agenten in Richtung der Optimierung eines Gütefunktionals (Bewertungsfunktion) durch Erfahrung.

5.1.2. Warum Lernen?

- Nicht alle Situationen vorhersehbar
- Nicht alle Details modellierbar
- Lösung oder Lösungsweg unbekannt, nicht explizit programmierbar
- Data Mining: Entdeckung neuen Wissens durch Analyse der Daten
- Selbstanpassende Programme

→ Lernen wichtige Eigenschaft lebender Wesen :-)

5.1.3. Learning Agent



5.1.4. Feedback während des Lernens

- **Überwachtes Lernen**
  - Lernen durch Beobachtung
  - Vorgabe von Beispielen: Ein- und Ausgabewerte

  → Regression, Klassifikation

- **Unüberwachtes Lernen**
  - Erkennen von Mustern in den Inputdaten, Clustering
  - Kein Feedback (!)

- **Reinforcement Lernen**
  - Bewertung der Aktionen des Agenten am Ende einer Aktionsfolge

Beispiel Kleinkind: Lernen von Klassen/Konzepten durch Beispiele

- Zuerst ist alles "Katze" (Übergeneralisierung)
- Differenzierung durch Feedback der Umwelt; Erkennung unterschiedlicher Ausprägungen

5.1.5. Beispiel: Kreditrisiko

- Bankkunde beantragt Kredit
- Soll er aus Sicht der Bank den Kredit bekommen?

- Bankangestellter betrachtet (relevante) Merkmale des Kunden:
  - Alter, Einkommen, sozialer Status
  - Kundenhistorie bei der Bank
  - Höhe des Kredits

- Bewertung des Kreditrisikos:
  - **Klassifikation**: Guter oder schlechter Kunde (Binäre Entscheidung: 2 Klassen)
  - **Regression**: Vorhersage Gewinn/Verlust für die Bank (Höhe des Gewinns/Verlusts interessant)

5.1.6. Beispiel: Autoreparatur

- **Gegeben**: Eigenschaften eines Autos

  → Eigenschaften: Ausprägungen der Merkmale

- **Gesucht**: Diagnose und Reparaturanleitung

  → Hypothese über den Merkmalen (Funktion h)

5.1.7. Lernen durch Beobachten: Lernen einer Funktion f

Funktionsapproximation: Lernen einer Funktion f anhand von Beispielen

- Ein Beispiel ist ein Tupel $(\mathbf{x}, \mathrm{f}(\mathbf{x}))$, etwa

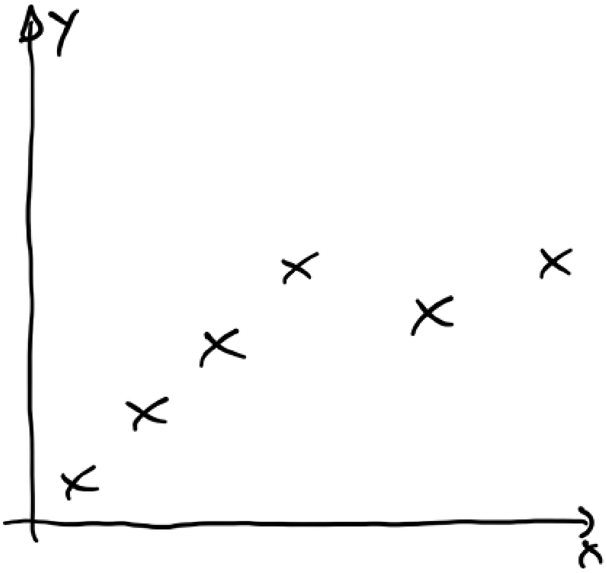
$$(\mathbf{x}, \mathrm{f}(\mathbf{x})) = \left(\begin{array}{ccc} O & O & X \\ . & X & . \\ X & . & . \end{array}, +1 \right)$$

- Aufgabe: Baue Hypothese h auf, so dass h $\approx$ f.
  - Benutze dazu Menge von Beispielen $\rightarrow$ **Trainingsdaten**.

- Ziele:
  1. **Konsistente Hypothese**: Übereinstimmung bei Trainingsdaten
  2. **Generalisierende Hypothese**: Korrekte Vorhersage bei unbekannten Daten

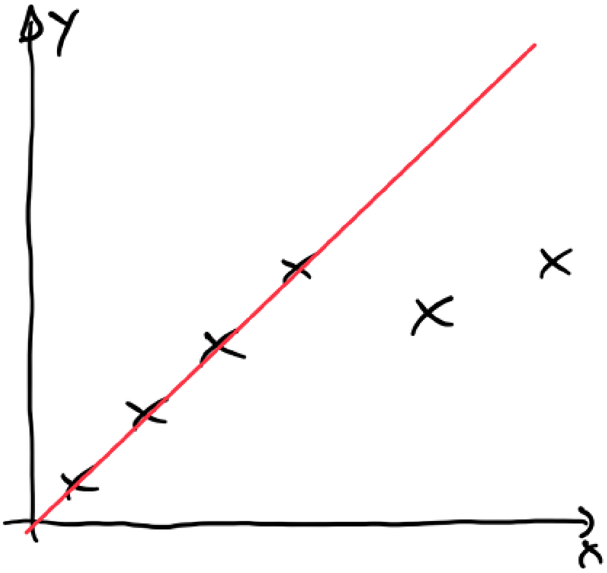
Anmerkung: Stark vereinfachtes Modell realen Lernens!

5.1.8. Konstruieren einer konsistenten Hypothese



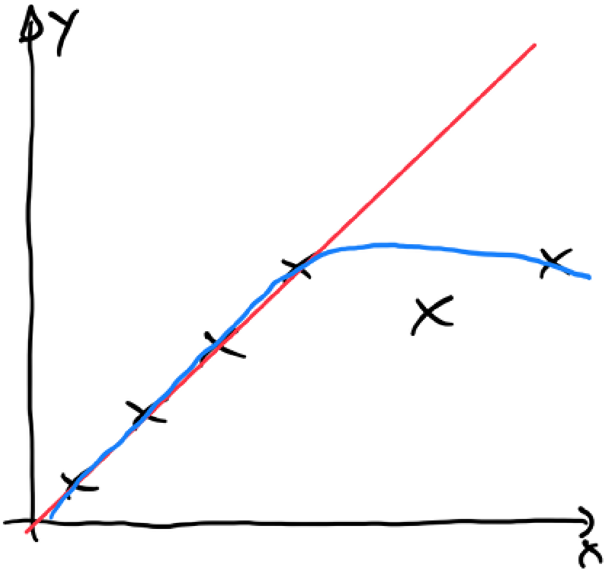
Welcher Zusammenhang ist hier dargestellt? Offenbar eine Art Funktionsverlauf … Wir haben für einige x-Werte die zugehörigen y-Werte vorgegeben.

5.1.9. Konstruieren einer konsistenten Hypothese (cnt.)



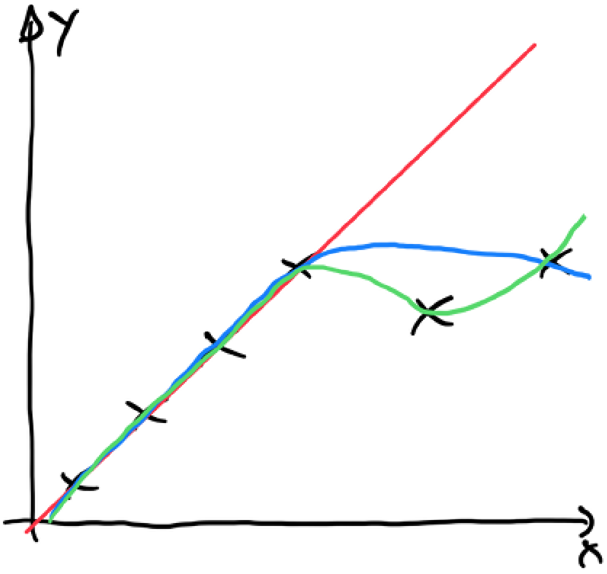
Die einfachste Approximation wäre eine lineare Funktion. Allerdings werden hierbei einige Werte mehr oder weniger stark nicht korrekt widergegeben, d.h. man hat einen relativ hohen (Trainings-) Fehler.

5.1.10. Konstruieren einer konsistenten Hypothese (cnt.)



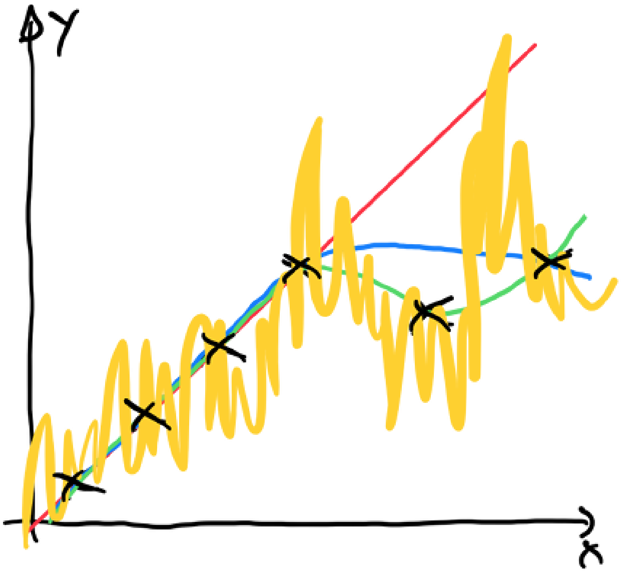
Die Hyperbel erklärt die Trainingsdaten bis auf den einen Punkt sehr gut. Die Frage ist, ob dieser eine Punkt zum zu lernenden Zusammenhang gehört oder ein Ausreißer ist, den man gefahrlos ignorieren kann?

5.1.11. Konstruieren einer konsistenten Hypothese (cnt.)



Die grüne Hypothese ist von allen bisher gezeigten die komplexeste, erklärt aber alle Datenpunkte. D.h. hier wäre der Trainingsfehler Null. Zwischen den Trainingsdaten zeigt das Modell eine "glatte" Approximation, d.h. es wird auch neue Daten, die es beim Training nicht gesehen hat, relativ gut erklären. (Dabei liegt freilich die Annahme zugrunde, dass alle relevanten Daten in der Trainingsmenge vorhanden sind, d.h. dass es insbesondere zwischen den Datenpunkten keine Ausreißer o.ä. gibt.)

5.1.12. Konstruieren einer konsistenten Hypothese (cnt.)



Diese Hypothese erklärt ebenfalls sämtliche Trainingsdaten. Allerdings schwingt die Funktion zwischen den Daten stark hin und her. Vermutlich entspricht dies nicht dem zu lernenden Funktionsverlauf. Der Trainingsfehler wäre wie bei der deutlich einfacheren Hypthese aus dem letzten Schritt Null. Der Generalisierungsfehler (sprich die Abweichung, wenn man das Modell nach Daten zwischen den Trainingspunkten fragt) dürfte erheblich höher liegen.

D.h. hier hat das Modell einfach die Trainingsdaten auswendig gelernt, aber nicht den Zusammenhang zwischen den Daten! Dies ist in der Regel unerwünscht!

5.1.13. Occam's Razor

Bevorzuge die einfachste konsistente Hypothese!

1. Wenn es mehrere mögliche Erklärungen für einen Sachverhalt gibt, ist die einfachste Erklärung allen anderen vorzuziehen.
2. Eine Erklärung ist "einfach", wenn sie möglichst wenige Variablen und Annahmen enthält und wenn diese in klaren logischen Beziehungen zueinander stehen, aus denen der zu erklärende Sachverhalt logisch folgt.

5.1.14. Trainingsdaten und Merkmalsvektoren

Lehrer gibt Beispiele vor: Eingabe $\mathbf{x}$ und passende Ausgabe $\mathrm{f}(\mathbf{x})$

- Ausgabe: typischerweise Skalar (Funktionswert oder Klasse)
  $\rightarrow$ Beispiel: Bewertung eines Spielstandes bei TicTacToe

- Eingabe: (Beschreibung des) Objekt(s) oder Situation, die zur Ausgabe gehört
  $\rightarrow$ Beispiel: Spielstand bei TicTacToe

Merkmalsvektoren:

- Zusammenfassen der relevanten Merkmale zu Vektoren

5.1.15. Beispiel: Schwimmen im See

Beschreibung der Faktoren, wann ich im See schwimmen möchte:

1. Scheint die Sonne?
2. Wie warm ist das Wasser?
3. Wie warm ist die Luft?

- Trainingsbeispiel:

  - Eingabe: Merkmalsvektor `(sonnig, warm, warm)`
  - Ausgabe: Klasse `ja`

Dabei wird davon ausgegangen, dass jeder Faktor (jedes Merkmal) an einer bestimmten Stelle im Merkmalsvektor aufgeführt ist. Beispielsweise gehört das `sonnig` zur Frage "Scheint die Sonne", `warm` jeweils zur Wasser- und zur Lufttemperatur.

Damit hat man in einem Vektor eine Situation komplett beschrieben, d.h. einen Zustand der Welt mit den relevanten Dingen beschrieben. Diesem Zustand kann man beispielsweise ein Label (Klasse) verpassen, hier in diesem Fall "ja, in dieser Welt möchte ich schwimmen".

Die Trainingsmenge baut sich dann beim überwachten Lernen aus vielen solcher Paare (Merkmalsvektor, Klasse) auf, und die Algorithmen sollen diese Zuordnung lernen, d.h. ein Modell für diese Daten erzeugen, welches die Daten gut erklärt und darüber hinaus für neue Daten aus der selben Datenquelle gute Vorhersagen macht.

5.1.16. Trainingsdaten – Merkmalsvektoren

Generell: Merkmalsvektor für Objekt v:

$$\mathbf{x}(v) = (x_1, x_2, \ldots, x_n)$$

- n Merkmale (Attribute)
- Attribut x_t hat m_t mögliche Ausprägungen
- Ausprägung von v bzgl. x_t: $\quad x_t(v) = i \quad$ (mit $i = 1 \ldots m_t$)

Anmerkung: Stellen Sie sich den Merkmalsvektor $\mathbf{x}$ vielleicht wie einen Konstruktor einer Klasse `x` vor: Die einzelnen Attribute x_t sind die Parameter, aus denen der Merkmalsvektor aufgebaut ist/wird. Jedes der Attribute hat einen Typ und damit eine bestimmte Anzahl erlaubter Werte ("Ausprägungen") …

Trainingsbeispiel:

- Tupel aus Merkmalsvektor und zugehöriger Klasse: $(\mathbf{x}(v), k)$

5.1.17. Wrap-Up

- Lernen ist Verhaltensänderung, Ziel: Optimierung einer Gütefunktion
  - Aufbau einer Hypothese, die beobachtete Daten erklären soll
  - Arten: Überwachtes Lernen, Unüberwachtes Lernen, Reinforcement Lernen
- Merkmalsvektoren gruppieren Eigenschaften des Problems bzw. der Objekte
- Trainingsdaten: Beispielobjekte (durch Merkmalsvektoren beschrieben) plus Vorgabe vom Lehrer

Zum Nachlesen

- Russell und Norvig (2021): Lernen: Abschnitte 19.1 und 19.2

Lernziele

- k1: Ich kann den Begriff "(maschinelles) Lernen" definieren und verschiedene Arten unterscheiden
- k2: Ich kann "Überwachtes Lernen" erklären: Lernen durch Beobachten (mit Lehrer)
- k2: Ich kann wichtige Elemente des (maschinellen) Lernens erklären: Merkmalsvektoren, Eigenschaften, Ausprägung, Objekte, Trainingsmenge

Quizzes

- Selbsttest Intro ML (ILIAS)

Challenges

Modellierung
Sie stehen vor der Entscheidung, ob Sie sich zur Vorbereitung auf die Flipped-Classroom-Sitzung noch das Skript anschauen. Welche Attribute benötigen Sie, um die Situation zu beschreiben?
Metriken für Klassifikatoren
Es ist wieder Wahlkampf: Zwei Kandidaten O und M bewerben sich um die Kanzlerschaft. Die folgende Tabelle zeigt die Präferenzen von sieben Wählern.

Nr.	Alter	Einkommen	Bildung	Kandidat	Vorhersage
1	≥ 35	hoch	Abitur	O	O
2	< 35	niedrig	Master	O	O
3	≥ 35	hoch	Bachelor	M	M
4	≥ 35	niedrig	Abitur	M	M
5	≥ 35	hoch	Master	O	O
6	< 35	hoch	Bachelor	O	M
7	< 35	niedrig	Abitur	M	O

Auf diesem Datensatz wurde ein Klassifikator trainiert, die Trainingsergebnisse sind in der Tabelle unter "Vorhersage" angegeben.
Bewerten Sie den Klassifikator.

5.2. DTL2: CAL2

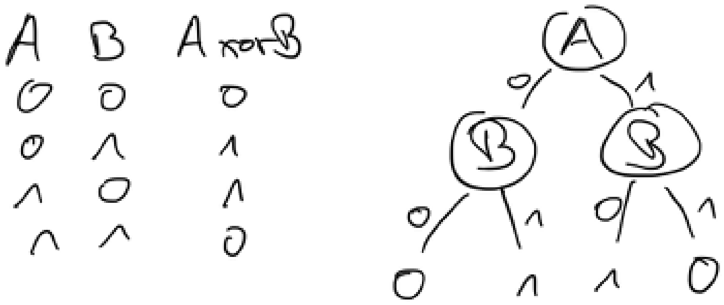
TL;DR

Eine Hypothese kann im einfachsten Fall als Entscheidungsbaum dargestellt werden. Die Merkmale bilden dabei die Knoten im Baum, und je Ausprägung gibt es eine Kante zu einem Nachfolgerknoten. Ein Merkmal bildet die Wurzel des Baums, an den Blättern sind die Klassen zugeordnet.
Einen Entscheidungsbaum kann man zur Klassifikation eines Objekts schrittweise durchlaufen: Für jeden

> Knoten fragt man die Ausprägung des Merkmals im Objekt ab und wählt den passenden Ausgang aus dem Knoten. Wenn man am Blatt angekommen ist, hat man die Antwort des Baumes auf das Objekt, d.h. üblicherweise die Klasse.
> Den Baum kann man mit dem Algorithmus CAL2 schrittweise aufbauen. Man startet mit "Nichtwissen" (symbolisiert mit einem "*") und iteriert durch alle Trainingsbeispiele, bis der Baum sich nicht mehr verändert. Wenn der Baum auf ein Beispiel einen "*" ausgibt, dann ersetzt man diesen "*" mit der Klasse des eben betrachteten Beispiels. Wenn der Baum bei einem Beispiel die passende Klasse ausgibt, macht man mit dem nächsten Beispiel weiter. Wenn der Baum bei einem Beispiel eine andere Klasse ausgibt, muss das Klassensymbol im Baum (an der Stelle, wo das Objekt gelandet ist) durch den nächsten Test ersetzt werden: Hierzu nimmt man das nächste, auf diesem konkreten Pfad noch nicht verwendete Merkmal. CAL2 kann nur mit diskreten Attributen und disjunkten Klassen einen fehlerfreien Baum erzeugen.

> **Videos**
>
> Vorlesung [YT], [HSBI]

5.2.1. Entscheidungsbäume: Klassifikation



- Attribute als Knoten im Baum
- Ausprägungen als Test (Ausgang, Verzweigung)
- Klasse (Funktionswert) als Blatt

Erinnern Sie sich an das Beispiel mit der Auto-Reparatur aus der letzten Sitzung.

Die relevanten Eigenschaften (Merkmale) eines Autos würden als Knoten im Baum repräsentiert. Beispiel: "Motor startet" oder "Farbe".

Jedes Merkmal hat eine Anzahl von möglichen Ausprägungen, diese entsprechen den Verzweigungen am Knoten. Beispiel: "startet", "startet nicht" oder "rot", "weiß", "silber", … .

Entsprechend kann man durch Abarbeiten des Entscheidungsbaumes am Ende zu einer Diagnose gelangen (Klasse).

Eine andere Sichtweise ist die Nutzung als Checkliste für eine Reparatur …

5.2.2. Definition Entscheidungsbaum

- Erinnerung: **Merkmalsvektor** für Objekt v:

$$\mathbf{x}(v) = (x_1, x_2, \ldots, x_n)$$

  - n Merkmale (Attribute)
  - Attribut x_t hat m_t mögliche Ausprägungen
  - Ausprägung von v bzgl. x_t: $\quad x_t(v) = i \quad$ (mit $i = 1 \ldots m_t$)

- **Alphabet** für Baum:

$$\{x_t | t = 1, \ldots, n\} \cup \{\kappa | \kappa = *, A, B, C, \ldots\} \cup \{(,)\}$$

- **Entscheidungsbaum** α:

$$\alpha = \begin{cases} \kappa & \text{Terminalsymbole: } \kappa = *, A, B, \ldots \\ x_t(\alpha_1, \alpha_2, \ldots, \alpha_{m_t}) & x_t \text{ Testattribut mit } m_t \text{ Ausprägungen} \end{cases}$$

Anmerkung: Stellen Sie sich die linearisierte Schreibweise wieder wie den (verschachtelten) Aufruf von Konstruktoren vor. Es gibt die Oberklasse `Baum`, von der für jedes Attribut eine Klasse abgeleitet wird. D.h. der Konstruktor für eine Attributklasse erzeugt letztlich ein Objekt vom Obertyp `Baum`. Außerdem sind die Terminalsymbole `A`, `B`, … Objekte vom Typ `Blatt`, welches eine Unterklasse von `Baum` ist …

Dabei wird die Anzahl der möglichen Ausprägungen für ein Attribut berücksichtigt: Jede Ausprägung hat einen Parameter im Konstruktor. Damit werden die Unterbäume beim Erzeugen des Knotens übergeben.

5.2.3. Induktion von Entscheidungsbäumen: CAL2

1) Anfangsschritt: $\alpha^{(0)} = *$ (totales Unwissen)
2) n-ter Lernschritt: Objekt v mit Klasse k, Baum $\alpha^{(n-1)}$ gibt κ aus
   - $\kappa = *$: ersetze $*$ durch k
   - $\kappa = k$: keine Aktion nötig
   - $\kappa \neq k$: Fehler
     - Ersetze κ mit neuem Test: $\kappa \leftarrow x_{t+1}(*, \ldots, *, k, *, \ldots, *)$
     - x_{t+1}: nächstes Attribut, auf dem aktuellen Pfad noch nicht verwendet
     - Symbol k an Position i wenn $x_{t+1}(v) = i$

$\alpha^{(n)}$ bezeichnet den Baum im n-ten Lernschritt.

CAL2 ist ein **Meta-Algorithmus**: Es ist ein Algorithmus, um einen Algorithmus zu lernen :-)

5.2.4. Beispiel mit CAL2

x_1	x_2	x_3	k
0	0	1	A
1	0	0	A
0	1	4	B
1	1	2	B
0	0	3	A

Ergebnis: $x_1(x_2(A, B), x_2(A, B))$

Anmerkung: Denken Sie an die Analogie von oben. x_1 kann als Konstruktor einer Klasse `x1` betrachtet werden, die eine Unterklasse von `Baum` ist. Durch den Aufruf des Konstruktors wird als ein `Baum` erzeugt.

Es gibt in x_1 zwei mögliche Ausprägungen, d.h. der Baum hat in diesem Knoten zwei alternative Ausgänge. Diese Unterbäume werden dem Konstruktor von `x1` direkt beim Aufruf übergeben (müssen also Referenzen vom Typ `Baum` sein).

5.2.5. CAL2: Bemerkungen

- Nur für diskrete Merkmale und disjunkte Klassen

- Zyklischer Durchlauf durch Trainingsmenge
- Abbruch:
  - Alle Trainingsobjekte richtig klassifiziert
    → Kein Fehler in einem kompletten Durchlauf
  - (Differenzierung nötig, aber alle Merkmale verbraucht)
  - (Lernschrittzahl überschritten)

5.2.6. Wrap-Up

- Darstellung der Hypothese als Entscheidungsbaum
- CAL2: diskrete Attribute, disjunkte Klassen

Zum Nachlesen

- Unger und Wysotzki (1981): Der Vollständigkeit halber aufgeführt (Werk ist leider vergriffen und wird nicht mehr verlegt)

Lernziele

- k3: Ich kann den Entscheidungsbaumlerner CAL2 auf ein Beispiel anwenden

Quizzes

- Selbsttest CAL2 (ILIAS)

Challenges

Modellierung
Sie stehen vor der Entscheidung, ob Sie sich zur Vorbereitung auf die Flipped-Classroom-Sitzung noch das Skript anschauen.
Zeichnen Sie einen Entscheidungsbaum, der Ihnen bei der Entscheidung hilft.

Textklassifikation
Betrachten Sie die folgenden Aussagen:

> - Patient A hat weder Husten noch Fieber und ist gesund.
> - Patient B hat Husten, aber kein Fieber und ist gesund.
> - Patient C hat keinen Husten, aber Fieber. Er ist krank.
> - Patient D hat Husten und kein Fieber und ist krank.
> - Patient E hat Husten und Fieber. Er ist krank.

Aufgaben:

1. Trainieren Sie auf diesem Datensatz einen Klassifikator mit CAL2.
2. Ist Patient F krank? Er hat Husten, aber kein Fieber.

Handsimulation CAL2
Zeigen Sie mit einer Handsimulation, wie CAL2 mit dem folgenden Trainingsdatensatz schrittweise einen Entscheidungsbaum generiert. Nutzen Sie die linearisierte Schreibweise.

Beispiel	x_1	x_2	x_3	Klasse
1	a	a	a	1
2	a	b	a	2
3	a	a	b	1
4	b	a	b	1
5	a	a	c	1
6	b	b	b	2

Welchen Entscheidungsbaum würde CAL2 lernen, wenn dem Trainingsdatensatz der Vektor $((a, a, b), 2)$ als Beispiel Nr. 7 hinzugefügt werden würde?

5.3. DTL5: Entropie

TL;DR

Die Entropie kann als Maß für den Informationsgehalt einer Trainingsmenge betrachtet werden: Wieviele Ja/Nein-Entscheidungen sind nötig, um die Daten fehlerfrei zu repräsentieren?
Nach der Wahl eines Attributs kann die verbleibende mittlere Entropie berechnet werden. Damit hat man ein Kriterium für die Auswahl von Attributen beim Aufbau von Entscheidungsbäumen: Nimm das Attribut, welches einen möglichst hohen Informationsgehalt hat. Oder andersherum: Wähle das Attribut, bei dem die verbleibende mittlere Entropie der Trainingsmenge nach der Wahl des Attributs am kleinsten ist.

Videos

Vorlesung [YT], [HSBI]

5.3.1. Wie Attribute wählen?

5.3.1.1. Erinnerung: CAL2

- Zyklische Iteration durch die Trainingsmenge
- Ausschließlich aktuelles Objekt betrachtet
- Reihenfolge der "richtigen" Attributwahl bei Verzweigung unklar

→ Betrachte stattdessen die komplette Trainingsmenge!

5.3.1.2. Relevanz → Informationsgehalt

- Shannon/Weaver (1949): **Entropie**
  - Maß für die Unsicherheit einer Zufallsvariablen
  - Anzahl der Bits zur Darstellung der Ergebnisse eines Zufallsexperiments

5.3.1.3. Beispiele

- Münze, die immer auf dem Rand landet: keine Unsicherheit, 0 Bit
- Faire Münze: Kopf oder Zahl: Entropie 1 Bit
- Fairer 4-seitiger Würfel: 4 mögliche Ausgänge: Entropie 2 Bit
- Münze, die zu 99% auf einer Seite landet: Entropie nahe Null

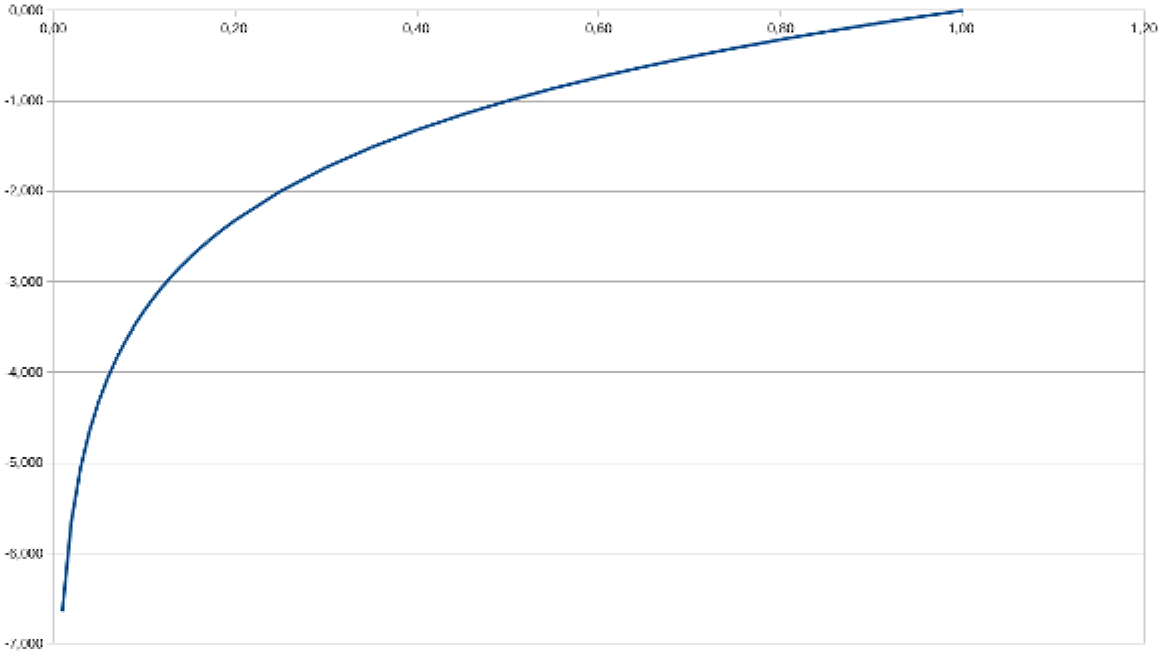
→ Anzahl der Ja/Nein-Fragen, um zur gleichen Information zu kommen

5.3.2. Definition der Entropie $H(V)$ für Zufallsvariable V

- Zufallsvariable $V \rightarrow$ mögliche Werte v_k
- Wahrscheinlichkeit für v_k sei $p_k = P(v_k)$

$$H(V) = -\sum_k p_k \log_2 p_k$$

Hinweis: $\log_2 x = \frac{\log_{10} x}{\log_{10} 2} = \frac{\log x}{\log 2}$



- Nur eine Klasse: $\log_2 1 = 0 \rightarrow H(V) = 0$ Bit
- Zwei Klassen, gleichwahrscheinlich: $\log_2 0.5 = -1 \rightarrow H(V) = 1$ Bit

5.3.3. Beispiele Entropie: faire Münze

Entropie:

$$H(V) = -\sum_k p_k \log_2 p_k$$

- $v_1 =$ Kopf, $v_2 =$ Zahl

- $p_1 = 0.5, p_2 = 0.5$
- $H(\text{Fair}) = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = 1$ Bit

$\log_2 0.5 = -1$

5.3.4. Beispiele Entropie: unfaire Münze

Entropie:

$$H(V) = -\sum_k p_k \log_2 p_k$$

- $v_1 = \text{Kopf}, v_2 = \text{Zahl}$
- $p_1 = 0.99, p_2 = 0.01$
- $H(\text{UnFair}) = -(0.99 \log_2 0.99 + 0.01 \log_2 0.01)$

  $H(\text{UnFair}) \approx 0.08$ Bit

$\log_2 0.01 \approx -6.64$

$\log_2 0.99 \approx -0,014$

5.3.5. Beispiele Entropie: 4-seitiger Würfel

Entropie:

$$H(V) = -\sum_k p_k \log_2 p_k$$

- $v_1 = 1, v_2 = 2, v_3 = 3, v_4 = 4$
- $p_1 = p_2 = p_3 = p_4 = 0.25$
- $H(\text{Wuerfel}) = -4 \cdot (0.25 \log_2 0.25) = 2$ Bit

$\log_2 0.25 = -2$

5.3.6. Entropie der Trainingsmenge: Häufigkeit der Klassen zählen

Nr.	x_1	x_2	x_3	k
1	0	0	0	A
2	1	0	2	A
3	0	1	1	A

Nr.	x_1	x_2	x_3	k
4	1	1	0	B
5	0	1	1	B
6	0	1	0	A

- Anzahl Klasse A: 4
- Anzahl Klasse B: 2
- Gesamtzahl Beispiele: 6

Wahrscheinlichkeit für A: $p_A = 4/6 = 0.667$

Wahrscheinlichkeit für B: $p_B = 2/6 = 0.333$

$$\begin{aligned} H(S) &= -\sum_k p_k \log_2 p_k && (5.1)\\ &= -(4/6 \cdot \log_2 4/6 + 2/6 \cdot \log_2 2/6) && (5.2)\\ &= -(-0.39 - 0.53) = 0.92 \text{ Bit} && (5.3) \end{aligned}$$

5.3.7. Mittlere Entropie nach Betrachtung von Attribut A

$$R(S, A) = \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

- Auswahl von Attribut A partitioniert die Trainingsmenge:
  Je Ausprägung v von A erhält man eine Submenge S_v
- $R(S, A)$ berechnet die mittlere Entropie der Trainingsmenge, nachdem Attribut A ausgewählt wurde: Unsicherheit/nötige Bits nach Auswahl von Attribut A

5.3.8. Entropie der Trainingsmenge nach Attributwahl

Nr.	x_1	x_2	x_3	k
1	0	0	0	A
2	1	0	2	A
3	0	1	1	A

Nr.	x_1	x_2	x_3	k
4	1	1	0	B
5	0	1	1	B
6	0	1	0	A

- Sei Attribut x_1 ausgewählt
- x_1 partitioniert die Trainingsmenge
  - $x_1 = 0$ liefert $S_0 = \{1, 3, 5, 6\}$
  - $x_1 = 1$ liefert $S_1 = \{2, 4\}$
  - Häufigkeit für $x_1 = 0$: $4/6$
  - Häufigkeit für $x_1 = 1$: $2/6$
  - Gesamtzahl Beispiele: 6

$$\begin{aligned}
R(S, A) &= \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v) && (5.4) \\
&= 4/6 \cdot H(\{1, 3, 5, 6\}) + 2/6 \cdot H(\{2, 4\}) && (5.5) \\
&= 4/6 \cdot (-3/4 \cdot \log_2 3/4 - 1/4 \cdot \log_2 1/4) + && (5.6) \\
&\quad 2/6 \cdot (-1/2 \cdot \log_2 1/2 - 1/2 \cdot \log_2 1/2) && (5.7) \\
&= 0.54 + 0.33 = 0.87 \text{ Bit} && (5.8)
\end{aligned}$$

5.3.9. Ausblick: Gini Impurity

Wir haben hier die Entropie als Maß für den Informationsgehalt einer Trainingsmenge genutzt. $R(S, A)$ als die mittlere Entropie nach Betrachtung von Attribut A wird von typischen Entscheidungsbaumverfahren wie ID3 und C4.5 genutzt, um bei einer Verzweigung das nächste möglichst aussagekräftige Merkmal auszuwählen.

In anderen Entscheidungsbaumlernern wird stattdessen die Gini Impurity zur Bestimmung des Informationsgehalts eingesetzt (u.a. CART). Dieses Maß sagt aus, wie oft man ein zufällig gezogenes Element des Datensatzes falsch klassifizieren würde, wenn man es mit einer zufälligen Klasse basierend auf der Verteilung der Klassen im Datensatz labeln würde.

Hierzu drei lesenswerte Blog-Einträge:

- Deep dive into the basics of Gini Impurity in Decision Trees with math Intuition
- Decision Trees, Explained
- Decision Tree Algorithm With Hands-On Example

5.3.10. Wrap-Up

- Begriff und Berechnung der Entropie: Maß für die Unsicherheit
- Begriff und Berechnung des Informationsgewinns
    - Entropie für eine Trainingsmenge
    - Mittlere Entropie nach Wahl eines Attributs

Zum Nachlesen

- Ertel (2025): Entscheidungsbäume: Abschnitt 8.4
- Russell und Norvig (2021): Entscheidungsbäume: Abschnitt 19.3
- Mitchell (2010): ID3: Kapitel 3

Lernziele

- k3: Ich kann für konkrete Beispiele die Entropie und den Information Gain ausrechnen

Quizzes

- Selbsttest Entropie (ILIAS)

Challenges

Entropie einer Trainingsmenge

Betrachten Sie die folgenden Aussagen:

> - Patient A hat weder Husten noch Fieber und ist gesund.
> - Patient B hat Husten, aber kein Fieber und ist gesund.
> - Patient C hat keinen Husten, aber Fieber. Er ist krank.
> - Patient D hat Husten und kein Fieber und ist krank.
> - Patient E hat Husten und Fieber. Er ist krank.

Aufgaben:

1. Geben Sie die Entropie $H(S)$ der Trainingsmenge an.
2. Berechnen Sie $R(H, A)$ (die mittlere Entropie der Trainingsmenge, nachdem Attribut A gesehen wurde) für die einzelnen Attribute.

5.4. DTL6: ID3 und C4.5

TL;DR

Der Entscheidungsbaum-Lernalgorithmus **ID3** nutzt den Informationsgehalt für die Entscheidung bei der Attributwahl: Nimm das Attribut, welches einen möglichst hohen Informationsgehalt hat. Oder andersherum: Wähle das Attribut, bei dem die verbleibende mittlere Entropie der Trainingsmenge nach der Wahl des Attributs am kleinsten ist. Oder noch anders formuliert: Nimm das Attribut, bei dem die Differenz

> zwischen der Entropie der Trainingsmenge (vor der Wahl des Attributs) und der verbleibenden mittleren Entropie (nach der Wahl des Attributs) am größten ist (die Differenz nennt man auch "*Information Gain*").
> Die Trainingsmenge wird entsprechend der Ausprägung in Bezug auf das eben gewählte Merkmal aufgeteilt und an die Kinder des Knotens weiter gereicht; dort wird der Baum rekursiv weiter aufgebaut.
> Durch eine Normierung des *Information Gain* kann eine Verbesserung in Bezug auf mehrwertige Attribute erreicht werden, dies führt zum Algorithmus **C4.5**.

> **Videos**
>
> Vorlesung [YT], [HSBI]

5.4.1. Wie Attribute wählen?

Erinnerung: CAL2

- Zyklische Iteration durch die Trainingsmenge
- Ausschließlich aktuelles Objekt betrachtet
- Reihenfolge der "richtigen" Attributwahl bei Verzweigung unklar

$\rightarrow$ Betrachte stattdessen die **komplette** Trainingsmenge!

5.4.2. Erinnerung Entropie: Maß für die Unsicherheit

- Entropie $H(S)$ der Trainingsmenge S: relative Häufigkeit der Klassen zählen
- Mittlere Entropie nach Betrachtung von Attribut A

$$R(S, A) = \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

- Informationsgewinn durch Betrachtung von Attribut A

$$\begin{aligned} \text{Gain}(S, A) &= H(S) - R(S, A) && (5.9) \\ &= H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v) && (5.10) \end{aligned}$$

$R(S, A)$ ist die Unsicherheit/nötige Bits nach Auswahl von Attribut A. Je kleiner $R(S, A)$, um so kleiner die **verbleibende Unsicherheit** bzw. um so kleiner die Anzahl der nötigen Bits zur Darstellung der partitionierten Trainingsmenge **nach** Betrachtung von Attribut A …

$\rightarrow$ Je kleiner $R(S, A)$, um so größer der Informationsgewinn

5.4.3. Informationsgewinn: Kriterium zur Auswahl von Attributen

1) Informationsgewinn für alle Attribute berechnen
2) Nehme Attribut mit größtem Informationsgewinn als nächsten Test

Nr.	x_1	x_2	x_3	k
1	0	0	0	A
2	1	0	2	A
3	0	1	1	A
4	1	1	0	B
5	0	1	1	B
6	0	1	0	A

$H(S) = 0.92$ Bit

$$\begin{aligned}
\text{Gain}(S, x_1) &= 0.92 - 0.87 = 0.05 \text{ Bit} && (5.11)\\
\text{Gain}(S, x_2) &= 0.92 - 2/6 \cdot 0 - 4/6 \cdot 1 && (5.12)\\
&= 0.25 \text{ Bit} && (5.13)\\
\text{Gain}(S, x_3) &= 0.92 - 3/6 \cdot 0.92 - 2/6 \cdot 1 - 1/6 \cdot 0 && (5.14)\\
&= 0.13 \text{ Bit} && (5.15)
\end{aligned}$$

Informationsgewinn für x_2 am höchsten → wähle x_2 als nächsten Test

5.4.4. Entscheidungsbaumlerner ID3 (Quinlan, 1986)

```
def ID3(examples, attr, default):
    # Abbruchbedingungen
    if examples.isEmpty():  return default
    if examples.each(class == A):  return A  # all examples have same class
    if attr.isEmpty():  return examples.MajorityValue()

    # Baum mit neuem Test erweitern
    test = MaxInformationGain(examples, attr)
    tree = new DecisionTree(test)
    m = examples.MajorityValue()
    for v_i in test:
        ex_i = examples.select(test == v_i)
        st = ID3(ex_i, attr - test, m)
```

```
            tree.addBranch(label=v_i, subtree=st)
        return tree
```

Russell und Norvig (2021): Man erhält aus dem “Learn-Decision-Tree”-Algorithmus (Russell und Norvig 2021, 678, Fig. 19.5) den hier vorgestellten ID3-Algorithmus, wenn man die Funktion Importance$(a, examples)$ als InformationGain$(examples, attr)$ implementiert/nutzt.

Hinweis: Mit der Zeile `if examples.each(class == A): return A` soll ausgedrückt werden, dass alle ankommenden Trainingsbeispiele die selbe Klasse haben und dass diese dann als Ergebnis zurückgeliefert wird. Das “`A`” steht im obigen Algorithmus nur symbolisch für die selbe Klasse! Es kann also auch ein anderes Klassensymbol als “`A`” sein …

5.4.4.1. Beispiel ID3

Nr.	x_1	x_2	x_3	k
1	0	0	0	A
2	1	0	2	A
3	0	1	1	A
4	1	1	0	B
5	0	1	1	B
6	0	1	0	A

- $x2$ höchsten Information Gain
- $x2 = 0$ → Beispiele 1,2 → A
- $x2 = 1$ → Beispiele 3,4,5,6 → Information Gain berechnen, weiter teilen und verzweigen

5.4.5. Beobachtung: Gain ist bei mehrwertigen Attributen höher

- Faire Münze:
    - Entropie = $H(\text{Fair}) = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = 1$ Bit
- 4-seitiger Würfel:
    - Entropie = $H(\text{Dice}) = -4 \cdot (0.25 \log_2 0.25) = 2$ Bit

→ Gain ist bei mehrwertigen Attributen höher

Damit würden Attribute bei der Wahl bevorzugt, nur weil sie mehr Ausprägungen haben als andere.

Anmerkung: Im obigen Beispiel wurde einfach die Entropie für zwei “Attribute” mit unterschiedlich vielen Ausprägungen betrachtet, das ist natürlich kein Gain(S, A). Aber es sollte deutlich machen, dass Merkmale mit mehr Ausprägungen bei der Berechnung des Gain für eine Trainingsmenge einfach wegen der größeren Anzahl an Ausprägungen rechnerisch bevorzugt würden.

5.4.6. C4.5 als Verbesserung zu ID3

Normierter Informationsgewinn: $\text{Gain}(S, A) \cdot \text{Normalisation}(A)$

$$\text{Normalisation}(A) = \frac{1}{\sum_{v \in \text{Values}(A)} p_v \log_2 \frac{1}{p_v}}$$

C4.5 kann zusätzlich u.a. auch noch mit kontinuierlichen Attributen umgehen, vgl. en.wikipedia.org/wiki/C4.5_algorithm.

In einem Paper (DOI 10.1007/s10115-007-0114-2) wurde der Algorithmus zu den "Top 10 algorithms in data mining" ausgewählt.

Im Wikipedia-Artikel Information Gain finden Sie weitere Informationen zum "Informationsgewinn" (*Information Gain*).

Ein anderer, relativ ähnlich arbeitender Entscheidungsbaumlerner ist der CART (Classification And Regression Tree)-Algorithmus, wobei der Begriff "CART" allerdings oft auch einfach allgemein für "Entscheidungsbaumlerner" genutzt wird.

Hierzu drei lesenswerte Blog-Einträge:

- Deep dive into the basics of Gini Impurity in Decision Trees with math Intuition
- Decision Trees, Explained
- Decision Tree Algorithm With Hands-On Example

5.4.7. Beispiele zur Normierung bei C4.5

- Faire Münze:
  - Entropie = $H(\text{Fair}) = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = 1$ Bit
  - Normierung: $1/(0.5 \log_2(1/0.5) + 0.5 \log_2(1/0.5)) = 1/(0.5 \cdot 1 + 0.5 \cdot 1) = 1$
  - Normierter Informationsgewinn: $\text{Gain}(S, A) \cdot \text{Normalisation}(A) = 1$ Bit $\cdot 1 = 1$ Bit
- 4-seitiger Würfel:
  - Entropie = $H(\text{Dice}) = -4 \cdot (0.25 \log_2 0.25) = 2$ Bit
  - Normierung: $1/(4 \cdot 0.25 \log_2(1/0.25)) = 1/(4 \cdot 0.25 \cdot 2) = 0.5$
  - Normierter Informationsgewinn: $\text{Gain}(S, A) \cdot \text{Normalisation}(A) = 2$ Bit $\cdot 0.5 = 1$ Bit

$\rightarrow$ Normierung sorgt für fairen Vergleich der Attribute

Anmerkung: Auch hier ist die Entropie natürlich kein $\text{Gain}(S, A)$. Das Beispiel soll nur übersichtlich deutlich machen, dass der "Vorteil" von Attributen mit mehr Ausprägungen durch die Normierung in C4.5 aufgehoben wird.

5.4.8. Wrap-Up

- Entscheidungsbaumlerner **ID3**
    - Nutze *Information Gain* zur Auswahl des nächsten Attributs
    - Teile die Trainingsmenge entsprechend auf ("nach unten hin")
- Verbesserung durch Normierung des *Information Gain*: **C4.5**

Zum Nachlesen

Sie können zum Thema Entscheidungsbäume und ID3 in Russell und Norvig (2021) (Abschnitt 19.3) nachlesen.
Insbesondere zu ID3 finden Sie gute Darstellungen in Ertel (2025) (Abschnitt 8.4) und Mitchell (2010) (Kapitel 3). Für weiterführende Informationen (Gini Impurity, CART, …) lesen Sie bitte Kapitel 6 "Decision Trees" im Geron (2023) nach.

Lernziele

- k3: Ich kann die Entscheidungsbaumalgorithmen ID3 und C4.5 auf konkrete Daten anwenden.

Challenges

Games: Behaviour Trees

In einem Dungeon-Crawler wurden über mehrere Spiele Daten für die Aktionen eines Monsters aufgezeichnet:

Nr.	Distanz	HP	Mana	Aktion
01	nah	niedrig	genug	heilen
02	fern	niedrig	gering	fliehen
03	nah	hoch	gering	angreifen
04	nah	hoch	genug	angreifen
05	mittel	mittel	genug	angreifen
06	fern	mittel	gering	fliehen

> Trainieren Sie mit diesen Daten einen Entscheidungsbaum als *Behaviour Tree* für das Monster, so dass es in einer konkreten Spielsituation von nun an die optimale Entscheidung treffen kann. Nutzen Sie dafür ID3.
>
> **Textklassifikation**
>
> Betrachten Sie die folgenden Aussagen:
>
> > - Patient A hat weder Husten noch Fieber und ist gesund.
> > - Patient B hat Husten, aber kein Fieber und ist gesund.
> > - Patient C hat keinen Husten, aber Fieber. Er ist krank.
> > - Patient D hat Husten und kein Fieber und ist krank.
> > - Patient E hat Husten und Fieber. Er ist krank.
>
> Aufgaben:
>
> 1. Trainieren Sie auf diesem Datensatz einen Klassifikator mit ID3.
> 2. Ist Patient F krank? Er hat Husten, aber kein Fieber.

5.5. DTL7: Random Forest

> **TL;DR**
>
> Ein Random Forest ist ein Ensemble vieler Entscheidungsbäume, die auf zufälligen Teilmengen der Trainingsdaten und Merkmale trainiert werden. Jeder Baum liefert eine eigene Vorhersage, und für neue Objekte entsteht die Gesamtvorhersage durch Mehrheitsentscheidung (Klassifikation) oder Mittelwertbildung (Regression). Durch das Bootstrapping der Daten und die zufällige Merkmalsauswahl entstehen bewusst unterschiedliche Bäume, deren Fehler sich teilweise gegenseitig ausgleichen. Insgesamt reduziert der Random Forest im Vergleich zu einem einzelnen Baum die Varianz und führt oft zu stabileren und genaueren Vorhersagen, ist aber weniger gut interpretierbar.

> **Videos**
>
> Vorlesung [YT], [HSBI]

5.5.1. Vom einzelnen Baum zum Wald

- Entscheidungsbaum:
  - verständlich, interpretierbar
  - kann aber “sehr speziell” auf den Trainingsdatensatz passen
  - kleine Änderungen in den Daten → oft ganz anderer Baum
- Idee:
  - Statt **einem** Baum viele **verschiedene** Bäume trainieren
  - Jeder Baum trifft eine eigene Vorhersage
  - Am Ende: **Mehrheitsentscheidung** (Klassifikation) bzw. **Mittelwert** (Regression)

- Intuition:
  - "Viele verschiedene Meinungen" kombinieren $\rightarrow$ robustere Entscheidung
  - Fehler einzelner Bäume können sich gegenseitig ausgleichen

Ein einzelner Entscheidungsbaum ist sehr anfällig dafür, die Trainingsdaten zu "überlernen" und reagiert empfindlich auf kleine Datenänderungen.

Der Random-Forest-Ansatz sagt: Wir verlassen uns nicht auf einen *einzelnen* Baum, sondern lassen ein *Gremium* aus mehreren Bäumen abstimmen (*Ensemble*). Jeder Baum sieht die Daten ein wenig anders und trifft eigene Entscheidungen; am Ende zählt die Mehrheit (oder der Mittelwert).

> **Important**
>
> Statt *einem einzigen* **starken** Klassifikator (*strong learner*) mit hoher Klassifikationsgüte (kleinem Fehler) setzen wir *viele* **schwache** Klassifikatoren (*weak learner*) (die oft nur wenig genauer als ein Münzwurf sind) als *Ensemble* ein.

5.5.2. Wie funktioniert ein Random Forest?

1. **Datenbasis**: Trainingsdatensatz D

2. **Viele (Teil-) Trainingsdatensätze durch Zufall (Bootstrapping/Bagging)**
   - Für jeden Baum $B_1, B_2, \ldots, B_k$:
     - Zufälliges Ziehen (mit Zurücklegen) der Trainingsbeispiele aus D
       $\rightarrow$ jeder Baum erhält eine leicht andere Stichprobe D_k

3. **Zufällige Merkmalsauswahl pro Split (Feature-Subsampling)**
   - Beim Training eines Baums:
     - an jedem Knoten nur zufällige Teilmenge der Attribute für den Split betrachten
     - daraus das "beste" Attribut wählen (Entropie)
   - Alternativ: Anzahl der Splits begrenzen

4. **Vorhersage**
   - Klassifikation: Mehrheitsentscheid der Vorhersagen der Bäume
   - Regression: Mittelwert der Vorhersagen der Bäume

Zwei Arten von Zufall:

1. Jeder Baum sieht eine andere Zufallsstichprobe der Daten.
2. Jeder Baum betrachtet an jedem Split nur eine zufällige Auswahl von Merkmalen.

Dadurch entstehen bewusst unterschiedliche Bäume. Wenn wir dann über viele solcher Bäume abstimmen lassen, ist die Gesamtsicht deutlich stabiler als die eines einzelnen Baums.

Das Ziehen mit Zurücklegen nennt man in der Literatur auch *bagging* (von “bootstrap aggregating”). Es gibt auch eine Variante ohne Zurücklegen (d.h. einmal gezogene Datenvektoren stehen nicht mehr für andere Bäume zur Verfügung), die dann *pasting* genannt wird.

In vielen Implementierungen von Random Forests wird statt der Entropie auch der Gini-Index (als Maß für die Unreinheit) genutzt. Für die Grundidee spielt das aber keine Rolle - es geht immer darum, an jedem Knoten “gute” Splits zu finden.

> **Tip**
>
> Im klassischen Random Forest nach Breiman (2001) werden das Bootstrapping und die zufällige Merkmalsauswahl pro Split eingesetzt und dann auf den Teildaten normal trainiert, d.h. die Splits werden wie üblich berechnet. Damit erhält man einzelne, relativ starke Entscheidungsbäume, die sich durch die zufällige Datenstichprobe und die zufällige Merkmalsauswahl voneinander unterscheiden. Die gewünschte “Weakness” im Ensemble entsteht hier nicht durch besonders flache Bäume, sondern durch diese beiden Zufallskomponenten, die die Bäume diverser machen. Viele Implementierungen (beispielsweise der `RandomForestClassifier` aus Scikit-Learn) folgen diesem Schema.
> Beim verwandten *Boosting* erzeugt man oft “Decision Stumps” bzw. “Stubs”, d.h. man beschränkt die Tiefe des Baumes künstlich und erzeugt Bäume mit nur ein oder zwei Ebenen. Dadurch geht das Training sehr schnell. Solche flachen Bäume werden traditionell vor allem im Boosting eingesetzt; in einem Random Forest kann man die Baumtiefe über Hyperparameter aber ebenfalls begrenzen und so kleinere Bäume erzwingen. Der `RandomForestClassifier` hat u.a. auch einen Hyperparameter, mit dem man die Tiefe der Bäume steuern kann.
> Bei Entscheidungsbäumen wird das Verfahren “Random Forest” genannt, aber das generelle Vorgehen (Trainieren von *weak learners* auf Teilen der Daten und Gruppierung als *Ensemble*) lässt sich auch auf andere ML-Verfahren anwenden.

5.5.3. Beispiel: Random Forest in Aktion (Klassifikation)

Wir betrachten eine einfache binäre Klassifikation mit zwei Merkmalen:

- Merkmal 1: “Größe” (klein / groß)
- Merkmal 2: “Farbe” (hell / dunkel)
- Klassen: “Klasse A” oder “Klasse B”

Bäume:

```
      Größe                    Farbe                      Größe
      /g  \k                   /d  \h                     /g  \k
     A     B                  A     B                  Farbe   B
                                                       /d  \h
                                                      B     A
```

Testobjekt: (Größe = groß, Farbe = hell)

Vorhersage:

```
        A                        B                          A
```

Mehrheitsentscheidung: 2x A, 1x B → **A**

Jeder einzelne Baum kann "falsch liegen" oder stark vereinfacht sein.

Im Beispiel sehen wir, dass Baum 2 sich anders entscheidet als die beiden anderen.

Der Random Forest kombiniert diese unterschiedlichen Sichtweisen sehr einfach: per Mehrheitsvotum. In der Praxis nutzt man oft Dutzende oder Hunderte Bäume. Die Stärke kommt also aus der Kombination vieler relativ einfacher Modelle, nicht aus einem extrem komplexen Einzelbaum.

5.5.4. Pseudocode: Random Forest

Training:

```
Input: Trainingsdaten D, Anzahl Bäume K,
       Anzahl Zufallsattribute pro Split m

For k = 1 .. K:
    Ziehe Zufallsstichprobe D_k aus D (mit Zurücklegen)    // bagging
    Trainiere Entscheidungsbaum T_k auf D_k:
        An jedem Knoten:
            Wähle zufällig m Attribute aus allen Attributen
            Finde unter diesen m Attributen den besten Split (Entropie)
Output: Ensemble {T_1, T_2, ..., T_K}
```

Vorhersage für ein neues Objekt x:

- Klassifikation: $\hat{y}_{\text{class}}(x) = \text{argmax}_{c \in C} \sum_{k=1}^{K} \mathbf{1}\left[T_k(x) = c\right]$
- Regression: $\hat{y}_{\text{reg}}(x) = \frac{1}{K} \sum_{k=1}^{K} T_k(x)$

Wichtig ist: Keine Änderungen am Baum-Algorithmus selbst (z.B. C4.5/ID3). Die "Magie" kommt aus den zwei Zufallsquellen: zufällige Datenstichproben und zufällige Attributauswahl.

Bei der Klassifikation erhält man die **Klassenwahrscheinlichkeiten**, indem man die Häufigkeiten der Klassen über alle Bäume ermittelt:

$$p(C = c \mid x) \approx \frac{1}{K} \sum_{k=1}^{K} \mathbf{1}\left[T_k(x) = c\right]$$

Dabei ist $\mathbf{1}(A)$ wieder die Indikatorfunktion:

$$\mathbf{1}[A] = \begin{cases} 1, & \text{wenn } A \text{ wahr ist} \\ 0, & \text{sonst} \end{cases}$$

Tip

Typische Hyperparameter (z.B. bei Scikit-Learn mit `RandomForestClassifier`):

- Anzahl der Bäume K
- Anzahl der zufälligen Attribute pro Split m (etwa $\sqrt{d}$ bei d Gesamtattributen)
- Maximale Tiefe oder Blattanzahl pro Baum

5.5.5. Bias und Varianz und Grenzen von Entscheidungsbäumen / Random Forest

- **Bias** (Verzerrung): Fehler, der dadurch entsteht, dass das Modell zu *einfach* ist und die wahren Zusammenhänge systematisch nicht trifft.

  Beispiel: Ein sehr flacher Baum (oder nur ein Stub) kann nur sehr grobe Regeln ausdrücken. Das führt zu vielen systematischen Fehlentscheidungen bzw. einem hohen Bias. Dafür ist der Baum stabiler gegenüber Datenänderungen.

- **Varianz**: Empfindlichkeit gegenüber Zufallsschwankungen in den Trainingsdaten.

  Beispiel: Ein sehr tiefer Baum kann die Trainingsdaten nahezu perfekt erklären, d.h. hier sprechen wir von "auswendig lernen" (Overfitting). Bereits kleine Änderungen im Datensatz (ein paar Punkte dazu/weg) können dann zu einem ganz anderen Baum führen (hohe Varianz).

Entscheidungsbäume zeigen typischerweise:

- Tiefer Baum: niedriger Bias, hohe Varianz (Overfitting-Risiko, instabil bei Datenänderungen)
- Flacher Baum: höherer Bias, geringere Varianz (stark vereinfachtes Modell der Daten, kann wichtige Muster verpassen)

Ein Random Forest hält den (relativ) niedrigen Bias tiefer Bäume, reduziert aber die Varianz, indem viele leicht unterschiedliche Bäume konstruiert und über einen Mehrheitsentscheid kombiniert werden. Beim Einsatz von Stubs nutzt man Bäume mit eher hohem Bias und gleichzeitig reduzierter Varianz.

Grenzen

- Interpretierbarkeit
  - Ein einzelner Baum ist leicht zu visualisieren und zu erklären
  - Ein Random Forest mit 100 oder mehr Bäumen ist für Menschen praktisch nicht mehr "durchschaubar"
- Datenarten
  - Für tabellarische, strukturierte Daten mit kategorialen Merkmalen (diskrete Ausprägungen), insbesondere nominalen Merkmalen (keine natürliche Reihenfolge, etwa Farben oder Städtenamen) und ordinalen Merkmalen (mit natürlicher Reihenfolge, etwa Schulnoten), sind Entscheidungsbäume und Random Forests sehr gut geeignet
  - Für extrem hochdimensionale Eingaben (z.B. Text, Bilder) sind spezialisierte Modelle (lineare Modelle mit guter Regularisierung, CNNs, Transformer usw.) oft im Vorteil

- Ressourcen
  - Viele und tiefe Bäume lassen Speicherbedarf und Rechenzeit (Training, "Inferenz") steigen
  - Praxis: Kompromiss durch Einsatz von Stubs (extrem flachen Bäumen)

5.5.6. Wrap-Up

- **Random Forest**:
  - Kombiniert viele (kleine) Entscheidungsbäume
  - Training auf zufälligen Teilmengen der Daten und Merkmalen (Bagging + Feature-Subsampling)
- Gesamtvorhersage: **Mehrheitsvotum** der Bäume bei Klassifikation bzw. Mittelwertbildung bei Regression
- Ensemble **reduziert** im Vergleich zu einem einzelnen Entscheidungsbaum die **Varianz** und wird damit robuster gegenüber Rauschen und kleinen Datenänderungen
- Wichtige Hyperparameter:
  - Anzahl der Bäume
  - Anzahl der Merkmale pro Split
  - Maximale Baumtiefe

Zum Nachlesen

Lesen Sie zu Random Forest im Kapitel 7 "Ensemble Learning and Random Forests" im Geron (2023) nach. Die Original-Arbeit zu "Random Forest" ist (Breiman 2001).

Lernziele

- k1: Ich kann kann den Begriff "Random Forest" definieren und die beiden zentralen Zufallskomponenten (Bootstrapping der Daten und zufällige Merkmalsauswahl pro Split) an einem selbst gewählten Beispiel benennen.
- k2: Ich kann erklären, warum ein Ensemble aus vielen Entscheidungsbäumen im Random Forest typischerweise stabilere Vorhersagen liefert als ein einzelner Entscheidungsbaum, und dies an einem einfachen Beispieldatensatz erläutern.
- k3: Ich kann für einen gegebenen kleinen Datensatz skizzieren, wie unterschiedliche Bootstrap-Stichproben zu verschiedenen Bäumen führen und wie daraus per Mehrheitsentscheidung eine Random-Forest-Vorhersage entsteht.
- k4: Ich kann die wichtigsten Hyperparameter eines Random Forest (z.B. Anzahl der Bäume, maximale Tiefe, Anzahl Merkmale pro Split) benennen und qualitativ erklären, wie sich deren Variation auf Overfitting, Rechenaufwand und Vorhersagegüte auswirkt.

Challenges

Mini-Beispiel

Beispiel	Wetter	Laune	Joggen?
1	sonnig	gut	ja
2	sonnig	schlecht	nein
3	regnerisch	gut	nein
4	regnerisch	schlecht	nein

Wir trainieren 3 Entscheidungsbäume (jeweils **nur ein Split**, also Stubs) mit **Bootstrap-Stichproben** aus diesen 4 Beispielen. Als Kriterium nehmen wir (wie bei ID3) die Maximierung des **Informationsgewinns** (Entropie).

- Baum 1: Beispiele 1, 2, 3
- Baum 2: Beispiele 1, 1, 4
- Baum 3: Beispiele 2, 3, 4

Vorhersage für ein neues Objekt:

- Wetter = sonnig
- Laune = gut

6. NN: Einführung in Neuronale Netze

Das Perzeptron kann als die Nachahmung einer biologischen Nervenzelle betrachtet werden. Durch das Zusammenschließen dieser “künstlichen”Nervenzellen” entstehen künstliche **Neuronale Netze** (NN), die ähnlich wie das Gehirn **lernen** sollen, komplexe Aufgaben zu bewerkstelligen.

6.1. NN01: Das Perzeptron

Videos

- NN1.1 - Einführung
- NN1.2 - Fallstudie und Formalisierung
- NN1.3 - Das Perzeptron Modell
- NN1.4 - Perzeptron Beispiel
- NN1.5 - Der Perzeptron Lernalgorithmus

Weitere Unterlagen

- NN01-Das_Perzeptron.pdf
- Perzeptron_Lernalgorithmus.ipynb

6.1.1. Kurze Übersicht

6.1.1.1. Definition “Maschinelles Lernen”

Fähigkeit zu lernen, ohne explizit programmiert zu werden. (Arthur Samuel, 1959)

6.1.1.2. Arten des Lernens

- Überwachtes Lernen (e.g. Klassifizierung, Regression)
- Unüberwachtes Lernen (e.g. Clustering, Dimensionsreduktion)
- Bestärkendes Lernen (e.g. Schach spielen)

Vergleiche auch DTL: Machine Learning 101.

6.1.1.3. Formalisierung

- Zielfunktion f
- Merkmalraum (input space)
- Ausgaberaum (output space)
- Datensatz $\mathcal{D}$
- Hypothesenmenge $\mathcal{H}$
- Lernalgorithmus $\mathcal{A}$

6.1.1.4. Das Perzeptron

Das Perzeptron ist ein einfaches Modell für die **binäre Klassifizierung**.

Hypothesenfunktion

- Die **Hypothesenfunktion** ist eine gewichtete Summe der Eingabemerkmale x_i plus eine Konstante w_0, auf die als Aktivierung die Signumfunktion angewendet wird:

$$h(\mathbf{x}) = \text{sgn}(w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n)$$

$$= \text{sgn}(\mathbf{w}^T\mathbf{x})$$

$$= \begin{cases} 1, & \text{wenn } \mathbf{w}^T\mathbf{x} \geq 0, \\ -1, & \text{wenn } \mathbf{w}^T\mathbf{x} < 0 \end{cases}$$

- Die **Ausgabe** $\hat{y}$ des Modells ist somit eine Zahl aus der Menge $\{1, -1\}$.
- $w_0, w_1, w_2, \ldots, w_n$ sind die Parameter des Modells, die es zu optimieren gilt.

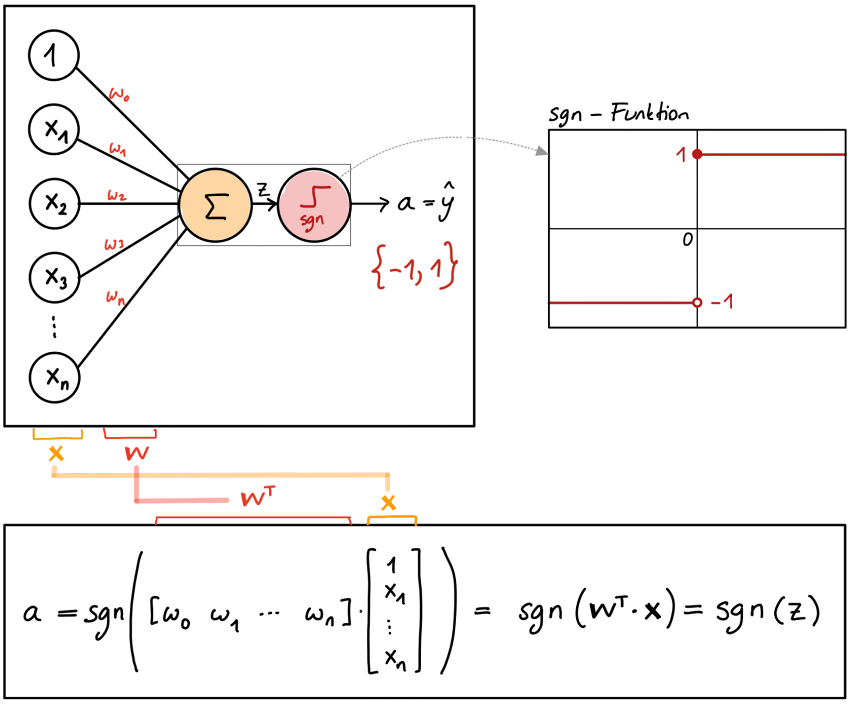

1
x1
x2
x3
xn
w0
w1
w2
w3
wn
Σ
z
sgn
a = ŷ
{-1, 1}
sgn – Funktion
1
0
-1
x
w
wT
x
a = sgn([w0 w1 ··· wn] · [1 x1 ⋮ xn]) = sgn(wT · x) = sgn(z)

Abbildung 6.1.: Perzeptron Aufbau und Berechnung

Lineare Entscheidungsgrenze

- Das Perzeptron erzeugt eine **lineare Entscheidungsgrenze** im Merkmalraum, die durch die folgende Gleichung definiert ist:

$$\mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n = 0$$

- Für einen gegebenen Datensatz müssen nun jene Parameter identifiziert werden, die eine Entscheidungsgrenze definieren, die alle Datenpunkte korrekt klassifiziert. Dies ist natürlich nur dann möglich, sofern die Daten **linear separierbar** sind.
- Im zweidimensionalen Merkmalraum $\mathbb{R}^2$ entspricht die Entscheidungsgrenze einer **Geraden**. Die folgende Abbildung zeigt beispielsweise die Entscheidungsgrenze für die Parameter $w_0 = 0, w_1 = 2$ und $w_3 = -1$, entsprechend dem Modell $h(\mathbf{x}) = \text{sgn}(2x_1 - x_2)$.

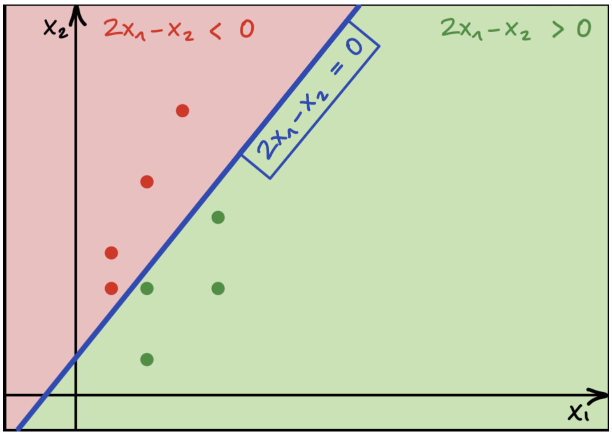


Abbildung 6.2.: Perzeptron Entscheidungsgrenze

6.1.1.5. Der Perzeptron Lernalgorithmus

Der Perzeptron Lernalgorithmus (PLA) startet mit zufälligen Gewichtsparametern und aktualisiert methodisch die Gewichte, bis die Daten alle richtig klassifiziert sind (oder eine festgelegte obere Schranke von Aktualisierungsschritten erreicht wird).

Ablauf des PLA:

1. Starte mit zufälligen Gewichtsparametern $\mathbf{w}$.
2. Wiederhole die folgenden Schritte, solange es falsch klassifizierte Punkte gibt:
    - Wähle einen beliebigen, falsch klassifizierten Punkt $\mathbf{x}$ aus. Sei y das zugehörige Label (Zielwert).
    - **Aktualisiere den Gewichtsvektor** wie folgt:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \cdot y \cdot \mathbf{x}$$

Hier bezeichnet α die **Lernrate** (auch Schrittweite genannt). Sie steuert, wie stark die Parameter in jedem Schritt in Richtung $y \cdot \mathbf{x}$ aktualisiert werden, und beeinflusst somit maßgeblich die Konvergenzgeschwindigkeit und Stabilität des Lernprozesses. Es existiert kein universell optimaler Wert für α; dieser muss problemabhängig empirisch ermittelt werden. Typische Startwerte für Experimente liegen bei 0.1, 0.01 oder 0.001.

Die **Gewichtsaktualisierung** ist der Ort, wo das Lernen stattfindet.

6.1.1.6. Nicht-Lineare Entscheidungsgrenze

Um **nicht-lineare Entscheidungsgrenzen** zu ermöglichen, können die Originalmerkmale auf nicht-lineare Weise kombiniert (z. B. x_1^2,x_2^2, $x_1 x_2, \ldots$) und diese neuen Werte als zusätzliche Merkmale in den Vektor $\mathbf{x}$ aufgenommen werden. Die Hypothesenfunktion bleibt dabei weiterhin **linear in den Gewichten** w_i.

Im Fall von zwei Merkmalen kann beispielsweise die folgende Hypothesenfunktion verwendet werden:

$$h(\mathbf{x}) = \text{sgn}(w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2).$$

Durch die Einbeziehung quadratischer Terme lassen sich nichtlineare Entscheidungsgrenzen modellieren, darunter insbesondere kreisförmige Entscheidungsgrenzen der Form

$$-4 + x_1^2 + x_2^2 = 0.$$

Lernziele

- k2: Ich kann verschiedene Arten des maschinellen Lernens erklären
- k2: Ich kann die Formalisierung eines ML-Problems, insbesondere der Klassifizierung erklären und dabei insbesondere die Begriffe Datensatz, Merkmalsraum, Hypothesenfunktion und Zielfunktion an einem Beispiel erläutern
- k2: Ich kann das Perzeptron als linearen Klassifikator verstehen und erklären
- k2: Ich kann den Begriff der Entscheidungsgrenze erklären
- k3: Ich kann die Entscheidungsgrenze für ein konkretes Beispiel berechnen
- k3: Ich kann den Perzeptron Lernalgorithmus implementieren und anwenden

Quizzes

- Selbsttest Intro ML (ILIAS)

6.2. NN02: Lineare Regression und Gradientenabstieg

Videos

- NN2.1 - Lineare Regression - Intro
- NN2.2 - Vergleich Perzeptron und Bsp
- NN2.3 - Kostenfunktiıon und Gradientenvektor
- NN2.4 - Berechnung Gradientenvektor - Beispiel
- NN2.5 - Berechnung Gradientenvektor - Allgemein
- NN2.6 - Skalierung der Merkmale

Weitere Unterlagen

- NN02-Lineare_Regression.pdf
- Lineare_Regression.ipynb

6.2.1. Kurze Übersicht

6.2.1.1. Formalisierung

- Ausgabe y ist reelle Zahl aus einem stetigen Bereich (zum Beispiel Hauspreis)

- Die **Hypothesenfunktion** ist eine gewichtete Summe der Merkmale x_i plus eine Konstante w_0:

$$h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$

- Der **Verlust** (engl. loss) für einen Datenpunkt $\mathbf{x}$ ist das **Fehlerquadrat**:

$$\mathcal{L} = (\hat{y} - y)^2 = (h(\mathbf{x}) - y)^2$$

- Die Kosten (engl. cost) sind der durchschnittliche Verlust über alle Datenpunkte:

$$J = \frac{1}{2m}\sum_{i=1}^{m}(\hat{y} - y)^2 = \frac{1}{2m}\sum_{i=1}^{m}(h(\mathbf{x}) - y)^2$$

6.2.1.2. Der Gradient

- Der **Gradientenvektor** $\nabla J(\mathbf{w})$ setzt sich zusammen aus den partiellen Ableitungen der Kostenfunktion J nach den Gewichten w_i und zeigt in jedem Punkt $\mathbf{w}$ in die **Richtung des steilsten Aufstiegs**:

$$\nabla J = [\partial J/\partial w_0 \quad \partial J/\partial w_1 \quad \ldots \quad \partial J/\partial w_n]^T$$

- **Schlussfolgerung**: In die entgegengesetzte Richtung, i.e. in Richtung $-\nabla J(\mathbf{w})$ geht es am *steilsten bergab!*
- **IDEE**: Bewege $\mathbf{w}$ in Richtung $-\nabla J(\mathbf{w})$, um die Kosten J möglichst schnell zu senken.

6.2.1.3. Der Gradientenabstieg (engl. Gradient Descent)

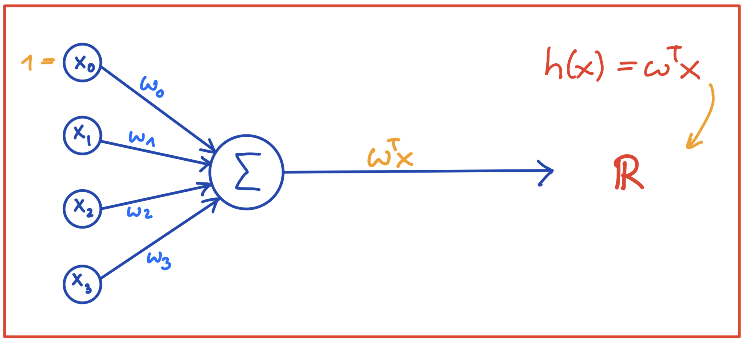
1. Starte mit zufälligen Gewichten $\mathbf{w}$
2. Berechne den Gradientenvektor im aktuellen Punkt $\mathbf{w}$
3. **Gewichtsaktualisierung**: Gehe einen *kleinen* Schritt in Richtung $-\nabla J(\mathbf{w})$

$$\mathbf{w}_{neu} := \mathbf{w}_{alt} - \alpha \cdot \nabla J(\mathbf{w}_{alt})$$

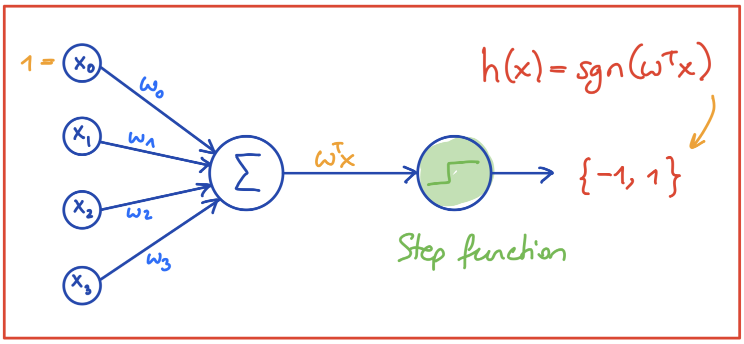
   (α: Lernrate/Schrittweite).
4. Wiederhole Schritte 2-3, bis das globale Minimum von J erreicht ist.

6.2.1.4. Graphische Übersicht

- **Lineare Regression**


1 = x0
x1
x2
x3
w0
w1
w2
w3
Σ
wTx
h(x) = wTx
ℝ

- **Perzeptron**


1 = x0
x1
x2
x3
w0
w1
w2
w3
Σ
wTx
h(x) = sgn(wTx)
{−1, 1}
Step function

Lernziele

- k2: Ich kann die lineare Regression aus Sicht neuronaler Netze erklären und dabei auf die graphische Darstellung und den Vergleich mit dem Perzeptron eingehen
- k2: Ich kann die Formalisierung von linearer Regression erklären
- k2: Ich kann den Begriff Verlust- und Kostenfunktion erklären
- k2: Ich kann den Gradientenvektor an einem Beispiel erklären
- k2: Ich kann den Begriff Lernrate erklären
- k3: Ich kann den Gradientenabstieg an einem Beispiel anwenden

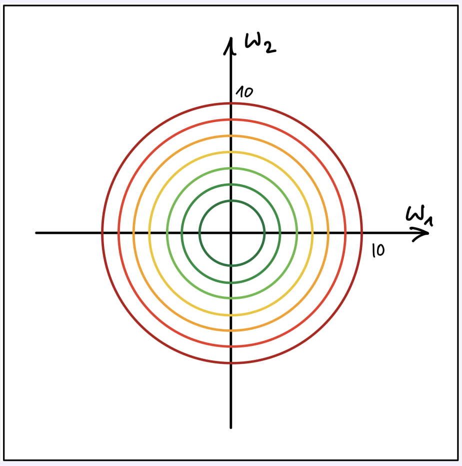
Quizzes

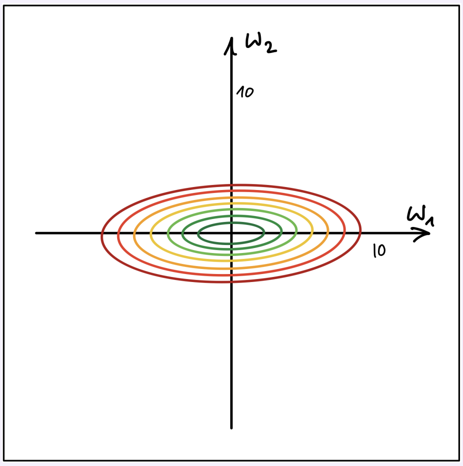
- Selbsttest Lineare Regression (ILIAS)

Challenges

Skalierung der Merkmale

Abbildung 1 und Abbildung 2 zeigen die Höhenlinien (Contour Lines) von zwei Kostenfunktionen.





- Erklären Sie, welcher der beiden Fälle nachteilhaft für den Gradientenabstieg Algorithmus ist. Wo liegt der Nachteil? Wie kann die Merkmalskalierung dem genannten Nachteil entgegenwirken?
- Zeigen Sie unter Verwendung Ihrer eigenen, zufällig generierten Datenpunkte aus dem Bereich $[100, 300] \times [0, 2]$, wie sich Standardisierung, Min-Max Skalierung und Normalisierung auf die Daten auswirken. Vergleichen Sie dazu die jeweiligen Streudiagramme (scatterplots). Sie können hierzu das folgende **Jupyter Notebook** als Startpunkt benutzen.

6.3. NN03: Logistische Regression

Videos

- NN3.1 - Logistische Regression - Intro
- NN3.2 - Logistische Regression - Hypothesenfunktion und Bsp
- NN3.3 - Logistische Regression - Verlust und Kosten
- NN3.4 - Logistische Regression - Gradientenabstieg

Weitere Unterlagen

- NN03-Logistische_Regression.pdf
- Logistische_Regression.ipynb

6.3.1. Kurze Übersicht

6.3.1.1. Formalisierung

- Ausgabe y ist reelle Zahl aus dem stetigen Bereich $(0, 1)$
- Die **Hypothesenfunktion** ist:

$$h(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x}) = \sigma(w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n) \tag{1}$$

- Der **Kreuzentropie Verlust** (engl. Cross-Entropy) für einen Datenpunkt $\mathbf{x}$:

$$\mathcal{L}(a, y) = -y\log(a) - (1-y)\log(1-a) \tag{2}$$

  wobei hier $a := \hat{y}$ die Vorhersage ist.

- Die Kosten als durchschnittlicher Verlust über alle Datenpunkte $x^{(1)}, \ldots, x^{(m)}$:

$$J = \frac{1}{m}\sum_{i=1}^{m}\mathcal{L}(a^{(i)}, y^{(i)}) \tag{3}$$

6.3.1.2. Gradientenabstieg

- Der Gradient für einen Datenpunkt $\mathbf{x}$:

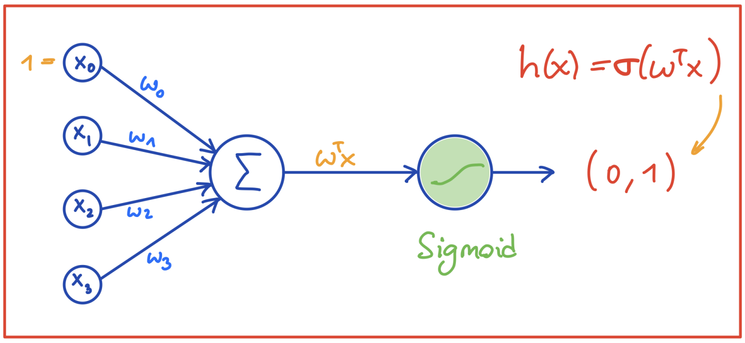
$$\frac{\partial\mathcal{L}}{\partial w} = (a-y)x \tag{4}$$

- Der Gradient für alle Datenpunkte X in Matrix-Notation:

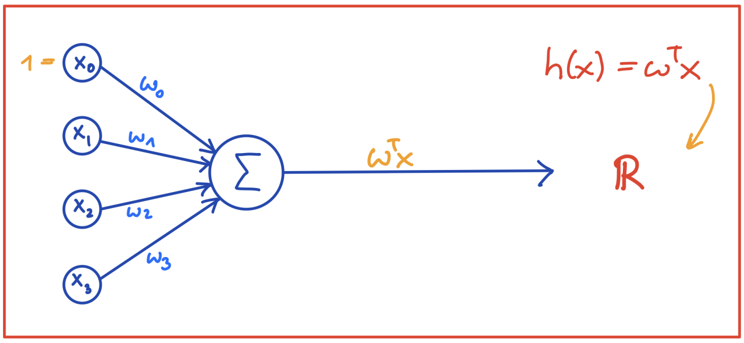
$$\nabla J = \frac{\partial J}{\partial w} = \frac{1}{m}X(A-Y)^T \tag{5}$$

6.3.1.3. Graphische Übersicht

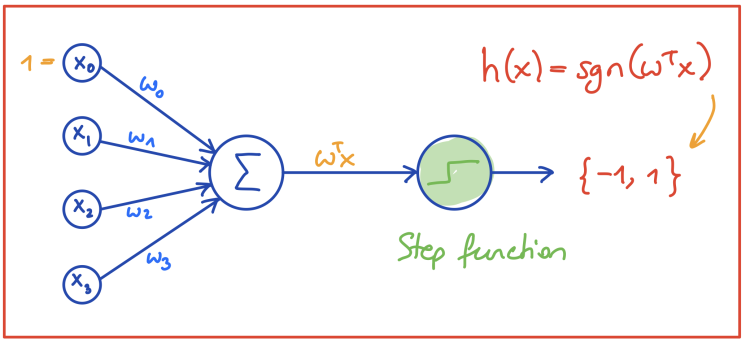
- **Logistische Regression**


$1 =$
x_0
x_1
x_2
x_3
w_0
w_1
w_2
w_3
Σ
$w^T x$
$h(x) = \sigma(w^T x)$
$(0, 1)$
Sigmoid

- **Lineare Regression**


$1 =$
x_0
x_1
x_2
x_3
w_0
w_1
w_2
w_3
Σ
$w^T x$
$h(x) = w^T x$
$\mathbb{R}$

- **Perzeptron**


$1 =$
x_0
x_1
x_2
x_3
w_0
w_1
w_2
w_3
Σ
$w^T x$
$h(x) = \text{sgn}(w^T x)$
$\{-1, 1\}$
Step function

Lernziele

- k2: Ich kann die logistische Regression aus Sicht neuronaler Netze erklären und dabei auf die graphische Darstellung und den Vergleich mit Perzeptron und linearer Regression eingehen
- k2: Ich kann die Formalisierung der logistische Regression erklären
- k2: Ich kann die Sigmoid-Aktivierungsfunktion erklären und an einem Beispiel demonstrieren
- k2: Ich kann Verlust- und Kosten (Cross-Entropy Loss) erklären
- k3: Ich kann den Gradientenabstieg für logistische Regression an einem Beispiel anwenden

Quizzes

- Selbsttest Logistische Regression (ILIAS)

6.4. NN05: Multilayer Perzeptron

Videos

- NN5.1 - MLP Forward Propagation

Weitere Unterlagen

- NN05-MLP.pdf

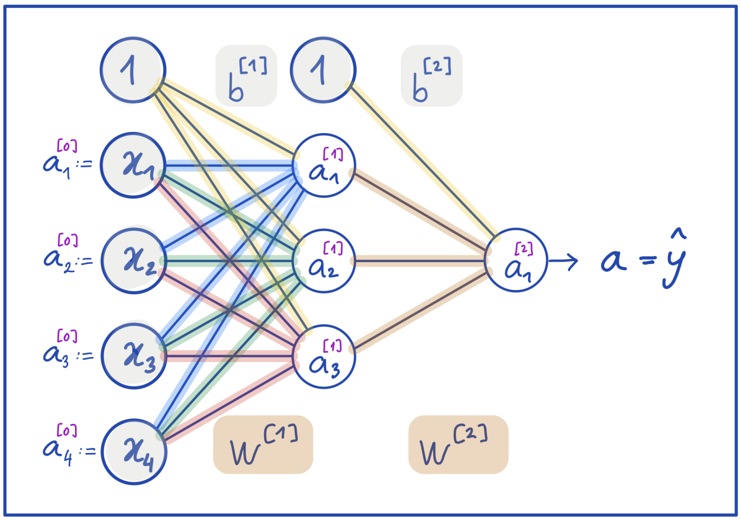
6.4.1. Kurze Übersicht

6.4.1.1. Multilayer Perzeptron (MLP)

- Das Perzeptron kann nur linear separable Daten korrekt klassifizieren.
- Durch das Zusammenschließen von mehreren Perzeptronen kann man ein mehrschichtiges Perzeptron (engl. Multilayer Perceptron) aufstellen, das komplexere Funktionen modellieren kann.
- Ein MLP wird oft auch als **Feed Forward Neural Network** oder als **Fully Connected Neural Network** bezeichnet.
- Die "inneren" Schichten eines solchen Netzwerkes sind sogenannte **versteckte Schichten** (engl. hidden layer). Das sind alle Schichten ausgenommen die Eingangs- und Ausgangsschicht.

6.4.1.2. Graphische Übersicht und Vorwärtslauf

- Ein Multi-Layer Perzeptron



Ein Vorwärtslauf (forward pass):

$$a^{[1]} = ReLU\left(W^{[1]} \cdot \curvearrowleft + b^{[1]}\right) \tag{1}$$

$$\hat{y} := a^{[2]} = \sigma\left(W^{[2]} \cdot a^{[1]} + b^{[2]}\right) \tag{2}$$

Lernziele

- k2: Ich kann das Multi-Layer Perzeptron (MLP) erläutern und dabei auf die graphische Darstellung eingehen und den Vorwärtslauf erklären
- k2: Ich kann verschiedene Aktivierungsfunktionen (insbesondere ReLU) erklären
- k3: Ich kann den Vorwärtslauf (forward pass) für ein gegebenes MLP anwenden
- k3: Ich kann die Berechnung der einzelnen Aktivierungen durchführen

Quizzes

- Selbsttest Multilayer Perzeptron (ILIAS)

Challenges

Lineares MLP

Gegeben sei ein MLP mit linearen Aktivierungsfunktionen, d.h. für jedes Neuron berechnet sich der Output durch die gewichtete Summe der Inputs: $y = g(w^T x)$, wobei $g(z) = z$ gilt, also $y = w^T x$. Zeigen Sie, dass dieses Netz durch eine einzige Schicht mit linearen Neuronen ersetzt werden kann.

Betrachten Sie dazu ein zwei-schichtiges Netz (i.e. bestehend aus Eingabe-Schicht, Ausgabe-Schicht und einer versteckten Schicht) und schreiben Sie die Gleichung auf, die die Ausgabe als Funktion der Eingabe darstellt.

Als Beispiel sei das zwei-schichtige MLP mit den folgenden Gewichten und Bias-Werten gegeben:

Schicht 1: $W_1 = [[2, 2], [3, -2]]$, $b_1 = [[1], [-1]]$

Schicht 2: $W_2 = [[-2, 2]]$, $b_2 = [[-1]]$

- Stellen Sie dieses Netzwerk graphisch dar. Was ist die Anzahl der Zellen in den einzelnen Schichten?
- Berechnen Sie die Ausgabe für eine Beispiel-Eingabe Ihrer Wahl.
- Stellen Sie ein ein-schichtiges Netz auf, das für jede Eingabe die gleiche Ausgabe wie das obige Netzwerk berechnet und es somit ersetzen könnte.

6.5. NN06: Backpropagation

Videos

- NN6.1 - MLP Backpropagation 1
- NN6.2 - MLP Backpropagation 2
- NN6.3 - MLP Zusammenfassung

Weitere Unterlagen

- NN06-MLP_Backpropagation.pdf
- NN06.2-MLP_Backpropagation_Beispiel.pdf

6.5.1. Kurze Übersicht

6.5.1.1. Forwärts- und Rückwärtslauf

- Im Forwärtslauf (engl. forward pass oder forward propagation) wird ein einzelner **Forwärtsschritt** von Schicht $[l-1]$ auf Schicht $[l]$ wie folgt berechnet:

  $$Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]} \tag{1}$$

  $$A^{[l]} = g(Z^{[l]}) \tag{2}$$

  Dabei bezeichnet g die Aktivierungsfunktion (z.B. Sigmoid oder ReLU).

- Im Rückwärtslauf (engl. *backpropagation*) werden in einem einzelnen **Rückwärtsschritt** von Schicht $[l]$ auf Schicht $[l-1]$ die folgenden Gradienten berechnet:

  $$dZ^{[l]} := \frac{\partial J}{\partial Z^{[l]}} = dA^{[l]} * g'(Z^{[l]}) \tag{3}$$

  $$dW^{[l]} := \frac{\partial J}{\partial W^{[l]}} = \frac{1}{m} dZ^{[l]} A^{[l-1]T} \tag{4}$$

  $$db^{[l]} := \frac{\partial J}{\partial b^{[l]}} = \frac{1}{m} \sum_{i=1}^{m} dZ^{[l](i)} \tag{5}$$

  $$dA^{[l-1]} := \frac{\partial J}{\partial A^{[l-1]}} = W^{[l]T} dZ^{[l]} \tag{6}$$

  Dabei steht “$*$” für die elementweise Multiplikation.

- Beachten Sie:
  - Der Forwärtsschirtt übernimmt $A^{[l-1]}$ von dem vorherigen Schritt und gibt $A^{[l]}$ an den nächsten Schritt weiter.
  - Der Rückwärtschritt übernimmt $dA^{[l]}$ von dem vorherigen Schritt und gibt $dA^{[l-1]}$ an den nächsten Rückwärtsschritt weiter.

6.5.1.2. Parameteraktualisierung

- Die Aktualisierung der Parameter in Schicht l erfolgt wie gewohnt durch:

  $$W^{[l]} = W^{[l]} - \alpha\, dW^{[l]} \tag{7}$$

  $$b^{[l]} = b^{[l]} - \alpha\, db^{[l]} \tag{8}$$

  Dabei bezeichnet α die Lernrate.

Lernziele

- k2: Ich kann den Forwärts- und Rückwärtslauf in Matrix Notation mit mehreren Datenpunkten als Eingabe erklären
- k3: Ich kann Aktivierungsfunktionen ableiten
- k3: Ich kann die Berechnung der partiellen Ableitungen durchführen
- k3: Ich kann den Rückwärtslauf (backpropagation) für ein gegebenes MLP durchführen

Quizzes

- Selbsttest Backpropagation (ILIAS)

6.6. NN04: Overfitting und Regularisierung

Videos

- NN4.1 - Nichtlineare Modelle
- NN4.2 - Overfitting und Regularisierung

Weitere Unterlagen

- NN04-Nichtlineare_Modelle_und_Overfitting.pdf

6.6.1. Kurze Übersicht

6.6.1.1. Nichtlineare Modelle

- Einführung von neuen Merkmalen in Form von nichtlienaren Kombinationen der ursprünglichen Merkmale
- Erhöhung der Komplexität des Modells ermöglicht das Erfassen von nichtlinearen Beziehungen
- **Bemerkung**: Die Hypothesenfunktion bleibt linear in den Gewichten, es wird weiterhin logistische Regression in einem **erweiterten** Merkmalraum durchgeführt.

6.6.1.2. Überanpassung und Regularisierung

- Die **Überanpassung** (engl. Overfitting) ist eines der häufigsten und wichtigsten Probleme in ML und DL

> "Was im Bereich des maschinellen Lernens Professionelle von Amateuren unterscheidet, ist ihre Fähigkeit mit Überanpassung umzugehen."
>
> – Quelle: (Abu-Mostafa u. a. 2012, 119)

- Anzeichen von Überanpassung sind geringe Trainingskosten und hohe **Testkosten** (Kosten auf nichtgesehenen Daten).

- Regularisierung ist eine Maßnahme gegen Überanpassung. Man kann es sich als eine Reduktion in der Komplexität des Modells vorstellen.
- Der Regularisierungsparameter λ ist ein Hyperparameter. Je größer der λ-Wert, desto größer der Regularisierungseffekt.
- Die **Kostenfunktion** bei regularisierter logistischer Regression:

$$J = \frac{1}{m}\left[\sum_{i=1}^{m}\left(-y^{[i]}log(a^{[i]}) - (1-y^{[i]})log(1-a^{[i]})\right) + \frac{\lambda}{2}\sum_{j=1}^{n}(w_j^2)\right] \tag{1}$$

- Die **Gewichtsaktualisierung** mit Regularisierungsterm:

$$w_j := w_j - \frac{\alpha}{m}\left[\sum_{i=1}^{m}\left((a^{[i]} - y^{[i]})x_j^{[i]}\right) + \lambda w_j\right] \tag{2}$$

Zum Nachlesen

- Abu-Mostafa u. a. (2012): Kapitel 4

Lernziele

- k2: Ich kann die Erhöhung der Modell-Komplexität durch Einführung von Merkmalen höherer Ordnung erklären
- k2: Ich kann die Begriffe Unter- und Überanpassung an einem Beispiel erklären
- k2: Ich kann den Begriff Regularisierung erklären und die Auswirkung auf die Gewichte und das Modell erläutern
- k3: Ich kann den Gradientenabstieg für regularisierte logistische Regression an einem Beispiel anwenden

Quizzes

- Selbsttest Overfitting (ILIAS)

6.7. NN07: Training & Testing

Videos

- NN7.1 - Training, Testing, Validierung
- NN7.2 - Kreuzvalidierung
- NN7.3 - Beispiel

Weitere Unterlagen

- NN07-Testing-Validierung.pdf

6.7.1. Kurze Übersicht

6.7.1.1. Training und Testing

- Der tatsächliche **Erfolg** eines Modells wird nicht durch niedrige Trainingskosten gemessen, sondern durch geringe Kosten auf ungesehenen Daten, d.h. **hohe Vorhersagekraft, gute Generalisierung**!
- Die Menge aller gelabelten Daten in **Trainingsset und Testset** aufteilen, Testset nicht während des Trainings einsetzen!.
    - E_{in} bezeichnet den Fehler auf dem Trainingsset, auch **in-sample error**.
    - E_{out} bezeichnet den Fehler auf dem gesamten Eingaberaum X, auch **out-of-sample error**. E_{out} ist der eigentliche Indikator für den zukünftigen Erfolg des Modells, ist uns aber nicht zugänglich.
    - E_{test} bezeichnet den Fehler auf dem Testset und ist eine **Näherung** für E_{out}.

> **Tip**
>
> **Analogie**:
> E_{in} : Erfolg in Übungsaufgaben und Probeprüfungen.
> E_{test} : Erfolg in Endprüfung.

- Die Näherung E_{test} sollte möglichst genau sein, damit es als ein verlässliches **Gütesiegel** dienen kann.
    - Das Testset sollte genug Daten enthalten. Üblicher Anteil an Testdaten:
        - bei $|D| \approx 100.000 \rightarrow$ ca. 20%
        - bei $|D| \approx 10.000.000 \rightarrow$ ca. 1%
        - Beispiel: Hat man 1000 Beispiele im Testset, wird E_{test} mit $\geq 98\%$ Wahrscheinlichkeit in der $\pm 5\%$ Umgebung von E_{out} liegen (für theoretische Grundlagen und Herleitung siehe (Abu-Mostafa u. a. 2012, 39–69)).
    - Trainingsdaten und Testdaten sollten möglichst aus derselben Verteilung kommen, wie die zukünftigen **Real-World-Daten**.
- **Wichtige Bemerkung**:
    - Testdaten nicht anfassen, bis das Modell Einsatzbereit ist!
    - Die Testdaten dürfen in **keinster Weise** bei der Auswahl der endgültigen Hypothese eingesetzt werden, weder bei der Berechnung der Parameter (Training), noch bei der Bestimmung der Hyperparameter (Hyperparameter-Tuning).
    - Sobald der Testfehler die Auswahl der endgültigen Hypothese beeinflusst, kann sie nicht mehr als "Gütesiegel" eingesetzt werden.
      CHECK: Hätte man zufällig andere Testdaten gewählt, könnte sich dadurch die endgültige Hypothese ändern?

6.7.1.2. Validierung und Modellauswahl

- Das Ziel ist es, das Modell mit bester Generalisierung, also kleinstem E_{out} zu bestimmen. E_{out} ist jedoch unbekannt und die Näherung E_{test} *darf nicht* bei der Modellauswahl eingesetzt werden.

- LÖSUNG: Einen weiteren Teil der Daten als **Validierungsset** (auch *development set*) beiseitelegen und nicht für das Training (i.e. Minimierung des Trainingsfehlers E_{in}) verwenden!
- **Bemerkung**:

  Das Wort **Modell** kann je nach Kontext unterschiedliche Bedeutungen annehmen.
  Ein Modell im aktuellen Kontext ist als ein Paar $(\mathcal{H}, \mathcal{A})$ von Hypothesenraum (bzw. **Modellarchitektur**) und **Lernalgorithmus** definiert.

  - Die Auswahl eines Modells kann aus einer Menge von Modellen unterschiedlicher Art erfolgen (z.B. lineare Modelle, polynomiale Modelle, neuronale Netze), oder von Modellen derselben Art aber mit unterschiedlichen Hyperparametern (z.B. Neuronale Netze mit unterschiedlicher Anzahl von versteckten Schichten).
  - Außerdem kann dieselbe Modellarchitektur $\mathcal{H}$ mit unterschiedlichen Lernalgorithmen trainiert werden, was wiederum die endgültige Hypothese beeinflussen kann. Die Bestimmung der Hyperparameter von $\mathcal{A}$ (wie z.B. Optimierungsfunktion, Lernrate, Kostenfunktion, Regularisierungsparameter usw.) sind daher auch Teil der Modellauswahl.
- Der **Validierungsfehler** E_{val} kann nun als Entscheidungsgrundlage an verschiedenen Stellen des Lernrpozesses eingesetzt werden, wie zum Beispiel:

  - Bei der **Auswahl geeigneter Hyperparameter** wie z.B. Anzahl Schichten, Anzahl Zellen/Schicht, Aktivierungsfunktion, Regularisierungsparameter (siehe Abbildung 1).

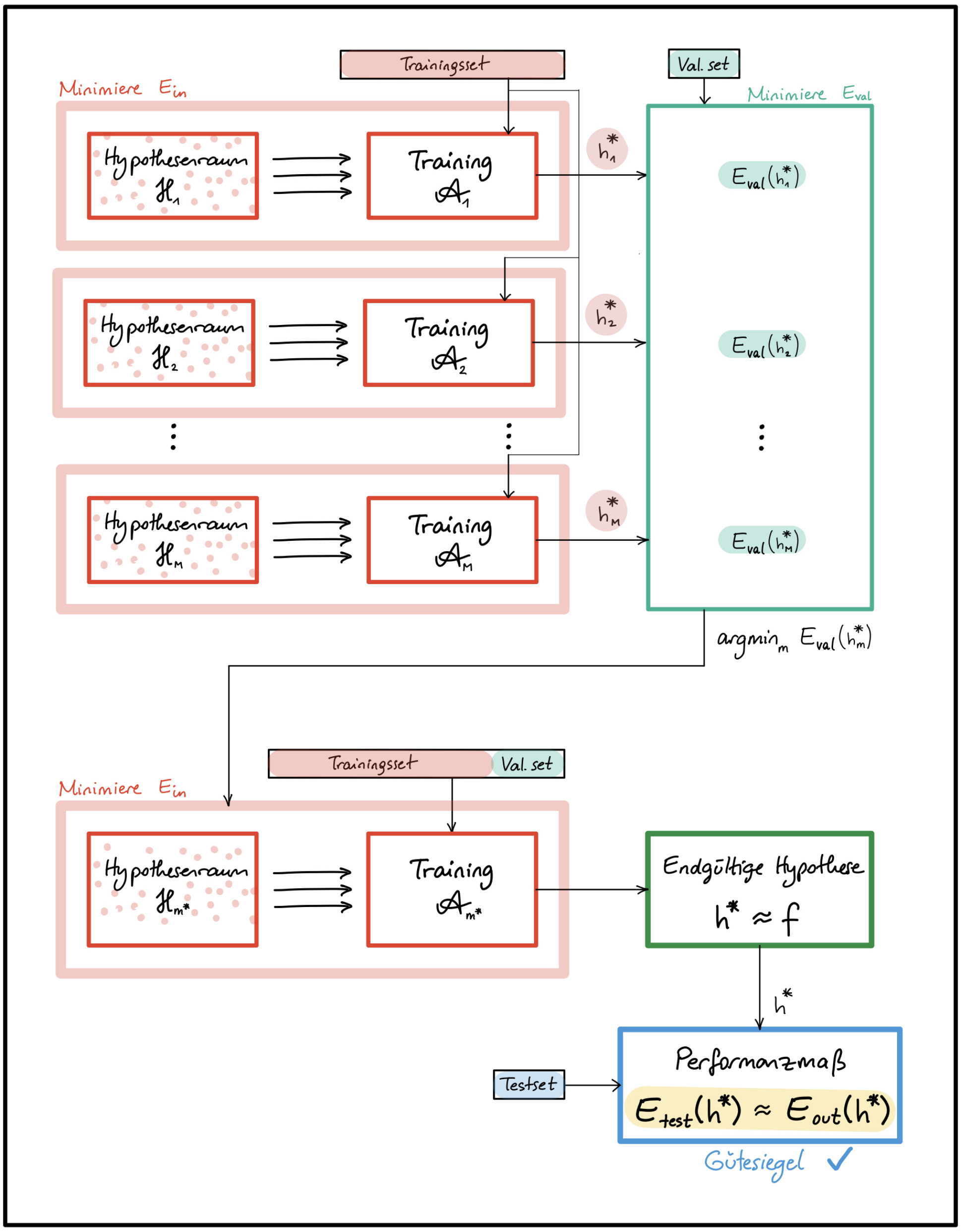

Trainingsset
Val. set
Minimiere E_{in}
Minimiere E_{val}
Hypothesenraum $\mathcal{H}_1$
Training $\mathcal{A}_1$
h_1^*
$E_{val}(h_1^*)$
Hypothesenraum $\mathcal{H}_2$
Training $\mathcal{A}_2$
h_2^*
$E_{val}(h_2^*)$
Hypothesenraum $\mathcal{H}_M$
Training $\mathcal{A}_M$
h_M^*
$E_{val}(h_M^*)$
$\mathrm{argmin}_m\ E_{val}(h_m^*)$
Trainingsset
Val. set
Minimiere E_{in}
Hypothesenraum $\mathcal{H}_{m^*}$
Training $\mathcal{A}_{m^*}$
Endgültige Hypothese
$h^* \approx f$
h^*
Testset
Performanzmaß
$E_{test}(h^*) \approx E_{out}(h^*)$
Gütesiegel ✓

Abbildung 6.3.: Abbildung 1 - Einsatz der Validierung für das Hyperparameter-Tuning

- Bei der **Auswahl der endgültigen Hypothese** (→ Parameterauswahl!): unter allen Hypothesen, die während des Trainings durchlafen werden, wähle jene mit kleinstem E_{val} (siehe Abbildung 2).

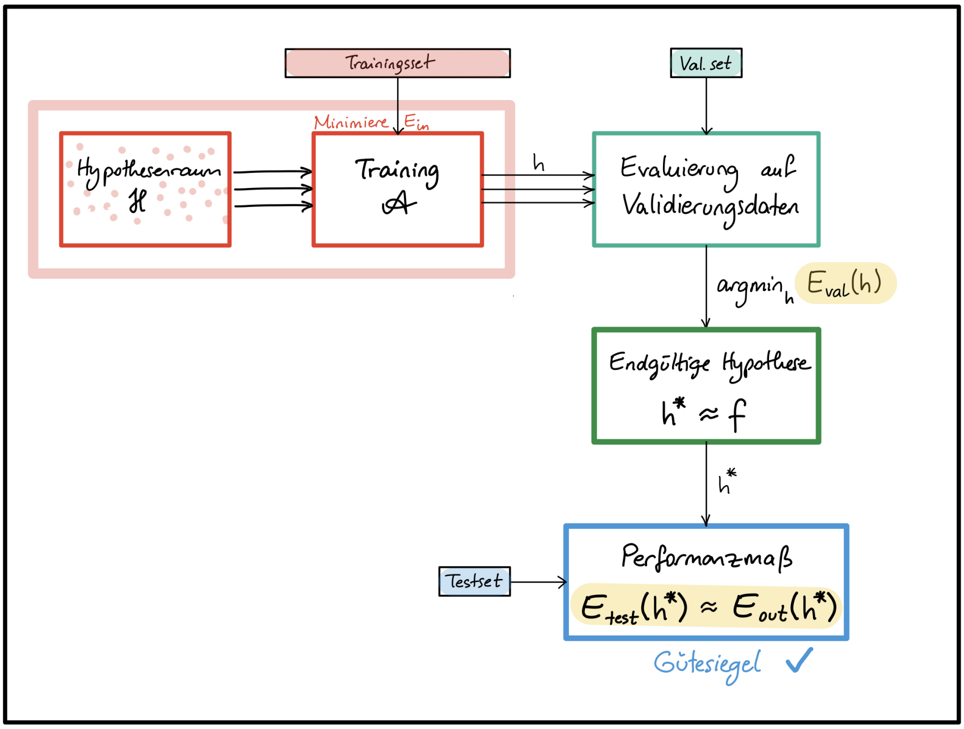

Trainingsset
Val. set
Minimiere E_{in}
Hypothesenraum $\mathcal{H}$
Training $\mathcal{A}$
h
Evaluierung auf Validierungsdaten
$\text{argmin}_h \; E_{val}(h)$
Endgültige Hypothese
$h^* \approx f$
h^*
Testset
Performanzmaß
$E_{test}(h^*) \approx E_{out}(h^*)$
Gütesiegel ✓

Abbildung 6.4.: Abbildung 2 - Einsatz der Validierung bei der Auswahl der entgültigen Hypothese

  - Bei der graphischen **Darstellung von Lernkurven** für die Diagnose von Über- und Unteranpassung (siehe Abbildung 3).

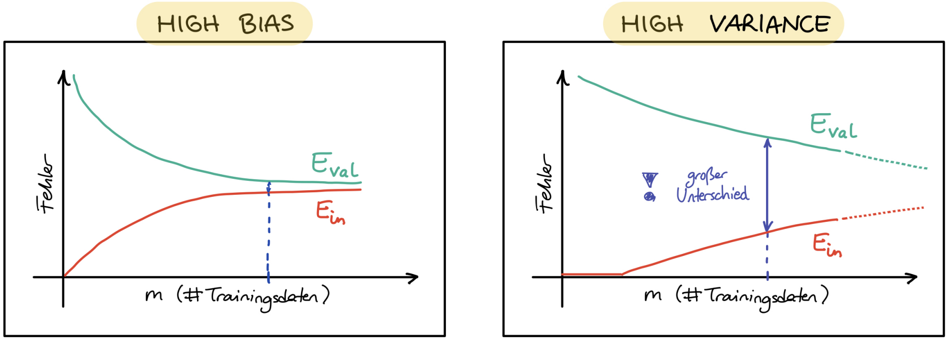


Abbildung 6.5.: Abbildung 3 - Lernkurven

- Übliche train/val/test Aufteilung der Daten (in Prozent):
  - bei $|D| \approx 100.000$ → ca. 60/20/20
  - bei $|D| \approx 10.000.000$ → ca. 98/1/1
- **Bemerkung**:

Das Modell ist trainiert für gute Ergebnisse auf Trainingsdaten und "fine-tuned" für gute Ergebnisse auf den Validierungsdaten. Ergebnisse auf Testdaten werden mit hoher wahrscheinlichkeit schlechter ausfallen, als auf Validierungsdaten (E_{val} ist eine zu optimistische Näherung).

- Sind Validierungs- und/oder Trainingsset zu klein, führt das zu schlechten Näherungen E_{val} und folglich zu schlechten Entscheidungen.
  - Bei der Aufteilung muss ein gutes Trade-off gefunden werden.
  - Wenn kein Gütesiegel notwendig ist, kann man auf das Testset verzichten und die Daten in Trainings- und Validierungsset aufteilen.
  - Für eine bessere Näherung mit weniger Validierungsdaten kann k-fache Kreuzvalidierung eingesetzt werden (wenn genug Rechenkapazität vorhanden ist).

6.7.1.3. K-fache Kreuzvalidierung (engl. k-fold cross-validation):

- Das Modell $(\mathcal{H}_{\Updownarrow}, \mathcal{A}_{\Updownarrow})$ wird k mal trainiert und validiert, jedes mal mit unterschiedlichen Trainings- und Validierungsmengen:
  - Die Trainingsdaten werden in k disjunkte Teilmengen $D_1, D_2, ..., D_k$ aufgeteilt.
  - Bei dem i-ten Training werden die Teilmenge D_i für die Berechnung des Validierungsfehlers $e_i := E_{val}(h_m^{*(i)})$ und die restlichen $k-1$ Teilmengen für das Training verwendet.
  - Der **Kreuzvalidierungsfehler** des Modells $(\mathcal{H}_{\Updownarrow}, \mathcal{A}_{\Updownarrow})$ ist der Durchschnitt der k Validierungsfehler $e_1, e_2, ..., e_k$ (siehe Abbildung 4).

$$E_{CV}(m) := \frac{1}{k}\sum_{i=1}^{k} e_i = \frac{1}{k}\sum_{i=1}^{k} E_{val}(h_m^{*(i)})$$

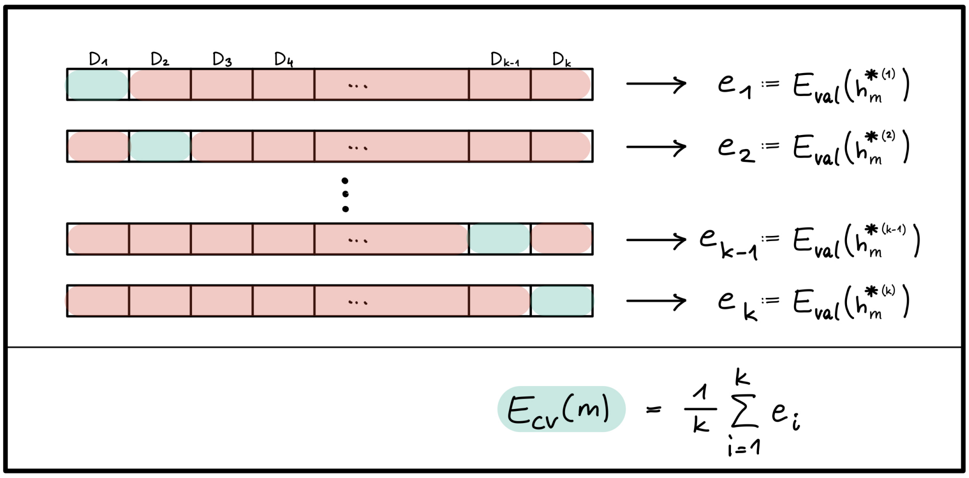

D1
D2
D3
D4
Dk-1
Dk
...
$e_1 := E_{val}(h_m^{*(1)})$
$e_2 := E_{val}(h_m^{*(2)})$
$e_{k-1} := E_{val}(h_m^{*(k-1)})$
$e_k := E_{val}(h_m^{*(k)})$
$E_{cv}(m) = \frac{1}{k}\sum_{i=1}^{k} e_i$

Abbildung 6.6.: Abbildung 4 - Kreuzvalidierung

- Bemerkung: Die Kreuzvalidierung wird nur bei der Modellauswahl eingesetzt: es liefert verlässlichere Näherungen für E_{out} und führt daher zu besseren Entscheidungen. Das zuletzt ausgewählte Modell wird danach wie gewohnt auf den gesamten Trainigsdaten (ausgenommen Testdaten) trainiert und zum Schluss mit den Testdaten evaluiert.

Lernziele

- k2: Ich kann die Begriffe Trainings-, Validierungs- und Testfehler erklären
- k2: Ich kann den Zweck einer Testmenge erläutern
- k2: Ich kann das Verfahren der Kreuzvalidierung erklären
- k2: Ich kann den Begriff Hyperparameter-Tuning erklären
- k3: Ich kann verschiedene Lernkurven interpretieren

Quizzes

- Selbsttest Training & Testing (ILIAS)

6.8. NN08: Performanzanalyse

Videos

- NN8.1 - Confusion Matrix
- NN8.2 - Precision und Recall
- NN8.3 - Precision Recall Trade-off
- NN8.4 - F1-Score
- NN8.5 - Harmonisches Mittel- Intuition

Weitere Unterlagen

- NN08-Performanzanalyse.pdf

6.8.1. Kurze Übersicht

6.8.1.1. Performanzmetriken für Klassifizierungsprobleme

Wahrheitsmatrix (engl. Confusion Matrix)

- Gibt eine Übersicht über die Anzahl von richtig und falsch klassifizierten Datenpunkten (bei binärer Klassifizierung)
  - $TP =$ # True Positives $=$ Anzahl richtiger 1-Vorhersagen
  - $FP =$ # False Positives $=$ Anzahl falscher 1-Vorhersagen
  - $FN =$ # False Negatives $=$ Anzahl falscher 0-Vorhersagen
  - $TN =$ # True Negatives $=$ Anzahl richtiger 0-Vorhersagen
- Bei Klassifizierungsproblemen mit N Klassen hat man eine $N \times N$ Matrix, die in Position (i, j) die Anzahl der Klasse-j-Beispiele enthält, die als Klasse-i vorhergesagt wurden.

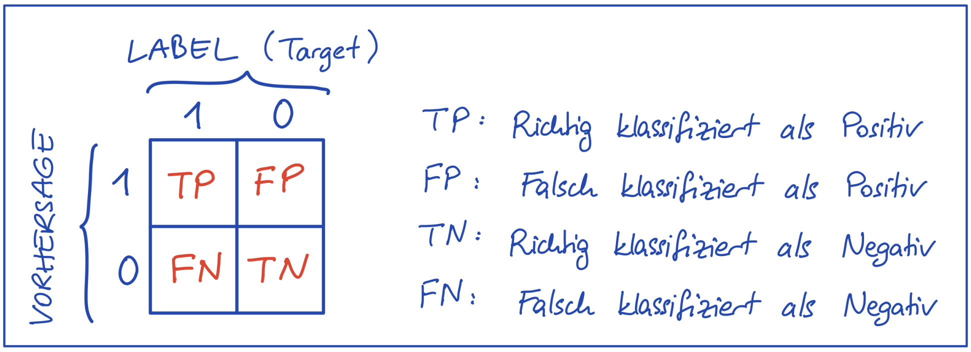

LABEL (Target)
1
0
VORHERSAGE
1
0
TP
FP
FN
TN
TP: Richtig klassifiziert als Positiv
FP: Falsch klassifiziert als Positiv
TN: Richtig klassifiziert als Negativ
FN: Falsch klassifiziert als Negativ

Abbildung 6.7.: Abbildung 1 - Wahrheitsmatrix bei binärer Klassifizierung

Treffergenauigkeit (engl. Accuracy)

- Anzahl richtig klassifizierter Datenpunkte, Erfolgsrate (engl. correct rate)

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- Accuracy vermittelt ein falsches Bild des Erfolges bei unausgewogenen Datensätzen

  Beispiel:

  - Klasse 1 hat 10, Klasse 0 hat 990 Beispiele.
  - Ein Modell, das immer 0 ausgibt, hat $990/1000 = 0.99$ Treffergenauigkeit, ist aber offensichtlich kein gutes Modell!

Precision

- Positive Predictive Value (PPV)
- Antwort auf: Von allen **positiven Vorhersagen**, wie viele sind richtig?

$$Precision = \frac{TP}{TP + FP}$$

- Wahrscheinlichkeit, dass ein positiv klassifiziertes Beispiel auch tatsächlich positiv ist.
- Je näher an 1, desto besser.
- Accuracy of **positive predictions**.

Recall

- True Positive Rate, auch Sensitivität (engl. Sensitivity)
- Antwort auf: Von allen **positiven Beispielen**, wie viele wurden richtig klassifiziert?

$$Recall = \frac{TP}{TP + FN}$$

- Wahrscheinlichkeit, dass ein positives Beispiel tatsächlich als solches erkannt wird.
- Je näher an 1, desto besser.
- Accuracy of **positive examples**.

Precision-Recall Trade-off

- Ein gutes Modell sollte hohe Precision und zugleich hohes Recall haben.
- Man kann die Precision eines Modells beliebig erhöhen (durch das Vergrößern des Schwellenwertes bei der Klassifizierung), jedoch wird dabei der Recall abnehmen.
- Genau so kann man den Recall eines Modells beliebig erhöhen (durch das Verkleinern des Schwellenwertes bei der Klassifizierung), jedoch wird dabei die Precision abnehmen.
- Es gilt ein gutes Trade-off zu finden.
- Eine Zwei-Zahlen-Metrik erschwert den Entscheidungsprozess bei Evaluierung und Modellauswahl.

F_1-Score (Harmonisches Mittel)

- Fasst Precision (P) und Recall (R) in einer Metrik zusammen (Harmonisches Mittel von P und R):

$$F_1 - Score = \frac{2}{\frac{1}{P} + \frac{1}{R}} = 2 \cdot \frac{PR}{P + R}$$

- Der F_1-Score wird nur dann hoch sein, wenn P und R beide hoch sind.
- Je näher an 1, desto besser.
- Sehr kleine P und R Werte ziehen den F_1-Score sehr stark herunter. In dieser Hinsicht gibt diese Metrik ein akkurates Bild über den Erfolg eines Modells.

Lernziele

- k2: Ich kann die betrachteten Performanzmetriken für die Evaluierung von Klassifizierungsmodellen erläutern
- k2: Ich kann die Wahrheitsmatrix (engl. Confusion Matrix) erklären
- k2: Ich kann den Begriff Treffergenauigkeit (engl. Accuracy) erklären
- k2: Ich kann den Begriff Precision (engl. Precision) erklären
- k2: Ich kann den Begriff Recall erklären
- k2: Ich kann den Begriff F_1-Score (Harmonisches Mittel) erklären
- k3: Ich kann Precision und Recall berechnen und deuten
- k3: Ich kann den F_1-Score berechnen und deuten
- k3: Ich kann die Performanzmetriken bei der Evaluierung und Auswahl von Modellen einsetzen

6.9. NN11: RNN

Videos

- NN11.1 - TODO
- NN11.2 - TODO

Weitere Unterlagen

- NN11-CNN.pdf

6.9.1. Kurze Übersicht

6.9.1.1. TODO

TODO

Lernziele

- k2: TODO
- k2: TODO
- k3: TODO
- k3: TODO

6.10. NN12: Transformer

Videos

- NN12.1 - TODO
- NN12.2 - TODO

Weitere Unterlagen

- NN12-CNN.pdf

6.10.1. Kurze Übersicht

6.10.1.1. TODO

TODO

Lernziele

- k2: TODO
- k2: TODO
- k3: TODO

- k3: TODO

7. Constraintsolving

Was haben Typ-Inferenz, Sudoku, das 8-Queens-Problem, das Einfärben von Landkarten und das Erstellen von Stundenplänen gemeinsam?

Es handelt sich um eine bestimmte Art von Suchproblemen, wobei den Parametern (Variablen) Werte so zugewiesen werden müssen, dass Einschränkungen bzw. Relationen zwischen den Variablen gelten.

7.1. CSP1: Einführung Constraints

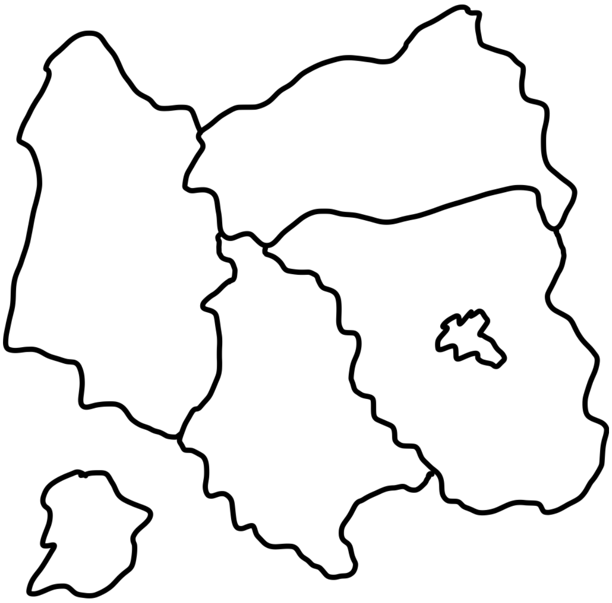
TL;DR

Ein Constraintproblem (CSP) besteht aus Variablen, die über Einschränkungen ("Constraints") verbunden sind. Jeder Variable wird eine Domäne (Wertebereich) zugeordnet.
Die Constraints können sich auf eine einzelne Variable beziehen ("unäre Constraints"), auf zwei Variablen ("binäre Constraints") oder auf mehr Variablen. Ein CSP kann als Graph dargestellt werden.
Eine "Belegung" ist eine Zuweisung von Werten an Variablen aus deren Domäne. Eine "konsistente" Belegung erfüllt die Constraints, eine "vollständige" Belegung belegt alle Variablen.
Eine Lösung für ein CSP ist eine vollständige und konsistente Belegung.

Videos

Vorlesung [YT], [HSBI]

7.1.1. Motivation: Einfärben von Landkarten



Die Skizze soll eine Landkarte mit verschiedenen Ländern darstellen. Die Aufgabe lautet: Färbe jedes Land mit einer Farbe ein, um die Übersichtlichkeit zu erhöhen. Verwende dabei so wenig wie möglich unterschiedliche Farben. Aneinander grenzende Länder müssen unterschiedliche Farben bekommen ($\rightarrow$ *Constraint*).

7.1.2. Einfärben von Landkarten: Formalisierung

- **Variablen**: A, B, C, D, E, F
- **Werte**: $\{red, green, blue\}$
- **Constraints**: Benachbarte Regionen müssen unterschiedliche Farben haben

- **Mögliche Lösung**: Zuweisung an Variablen ("Belegung") $\{$A $= red,$ B $= blue,$ C $= green,$ D $= red,$ E $= blue,$ F $= blue\}$

7.1.3. Definition: Constraint Satisfaction Problem (CSP)

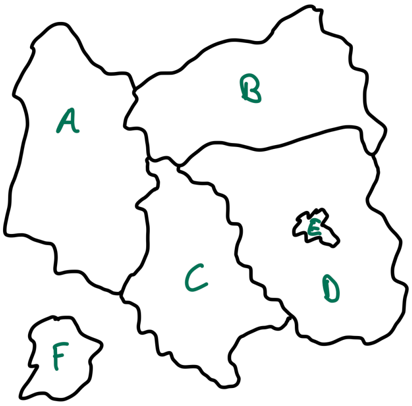
- Ein CSP $\langle V, D, C \rangle$ besteht aus:
  - Menge von **Variablen** $V = \{V_1, V_2, \ldots, V_n\}$
  - Je V_i nicht leere **Domäne** $D_i = \{d_{i,1}, d_{i,2}, \ldots, d_{i,m_i}\}$
  - Menge von **Constraints** $C = \{C_1, C_2, \ldots, C_p\}$ (Randbedingungen, Abhängigkeiten zwischen Variablen)

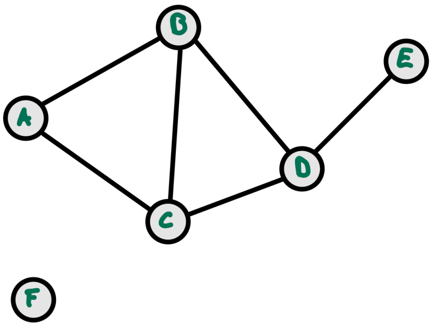
- Zuweisung/Belegung (*Assignment*) α:

  - Zuweisung von Werten an (einige/alle) Variablen: $\alpha = \{X = a, Y = b, \dots\}$
    (aus den jeweiligen Wertebereichen)
  - **Konsistente Belegung**: Randbedingungen sind nicht verletzt
  - **Vollständige Belegung**: Alle Variablen sind belegt

- **Lösung** eines CSP: Vollständige und konsistente Belegung

7.1.4. Constraint-Graph





Ein CSP kann man auch als Constraint-Graph darstellen. Die Variablen werden zu Knoten im Graph, die Constraints zu Kanten zwischen den Knoten. Dadurch kann man die aus dem Problemlösen bekannten Algorithmen anwenden …

7.1.5. Constraints – Arität

Die *Arität* betrifft hier die "Stelligkeit": Wie viele Variablen stehen in einem Constraint miteinander in Beziehung? (Also wie viele Parameter hat ein Constraint?)

- **unär**: betrifft einzelne Variablen
  Beispiel: A $\neq red$

- **binär**: betrifft Paare von Variablen
  Beispiel: A $\neq$ B

- **höhere Ordnung**: betrifft 3 oder mehr Variablen

- **Präferenzen**: "soft constraints"
  Beispiel: "rot ist besser als grün"

  Abbildung über Gewichtung $\rightarrow$ Constraint-Optimierungsproblem (COP)

7.1.6. Constraints – Wertebereiche

- **Endliche Domänen**: d Werte $\rightarrow O(d^n)$ mögliche Zuweisungen (exponentiell in der Zahl der Variablen)

- **Unendliche Domänen**: reelle Zahlen, natürliche Zahlen
  $\rightarrow$ Keine Auflistung der erlaubten Wertekombinationen mehr möglich
  $\rightarrow$ Übergang zu Gleichungen/Ungleichungen: $job_1 + 5 < job_2$
    - lineare Constraints
    - nichtlineare Constraints

Historische Unterscheidung:

- **Constraint Satisfaction**: endliche Domänen, kombinatorische Methoden
- **Constraint Solving**: unendliche Domänen

7.1.7. CSP sind überall …

- Stundenpläne (Klassen, Räume, Zeiten)
- Konfiguration (Computer, Autos, …)
- Fahrpläne (Zug, Flug, …)
- Planung von komplexen Projekten
- Sudoku :-)
- …

7.1.8. Wrap-Up

- Definitionen und Begriffe:
    - Variable, (un-) endliche Domänen, Wertemenge
    - Constraint, Arität, CSP
    - Zuweisung, Lösung, …

Zum Nachlesen

Lesen Sie in (Russell und Norvig 2021) bitte den Abschnitt 5.1 “Defining Constraint Satisfaction Problems”.

Lernziele

- k2: Ich kann die Definitionen im Zusammenhang mit Constraints und CSP erklären: Variable, Domäne, Constraint, Arität, CSP, Zuweisung
- k3: Ich kann Probleme als CSP formulieren

Quizzes

- Selbsttest Intro CSP (ILIAS)

Challenges

Problem 1
Drei Personen - Paul, George und John - sind zu einem Abendessen eingeladen und sollen an einer Seite des Tisches auf drei nebeneinander liegenden Plätzen platziert werden. George und John können sich nicht besonders gut leiden, weshalb sie nicht direkt nebeneinander gesetzt werden dürfen.
Formulieren Sie das Platzierungsproblem als Constraint-Problem. Geben Sie Variablen, Domänen und Constraints an.
Problem 2
Betrachten Sie das Problem ex_2 aus den AIMA Excercises.
Sei $n = 3$. Diskutieren Sie die Fragen (1), (2) und (3) und geben Sie Lösungsvorschläge an.
Problem 3
Formulieren Sie Sudoku als CSP und geben Sie Variablen, Domänen und Constraints an.

7.2. CSP2: Lösen von diskreten CSP (Backtracking-Search)

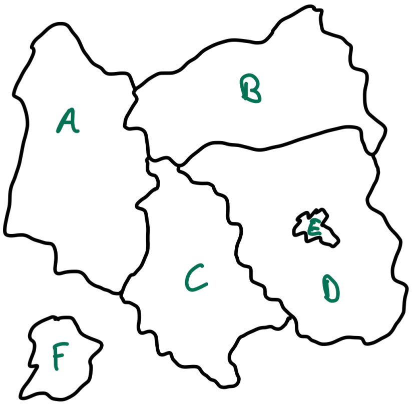
TL;DR

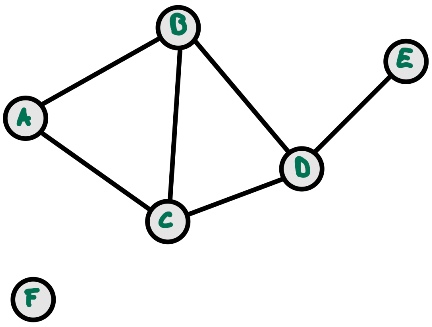
CSP's mit endlichen Domänen lassen sich mit einer Backtracking-Suche lösen. Dabei wird schrittweise eine Variablen ausgewählt und dann ein Wert aus deren Wertebereich für die Belegung ausgewählt. Danach ruft sich die Backtracking-Suche rekursiv auf. Falls dabei keine Lösung gefunden werden kann, erfolgt Backtracking und die Belegung wird schließlich rückgängig gemacht und durch die nächste Möglichkeit ersetzt.

Videos

Vorlesung [YT], [HSBI]

7.2.1. Einfärben von Landkarten als CSP





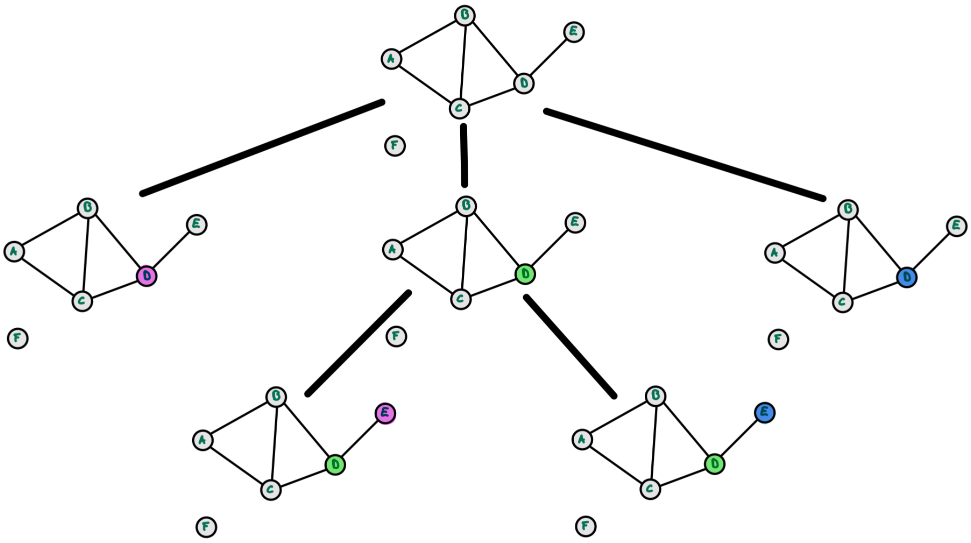
7.2.2. Endliche Domänen: Formulierung als Suchproblem

```
def BT_Search(assignment, csp):
    if complete(assignment, csp): return assignment

    var = VARIABLES(csp, assignment)

    for value in VALUES(csp, var):
        if consistent(value, var, assignment, csp):
            assignment += {var = value}

            if INFERENCE(csp, assignment, var) != failure:
                result = BT_Search(assignment, csp)
                if result != failure: return result

            assignment -= {var = value}

    return failure
```

Quelle: Backtracking-Search: Eigener Code basierend auf einer Idee nach (Russell und Norvig 2021, p. 176, fig. 5.5)

Hierbei handelt es sich um eine etwas angepasste Tiefensuche: Starte mit leerem Assignment und weise schrittweise Variablen passende Werte zu und mache notfalls Backtracking.

7.2.3. BT-Suche für CSP am Beispiel Landkartenfärbeproblem



7.2.4. Wrap-Up

- Lösung von CSP mit endlichen Domänen mit Hilfe der Backtracking-Suche

Zum Nachlesen

Lesen Sie in (Russell und Norvig 2021) bitte den Abschnitt 5.3 “Backtracking Search for CSPs”.

Lernziele

- k3: Ich kann CSP mit endlichen Domänen mit Hilfe der BT-Suche lösen

Challenges

Sei ein Constraintproblem definiert durch $V = \{a, b, c, d, e\}$, $D_a = D_b = D_c = D_e = \{1, 2, 3\}$, $D_d = \{1, 2\}$, und $C = \{c_1, c_2, c_3, c_4, c_5\}$
mit

- $c_1 = ((a, b), \neq)$,
- $c_2 = ((b, c), \neq)$,
- $c_3 = ((a, c), \neq)$,
- $c_4 = ((c, d), =)$, und
- $c_5 = ((b, e), <)$.

1. Zeichen Sie den Constraint-Graph.
2. Zeigen Sie, wie der BT-Search-Algorithmus eine Lösung für das Problem findet.

7.3. CSP3: Heuristiken

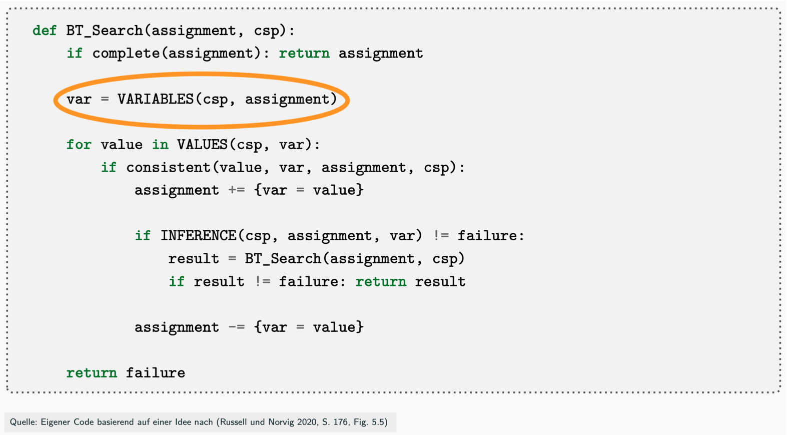
TL;DR

CSP's mit endlichen Domänen lassen sich mit einer Backtracking-Suche lösen. Dabei gibt es einige Freiheitsgrade: Auswahl der nächsten Variable und Wahl des nächsten Werts. Hier können Heuristiken die Suche beschleunigen.
Zur Wahl der als nächstes zu betrachtenden Variable kann die **Minimum Remaining Values (MRV)**-Heuristik eingesetzt werden: Wähle die Variable mit wenigsten freien Werten. Bei Gleichstand bei der MRV kann man mit der **Gradheuristik** die Variable mit den meisten Constraints zu offenen (noch nicht belegten) Variablen wählen.
Bei der Wahl des Wertes kann die **Least Constraining Value (LCV)**-Heuristik genutzt werden: Wähle den Wert, der für die verbleibenden Variablen die wenigsten Werte ungültig macht.
Während die MRV relativ leicht umzusetzen ist, muss man für die LCV alle Constraints zu den Nachbarn auswerten.

Videos

Vorlesung [YT], [HSBI]

7.3.1. VARIABLES: Variablen-Sortierung, Welche Variable soll betrachtet werden?



```
def BT_Search(assignment, csp):
    if complete(assignment): return assignment

    var = VARIABLES(csp, assignment)

    for value in VALUES(csp, var):
        if consistent(value, var, assignment, csp):
            assignment += {var = value}

            if INFERENCE(csp, assignment, var) != failure:
                result = BT_Search(assignment, csp)
                if result != failure: return result

            assignment -= {var = value}

    return failure
```

Quelle: Eigener Code basierend auf einer Idee nach (Russell und Norvig 2020, S. 176, Fig. 5.5)

VARIABLES: Welche Variable zuerst ausprobieren?

Minimum Remaining Values (MRV): (vgl. (Russell und Norvig 2021, 177))

- Wähle Variable mit wenigsten freien Werten (die am meisten eingeschränkte Variable)

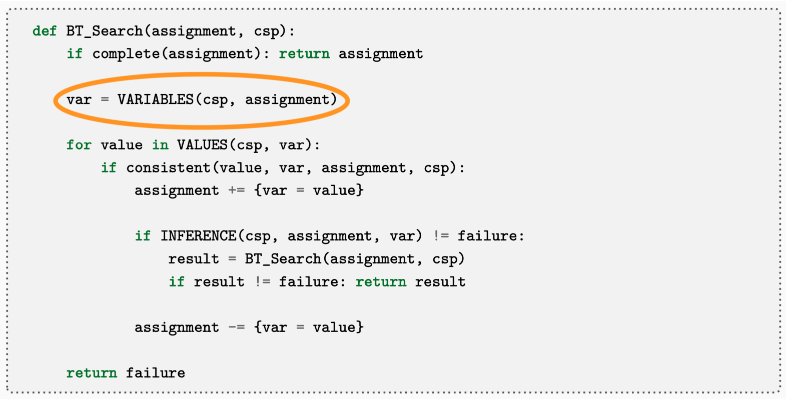
  → reduziert den Verzweigungsgrad

Beispiel:

1. Freie Auswahl, alle haben gleich viele freie Werte (jeweils 3) → wähle A

2. B und C haben nur noch zwei freie Werte → wähle B (oder C)
3. C hat nur noch einen Wert, D noch zwei, der Rest drei → wähle C

7.3.2. VARIABLES: Gleichstand bei MRV



```
def BT_Search(assignment, csp):
    if complete(assignment): return assignment

    var = VARIABLES(csp, assignment)

    for value in VALUES(csp, var):
        if consistent(value, var, assignment, csp):
            assignment += {var = value}

            if INFERENCE(csp, assignment, var) != failure:
                result = BT_Search(assignment, csp)
                if result != failure: return result

            assignment -= {var = value}

    return failure
```

Quelle: Eigener Code basierend auf einer Idee nach (Russell und Norvig 2020, S. 176, Fig. 5.5)

VARIABLES: Welche Variable zuerst ausprobieren?

Gradheuristik: Erweiterung von *MRV* bei *Gleichstand* (vgl. (Russell und Norvig 2021, 177))

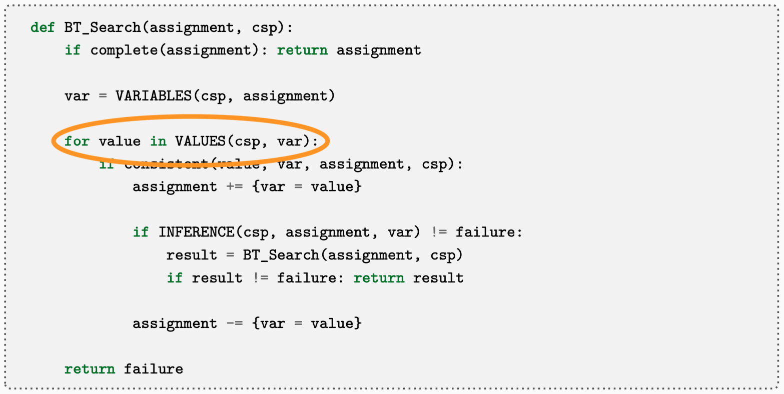
- Wähle Variable mit meisten Constraints auf offene (noch nicht zugewiesene) Variablen

  → reduziert den Verzweigungsgrad in späteren Schritten

Beispiel:

1. MRV: Alle haben gleich viele freie Werte (jeweils 3) → Gradheuristik: B, C und D haben die meisten Verbindungen (Constraints) auf offene Variablen → wähle B (oder C oder D)
2. MRV: A, C und D haben nur noch zwei freie Werte → Gradheuristik: C und D haben je zwei Constraints auf noch offene Variablen → wähle C (oder D)
3. MRV: A und D haben beide nur noch einen Wert → Gradheuristik: D hat die meisten Verbindungen (Constraints) auf offene Variablen → wähle D

7.3.3. VALUES: Werte-Sortierung, Welchen Wert soll ich ausprobieren?



```
def BT_Search(assignment, csp):
    if complete(assignment): return assignment

    var = VARIABLES(csp, assignment)

    for value in VALUES(csp, var):
        if consistent(value, var, assignment, csp):
            assignment += {var = value}

            if INFERENCE(csp, assignment, var) != failure:
                result = BT_Search(assignment, csp)
                if result != failure: return result

            assignment -= {var = value}

    return failure
```

Quelle: Eigener Code basierend auf einer Idee nach (Russell und Norvig 2020, S. 176, Fig. 5.5)

VALUES: Welchen Wert zuerst ausprobieren?

Least Constraining Value (LCV): (vgl. (Russell und Norvig 2021, 177))

- Wähle Wert, der für verbleibende Variablen die wenigsten Werte ungültig macht

  → verringert die Wahrscheinlichkeit für Backtracking

Beispiel:

1. Sei A gewählt: Alle Werte machen in den anderen Variablen einen Wert ungültig → freie Wahl des Wertes → wähle beispielsweise rot
2. Sei B gewählt: Alle Werte machen in den anderen Variablen einen Wert ungültig → freie Wahl des Wertes → wähle beispielsweise grün
3. Sei D gewählt: Verbleibende Werte rot und blau
    - Wahl von rot würde für C einen Wert übrig lassen (blau)
    - Wahl von blau würde für C **keinen** Wert übrig lassen → LCV: Wahl von rot!

Hinweis: Diese Heuristik ist in der Praxis sehr aufwändig zu berechnen! Man müsste für jeden Wert die noch offenen Constraints anschauen und berechnen, wie viele Werte damit jeweils ungültig gemacht werden. Die Idee ist aber dennoch interessant, und möglicherweise kann man sie für ein reales Problem so adaptieren, dass bei der Umsetzung nur wenig zusätzlicher Aufwand entsteht.

7.3.4. Wrap-Up

- Verbesserung der BT-Suche mit Heuristiken: MRV, Gradheuristik, LCV

Zum Nachlesen

Lesen Sie in (Russell und Norvig 2021) bitte den Abschnitt 5.3 “Backtracking Search for CSPs”.

Lernziele

- k3: Ich kann die BT-Suche mit verschiedenen Heuristiken verbessern: MRV, Gradheuristik, LCV

Quizzes

- Selbsttest CSP, Heuristiken (ILIAS)

Challenges

Problem 1

Sei $D = \{0, \ldots, 5\}$, und ein Constraintproblem definiert durch $\langle \{v_1, v_2, v_3, v_4\}, \{D_{v_1} = D_{v_2} = D_{v_3} = D_{v_4} = D\}, \{c_1, c_2, c_3, c_4\} \rangle$ mit

- $c_1 = \big((v_1, v_2), \{(x, y) \in D^2 | x + y = 3\}\big)$,
- $c_2 = \big((v_2, v_3), \{(x, y) \in D^2 | x + y \leq 3\}\big)$,
- $c_3 = \big((v_1, v_3), \{(x, y) \in D^2 | x \leq y\}\big)$ und
- $c_4 = \big((v_3, v_4), \{(x, y) \in D^2 | x \neq y\}\big)$.

1. Zeichen Sie den Constraint-Graph.
2. Welche Variable würde bei der Anwendung von *MRV* und *Gradheuristik* im ersten Schritt bei der Suche mit der BT-Search ausgewählt?
3. Geben Sie eine Lösung für das Problem an.

Problem 2

Sei ein Constraintproblem definiert durch $V = \{a, b, c, d, e\}, D_a = D_b = D_c = D_e = \{1, 2, 3\}, D_d = \{1, 2\}$, und $C = \{c_1, c_2, c_3, c_4, c_5\}$ mit

- $c_1 = ((a, b), \neq)$,
- $c_2 = ((b, c), \neq)$,
- $c_3 = ((a, c), \neq)$,
- $c_4 = ((c, d), =)$, und
- $c_5 = ((b, e), <)$.

1. Zeichen Sie den Constraint-Graph.
2. Welche Variable würde bei der Anwendung der Heuristik *MRV* im ersten Schritt bei der Suche mit der BT-Search ausgewählt?
3. Gegeben die (partielle) Belegung $d = 1$ und $c = 1$, welche Variable würde mit *MRV* ausgewählt werden?
4. Was würde die *Gradheuristik* jeweils ändern?

7.4. CSP4: Kantenkonsistenz und AC-3

TL;DR

Bei der Backtracking-Suche werden schrittweise Variablen belegt. Dabei kann eine Belegung eine Lösung im weiteren Verlauf der Suche unmöglich machen, so dass (viel) Backtracking notwendig wird.
Beim **Forward Checking** entfernt man nach der Belegung einer Variablen in allen Nachbarvariablen die durch die aktuelle Belegung inkonsistent gewordenen Werte. Wenn dabei ein Wertebereich leer wird, führt die aktuelle Belegung nicht zu einer Lösung und kann sofort zurückgenommen werden. Allerdings findet man mit Forward Checking nicht alle Inkonsistenzen.
Bei der **Kantenkonsistenz** prüft man, ob zu jedem Wert aus dem Wertebereich einer Variablen in den Nachbarvariablen mindestens ein passender (konsistenter) Wert existiert. Dabei werden die Constraints nacheinander betrachtet (nicht gleichzeitig). Wenn dies nicht der Fall ist, wird der Wert aus dem Wertebereich der betrachteten Variablen entfernt. Der AC-3-Algorithmus erzeugt schrittweise Kantenkonsistenz für ein CSP.
Man kann den AC-3 als Vorverarbeitung nutzen und die Wertemengen *vor* der BT-Suche reduzieren. Eventuell findet man dabei bereits eine Lösung oder kann eine Lösung ausschließen. Man kann den AC-3 auch als Inferenzschritt in die BT-Suche einbetten ("MAC").

Videos

Vorlesung [YT], [HSBI]

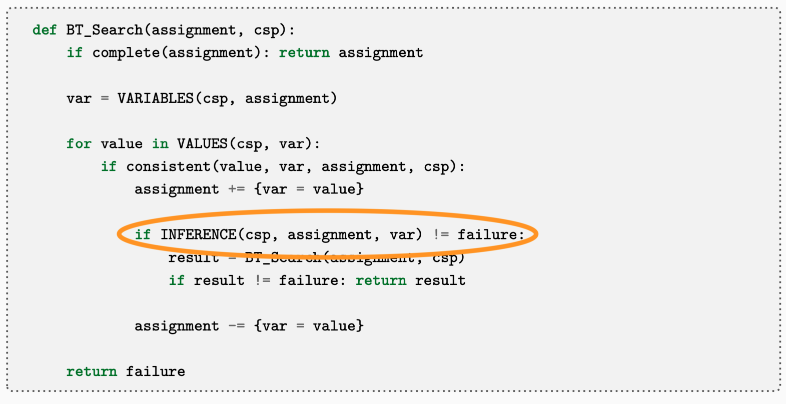
7.4.1. Problem bei BT-Suche

Zuweisung eines Wertes an Variable X:

- Passt zu aktueller Belegung
- Berücksichtigt aber nicht **restliche** Constraints
  $\rightarrow$ macht weitere Suche u.U. unmöglich/schwerer

Lösung: Nach Zuweisung alle *nicht zugewiesenen Nachbarvariablen* prüfen

7.4.2. INFERENCE: Vorab-Prüfung (Forward Checking)



```
def BT_Search(assignment, csp):
    if complete(assignment): return assignment

    var = VARIABLES(csp, assignment)

    for value in VALUES(csp, var):
        if consistent(value, var, assignment, csp):
            assignment += {var = value}

            if INFERENCE(csp, assignment, var) != failure:
                result = BT_Search(assignment, csp)
                if result != failure: return result

            assignment -= {var = value}

    return failure
```

Quelle: Eigener Code basierend auf einer Idee nach (Russell und Norvig 2020, S. 176, Fig. 5.5)

Inference: Frühzeitiges Erkennen von Fehlschlägen! (vgl. (Russell und Norvig 2021, 178))

Nach Zuweisung eines Wertes an Variable X:

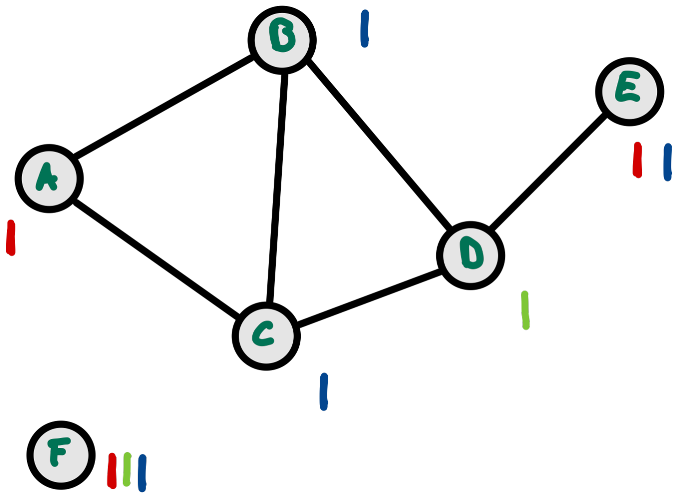
- Betrachte alle nicht zugewiesenen Variablen Y:
  - Falls Constraints zw. X und Y, dann …
  - … entferne alle inkonsistenten Werte aus dem Wertebereich von Y.

Beispiel:

1. Sei A auf rot gesetzt → entferne rot in B und C
2. Sei D auf grün gesetzt → entferne grün in B und C und E

Problem: Für B und C bleibt nur noch blau; sind aber benachbart!

7.4.3. Forward Checking findet nicht alle Inkonsistenzen!



- Nach $\{A = red, D = green\}$ bleibt für B und C nur noch blue
- B und C sind aber benachbart

7.4.4. Übergang von Forward Checking zu Kantenkonsistenz

- Forward Checking erzeugt Konsistenz für alle Constraints der **gerade betrachteten (belegten) Variablen**.

- Idee: Ausdehnen auf alle Kanten … → Einschränken der Wertemengen

7.4.5. Definition Kantenkonsistenz (Arc Consistency)

> Eine Kante von X nach Y ist "konsistent", wenn für jeden Wert $x \in D_X$ und für alle Constraints zwischen X und Y jeweils ein Wert $y \in D_Y$ existiert, so dass der betrachtete Constraint durch (x, y) erfüllt ist.

Ein CSP ist kanten-konsistent, wenn für alle Kanten des CSP Konsistenz herrscht.

7.4.6. Beispiel Kantenkonsistenz

$V = \{a, b, c, d, e\}$

$\mathrm{C} = \{((a, b), \neq), ((b, c), \neq), ((a, c), \neq), ((c, d), =), ((b, e), <)\}$

$D_a = D_b = D_c = \{1, 2, 3\}, D_d = \{1, 2\}, D_e = \{1, 2, 3\}$

Einschränkung der Ausgangswertemengen (kanten-konsistent)

$D_a = \{1, 2, 3\}, D_b = \{1, 2\}, D_c = \{1, 2\}, D_d = \{1, 2\}, D_e = \{2, 3\}$

→ Kantenkonsistenz ist nur **lokale** Konsistenz!

Anmerkung: $((a, b), \neq)$ ist Kurzform für $((a, b), \{(x, y) \in D_a \times D_b | x \neq y\})$

7.4.7. AC-3 Algorithmus: Herstellen von Kantenkonsistenz

```
def AC3(csp):
    queue = Queue(csp.arcs)
    while not queue.isEmpty():
        (x,y) = queue.dequeue()
        if ARC_Reduce(csp,x,y):
            if D_x.isEmpty(): return false
            for z in x.neighbors(): queue.enqueue(z,x)
    return true
```

```

def ARC_Reduce(csp, x, y):
    change = false
    for v in D_x:
        if not (any w in D_y and csp.C_xy(v,w)):
            D_x.remove(v);  change = true
    return change
```

Quelle: AC-3 Algorithmus: Eigener Code basierend auf einer Idee nach (Russell und Norvig 2021, p. 171, fig. 5.3)

Anmerkung: Die Queue in AC-3 ist wie eine (mathematische) Menge zu betrachten: Jedes Element kann nur genau einmal in einer Menge enthalten sein. D.h. wenn man bei `queue.enqueue(z,x)` die Rückkanten von den Nachbarn in die Queue aufnimmt, sorgt die Queue eigenständig dafür, dass es keine doppelten Vorkommen einer Kante in der Queue gibt. (Falls die verwendete Queue in einer Programmiersprache das nicht unterstützt, müsste man bei `queue.enqueue(z,x)` stets abfragen, ob die Kante `(z,x)` bereits in der Queue ist und diese dann nicht erneut hinzufügen.) AC-3 hat eine Laufzeit von $O(d^3n^2)$ (n Knoten, maximal d Elemente pro Domäne). Leider findet auch AC-3 nicht alle Inkonsistenzen … (NP-hartes Problem).

Hinweis: In gewisser Weise kann man Forward Checking als ersten Schritt bei der Herstellung von Kantenkonsistenz interpretieren.

7.4.8. Einsatz des AC-3 Algorithmus

1. Vorverarbeitung: Reduktion der Wertemengen *vor* BT-Suche
    - Nach AC-3 evtl. bereits Lösung gefunden (oder ausgeschlossen)
2. Propagation: Einbetten von AC-3 als Inferenzschritt in BT-Suche (**MAC** – Maintaining Arc Consistency)
    - Nach jeder Zuweisung an X_i Aufruf von AC-3-Variante:
        - Initial nur Kanten von X_i zu allen noch nicht zugewiesenen Nachbarvariablen
    - Anschließend rekursiver Aufruf von BT-Suche

7.4.9. Wrap-Up

- Anwendung von Forward Checking und …
- … die Erweiterung auf alle Kanten: AC-3, Kantenkonsistenz

Zum Nachlesen

Lesen Sie in (Russell und Norvig 2021) bitte den Abschnitt 5.2 “Constraint Propagation”.

Lernziele

- k2: Ich kann “Forward Checking” (FC) erklären
- k2: Ich kann die Erweiterung von FC auf alle Kanten erklären (Kantenkonsistenz)

- k2: Ich kann an einem Beispiel erklären, dass Kantenkonsistenz nicht globale Konsistenz bedeutet
- k3: Ich kann den AC-3 Algorithmus anwenden

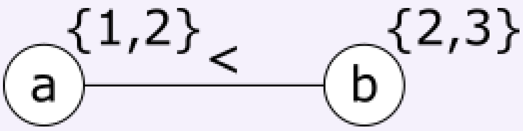
Quizzes

- Selbsttest CSP, AC-3 (ILIAS)

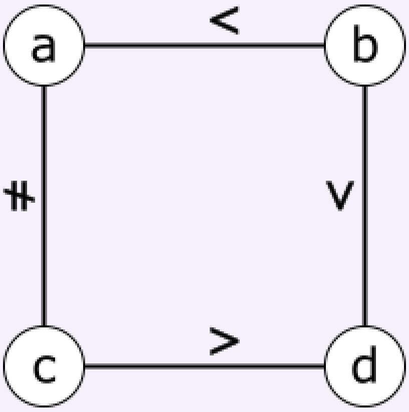
Challenges

Fingerübungen

1. Ist die Kante zwischen a und b konsistent?

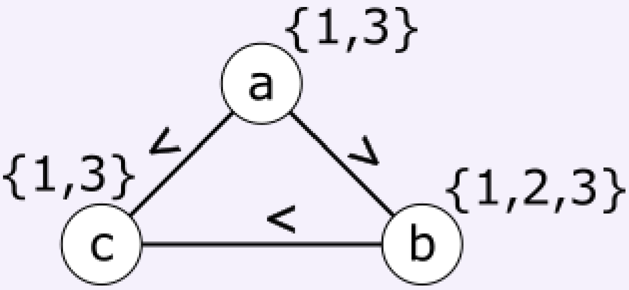


2. Wann ist der Graph lokal konsistent?



    - a {1,2}; b {2,3}; c {1,2,3}; d {1,2,3}
    - a {1,2}; b {2,3}; c {3}; d {1,2}
    - a {1,3}; b {2,3}; c {1,3}; d {1,2,3}
    - a {1,2}; b {2,3}; c {1,3}; d {1,2,3}

3. Wie sieht die Queue im nächsten Schritt mit AC3 aus?



Aktuelle Queue: [ab, ac, ba, bc, ca, cb]

- [bc, ba, ca, cb, ab, ac]
- [ab, ac, ba, bc, ca, cb]
- [ac, ba, bc, ca, cb]
- [ac, ba, bc, ca, cb, ba]

7.5. CSP5: Min-Conflicts Heuristik

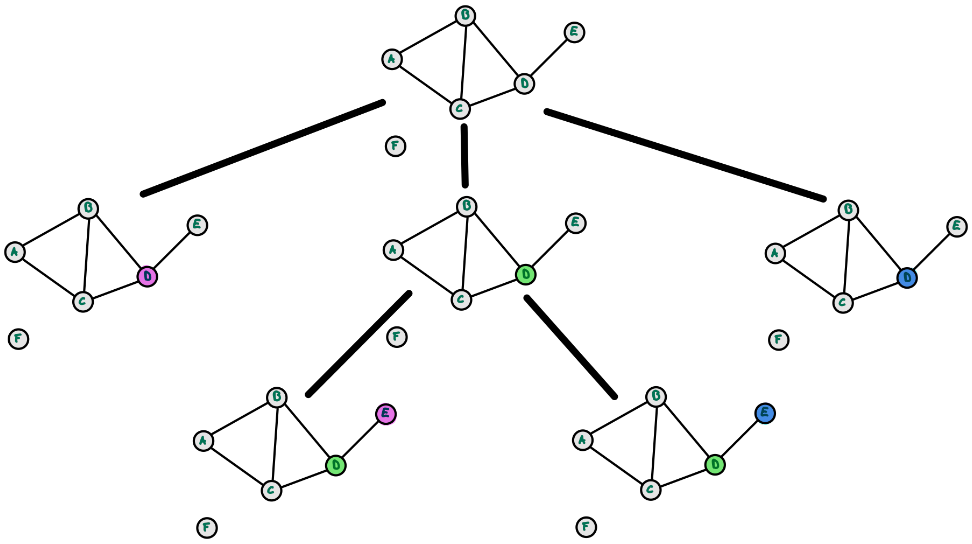
TL;DR

Die BT-Search geht schrittweise durch den Constraintgraphen, weist dem aktuellen Knoten einen Wert aus seiner Domäne zu und geht dann zum nächsten Knoten. Sobald es Konflikte gibt, wird Backtracking bis zum letzten Entscheidungspunkt gemacht und dort eine andere Zuweisung ausprobiert. Das dauert oft sehr lange.
Bei der Min-Conflicts Heuristik startet man stattdessen mit einer zufällig erzeugten, kompletten Belegung. So lange es dabei Konflikte gibt, wählt man eine der konfliktbehafteten Variablen und ändert ihren Wert, so dass die Anzahl der dadurch verletzten Constraints minimiert wird. Der Vorteil dieses Algorithmus ist seine Einfachheit und Schnelligkeit, allerdings ist der Algorithmus unvollständig.
Dieses Vorgehen erinnert stark an die Suche mit Genetischen/Evolutionären Algorithmen oder bei Simulated Annealing. Wie dort hat man bei der Min-Conflicts Heuristik Schwierigkeiten mit Plateaus, denen man durch *Noise* und Restarts entgegenzuwirken versucht.

Videos

Vorlesung [YT], [HSBI]

7.5.1. Problem bei BT-Suche



Die BT-Search würde den Graphen ablaufen und schrittweise dem jeweils ausgewählten Knoten eine Farbe zuweisen und dann weiter in die Tiefe gehen. Oft sieht man erst relativ spät, dass eine bestimmte Belegung nicht passt und weiter unten im Baum zu Konflikten führt und muss dann aufwändiges Backtracking machen. Dadurch dauert eine Suche mit der BT-Search u.U. relativ lange, selbst wenn man die in CSP: Heuristiken besprochenen Heuristiken einsetzt.

7.5.2. Idee: Würfeln und Schütteln

1. **Würfeln**: Erzeuge zufällige *vollständige* Belegung

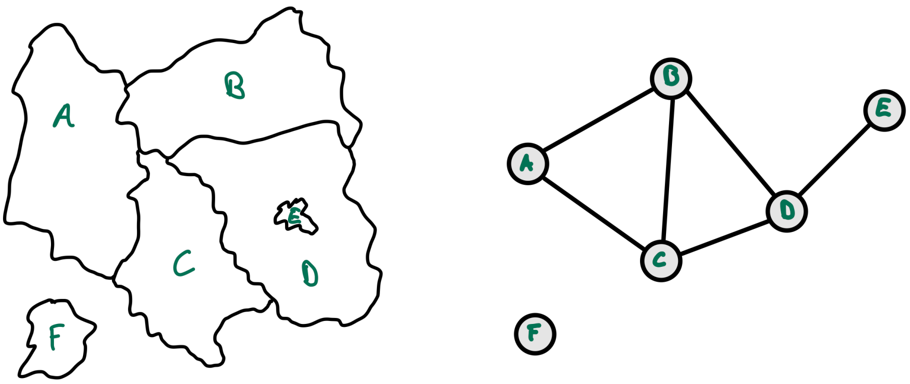
   Was würde passieren, wenn wir analog zu GA/EA eine *vollständige* Codierung wählen würden mit initial zufällig aus den Domänen ausgewählten Werten?

2. **Schütteln**: Verändere schrittweise Werte

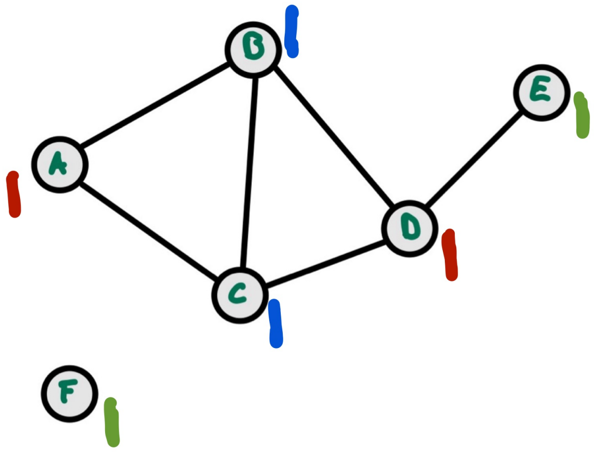
   Was würde passieren, wenn wir danach die Konflikte "*heraus schütteln*" (erinnert ein bisschen an Simulated Annealing)?

7.5.3. Beispiel: Einfärben von Landkarten

Ausgangszustand:



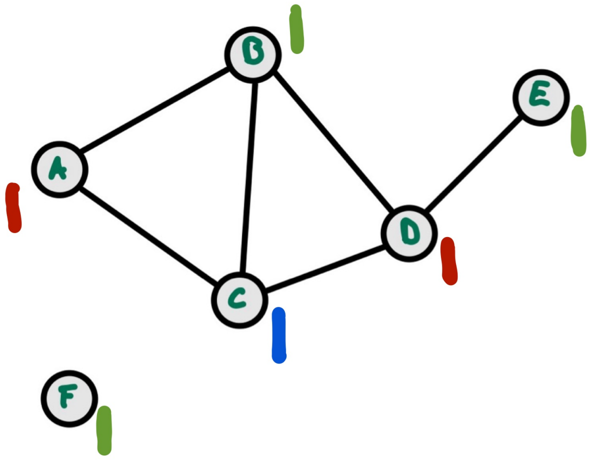
Schritt 1: "Würfeln" (zufällige vollständige Belegung)



Problem: Konflikt in Knoten B und C.

Schritt 2: “Schütteln”

- Auswahl von Knoten B
- Auswahl einer anderen Farbe (grün - Anzahl der verbleibenden Konflikte = 0)



Lösung erreicht.

7.5.4. Min-Conflicts Heuristik

```
Min-Conflicts(csp, maxSteps):
    assignment = random_complete_assignment(csp)

    for step in 1..maxSteps:
        if complete_consistent(assignment, csp): return assignment

        var = random_conflicted_variable(csp, assignment)
        value = choose_val_min_conflicts(csp, assignment, var)
        assignment += {var = value}

    return failure
```

Quelle: Min-Conflicts: Eigener Code basierend auf einer Idee nach (Russell und Norvig 2021, p. 182, fig. 5.8)

7.5.4.1. Hinweis

Wenn es mehrere gleichwertige Möglichkeiten gibt, nimm eine beliebige davon.

7.5.4.2. Beobachtungen

- Algorithmus braucht i.d.R. sehr wenige Schritte bis zur Lösung
- Plateaus können häufig auftreten
  - Zusätzliches Einführen von *Noise*: ersetze die deterministische Wahl des Wertes mit den wenigsten Konflikten (vgl. Simulated Annealing):
    - Mit einer Wahrscheinlichkeit < Noise: nimm zufälligen Wert aus der Domäne
    - Sonst: nimm den die Anzahl der Konflikte minimierenden Wert aus der Domäne
  - Restarts können zusätzlich helfen, dadurch komplett neue zufällige Belegung (vgl. GA/EA)
- Weitere Verbesserung: Statt einer beliebigen Variable diejenige mit den meisten Konflikten auswählen

7.5.4.3. Beispiel aus der Praxis

Mit der Min-Conflicts Heuristik konnte die Rechenzeit für die Planungen für eine Woche Beobachtungen für das Hubble-Weltraum-Teleskop von **drei Wochen** auf **10 Minuten** gesenkt werden. (Quelle: Wikipedia)

7.5.5. Diskussion

MAC (Maintaining Arc Consistency: BT-Search plus AC-3):

- Systematische Backtracking-Search
- Nach jeder Variablenzuweisung wird Arc-Consistency (AC-3) auf den verbleibenden Domänen durchgesetzt
- Vollständiger Algorithmus
- Gute Pruning-Eigenschaften bei starken/engen Constraints
- Hoher Rechenaufwand pro Schritt durch wiederholte Konsistenzprüfung

Min-Conflicts Heuristik:

- Lokale Repair-Heuristik
- Startet mit kompletter und zufälliger Belegung
- Ändert iterativ eine konfliktbehaftete Variable auf einen Wert, der die wenigsten Konflikte verursacht
- Sehr schnell in der Praxis für große, locker eingeschränkte Probleme

- Unvollständig, heuristisch, kann stecken bleiben

Ausblick: Verwandtschaft zu SAT (Erfüllbarkeitsproblem der Aussagenlogik)

Umformung der Constraints in aussagenlogische Formeln und Anwenden von SAT-Solvern wie *Conflict-Driven Clause Learning* (CDCL), *Davis-Putnam-Logemann-Loveland*-Algorithmus (DPLL), *WalkSAT* mit Hill Climbing, … vgl. Boolean satisfiability problem.

7.5.6. Wrap-Up

- Min-Conflicts ist ein skalierbares, einfach implementierbares Gegenstück zur vollständigen Suche
- Typische Probleme mit Plateaus

Zum Nachlesen

Lesen Sie in (Russell und Norvig 2021) bitte den Abschnitt 5.4 “Local Search for CSPs”.

Lernziele

- k3: Ich kann die Min-Conflicts Heuristik erklären und anwenden

Challenges

Betrachten Sie (erneut) das folgende Problem: Sei $D = \{0, \ldots, 5\}$, und ein Constraintproblem definiert durch $\langle\{v_1, v_2, v_3, v_4\}, \{D_{v_1} = D_{v_2} = D_{v_3} = D_{v_4} = D\}, \{c_1, c_2, c_3, c_4\}\rangle$
mit

- $c_1 = \big((v_1, v_2), \{(x, y) \in D^2 | x + y = 3\}\big)$,
- $c_2 = \big((v_2, v_3), \{(x, y) \in D^2 | x + y \leq 3\}\big)$,
- $c_3 = \big((v_1, v_3), \{(x, y) \in D^2 | x \leq y\}\big)$ und
- $c_4 = \big((v_3, v_4), \{(x, y) \in D^2 | x \neq y\}\big)$.

1. Zeichen Sie den Constraint-Graph.
2. Erzeugen Sie eine Lösung durch schrittweise Anwendung der Min-Conflicts Heuristik.

8. Naive Bayes

Ich habe Symptome beobachtet. Kann ich die Ursache (also die Krankheit) bestimmen, wenn ich etwas Hintergrundwissen habe:

- Wie häufig treten verschieden Krankheiten auf
- Welche Krankheit zeigt welche Symptome (und wie oft treten die dann auf)

Kann ich aus diesen Daten einen Klassifikator lernen?

8.1. NB1: Wiederholung Wahrscheinlichkeitstheorie

TL;DR

Diese Sitzung ist eine (relativ oberflächliche) Einführung/Wiederholung in die/der Grundlagen der Wahrscheinlichkeitstheorie.
Wir schauen uns die möglichen Ausgänge eines Zufallsexperiments an ("Ereignisse"). Wenn diese Ereignisse sich gegenseitig ausschließen und alle denkbaren Ergebnisse abdecken, dann nennt man diese Ereignisse auch **Elementarereignisse**. Die Wahrscheinlichkeit für ein Ereignis kann man angeben als Anzahl der möglichen Ergebnisse, die für dieses Ereignis günstig sind, geteilt durch die Anzahl aller Ausgänge. Über die Kolmogorov Axiome bekommt man die typischen Rechenregel für die Wahrscheinlichkeit.
Man kann eine **Verbundwahrscheinlichkeit** $P(A, B) = P(B, A)$ angeben, das ist die Wahrscheinlichkeit, dass A und B gleichzeitig auftreten.
Die **bedingte** Wahrscheinlichkeit für A gegeben B ist $P(A \mid B)$ und berechnet sich $P(A \mid B) = P(A, B)/P(B)$.
Daraus kann man die **Bayes-Regel** ableiten: $P(A \mid B) = P(B \mid A)P(A)/P(B)$.
Dabei nennt man

- $P(A)$ **Prior** oder **A-priori-Wahrscheinlichkeit** (die Wahrscheinlichkeit für A ohne weiteres Wissen),
- $P(B \mid A)$ **Likelihood** (Wie wahrscheinlich ist das Auftreten von B, gegeben A?),
- $P(A \mid B)$ **Posterior** oder **A-posteriori-Wahrscheinlichkeit** (Wie wahrscheinlich ist A, wenn B eingetreten ist?), und
- $P(B)$ ist ein Normierungsfaktor (Wie wahrscheinlich ist B an sich?).

Videos

Vorlesung [YT], [HSBI]

8.1.1. Ereignisse und Wahrscheinlichkeit

Hinweis: Die folgende Darstellung zur Einführung in die Wahrscheinlichkeitstheorie dient dem Verständnis des Naive Bayes Klassifikationsalgorithmus und ist teilweise eher oberflächlich gehalten. Sie kann und soll keine entsprechende mathematische Einführung ersetzen!

8.1.1.1. Ereignisse

- **Ereignisse** $\Omega = \{\omega_1, \omega_2, \ldots, \omega_n\}$: endliche Menge der Ausgänge eines Zufallsexperiments
- **Elementarereignis**: Die $\omega_i \in \Omega$
  - decken *alle* möglichen Versuchsergebnisse ab, und
  - schließen sich gegenseitig aus

8.1.1.2. Regeln

- Wenn A und B Ereignisse sind, dann auch $A \cup B$
- Ω wird als **sicheres Ereignis** bezeichnet: Enthält definitionsgemäß **alle** Versuchsausgänge, d.h. *ein* in der Menge enthaltenes Ereignis *muss* auftreten
- Die leere Menge $\emptyset$ wird als **unmögliches Ereignis** bezeichnet
- Die Variablen A und B heißen auch **Zufallsvariablen**

Im Rahmen dieser Veranstaltung betrachten wir nur diskrete Zufallsvariablen mit endlichem Wertebereich!

8.1.1.3. Wahrscheinlichkeit

- **Wahrscheinlichkeit**:

  Sei $\Omega = \{\omega_1, \omega_2, \ldots, \omega_n\}$ endlich. Die Wahrscheinlichkeit $P(A)$ für ein Ereignis A ist dann definiert als

  $$P(A) = \frac{|A|}{|\Omega|} = \frac{\text{Anzahl der für A günstigen Fälle}}{\text{Anzahl der möglichen Fälle}}$$

  Man könnte auch schreiben: $P(A) = \sum_{\omega \in A} P(\omega)$

  Hinweis: Diese Definition von Wahrscheinlichkeit geht von gleich wahrscheinlichen Elementarereignissen aus! Die allgemeine Definition geht über einen entsprechenden Grenzwert.

8.1.1.4. Verteilung

Den Vektor mit den Wahrscheinlichkeiten aller Elementarereignisse nennt man auch *Verteilung*.

Beispiel: $\mathbf{P}(A) = (P(A = 1), P(A = 2), \ldots, P(A = 6)) = (1/6, 1/6, \ldots, 1/6)$

Hinweis: Wir betrachten hier nur diskrete Zufallsvariablen. Für kontinuierliche Variablen wird die Verteilung mit Hilfe einer **Dichtefunktion** dargestellt, beispielsweise der Gauss'schen Funktion.

8.1.1.5. Beispiel

- Einmaliges Würfeln mit einem Spielwürfel: $\Omega = \{1, 2, 3, 4, 5, 6\}$
- Elementarereignisse: $\{1, 2, 3, 4, 5, 6\}$
- Das Würfeln einer geraden Zahl ($A = \{2, 4, 6\}$) ist *kein* Elementarereignis, ebenso wie das Würfeln einer Zahl kleiner 5 ($B = \{1, 2, 3, 4\}$), da $A \cap B = \{2, 4\} \neq \emptyset$
- Wahrscheinlichkeit, eine 1 zu würfeln: $P(A \in \{1\}) = P(A = 1) = \frac{1}{6}$.

  Anmerkung: Man schreibt statt $P(A \in \{1\})$ oft einfach $P(1)$.
- Wahrscheinlichkeit, eine gerade Zahl zu würfeln: $P(A \in \{2, 4, 6\}) = P(A = 2 \vee A = 4 \vee A = 6) = \frac{|\{2,4,6\}|}{|\{1,2,3,4,5,6\}|} = \frac{3}{6} = 0.5$

8.1.2. Rechenregeln: Kolmogorov Axiome

Sei A ein Ereignis, also $A \subseteq \Omega$:

- $0 \leq P(A) \leq 1$
- $\Omega = \{\omega_1, \omega_2, \ldots, \omega_n\}$: $\sum_i P(\omega_i) = 1$ (Normierungsbedingung: Summe über die Wahrscheinlichkeiten aller Elementarereignisse ist immer 1)
- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$

Daraus folgt (u.a.):

- $P(\Omega) = 1$
- $P(\emptyset) = 0$
- $P(A) = 1 - P(\neg A)$

- A und B *unabhängig*: $P(A \cup B) = P(A) + P(B)$
- $P(A \cap B)$ ist leer, wenn A und B sich nicht überlappen
- $A \subseteq B$: $P(A) \leq P(B)$

8.1.3. Verbundwahrscheinlichkeiten

$$P(A, B) = P(B, A) = \text{ Wahrscheinlichkeit, dass A und B gleichzeitig auftreten}$$

	Halsschmerzen	$\neg$ Halsschmerzen
Schnupfen	0.04	0.06

	Halsschmerzen	¬ Halsschmerzen
¬ Schnupfen	0.01	0.89

- $P(S, H) = 0.04$

Die Tabelle kann man so lesen: In 4 von 100 Fällen tritt das Ereignis “Schnupfen” gleichzeitig mit dem Ereignis “Halsschmerzen” auf, in 6 von 100 Fällen tritt “Schupfen” ohne Halsschmerzen auf. … In Summe kommt man wieder auf 100 Fälle (100 Prozent).

Nach diesen Zahlen liegt also die Verbundwahrscheinlichkeit für die Ereignisse “Schnupfen” und “Husten”, d.h. $P(S, H)$, bei 4 Prozent.

Hinweis: Die gezeigten Zahlen und Zusammenhänge sind **fiktiv** und dienen lediglich zur Verdeutlichung der Wahrscheinlichkeitsbegriffe!

8.1.4. Bedingte Wahrscheinlichkeit

Definition: Bedingte Wahrscheinlichkeit für A gegeben B:

$$P(A \mid B) = \frac{P(A, B)}{P(B)}$$

	Halsschmerzen	¬ Halsschmerzen
Schnupfen	0.04	0.06
¬ Schnupfen	0.01	0.89

- $P(\text{Schnupfen} \mid \text{Halsschmerzen}) = \frac{P(S,H)}{P(H)} = \frac{0.04}{0.04+0.01} = 0.8$
- $P(\text{Halsschmerzen} \mid \text{Schnupfen}) = \frac{P(H,S)}{P(S)} = \frac{0.04}{0.04+0.06} = 0.4$

Wegen $P(A \mid B) = \dfrac{P(A, B)}{P(B)}$ ist $P(A, B) = P(A \mid B)P(B) = P(B \mid A)P(A)$ (**Produkt-Regel**)!

8.1.5. Marginalisierung

	Halsschmerzen	¬ Halsschmerzen	$\sum$
Schnupfen	0.04	0.06	*0.1*
¬ Schnupfen	0.01	0.89	*0.9*
$\sum$	*0.05*	*0.95*	*1*

$P(S) = P(S,H) + P(S,\neg H)$

Allgemein: Seien $B_1, \ldots, B_n$ Elementarereignisse mit $\bigcup_i B_i = \Omega$. Dann ist

$$P(A) = \sum_i P(A, B_i) = \sum_i P(A \mid B_i) P(B_i)$$

Diesen Vorgang nennt man **Marginalisierung**. Die resultierende Verteilung $P(A)$ nennt man auch *“Randverteilung”*, da sie mit einer Projektion eines Quaders auf eine Seitenfläche vergleichbar ist.

8.1.6. Kettenregel

- **Produktregel**: Wegen $P(A \mid B) = \dfrac{P(A,B)}{P(B)}$ gilt $P(A,B) = P(A \mid B)P(B)$
- Verallgemeinerung (**Kettenregel**):

$$\begin{aligned}
P(A_1, A_2, \ldots, A_n) &= P(A_n, \ldots, A_2, A_1) && (8.1)\\
&= P(A_n \mid A_{n-1}, \ldots, A_1) P(A_{n-1}, \ldots, A_1) && (8.2)\\
&= P(A_n \mid A_{n-1}, \ldots, A_1) P(A_{n-1} \mid A_{n-2}, \ldots, A_1) P(A_{n-2}, \ldots, A_1) && (8.3)\\
&= \ldots && (8.4)\\
&= P(A_n \mid A_{n-1}, \ldots, A_1) \ldots P(A_2 \mid A_1) P(A_1) && (8.5)\\
&= \prod_i P(A_i \mid A_1, \ldots, A_{i-1}) && (8.6)
\end{aligned}$$

8.1.7. Bayes-Regel

Bedingte Wahrscheinlichkeit: $P(A,B) = P(A \mid B)P(B) = P(B \mid A)P(A)$

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$$

- $P(A)$ nennt man **Prior** oder **A-priori-Wahrscheinlichkeit** (Das ist die Wahrscheinlichkeit für A ohne weiteres Wissen)
- $P(B \mid A)$ nennt man **Likelihood** (Wie wahrscheinlich ist das Auftreten von B, gegeben A?)
- $P(A \mid B)$ nennt man **Posterior** oder **A-posteriori-Wahrscheinlichkeit** (Wie wahrscheinlich ist A, wenn B eingetreten ist?)
- $P(B)$ ist ein Normierungsfaktor

Wenn man (siehe später: Naive Bayes Klassifikator) A als Klasse und B als Daten betrachtet:

- $P(A)$: Wie wahrscheinlich ist eine bestimmte Klasse an sich (A-priori-Wahrscheinlichkeit der Klassen)?
- $P(B \mid A)$: Wie wahrscheinlich sind bestimmte Daten, gegeben die Klasse A? (Likelihood der Daten)
- $P(A \mid B)$: Gegeben die Daten B, wie wahrscheinlich ist die Klasse A? (Posterior)

In der Medizin hat sucht man i.d.R. die Ursache für beobachtete Symptome:

$$P(\text{Ursache} \mid \text{Symptome}) = \frac{P(\text{Symptome} \mid \text{Ursache})P(\text{Ursache})}{P(\text{Symptome})}$$

Aus der A-priori-Wahrscheinlichkeit für bestimmte Krankheiten und der Likelihood der Symptome (wie wahrscheinlich sind Symptome, gegeben eine Krankheit) kann man die Wahrscheinlichkeit für das Vorliegen einer Erkrankung gegeben bestimmte Symptome berechnen.

8.1.8. Beispiel Bayes

- Bei Arthrose wird in 80 Prozent der Fälle ein steifes Gelenk beobachtet
- Eine von 10.000 Personen hat Arthrose
- Eine von 10 Personen hat ein steifes Gelenk

$\rightarrow$ Ich habe ein steifes Gelenk. Habe ich Arthrose?

- Gegeben: $P(A) = 0.0001, P(S) = 0.1, P(S \mid A) = 0.8$
- Gesucht: $P(A \mid S)$

$$P(A \mid S) = \frac{P(S \mid A)P(A)}{P(S)} = \frac{0.8 \times 0.0001}{0.1} = 0.0008 = 0.08\%$$

Wenn ein steifes Gelenk vorliegt, ist die Wahrscheinlichkeit, dann an Arthrose erkrankt zu sein, bei nur 0.08%. Kein Grund zur Sorge in diesem Fall :-)

$\rightarrow$ Wie wahrscheinlich ist ein steifes Gelenk ohne Arthrose, also $P(S \mid \neg A)$?

Mit Marginalisierung: $P(S) = P(S \mid A)P(A) + P(S \mid \neg A)P(\neg A)$,
d.h. $0.1 = 0.8 \times 0.0001 + P(S \mid \neg A) \times (1 - 0.0001)$, d.h. $P(S \mid \neg A) = 0.0999$

In knapp 10 Prozent der Fälle würde man im obigen Beispiel bei der Diagnose "keine Arthrose" ein steifes Gelenk beobachten.

Hinweis: Die genannten Zahlen und Zusammenhänge sind rein fiktional und sollen lediglich zur Veranschaulichung der Bayes-Regel dienen!

Schauen Sie sich auch das Beispiel 7.9 in (Ertel 2025, Ex. 7.9, S. 135) an!

8.1.9. Unabhängige Ereignisse

- $P(\text{Halsschmerzen}, \text{ Regen}) = P(\text{Regen } \mid \text{ Halsschmerzen})P(\text{Halsschmerzen})$
- $P(\text{Regen } \mid \text{ Halsschmerzen}) = \text{ ?? } = P(\text{Regen})$

- Zwei Ereignisse A und B sind **unabhängig**, wenn

$$P(A \mid B) = P(A)$$

$\rightarrow P(A, B) = P(A \mid B)P(B) = P(A)P(B)$

Dies kann man verallgemeinern (**bedingte Unabhängigkeit**):

> X und Y sind *bedingt unabhängig* (gegeben Z), wenn $P(X \mid Y, Z) = P(X \mid Z)$ bzw. $P(Y \mid X, Z) = P(Y \mid Z)$

Daraus folgt:

$$P(X, Y \mid Z) = P(X \mid Y, Z)P(Y \mid Z) = P(X \mid Z)P(Y \mid Z)$$

8.1.10. Wrap-Up

- Grundlagen der Wahrscheinlichkeitstheorie
  - Elementarereignisse und Wahrscheinlichkeit
  - Rechenregeln
  - Bedingte Wahrscheinlichkeit und Verbundwahrscheinlichkeit
  - Marginalisierung
  - (Bedingte) Unabhängigkeit
  - Bayes’sche Regel

Zum Nachlesen

Lesen Sie in (Russell und Norvig 2021) bitte die Abschnitte 12.2 bis (einschließlich) 12.5. Darüber hinaus ist Abschnitt “7.1 Rechnen mit Wahrscheinlichkeiten” (Ertel 2025) sehr empfehlenswert.

Lernziele

- k2: Ich kann die Begriffe Elementarereignisse und Wahrscheinlichkeit erklären
- k2: Ich kann bedingte Wahrscheinlichkeit und Verbundwahrscheinlichkeit erklären
- k2: Ich kann (bedingte) Unabhängigkeit erklären
- k3: Ich kann die Rechenregeln für Wahrscheinlichkeit anwenden
- k3: Ich kann Marginalisierung an einem Beispiel anwenden
- k3: Ich verstehe die Bayes’sche Regel und kann sie an einem Beispiel demonstrieren

Quizzes

- Selbsttest Wahrscheinlichkeiten (ILIAS)

8.2. NB2: Klassifikation mit Naive Bayes

TL;DR

Mit Hilfe der (verallgemeinerten) Bayes-Regel kann man Klassifikation durchführen. Dazu werden beim "Training" die bedingten Wahrscheinlichkeiten aus den Trainingsdaten geschätzt. Die Anwendung (Klassifikation) erfolgt dann durch die Nutzung der beim "Training" berechneten bedingten Wahrscheinlichkeiten:

$$h_{MAP} = \underset{h \in H}{\text{argmax}}\ P(h) \prod_i P(D_i \mid h)$$

Für jede Hypothese h, d.h. für jede Klasse, wird der Posterior $P(h \mid D_1, \ldots, D_n)$ ausgerechnet. Die Klasse, deren Wert dabei am höchsten ist, "gewinnt", d.h. die Klasse mit dem größten Posterior wird ausgegeben. (Deshalb wird das Verfahren oft auch "MAP" – *Maximum a Posteriori* – genannt.)
Bei der Berechnung wird angenommen, dass die betrachteten Merkmale (bedingt) unabhängig sind (dies geht in die obige Formel ein). Diese Annahme trifft aber oft nicht zu, deshalb auch der Name "*Naive* Bayes Klassifikation". Man berechnet in diesem Fall falsche Werte. Dennoch zeigt der Algorithmus in der Praxis sehr gute Ergebnisse.
Durch den Einsatz der bedingten Wahrscheinlichkeiten in der Produktformel ergeben sich einige Schwierigkeiten:

1. Wenn beim "Training" Ausprägungen fehlen, ist die bedingte Wahrscheinlichkeit Null. Dadurch wird das gesamte Produkt Null. Zur Abhilfe kann man den **Laplace-Schätzer** nutzen, der (gesteuert über einen Parameter) gewissermaßen virtuelle Trainingsbeispiele beisteuert.
2. Durch das Produkt vieler kleiner Werte kann es schnell zu *Floating Point*-Underflows kommen. Hier kann man einen Trick nutzen: Man berechnet den Logarithmus der Produktformel. Dadurch ändern sich zwar die absoluten Werte, die Reihenfolge der Hypothesen bleibt aber erhalten. Da wir nur nach der Hypothese suchen, die einen höheren Wert als die anderen hat, und nicht den absoluten Wert an sich benötigen, kann man so vorgehen. Durch den Logarithmus wird aus dem Produkt eine Summe, wo die kleinen Werte der bedingten Wahrscheinlichkeiten nicht so starke Auswirkungen haben wie im Produkt.

Oft nimmt man zusätzlich an, dass für alle Hypothesen (Klassen) h der Prior $P(h)$ gleich ist. Dann kann man diesen Faktor ebenfalls aus der Berechnung entfernen. Dieses Verfahren nennt man auch **Maximum Likelihood**.
Der NB-Klassifikator wird gern für die Textklassifikation eingesetzt. Hier muss man einem Text ein Label zuordnen. In einer Vorverarbeitung wird zunächst eine Menge der relevanten Wörter über alle Trainingstexte gebildet (*Bag-of-Words*). Der Bag-of-Words entspricht einem Merkmalsvektor, wobei die Merkmale die einzelnen Wörter sind. Dann kann jeder Text der Trainingsmenge über so einen Merkmalsvektor dargestellt werden: Entweder man gibt pro Merkmal an, ob es da (1) oder nicht da (0) ist oder man zählt die Häufigkeit des Auftretens. Dann kann man mit dem NB-Klassifikator die bedingten Wahrscheinlichkeiten schätzen und einen neuen Text klassifizieren.

Videos

Vorlesung [YT], [HSBI]

8.2.1. Medizinische Diagnostik mit NB

- Bei Arthrose wird in 80 Prozent der Fälle ein steifes Gelenk beobachtet: $P(S \mid A) = 0.8$
- Eine von 10.000 Personen hat Arthrose: $P(A) = 0.0001$
- Eine von 10 Personen hat ein steifes Gelenk: $P(S) = 0.1$

$\rightarrow$ Ich habe ein steifes Gelenk. Habe ich Arthrose?

8.2.2. Textklassifikation mit NB

- Mails, manuell markiert:
  - D1: ("Sieben Zwerge fraßen sieben Ziegen", OK)
  - D2: ("Sieben Ziegen traten sieben Wölfe", SPAM)
  - D3: ("Sieben Wölfe fraßen sieben Böcke", OK)
  - D4: ("Sieben Böcke traten sieben Zwerge", SPAM)

- Neue Mails:
  - T1: ("Sieben Zwerge fraßen sieben Wölfe")
  - T2: ("Sieben Zwerge traten sieben Ziegen")

Lernen Sie mit Hilfe der Trainingsmenge einen Naive-Bayes-Klassifikator und wenden Sie diesen auf die beiden Test-Dokumente an.

8.2.3. Naive Bayes

- Verallgemeinerte Bayes Regel

$$P(H \mid D_1, \ldots, D_n) = \frac{P(D_1, \ldots, D_n \mid H)P(H)}{P(D_1, \ldots, D_n)}$$

- Annahme: D_i sind bedingt unabhängig

$$P(D_1, \ldots, D_n \mid H) = P(D_1 \mid H) \cdot \ldots \cdot P(D_n \mid H) = \prod_i P(D_i \mid H)$$

- Beobachtung: $P(D_1, \ldots, D_n)$ für alle Hypothesen $h \in H$ gleich

- **Naive Bayes Klassifikator** bzw. **MAP** ("Maximum a Posteriori")

  $$h_{MAP} = \underset{h \in H}{\text{argmax}}\ P(h \mid D_1, \ldots, D_n) = \underset{h \in H}{\text{argmax}}\ P(h) \prod_i P(D_i \mid h)$$

  Naive Bayes: Wähle die plausibelste Hypothese, die von den Daten unterstützt wird.

8.2.4. Bayes'sches Lernen

Naive Bayes Klassifikator/**MAP**:

$$h_{MAP} = \underset{h \in H}{\text{argmax}}\ P(h \mid D_1, \ldots, D_n) = \underset{h \in H}{\text{argmax}}\ P(h) \prod_i P(D_i \mid h)$$

Training: Bestimme die Wahrscheinlichkeiten aus Trainingsdaten $\mathbf{S}$

- Für jede Klasse h:
  - Schätze $P(h) = \dfrac{|S(h)|}{|S|}$
  - Für jedes Attribut D_i und jede Ausprägung $x \in D_i$:
    Schätze $P(D_i = x \mid h) = \dfrac{|S_{D_i}(x) \cap S(h)|}{|S(h)|}$

Klassifikation: Wähle wahrscheinlichste Klasse h_{MAP} für Vektor $\mathbf{x}$

- $h_{MAP} = \text{argmax}_{h \in H}\ P(h) \prod_{x \in \mathbf{x}} P(x \mid h)$

8.2.5. Beispiel Klassifikation mit NB

Nase läuft	Husten	Gerötete Haut	Fieber	Klasse
1	1	1	0	krank
1	1	0	0	krank
0	0	1	1	krank
1	0	0	0	gesund
0	0	0	0	gesund

- Eingabe: Person mit Husten und Fieber

Gesucht: $P(\text{krank})$, $P(\text{gesund})$, $P(\text{Nase=0} \mid \text{krank})$, $P(\text{Nase=0} \mid \text{gesund})$, $\ldots$

Wähle Klasse

$$h_{MAP} = \underset{h \in \{\text{gesund, krank}\}}{\text{argmax}} \quad P(h) \cdot P(\text{Nase=0} \mid h) \cdot P(\text{Husten=1} \mid h) \tag{8.7}$$

$$\cdot P(\text{Haut=0} \mid h) \cdot P(\text{Fieber=1} \mid h) \tag{8.8}$$

Ergebnis: (nur die für den zu klassifizierenden Beispiel-Vektor nötigen Werte, die restlichen müssten aber auch beim "Training" berechnet werden!)

```
P(gesund) = 2/5 = 0.4
P(krank)  = 3/5 = 0.6

P(Nase=0 | gesund) = 1/2 = 0.5
P(Nase=0 | krank)  = 1/3 = 0.333

P(Husten=1 | gesund) = 0/2 = 0
P(Husten=1 | krank)  = 2/3 = 0.667

P(Haut=0 | gesund) = 2/2 = 1
P(Haut=0 | krank)  = 1/3 = 0.333

P(Fieber=1 | gesund) = 0/2 = 0
P(Fieber=1 | krank)  = 1/3 = 0.333

h = gesund: P(gesund) * P(Nase=0 | gesund) * P(Husten=1 | gesund) * P(Haut=0 |
    gesund) * P(Fieber=1 | gesund) = 0.4*0.5*0*1*0              = 0
h = krank:  P(krank)  * P(Nase=0 | krank)  * P(Husten=1 | krank)  * P(Haut=0 |
    krank)  * P(Fieber=1 | krank)  = 0.6*0.333*0.667*0.33*0.333 = 0.015
```

→ Klasse "krank" gewinnt (Wert für $P(\text{krank})$ ist der höchste der beiden Hypothesen) …

8.2.6. Textklassifikation mit NB

- Texte als Trainingsmenge:
  - Text zerlegen in Terme (Wörter, sonstige relevante Token)
  - ggf. Entfernen von Stoppwörtern (beispielsweise Artikel u.ä.)
  - ggf. Stemming und Lemmatisierung für restliche Terme
  - ggf. weitere Vorverarbeitungsschritte (Groß-Klein-Schreibung, …)
  - Terme zusammenfassen als Menge: *"Bag of Words"* (mit Häufigkeit)

- Naive Bayes "trainieren":
  - A-priori-Wahrscheinlichkeit der Klassen: $P(c) = \frac{N_c}{N} = \frac{\text{Anzahl Dokumente in Klasse c}}{\text{Anzahl Dokumente}}$
  - Likelihood der Daten (Terme):

  - $P(t \mid c) = \dfrac{\text{count}(t, c)}{\sum_{v \in V} \text{count}(v, c)}$
    mit count(t, c) Anzahl der Vorkommen von Term t in allen Dokumenten der Klasse c und V die Vereinigung aller Terme aller Dokumente (als Menge)
  - Variante mit Laplace-Glättung (s.u.): $P(t \mid c) = \dfrac{\text{count}(t, c) + 1}{\sum_{v \in V} \text{count}(v, c) + |V|}$

8.2.7. Naivität im Naive Bayes

- Unabhängigkeit der Attribute oft nicht gegeben

  $\rightarrow P(D_1, \ldots, D_n \mid H) \neq \prod_i P(D_i \mid H)$
- A-posteriori-Wahrscheinlichkeiten oft unrealistisch nah an 1 oder 0

- Praxis: Dennoch häufig sehr gute Ergebnisse

  Wichtig: Solange die **Maximierung** über alle Hypothesen die selben Ergebnisse liefert, müssen die konkreten Schätzungen/Werte nicht exakt stimmen …

Wenn Attribute nicht (bedingt) unabhängig sind, kann sich der NB verschätzen, d.h. es kommt dann u.U. zu einer höheren Fehlerrate, da bestimmte Eigenschaften in der Trainingsmenge zu hoch gewichtet werden.

Tip

8.2.7.1. Beispiel

Gegebene Daten

Seien die beiden Merkmale x_1 und x_2 mit den folgenden Verteilungen gegeben:

- Klassen: $H \in \{0, 1\}$, $P(H = 0) = P(H = 1) = 0.5$
- Bedingte Verteilungen $P(x_1, x_2 \mid H)$:
  - Für $H = 0$:
    - $P(x_1 = 0, x_2 = 0 \mid 0) = 0.30$
    - $P(x_1 = 0, x_2 = 1 \mid 0) = 0.35$
    - $P(x_1 = 1, x_2 = 0 \mid 0) = 0.15$
    - $P(x_1 = 1, x_2 = 1 \mid 0) = 0.20$
  - Für $H = 1$:
    - $P(x_1 = 0, x_2 = 0 \mid 1) = 0.00$
    - $P(x_1 = 0, x_2 = 1 \mid 1) = 0.30$
    - $P(x_1 = 1, x_2 = 0 \mid 1) = 0.65$
    - $P(x_1 = 1, x_2 = 1 \mid 1) = 0.05$

Analyse der gegebenen Daten

Die Merkmale x_1 und x_2 sind *nicht* bedingt abhängig gegeben H.

Erinnerung: Zwei Ereignisse X und Y sind bedingt unabhängig gegeben Z, wenn gilt $P(X, Y \mid Z) = P(X \mid Y, Z)P(Y \mid Z) = P(X \mid Z)P(Y \mid Z)$.
Wir haben aber im Fall von $H = 1$:

- $P(x_1 = 0, x_2 = 0 \mid 1) = 0.00$ vs. $P(x_1 = 0 \mid 1)P(x_2 = 0 \mid 1) = (0.00 + 0.30) * (0.00 + 0.65) = 0.30 * 0.65 = 0.195$
- $P(x_1 = 0, x_2 = 1 \mid 1) = 0.30$ vs. $P(x_1 = 0 \mid 1)P(x_2 = 1 \mid 1) = (0.00 + 0.30) * (0.30 + 0.05) = 0.30 * 0.35 = 0.105$
- $P(x_1 = 1, x_2 = 0 \mid 1) = 0.65$ vs. $P(x_1 = 1 \mid 1)P(x_2 = 0 \mid 1) = (0.65 + 0.05) * (0.00 + 0.65) = 0.70 * 0.65 = 0.455$
- $P(x_1 = 1, x_2 = 1 \mid 1) = 0.05$ vs. $P(x_1 = 1 \mid 1)P(x_2 = 1 \mid 1) = (0.65 + 0.05) * (0.30 + 0.05) = 0.70 * 0.35 = 0.245$

(analog für $H = 0$)
Damit bekommen wir im Naive Bayes Klassifikator ein Problem. Dort wird von bedingt unabhängigen Merkmalen ausgegangen und deshalb die Vereinfachung von $P(x_1, x_2 \mid H)$ zu $P(x_1 \mid H)P(x_2 \mid H)$ vorgenommen. Da die Annahme nicht stimmt, werden die Merkmale falsch gewichtet und es kann zu Fehlklassifikationen kommen.

Klassifikation einer Beobachtung

Wir machen nun die folgende Beobachtung: $X = (x_1 = 1, x_2 = 1)$.
Die nötigen Marginalisierungen aus den Trainingsdaten für diese Beobachtung sind:

- Für $H = 0$: $P(x_1 = 1 \mid 0) = 0.15 + 0.20 = 0.35$, $P(x_2 = 1 \mid 0) = 0.35 + 0.20 = 0.55$
- Für $H = 1$: $P(x_1 = 1 \mid 1) = 0.65 + 0.05 = 0.70$, $P(x_2 = 1 \mid 1) = 0.30 + 0.05 = 0.35$

Anwendung der Naive Bayes Klassifikation (mit Annahme bedingt unabhängige Merkmale): Wir nutzen $h_{MAP} = \text{argmax}_{h \in H} \; P(h) \prod_i P(D_i \mid h)$ und setzen unsere beiden Merkmale ein: $h_{MAP} = \text{argmax}_{h \in H} \; P(h)P(x_1 \mid h)P(x_2 \mid h)$.
Damit bekommen wir folgende Entscheidung:

- $H = 0 : 0.5 * 0.35 * 0.55 = 0.09625$
- $H = 1 : 0.5 * 0.70 * 0.35 = 0.1225$
- Entscheidung für Klasse $H = 1$

Da die Merkmale nicht unabhängig sind, darf die Produktannahme nicht verwendet werden, sondern wir müssten eigentlich den Term $P(x_1, x_2 \mid h)$ nutzen: $h_{MAP} = \text{argmax}_{h \in H} \; P(h)P(x_1, x_2 \mid h)$.
Aus den gegebenen Daten haben wir (einfach oben ablesen):

- $P(x_1 = 1, x_2 = 1 \mid 0) = 0.20$
- $P(x_1 = 1, x_2 = 1 \mid 1) = 0.05$

Eingesetzt in die Formel $h_{MAP} = \text{argmax}_{h \in H} \; P(h)P(x_1, x_2 \mid h)$:

- $H = 0 : 0.5 * 0.20 = 0.10$
- $H = 1 : 0.5 * 0.05 = 0.025$

- Entscheidung für Klasse $H = 0$

D.h. der Naive Bayes Klassifikator würde hier $H = 1$ vorschlagen, während der korrekte Posterior für $H = 0$ spricht.

Interpretation

In diesem konstruierten Beispiel führt die Abhängigkeit der Merkmale zu einer Fehlkalibrierung der Posterior-Wahrscheinlichkeiten durch NB und damit zu einer falschen Klassifikation. In der Realität erweist sich der NB trotzdem relativ robust gegenüber der Abhängigkeit von Merkmalen: Durch die Korrelation kommen häufig A-posteriori-Wahrscheinlichkeiten nahe 0 oder nahe 1 heraus, aber da es nur auf das Maximum und nicht auf den konkreten Wert ankommt, erhält man häufig trotzdem noch eine korrekte Klassifikation.

8.2.8. Laplace-Schätzer

- Problem: Attribut-Ausprägung für bestimmte Klasse nicht in Trainingsmenge:
  - $\to$ Bedingte Wahrscheinlichkeit ist 0
  - $\to$ Produkt gleich 0

- Lösung: "Laplace-Schätzer" (auch "Laplace-Glättung")

  Statt $P(D_i = x \mid h) = \dfrac{|S_{D_i}(x) \cap S(h)|}{|S(h)|}$

  nutze $P(D_i = x \mid h) = \dfrac{|S_{D_i}(x) \cap S(h)| + m \cdot p_i}{|S(h)| + m}$

  - mit m: frei wählbarer Faktor, und
  - p_i: A-priori-Wahrscheinlichkeit für $P(D_i = x \mid h)$

    Hintergrundwissen oder einfach *uniforme Verteilung der Attributwerte*: $p_i = 1/|D_i|$ (Wahrscheinlichkeit für eine Attributausprägung ist 1/(Anzahl der Ausprägungen des Attributs))

  $\to$ "virtuelle" Trainingsbeispiele (m ist die Zahl der virtuellen Trainingsbeispiele)

8.2.9. Probleme mit Floating Point Underflow

- MAP berechnet Produkt mit vielen Termen
- Problem: Bei kleinen Zahlen kann **Floating Point Underflow** auftreten!

- Lösung: Logarithmus maximieren (Produkt geht in Summe über)

  Erinnerung: $\log(x \cdot y) = \log(x) + \log(y)$ und Logarithmus streng monoton

$$\begin{aligned} h_{MAP} &= \operatorname*{argmax}_{h \in H} P(h \mid D_1, \ldots, D_n) && (8.9) \\ &= \operatorname*{argmax}_{h \in H} P(h) \prod_i P(D_i \mid h) && (8.10) \\ &= \operatorname*{argmax}_{h \in H} [\log(P(h)) + \sum_i \log(P(D_i \mid h))] && (8.11) \end{aligned}$$

8.2.10. Maximum Likelihood

- **Maximum a Posteriori**

$$h_{MAP} = \operatorname*{argmax}_{h \in H} P(h \mid D_1, \ldots, D_n) = \operatorname*{argmax}_{h \in H} P(h) \prod_i P(D_i \mid h)$$

- Annahme: Klassen uniform verteilt $\rightarrow P(h_i) = P(h_j)$

  Maximum Likelihood

$$h_{ML} = \operatorname*{argmax}_{h \in H} \prod_i P(D_i \mid h)$$

  $\rightarrow$ Maximiere die Likelihood der Daten

8.2.11. Ausblick: Kontinuierliche Attribute

Bisher sind wir von diskreten Attributen ausgegangen. Bei kontinuierlichen Attributen hat man zwei Möglichkeiten:

- Diskretisierung der Attribute: Aufteilung in Intervalle und Bezeichnung der Intervalle mit einem Namen
- Einsatz einer Verteilungsannahme und deren Dichtefunktion, beispielsweise Annahme von **normalverteilten** Daten mit der Dichtefunktion

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

  wobei μ der Mittelwert und σ^2 die Varianz der Daten sind.

8.2.12. Hinweis zum Sprachgebrauch

In Abhängigkeit von der Verteilung der $P(D_i \mid h)$ spricht man von

- "multinominalem" NB: Attribute umfassen mehrere Kategorien (verschiedene Ausprägungen, wie im "Wahlkampf"-Beispiel: Attribut "Bildung" hat die Ausprägungen "Abitur", "Bachelor" und "Master")
- Bernoulli NB: Attribute sind binär (Ausprägung 0 oder 1), typischerweise bei der Textklassifikation
- Gauss'sches NB: Annahme einer Normalverteilung der Attribut-Ausprägungen

8.2.13. Wrap-Up

- Klassifikation mit Naive Bayes
  - Annahme von Unabhängigkeit → "Naive" Bayes Klassifikation
  - Schätzen der bedingten Wahrscheinlichkeiten aus den Trainingsdaten
  - Klassifikation durch Nutzung der geschätzten Wahrscheinlichkeiten
  - Hinweis auf Naivität der Annahme, dennoch sehr gute Erfolge in Praxis
  - Hinweis auf Probleme mit niedrigen Wahrscheinlichkeiten

Zum Nachlesen

Lesen Sie in (Russell und Norvig 2021) bitte den Abschnitt 12.6 "Naive Bayes Models". Darüber hinaus ist Abschnitt "8.10 Der Naive Bayes Klassifikator" (Ertel 2025) sehr empfehlenswert.

Lernziele

- k2: Ich kann die Unabhängigkeits-Annahme in der 'Naive' Bayes Klassifikation erklären
- k2: Ich kann die Probleme mit niedrigen Wahrscheinlichkeiten erklären
- k3: Ich kann die bedingten Wahrscheinlichkeiten aus konkreten Trainingsdaten schätzen
- k3: Ich kann die Klassifikation mit Naive Bayes durch Nutzung der geschätzten Wahrscheinlichkeiten durchführen

Quizzes

- Selbsttest Naive Bayes Klassifikation (ILIAS)

Challenges

Spam-Mails

Stellen Sie sich vor, Sie haben eine Sammlung von 100 E-Mails (60 Spam, 40 Nicht-Spam). Sie wissen, dass das Wort "Gewinn" in 45 Spam-E-Mails und in 5 Nicht-Spam-E-Mails vorkommt.

1. Berechnen Sie die Wahrscheinlichkeit, dass es eine E-Mail Spam ist, wenn das Wort "Gewinn" darin vorkommt.
2. Wie würde die E-Mail mit dem Wort "Gewinn" durch einen Naive Bayes Klassifikator bewertet?

Textklassifikation

Betrachten Sie die folgenden Aussagen:

> - Patient A hat weder Husten noch Fieber und ist gesund.
> - Patient B hat Husten, aber kein Fieber und ist gesund.
> - Patient C hat keinen Husten, aber Fieber. Er ist krank.
> - Patient D hat Husten und kein Fieber und ist krank.
> - Patient E hat Husten und Fieber. Er ist krank.

Aufgaben:

1. Trainieren Sie auf diesem Datensatz einen Klassifikator mit NB.

2. Ist Patient F krank? Er hat Husten, aber kein Fieber.

8.3. NB3: Text-Klassifikation mit Naive Bayes

TL;DR

Der NB-Klassifikator wird gern für die Textklassifikation eingesetzt. Hier muss man einem Text ein Label zuordnen.
In einer Vorverarbeitung wird zunächst eine Menge der relevanten Wörter über alle Trainingstexte gebildet (*Bag-of-Words*). Der Bag-of-Words entspricht einem Merkmalsvektor, wobei die Merkmale die einzelnen Wörter sind. Dann kann jeder Text der Trainingsmenge über so einen Merkmalsvektor dargestellt werden: Entweder man gibt pro Merkmal an, ob es da (1) oder nicht da (0) ist (Variante Bernoulli NB) oder man zählt die Häufigkeit des Auftretens der Merkmale (Variante Multinomial NB). Dann kann man mit dem NB-Klassifikator die bedingten Wahrscheinlichkeiten schätzen und einen neuen Text klassifizieren.
Bei beiden Varianten hat man das Problem mit Termen, die für eine Klasse im Training nicht vorkommen (Zero-Frequency-Problem). Dabei werden die geschätzten Wahrscheinlichkeiten zu 0, wodurch bei der Verwendung in der Klassifikation entsprechend die Scores ebenfalls zu 0 berechnen und eine sinnvolle Klassifikation nicht möglich ist. Diesem Problem kann man durch die Laplace-Glättung begegnen, welche wie künstliche Trainingsdaten wirkt.
Wenn im zu klassifizierenden Testdatensatz Wörter vorkommen, die nicht beim Training vorhanden waren, spricht man auch vom “Out-of-Vocabulary”-Problem. Am einfachsten lässt sich dieses Problem lösen, indem man bereits beim Training einen “UNK”-Term trainiert (UNK steht für “unknown”). Bei der Klassifikation werden OOV-Terme einfach auf den UNK-Term abgebildet und die dafür berechneten Wahrscheinlichkeiten genutzt.

Videos

Vorlesung [YT], [HSBI]

8.3.1. Textklassifikation mit NB

- Trainingsdaten: Mails, manuell markiert:
  - D1: (“Die sieben Zwerge fraßen die sieben Ziegen”, OK)
  - D2: (“Die sieben Ziegen traten die sieben Wölfe”, SPAM)
  - D3: (“Die sieben Wölfe fraßen die sieben Böcke”, OK)
  - D4: (“Die sieben Böcke traten die sieben Zwerge”, OK)

- Testdaten (neue Mails):
  - T1: (“Die sieben Zwerge fraßen die sieben Wölfe”)

Lernen Sie mit Hilfe der Trainingsmenge einen Naive-Bayes-Klassifikator und wenden Sie diesen auf die Test-Dokumente an.

8.3.2. Vorverarbeitung

- Vereinheitlichung: Kleinbuchstaben
- **Tokenisierung**: Aufteilung auf einzelne Wörter

  Manchmal gar nicht so einfach: Wie viele Wörter sind "aren't"?
- **Stop Words** entfernen: "der", "die", "das", …

  Artikel und Füllwörter enthalten oft keine wirkliche Information für die Klassifikationsaufgabe und werden deshalb entfernt.

  Dabei am besten auch Zeichensetzung mit entfernen.
- **Lemmatisierung**: "fraßen" → "fressen", "Wölfe" → "Wolf", …

  Die Wörter werden auf ihre "Grundform" gebracht: gebeugte Verben durch den Infinitiv ersetzt oder Plurale durch den Singular ersetzt etc. Erfolgt häufig mit Hilfe von Wörterbüchern.
- **Synonyme, Homonyme, Akronyme** ersetzen

  Synonyme: Verschiedene Wörter mit selber/ähnlicher Bedeutung: "Fahrstuhl", "Lift"

  Homonyme: Wort mit unterschiedlicher Bedeutung: "Bank" (Sitzgelegenheit, Geldinstitut)

  Akronyme: Abkürzungen: "SPO"
- **Stemming**

  Wörter auf ihre "Grundform" zurückbringen durch Abschneiden: Aus "closed" oder "closing" würde "clos" als gemeinsame Form. Eine etwas krude Variante der Lemmatisierung.
- **Terme**: Die erhaltenen Wörter nennen wir "Terme".

8.3.3. Merkmalsvektor: Bag of Words

Nach Vorverarbeitung:

- Trainingsdaten (**Terme**):
  - D1: (sieben, zwerg, fressen, sieben, ziege; OK)
  - D2: (sieben, ziege, treten, sieben, wolf; SPAM)
  - D3: (sieben, wolf, fressen, sieben, bock; OK)
  - D4: (sieben, bock, treten, sieben, zwerg; OK)
- Testdaten (**Terme**):
  - T1: (sieben, zwerg, fressen, sieben, wolf)

Vokabular: V = {sieben, zwerg, fressen, ziege, treten, wolf, bock}

Bag of Words: Merkmalsvektor mit fester Anordnung des Vokabulars

8.3.4. Bag of Words für Trainingsdaten

	sieben	zwerg	fressen	ziege	treten	wolf	bock	Klasse
D1	2	1	1	1	0	0	0	OK
D2	2	0	0	1	1	1	0	SPAM
D3	2	0	1	0	0	1	1	OK
D4	2	1	0	0	1	0	1	OK

Im *Bag of Words* (BoW) bekommt jeder Term des Vokabulars einen festen Platz. Die Trainingsdaten und die Testdaten können nun mit Hilfe dieses Vektors dargestellt werden, indem für jeder Term im BoW die Anzahl der Vorkommen im Trainings- oder Testvektor gezählt wird ("*Multinomial NB*").

Es gibt auch die einfachere Form, die lediglich das Vorkommen vermerkt, also mit 0 und 1 arbeitet und nicht durchzählt ("*Bernoulli NB*", s.u.).

8.3.5. Naive Bayes Training (Multinomial NB)

Bei Multinomial NB zählen wir die Häufigkeiten der Vorkommen der einzelnen Terme in den einzelnen Dokumenten der jeweiligen Klasse.

Nachfolgend sind die entsprechenden Häufigkeiten im Trainingsdatensatz zusammengefasst dargestellt.

Klasse	sieben	zwerg	fressen	ziege	treten	wolf	bock	Anzahl Wörter	Anzahl Dokumente
OK	6	2	2	1	1	1	2	15	3
SPAM	2	0	0	1	1	1	0	5	1

Für die Terme ist jeweils die Gesamt-Anzahl des Vorkommens des Terms in den Dokumenten der jeweiligen Klasse angegeben.

Naive Bayes "trainieren"

- A-priori-Wahrscheinlichkeit der Klassen: $P(c) = \frac{N_c}{N} = \frac{\text{Anzahl Dokumente in Klasse c}}{\text{Anzahl Dokumente}}$
- Likelihood der Daten (Terme): $P(t \mid c) = \frac{\text{count}(t, c)}{\sum_{v \in V} \text{count}(v, c)}$
  mit count(t, c) Anzahl der Vorkommen von Term t in allen Dokumenten der Klasse c und V die Vereinigung aller Terme aller Dokumente (Vokabular)
- A-priori-Wahrscheinlichkeit:
  - $P(\text{OK}) = 3/4 = 0.75$

  - $P(\text{SPAM}) = 1/4 = 0.25$
- Likelihood:
  - $P(\text{sieben} \mid \text{OK}) = (2+2+2)/(2+2+2+1+1+1+1+1+1+1+1+1) = 6/15 = 0.40$
  - $P(\text{sieben} \mid \text{SPAM}) = (2)/(2+1+1+1) = 2/5 = 0.40$
  - $P(\text{zwerg} \mid \text{OK}) = 2/15 = 0.133$
  - $P(\text{zwerg} \mid \text{SPAM}) = 0/5 = 0.00$
  - $P(\text{fressen} \mid \text{OK}) = 2/15 = 0.133$
  - $P(\text{fressen} \mid \text{SPAM}) = 0/5 = 0.00$
  - $P(\text{ziege} \mid \text{OK}) = 1/15 = 0.067$
  - $P(\text{ziege} \mid \text{SPAM}) = 1/5 = 0.20$
  - $P(\text{treten} \mid \text{OK}) = 1/15 = 0.067$
  - $P(\text{treten} \mid \text{SPAM}) = 1/5 = 0.20$
  - $P(\text{wolf} \mid \text{OK}) = 1/15 = 0.067$
  - $P(\text{wolf} \mid \text{SPAM}) = 1/5 = 0.20$
  - $P(\text{bock} \mid \text{OK}) = 2/15 = 0.133$
  - $P(\text{bock} \mid \text{SPAM}) = 0/5 = 0.00$

8.3.6. Naive Bayes Klassifikation (Multinomial NB)

$h_{MAP} = \text{argmax}_{h \in H}\ P(h) \prod_{x \in \mathbf{x}} P(x \mid h)$

Jedes Vorkommen eines Wortes im Testdatensatz ist ein x!

$$h_{MAP} = \underset{h \in H}{\text{argmax}}\ P(h) \prod_{w \in \mathbf{V}\colon \text{count}(w) > 0} P(w \mid h)^{\text{count}(w)}$$

T1: (sieben, zwerg, fressen, sieben, wolf)

- **h = OK**: $P(\text{OK}) \cdot P(\text{sieben} \mid \text{OK})^2 \cdot P(\text{zwerg} \mid \text{OK})^1 \cdot P(\text{fressen} \mid \text{OK})^1 \cdot P(\text{ziege} \mid \text{OK})^0 \cdot P(\text{treten} \mid \text{OK})^0 \cdot P(\text{wolf} \mid \text{OK})^1 \cdot P(\text{bock} \mid \text{OK})^0 = 0.75 * 0.40 * 0.40 * 0.133 * 0.133 * 1 * 1 * 0.067 * 1 = 0.00014221956$
- **h = SPAM**: $P(\text{SPAM}) \cdot P(\text{sieben} \mid \text{SPAM})^2 \cdot P(\text{zwerg} \mid \text{SPAM})^1 \cdot P(\text{fressen} \mid \text{SPAM})^1 \cdot P(\text{ziege} \mid \text{SPAM})^0 \cdot P(\text{treten} \mid \text{SPAM})^0 \cdot P(\text{wolf} \mid \text{SPAM})^1 \cdot P(\text{bock} \mid \text{SPAM})^0 = 0.25 * 0.40 * 0.40 * 0.00 * 0.00 * 1 * 1 * 0.20 * 1 = 0.00$

Entscheidung: OK

> **Tip**
>
> **Hinweis**: $P(w \mid h)^{\text{count}(w)}$ muss so gelesen werden:
>
> - Wenn ein Term w ein- oder mehrfach im Testvektor vorkommt, wird die zugehörige Likelihood entsprechend ein- bzw. mehrfach multipliziert.
> - Wenn ein Term w nicht im Testvektor vorhanden ist ($\text{count}(w) = 0$), verschwindet die Likelihood und es wird automatisch ein Faktor 1 aufmultipliziert. Alternativ lässt man alle Terme mit $\text{count}(w) = 0$

> einfach in der Formel weg.
>
> Nur tatsächlich vorhandene Terme im Testvektor (count$(w) > 0$) zahlen in das Ergebnis ein.
> Wenn die Likelihood eines Terms 0 ist (Zero-Frequency-Problem), dann wird der gesamte Ausdruck zu 0. Durch die beschriebene Lesart von $P(w \mid h)^{\text{count}(w)}$ ergibt sich dann bei count$(w) = 0$ auch kein 0^0-Problem, weil die Likelihood gar nicht erst ausgewertet wird bzw. automatisch mit 1 ersetzt wurde. Dies ist eine Art Implementierungsdetail (rein mathematisch lässt sich das so nicht ablesen)!

Beobachtungen:

1. Viele kleine Faktoren ergeben ein immer kleineres Produkt → Logarithmus nutzen und Übergang zu Summe statt Produkt!
2. Im Trainingsdatensatz nicht vorhandene Terme bei einer Klasse führen zu einem Faktor 0 (Zero-Frequency-Problem), wodurch der gesamte Score 0 wird. → Laplace-Glättung einsetzen!
3. Im Testdatensatz nicht vorhandene Terme des Vokabulars werden automatisch ignoriert, da der *count* 0 ist und als Exponent verwendet den neutralen Faktor 1 ergibt.

8.3.7. Naive Bayes mit Laplace-Glättung (Multinomial NB)

$$P(t \mid c) = \frac{\text{count}(t, c) + \alpha}{\sum_{v \in V} \text{count}(v, c) + \alpha \cdot |V|}, \qquad \text{bei Laplace: } \alpha = 1$$

- A-priori-Wahrscheinlichkeit:
  - $P(\text{OK}) = 3/4 = 0.75$
  - $P(\text{SPAM}) = 1/4 = 0.25$
- Likelihood:
  - $P(\text{sieben} \mid \text{OK}) = (2+2+2+\mathbf{1})/(2+2+2+1+1+1+1+1+1+1+1+1+1+\mathbf{7}) = 7/22 = 0.318$ (vorher: 0.40)
  - $P(\text{sieben} \mid \text{SPAM}) = (2+\mathbf{1})/(2+1+1+1+\mathbf{7}) = 3/12 = 0.25$ (vorher: 0.40)
  - $P(\text{zwerg} \mid \text{OK}) = (2+\mathbf{1})/(15+\mathbf{7}) = 0.136$ (vorher: 0.133)
  - $P(\text{zwerg} \mid \text{SPAM}) = (0+\mathbf{1})/(5+\mathbf{7}) = 0.083$ (vorher: **0.00**)
  - $P(\text{fressen} \mid \text{OK}) = (2+\mathbf{1})/(15+\mathbf{7}) = 0.136$ (vorher: 0.133)
  - $P(\text{fressen} \mid \text{SPAM}) = (0+\mathbf{1})/(5+\mathbf{7}) = 0.083$ (vorher: **0.00**)
  - $P(\text{ziege} \mid \text{OK}) = (1+\mathbf{1})/(15+\mathbf{7}) = 0.091$ (vorher: 0.067)
  - $P(\text{ziege} \mid \text{SPAM}) = (1+\mathbf{1})/(5+\mathbf{7}) = 0.167$ (vorher: 0.20)
  - $P(\text{treten} \mid \text{OK}) = (1+\mathbf{1})/(15+\mathbf{7}) = 0.091$ (vorher: 0.067)
  - $P(\text{treten} \mid \text{SPAM}) = (1+\mathbf{1})/(5+\mathbf{7}) = 0.167$ (vorher: 0.20)
  - $P(\text{wolf} \mid \text{OK}) = (1+\mathbf{1})/(15+\mathbf{7}) = 0.091$ (vorher: 0.067)
  - $P(\text{wolf} \mid \text{SPAM}) = (1+\mathbf{1})/(5+\mathbf{7}) = 0.167$ (vorher: 0.20)
  - $P(\text{bock} \mid \text{OK}) = (2+\mathbf{1})/(15+\mathbf{7}) = 0.136$ (vorher: 0.133)
  - $P(\text{bock} \mid \text{SPAM}) = (0+\mathbf{1})/(5+\mathbf{7}) = 0.083$ (vorher: **0.00**)

T1: (sieben, zwerg, fressen, sieben, wolf)

- **h = OK**: $P(\text{OK}) \cdot P(\text{sieben} \mid \text{OK})^2 \cdot P(\text{zwerg} \mid \text{OK})^1 \cdot P(\text{fressen} \mid \text{OK})^1 \cdot P(\text{ziege} \mid \text{OK})^0 \cdot P(\text{treten} \mid \text{OK})^0 \cdot P(\text{wolf} \mid \text{OK})^1 \cdot P(\text{bock} \mid \text{OK})^0 = 0.75 * 0.318 * 0.318 * 0.136 * 0.136 * 1 * 1 * 0.091 * 1 = 0.0001276540836$ (vorher: 0.00014221956)
- **h = SPAM**: $P(\text{SPAM}) \cdot P(\text{sieben} \mid \text{SPAM})^2 \cdot P(\text{zwerg} \mid \text{SPAM})^1 \cdot P(\text{fressen} \mid \text{SPAM})^1 \cdot P(\text{ziege} \mid \text{SPAM})^0 \cdot P(\text{treten} \mid \text{SPAM})^0 \cdot P(\text{wolf} \mid \text{SPAM})^1 \cdot P(\text{bock} \mid \text{SPAM})^0 = 0.25 * 0.25 * 0.25 * 0.083 * 0.083 * 1 * 1 * 0.167 * 1 = 0.00001797598438$ (vorher: **0.00**)

Entscheidung: OK

Beobachtungen:

Laplace-Glättung führt zu leicht veränderten Schätzungen, die Berechnung der Scores erscheint aber plausibler (keine Probleme durch Zero-Frequency-Terme im Training mehr).

8.3.8. Naive Bayes mit Out-of-Vocabulary-Termen (OOV, Multinomial NB)

Was passiert, wenn bei der Klassifikation eines Testvektors Wörter vorkommen, die es beim Training nicht gab?

T1: (sieben, zwerg, fressen, sieben, wolf, **lecker**)

- **h = OK**: $P(\text{OK}) \cdot P(\text{sieben} \mid \text{OK})^2 \cdot P(\text{zwerg} \mid \text{OK})^1 \cdot P(\text{fressen} \mid \text{OK})^1 \cdot P(\text{ziege} \mid \text{OK})^0 \cdot P(\text{treten} \mid \text{OK})^0 \cdot P(\text{wolf} \mid \text{OK})^1 \cdot P(\text{bock} \mid \text{OK})^0 \cdot \mathbf{P}(\text{lecker} \mid \text{OK})^{\mathbf{1}}$
- **h = SPAM**: $P(\text{SPAM}) \cdot P(\text{sieben} \mid \text{SPAM})^2 \cdot P(\text{zwerg} \mid \text{SPAM})^1 \cdot P(\text{fressen} \mid \text{SPAM})^1 \cdot P(\text{ziege} \mid \text{SPAM})^0 \cdot P(\text{treten} \mid \text{SPAM})^0 \cdot P(\text{wolf} \mid \text{SPAM})^1 \cdot P(\text{bock} \mid \text{SPAM})^0 \cdot \mathbf{P}(\text{lecker} \mid \text{SPAM})^{\mathbf{1}}$

$\mathbf{P}(\text{lecker} \mid \text{OK})$ und $\mathbf{P}(\text{lecker} \mid \text{SPAM})$ sind **unbekannt**!

Möglichkeiten:

1. Da im Training nicht vorgekommen: 0 als Wert für $P(\text{lecker} \mid c)$ annehmen
2. Term ignorieren
3. Ad-hoc Laplace einführen
4. Mit UNK-Term arbeiten im Training/Klassifikation

Wenn wir während der Klassifikation mit einem Term konfrontiert werden, der beim Training nicht aufgetaucht ist, bekommen wir ein Problem. Potentiell sind die vier oben dargestellten Möglichkeiten als Lösung denkbar. Diese haben aber unterschiedliche Auswirkungen:

- Problem bei (1): Sämtliche Scores werden zu 0, d.h. wir können uns nicht entscheiden.
- Problem bei (2): Der Term wurde durch die Vorverarbeitung nicht entfernt, d.h. er trägt höchstwahrscheinliche eine Bedeutung. Ihn einfach zu ignorieren würde bedeuten, dass wir diese Information wegwerfen und möglicherweise eine Fehlklassifikation durchführen.
- Problem bei (3): In sämtlichen Likelihoods steckt die Größe des Vokabulars drin - wir müssten also während der Klassifikation ad hoc alle Likelihoods neu berechnen!
- Variante (4) ist dagegen relativ elegant …

8.3.9. Naive Bayes mit UNK-Term (OOV, Multinomial NB)

Training

Wir führen einen zusätzlichen Term **UNK** (“unknown”) ein und führen das Training damit durch.

Achtung: Das Vokabular wird dadurch um einen Eintrag größer - alle Likelihoods müssen entsprechend angepasst werden!

Klasse	sieben	zwerg	fressen	ziege	treten	wolf	bock	UNK	Anzahl Wörter	Anzahl Dokumente
OK	6	2	2	1	1	1	2	0	15	3
SPAM	2	0	0	1	1	1	0	0	5	1

- A-priori-Wahrscheinlichkeit:
  - $P(\text{OK}) = 3/4 = 0.75$
  - $P(\text{SPAM}) = 1/4 = 0.25$
- Likelihood:
  - $P(\text{sieben} \mid \text{OK}) = (2+2+2+1)/(2+2+2+1+1+1+1+1+1+1+1+1+\mathbf{8}) = 7/23 = 0.304$ (vorher: 0.318)
  - $P(\text{sieben} \mid \text{SPAM}) = (2+1)/(2+1+1+1+\mathbf{8}) = 3/13 = 0.231$ (vorher: 0.25)
  - $\ldots$
  - $P(\text{UNK} \mid \text{OK}) = (0+1)/(15+\mathbf{8}) = 0.043$
  - $P(\text{UNK} \mid \text{SPAM}) = (0+1)/(5+\mathbf{8}) = 0.077$

Klassifikation

Unbekannte Terme werden auf **UNK** gemappt. Dabei für die Exponenten wieder die Anzahl beachten!

T1: (sieben, zwerg, fressen, sieben, wolf, **lecker**)

- **h = OK**: $P(\text{OK}) \cdot P(\text{sieben} \mid \text{OK})^2 \cdot P(\text{zwerg} \mid \text{OK})^1 \cdot P(\text{fressen} \mid \text{OK})^1 \cdot P(\text{ziege} \mid \text{OK})^0 \cdot P(\text{treten} \mid \text{OK})^0 \cdot P(\text{wolf} \mid \text{OK})^1 \cdot P(\text{bock} \mid \text{OK})^0 \cdot \mathbf{P}(\text{lecker} \mid \text{OK})^{\mathbf{1}}$
- **h = SPAM**: $P(\text{SPAM}) \cdot P(\text{sieben} \mid \text{SPAM})^2 \cdot P(\text{zwerg} \mid \text{SPAM})^1 \cdot P(\text{fressen} \mid \text{SPAM})^1 \cdot P(\text{ziege} \mid \text{SPAM})^0 \cdot P(\text{treten} \mid \text{SPAM})^0 \cdot P(\text{wolf} \mid \text{SPAM})^1 \cdot P(\text{bock} \mid \text{SPAM})^0 \cdot \mathbf{P}(\text{lecker} \mid \text{SPAM})^{\mathbf{1}}$

Mapping:

- $P(\text{lecker} \mid \text{OK}) \to P(\text{UNK} \mid \text{OK}) = 0.043$
- $P(\text{lecker} \mid \text{SPAM}) \to P(\text{UNK} \mid \text{SPAM}) = 0.077$

Mit dem UNK-Term kann man das Problem des Out-of-Vocabulary elegant lösen. Im Training berechnet man die Likelihood für diesen Term normal mit. In der Klassifikation werden Terme in den Daten, die nicht im Vokabular sind, auf diesen UNK-Term gemappt und dessen Likelihood entsprechend verwendet.

Erweiterung: Der Bag-of-Words-Ansatz führt sehr schnell zu riesigen Vektoren, die nur dünn besetzt sind. Um diesem Problem ein wenig entgegen zu wirken, führt man häufig bereits beim Training eine Schwelle für die Häufigkeit der Terme der Trainingsmenge ein: Terme, die in ihrer Häufigkeit über der Schwelle liegen, werden normal für die Schätzung verwendet. Terme, die in ihrer Häufigkeit unter der Schwelle liegen, werden auf den UNK-Term abgebildet und dort mitgezählt. Hier macht man sich das Zipf'sche Gesetz zunutze: Wenige Wörter kommen viel häufiger vor als andere, und die anderen Wörter sind sehr selten - wobei man sich durchaus darüber streiten kann, ob die seltenen Wörter eher eine Art Rauschen darstellen oder vielleicht gerade durch ihre Seltenheit mit zur Trennschärfe beitragen können.

8.3.10. Naive Bayes Training (Bernoulli NB, mit Laplace)

Bei Bernoulli NB prüfen wir das Vorkommen der einzelnen Terme. Der Merkmalsraum ist also binär - 0 oder 1.

Im Grunde haben wir hier die selben Varianten wie bei Multinomial NB - ich zeige hier nachfolgend nur die Kombination von Bernoulli NB und Laplace-Glättung. Für "ohne" Laplace kann man einfach das $\alpha = 0$ setzen. Analog zu der Multinomial NB Variante kann man auch bei Bernoulli NB mit UNK-Termen arbeiten.

	sieben	zwerg	fressen	ziege	treten	wolf	bock	Klasse
D1	1	1	1	1	0	0	0	OK
D2	1	0	0	1	1	1	0	SPAM
D3	1	0	1	0	0	1	1	OK
D4	1	1	0	0	1	0	1	OK

Zusammengefasst ergibt sich damit pro Klasse:

Klasse	sieben	zwerg	fressen	ziege	treten	wolf	bock	Anzahl Dokumente
OK	3	2	2	1	1	1	2	3
SPAM	1	0	0	1	1	1	0	1

Für die Terme ist jeweils die Gesamt-Anzahl der Dokumente mit dem Term und der jeweiligen Klasse angegeben.

Naive Bayes "trainieren"

- A-priori-Wahrscheinlichkeit der Klassen: $P(c) = \frac{N_c}{N} = \frac{\text{Anzahl Dokumente in Klasse c}}{\text{Anzahl Dokumente}}$
- Likelihood der Daten (Terme):
  - $P(t = 1 \mid c) = \frac{\text{Anzahl Dokumente in Klasse c mit Term t} + \alpha}{\text{Anzahl Dokumente in Klasse c} + 2 \cdot \alpha}$
  - $P(t = 0 \mid c) = 1 - P(t = 1 \mid c)$

- A-priori-Wahrscheinlichkeit:
  - $P(\text{OK}) = 3/4 = 0.75$
  - $P(\text{SPAM}) = 1/4 = 0.25$
- Likelihood $\alpha = 1$:
  - $P(\text{sieben} = 1 \mid \text{OK}) = (3+1)/(3+2) = 4/5 = 0.80$
  - $P(\text{sieben} = 1 \mid \text{SPAM}) = (1+1)/(1+2) = 2/3 = 0.667$
  - $P(\text{zwerg} = 1 \mid \text{OK}) = 3/5 = 0.60$
  - $P(\text{zwerg} = 1 \mid \text{SPAM}) = 1/3 = 0.333$
  - $P(\text{fressen} = 1 \mid \text{OK}) = 3/5 = 0.60$
  - $P(\text{fressen} = 1 \mid \text{SPAM}) = 1/3 = 0.333$
  - $P(\text{ziege} = 1 \mid \text{OK}) = 2/5 = 0.40$
  - $P(\text{ziege} = 1 \mid \text{SPAM}) = 2/3 = 0.667$
  - $P(\text{treten} = 1 \mid \text{OK}) = 2/5 = 0.40$
  - $P(\text{treten} = 1 \mid \text{SPAM}) = 2/3 = 0.667$
  - $P(\text{wolf} = 1 \mid \text{OK}) = 2/5 = 0.40$
  - $P(\text{wolf} = 1 \mid \text{SPAM}) = 2/3 = 0.667$
  - $P(\text{bock} = 1 \mid \text{OK}) = 3/5 = 0.60$
  - $P(\text{bock} = 1 \mid \text{SPAM}) = 1/3 = 0.333$

Bei Bernoulli NB wird auch explizit die Wahrscheinlichkeit für die Abwesenheit eines Terms berechnet, dies ist einfach $P(t = 0 \mid c) = 1 - P(t = 1 \mid c)$. Der Übersichtlichkeit halber ist dies oben nicht explizit angegeben/ausgerechnet.

8.3.11. Naive Bayes Klassifikation (Bernoulli NB)

$h_{MAP} = \text{argmax}_{h \in H}\ P(h) \prod_{x \in \mathbf{x}} P(x \mid h)$

Für jedes Wort w im Vokabular V gibt es die Beobachtung (Bernoulli-Variable) $lw \in \{0, 1\}$. ($lw = 1$ wenn Wort w im Testdokument vorkommt, $lw = 0$ sonst.)

$$h_{MAP} = \underset{h \in H}{\text{argmax}}\ P(h) \prod_{w \in \mathbf{V}} P(lw = 1 \mid h)^{lw} \cdot (1 - P(lw = 1 \mid h))^{1-lw}$$

Wenn ein Wort w im Testdatensatz vorkommt, wird es mit $P(lw = 1 \mid h)$ berücksichtigt. Sonst mit $P(lw = 0 \mid h) = (1 - P(lw = 1 \mid h))$.

T1: (sieben, zwerg, fressen, sieben, wolf)

- **h = OK**: $P(\text{OK}) \cdot P(\text{sieben} = 1 \mid \text{OK}) \cdot P(\text{zwerg} = 1 \mid \text{OK}) \cdot P(\text{fressen} = 1 \mid \text{OK}) \cdot P(\text{ziege} = 0 \mid \text{OK}) \cdot P(\text{treten} = 0 \mid \text{OK}) \cdot P(\text{wolf} = 1 \mid \text{OK}) \cdot P(\text{bock} = 0 \mid \text{OK}) = 0.75 * 0.80 * 0.60 * 0.60 * (1 - 0.40) * (1 - 0.40) * 0.40 * (1 - 0.60) = 0.0124416$

- **h = SPAM**: $P(\text{SPAM}) \cdot P(\text{sieben} = 1 \mid \text{SPAM}) \cdot P(\text{zwerg} = 1 \mid \text{SPAM}) \cdot P(\text{fressen} = 1 \mid \text{SPAM}) \cdot P(\text{ziege} = 0 \mid \text{SPAM}) \cdot P(\text{treten} = 0 \mid \text{SPAM}) \cdot P(\text{wolf} = 1 \mid \text{SPAM}) \cdot P(\text{bock} = 0 \mid \text{SPAM}) = 0.25 * 0.667 * 0.333 * 0.333 * (1 - 0.667) * (1 - 0.667) * 0.667 * (1 - 0.333) = 0.0009122091926$

Entscheidung: OK

Beobachtungen:

1. Viele kleine Faktoren ergeben ein immer kleineres Produkt → Logarithmus nutzen und Übergang zu Summe statt Produkt!
2. Im Trainingsdatensatz nicht vorhandene Terme bei einer Klasse führen zu einem Faktor 0 (Zero-Frequency-Problem), wodurch der gesamte Score 0 wird. → Laplace-Glättung einsetzen!
3. Im Testdatensatz nicht vorhandene Terme des Vokabulars werden automatisch korrekt mit $P(lw = 0 \mid h)$ bzw. $(1 - P(lw = 1 \mid h))$ berücksichtigt. (Multinomial NB würde diese Terme mit dem Faktor 1 ignorieren.)

8.3.12. Wrap-Up

- **Vorverarbeitung**: Tokenisierung, Vokabular, Bag of Words (BoW)
- **Multinomial NB**:
  - Zähle die Vorkommen eines Terms: $P(t \mid c) = \dfrac{\text{count}(t, c) + \alpha}{\sum_{v \in V} \text{count}(v, c) + \alpha \cdot |V|}$
  - Klassifikation mit $h_{MAP} = \text{argmax}_{h \in H} \; P(h) \prod_{w \in \mathbf{V}} P(w \mid h)^{\text{count}(w)}$
  - Im Test nicht vorhandene Terme t aus V werden ignoriert (Faktor 1)
- **Bernoulli NB**:
  - Prüfe das Vorkommen eines Terms: $P(t = 1 \mid c) = \dfrac{\text{Anzahl Dokumente in Klasse c mit Term t} + \alpha}{\text{Anzahl Dokumente in Klasse c} + 2 \cdot \alpha}$
  - Klassifikation mit $h_{MAP} = \text{argmax}_{h \in H} \; P(h) \prod_{w \in \mathbf{V}} P(lw = 1 \mid h)^{lw} \cdot (1 - P(lw = 1 \mid h))^{1-lw}$
  - Im Test nicht vorhandene Terme t aus V werden korrekt mit $P(lw = 0 \mid h)$ berücksichtigt

- Problem mit kleinen Faktoren: **Logarithmus**, Übergang zu Summe
- **Zero-Frequency-Problem**: Laplace-Glättung mit $\alpha = 1$
- Klassifikation von **Out-of-Vocabulary-Termen**: UNK-Term trainieren

Zum Nachlesen

Lesen Sie in (Russell und Norvig 2021) bitte den Abschnitt 12.6 “Naive Bayes Models”. Darüber hinaus ist Abschnitt “8.10 Der Naive Bayes Klassifikator” (Ertel 2025) sehr empfehlenswert.

Lernziele

- k2: Ich kann die verschiedenen Schritte der Vorverarbeitung erklären
- k2: Ich kann das Bag-of-Words-Modell erklären
- k3: Ich kann einen Text in das BoW-Modell überführen

- k3: Ich kann die bedingten Wahrscheinlichkeiten aus konkreten Trainingsdaten schätzen: Multinomial NB, Bernoulli NB, mit/ohne Laplace-Glättung
- k3: Ich kann die Klassifikation von Testdaten mit Naive Bayes durch Nutzung der geschätzten Wahrscheinlichkeiten durchführen: Multinomial NB, Bernoulli NB
- k3: Ich kann dem Out-of-Vocabulary-Problem durch das Arbeiten mit UNK-Termen entgegenwirken

9. Genetische Algorithmen

Lokale Suche mit Methoden, die der biologischen Evolution abgeschaut bzw. nachempfunden sind.

9.1. EA1: Einführung Evolutionäre Algorithmen

TL;DR

Lokale Suchverfahren: Nur das Ergebnis zählt!
Evolutionäre Algorithmen sind lokale Suchverfahren, wobei gleichzeitig an mehreren Stellen im Problemraum gesucht wird. Sie bedienen sich Mechanismen aus der Evolution: Es gibt eine Population von Individuen, die jedes das Problem kodieren ("vollständige Zustandsbeschreibung") und damit im Laufe der Suche zu einer möglichen Lösung werden können.

Videos

Vorlesung [YT], [HSBI]

9.1.1. Evolution sehr erfolgreich bei Anpassung

Quelle: Photo Evolution by Johannes Plenio on Unsplash.com (Unsplash License)

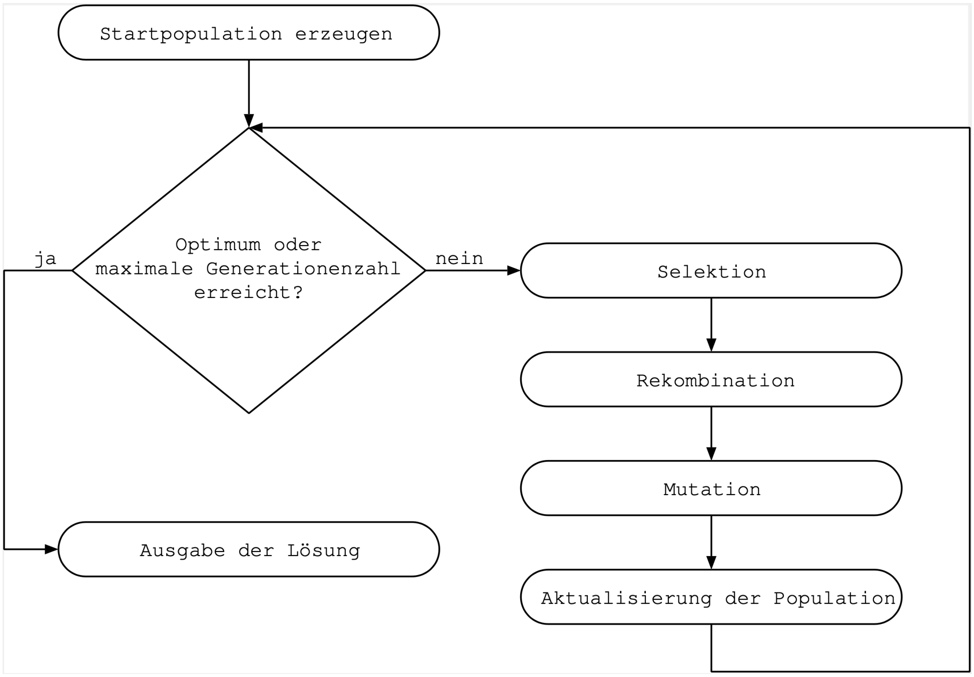
Wie funktioniert's?

9.1.2. EA – Zutaten und Mechanismen

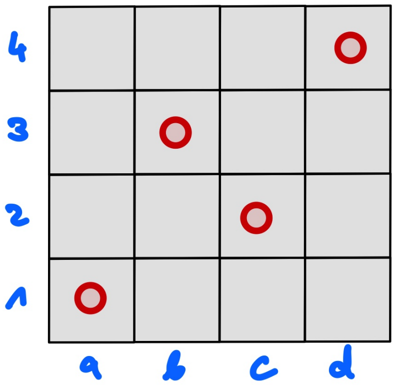
- Zutaten:
  - **Individuen**: Kodierung möglicher Lösungen
  - **Population** von Individuen
  - **Fitnessfunktion**: Bewertung der Angepasstheit

- Mechanismen ("Operatoren"):
  - Selektion
  - Rekombination (Crossover)
  - Mutation

9.1.3. EA – Allgemeiner Ablauf


Startpopulation erzeugen
Optimum oder maximale Generationenzahl erreicht?
ja
nein
Selektion
Rekombination
Mutation
Aktualisierung der Population
Ausgabe der Lösung

9.1.4. EA – Beispiel



Jedes Individuum kodiert ein Spielfeld mit einer konkreten Anordnung **aller** Königinnen → **Vollständige Zustandsbeschreibung**.

Dabei korrespondiert der Index in das Array des Individuums mit der jeweiligen Spalte des Spielfelds. Die Zahl an einer Arrayposition gibt dann an, in welcher Zeile in dieser Spalte eine Königin ist.

Crossover: Die ausgewählten Individuen werden an der selben Stelle aufgetrennt und die Hälften verkreuzt zu zwei neuen Individuen zusammengesetzt. Es entstehen zwei neue Anordnungen der Königinnen (zwei neue Spielfelder).

9.1.5. EA – Strömungen

1. **Genetische Algorithmen** (GA)
    - Holland (1975) (Holland (1992)) und Goldberg (1986)
    - Binäre Lösungsrepräsentation (Bitstring): $\mathbf{g} = (g_1, \ldots, g_m) \in \{0, 1\}^m$
    - Fitnessbasierte stochastische Selektion
    - μ Eltern erzeugen μ Kinder
2. **Evolutionsstrategien** (ES)
    - Rechenberg (1978) und Schwefel (1975) (Schwefel (1995))
    - Kodierung reellwertiger Parameter: $\mathbf{g} = (\mathbf{x}, \sigma)$ mit $\mathbf{x} = (x_1, \ldots, x_n) \in \mathbb{R}^n$
    - μ Eltern erzeugen λ Kinder mit $\mu \leq \lambda$
3. **Evolutionäre Programmierung** (EP)

Hinweis: Häufig finden sich Mischformen, beispielsweise GA mit reellwertigen Parametern

Hinweis: Im Folgenden werden **Genetische Algorithmen** (GA) betrachtet. Sie finden jeweils Hinweise auf die Gestaltung der Operatoren bei ES.

9.1.6. Anwendungsbeispiele für Evolutionäre Algorithmen

- Berechnung und Konstruktion komplexer Bauteile: beispielsweise Tragflächenprofile (Flugzeuge), Brücken oder Fahrzeugteile unter Berücksichtigung bestimmter Nebenbedingungen
- Scheduling-Probleme: Erstellung von Stunden- und Raumplänen oder Fahrplänen
- Berechnung verteilter Netzwerktopologien: Wasserversorgung, Stromversorgung, Mobilfunk
- Layout elektronischer Schaltkreise

9.1.7. Wrap-Up

Lokale Suchverfahren: Nur das Ergebnis zählt!

- Evolutionäre Algorithmen: Unterschied GA und ES (grober Überblick)

Zum Nachlesen

- Russell und Norvig (2021): GA: Abschnitt 4.1.4
- Weicker (2015)

Lernziele

- k2: Ich kann die Problematik der lokalen Minima bei Gradientenverfahren erklären
- k2: Ich kann einen Überblick über die verschiedenen Strömungen bei EA/GA darstellen
- k2: Ich kann den prinzipiellen Ablauf von Genetischen Algorithmen an einem Beispiel erklären

9.2. EA2: Modellierung mit Genetischen Algorithmen

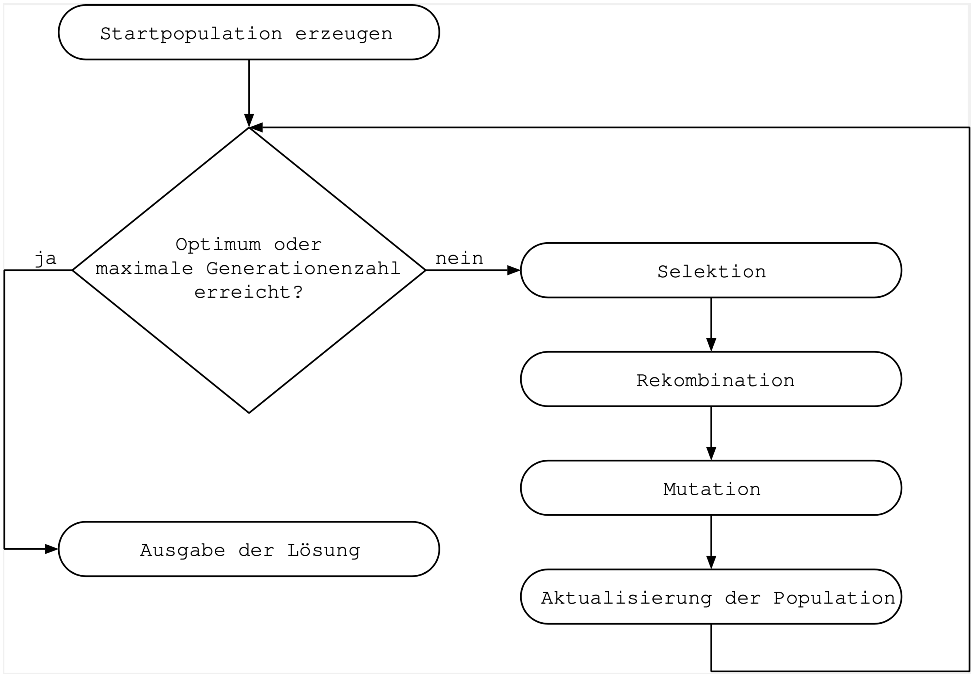
TL;DR

Lokale Suchverfahren: Nur das Ergebnis zählt!
Evolutionäre Algorithmen sind lokale Suchverfahren, wobei gleichzeitig an mehreren Stellen im Problemraum gesucht wird. Sie bedienen sich Mechanismen aus der Evolution: Es gibt eine Population von Individuen, die jedes das Problem kodieren ("vollständige Zustandsbeschreibung") und damit im Laufe der Suche zu einer möglichen Lösung werden können.
Die Individuen werden mit Hilfe einer Fitnessfunktion bewertet, wie gut sie bereits an das Problem angepasst sind (bzw. wie sehr sie bereits der gesuchten Lösung entsprechen). Über eine fitnessproportionale Selektion werden Individuen ausgewählt, aus denen mittels Rekombination (auch "Crossover" genannt) neue Individuen mit Eigenschaften der Eltern erzeugt werden. Über eine Mutation werden dann noch Elemente der neuen Individuen leicht verändert, bevor diese zur neuen Population werden …
Durch das Anwenden von Rekombination und Mutation springt man im Problemraum umher. Auch wenn als Basis die fitteren (angepassteren) Individuen dienen, kann es wie bei allen lokalen Suchverfahren vorkommen, dass sich der Algorithmus in lokalen Minima (bzw. lokalen Maxima, je nach Richtung der Optimierung) festfrisst.

Videos

Vorlesung [YT], [HSBI]

9.2.1. EA – Allgemeiner Ablauf


Startpopulation erzeugen
Optimum oder maximale Generationenzahl erreicht?
ja
nein
Selektion
Rekombination
Mutation
Aktualisierung der Population
Ausgabe der Lösung

9.2.2. Kodierung Individuen

- Binäre Lösungsrepräsentation (Bitstring): $\mathbf{g} = (g_1, \ldots, g_m) \in \{0,1\}^m$
    - String gliedert sich in n Elemente (mit $n \leq m$)
      $\rightarrow$ jedes Segment entspricht einer Problemvariablen
    - Dekodierungsfunktion $\Gamma : \{0,1\}^m \rightarrow \mathbb{R}^n$

  Alle relevanten Aspekte des Problems müssen in die Codierung einfließen!
  Bei ES hat man einen Vektor mit reellen Zahlen, wobei jeder Eintrag einen Parameter des Problems darstellt.
  Eine Dekodierungsfunktion benötigt man entsprechend nicht.
  Bei der Erzeugung der Startpopulation werden die Individuen **zufällig** (mit zufälligen Werten) initialisiert.

- Fitnessfunktion Φ ordnet jedem Individuum $\mathbf{g}_i$ eine reelle Zahl zu:

$$\Phi(\mathbf{g}_i) = F(\Gamma(\mathbf{g}_i)) - w \cdot \sum_j (Z_j(\Gamma(\mathbf{g}_i)))^2$$

    - Zielfunktion F: wie sehr genügt ein Individuum bereits dem Optimierungproblem
    - Strafterme Z_j: Anreicherung der Optimierung mit weiteren Informationen

  - Gewichte w: statisch oder dynamisch (Abkühlen)

  Die Wahl einer guten Fitnessfunktion ist oft eine Herausforderung, aber dennoch wichtig, da damit die Suche gesteuert wird!

9.2.3. Selektion: Erstelle Matingpool mit μ Individuen

- Fitnessproportionale Selektion (*Roulette Wheel Selection*):
  Auswahlwahrscheinlichkeit für Individuum $\mathbf{g}_k$:

  $$p_{sel}(\mathbf{g}_k) = \frac{\Phi(\mathbf{g}_k)}{\sum_j \Phi(\mathbf{g}_j)}$$

  $\rightarrow$ Voraussetzung: positive Fitnesswerte

- Turnier-Selektion (*Tournament Selection*):
  - Turniergröße ξ
  - Turnier: ziehe ξ Individuen gleichverteilt (mit Zurücklegen!) und kopiere fittestes Individuum in den Matingpool
  - Führe μ Turniere durch

Hinweis: Es gibt noch viele weitere Selektionsmechanismen. Die vorgestellten sind in der Praxis am gebräuchlichsten.

Über die Selektion wird der sogenannte "Selektionsdruck" aufgebaut: Wie gut muss ein Individuum sein (im Vergleich zu den restlichen Individuen in der Population), damit es eine Chance zur Reproduktion erhält? Dürfen sich nur die "Guten" fortpflanzen, oder erhalten auch die "Schlechten" eine gewisse Chance?

Da jedes Individuum einen Punkt im Suchraum darstellt, beeinflusst die Wahl der Selektion die Geschwindigkeit der Suche, begünstigt u.U. aber auch ein eventuelles Festfahren in lokalen Minima. Dies kann beispielsweise geschehen, wenn immer nur die "Guten" selektiert werden, aber die "Guten" der Population sich in der Nähe eines lokalen Minimums befinden. Dann werden auch die Nachfolger sich wieder dort aufhalten.

9.2.4. Crossover: Erzeuge zwei Nachkommen aus zwei Eltern

Festlegung der Crossover-Wahrscheinlichkeit p_{cross} (typisch: $p_{cross} \geq 0.6$)

1. Selektiere Eltern $\mathbf{g}_a$ und $\mathbf{g}_b$ **gleichverteilt** aus Matingpool

2. Zufallsexperiment:
   - mit $1 - p_{cross}$: Kinder identisch zu Eltern (kein Crossover)
   - mit p_{cross}: Crossover mit $\mathbf{g}_a$ und $\mathbf{g}_b$
     - Ziehe i gleichverteilt mit $1 < i < m$

- Kinder aus $\mathbf{g}_a$ und $\mathbf{g}_b$ zusammenbauen:

$$\mathbf{g}_c = (g_{a,1}, \ldots, g_{a,i},\ g_{b,i+1}, \ldots, g_{b,m})$$

und

$$\mathbf{g}_d = (g_{b,1}, \ldots, g_{b,i},\ g_{a,i+1}, \ldots, g_{a,m})$$

→ Trenne Eltern an gleicher Stelle auf, vertausche Bestandteile

3. Gehe zu Schritt 1, bis insg. μ Nachkommen

Anmerkung: Die Eltern werden jeweils in die Ausgangsmenge zurückgelegt.

Mit einer kleinen Wahrscheinlichkeit sind die Kinder also identisch zu den Eltern. Dies ist im Sinne der lokalen Suche wichtig, um bereits erreichte gute Positionen im Suchraum nicht zu verlieren: Es könnte sein, dass die Nachfolger alle schlechter sind …

Varianten: N-Punkt-Crossover, Shuffle-Crossover

Bei ES wird parameterweise gekreuzt. Dabei gibt es verschiedene Möglichkeiten: Übernahme eines Parameters von einem Elternteil, Verrechnen (beispielsweise Mitteln) der Werte beider Eltern, … Bei ES heißt "Crossover" deshalb oft "Rekombination".

9.2.5. Mutation

- Mutationswahrscheinlichkeit p_{mut}
  (typische Werte: $p_{mut} = 0.01$ oder $p_{mut} = 0.001$)

- Für alle Individuen:
  - Mutiere jedes Gen eines Individuums mit p_{mut}:

$$g_i^{(t+1)} = \begin{cases} \neg \quad g_i^{(t)} & \text{falls } \chi_i \leq p_{mut} \\ \quad g_i^{(t)} & \text{sonst} \end{cases}$$

→ χ_i gleichverteilte Zufallsvariable (Intervall $[0, 1]$), für jedes Bit g_i neu bestimmen

Anmerkung: Die optimale Mutationsrate p_{mut}^* ist von Länge m des Bitstrings abhängig; annäherbar durch $p_{mut}^* \approx 1/m$.

Die beim Crossover erstellten Nachfolger liegen im Suchraum in der Nähe der Eltern. Durch die Mutationsrate bestimmt man, ob und wie weit sich ein Kind entfernen kann. Dies entspricht dem Bild des "Schüttelns" der Zustandslandschaft.

Bei ES unterscheidet man Mutationswahrscheinlichkeit und Mutationsrate. Es wird parameterweise mutiert.

9.2.6. Bewertungskriterien

Vorsicht: Es handelt sich um Zufallsexperimente. Wenn man nicht nur direkt nach einer Lösung sucht, sondern beispielsweise Parametereinstellungen oder die Wahl der Fitnessfunktion für ein Problem vergleichen will, muss man jeweils mehrere Experimente mit der selben Einstellung machen und Kenngrößen berechnen.

Geschwindigkeit: AES *Average Evaluations to a Solution*

$$\text{AES} = \frac{\sum\limits_{i \in \text{erfolgreiche Läufe}} \text{Generationen von Lauf } i}{\text{Anzahl der erfolgreichen Läufe}}$$

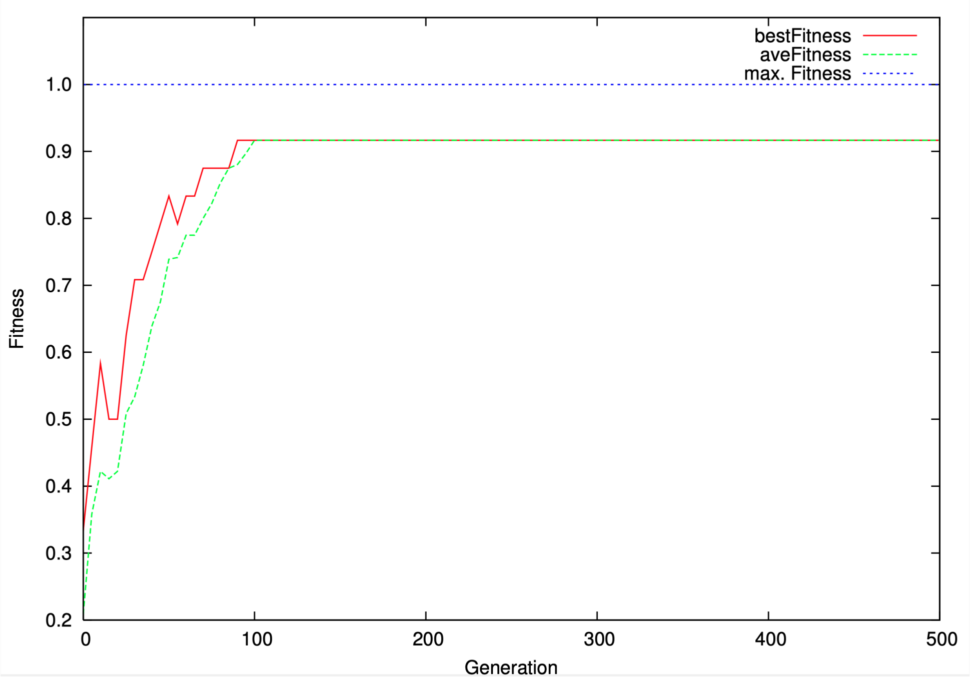
Die AES liegt im Intervall $[0, maxGen]$.

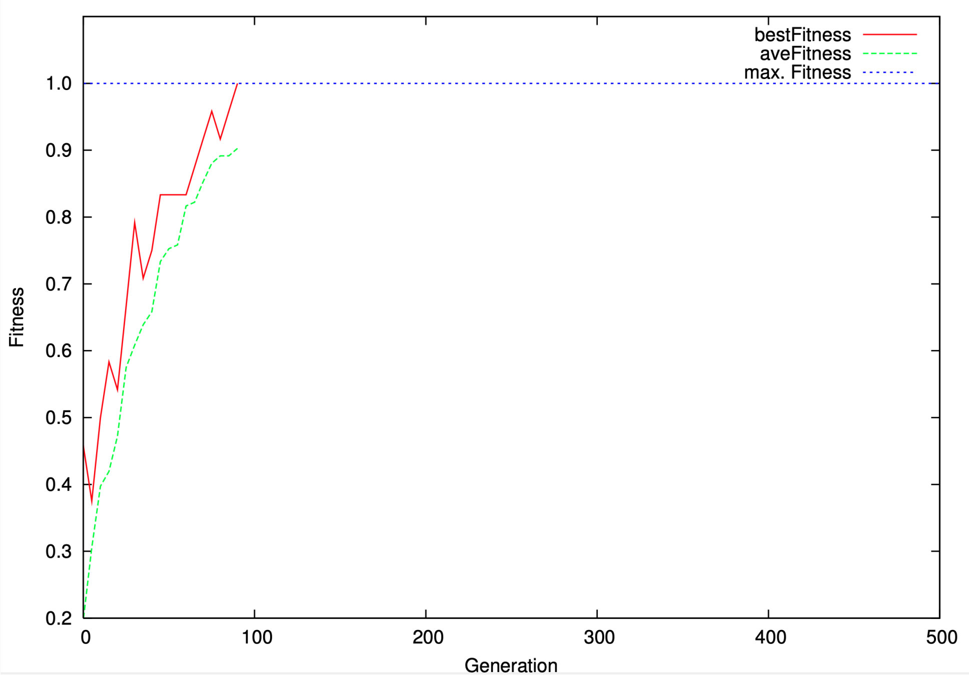
Lösungswahrscheinlichkeit: SR *Success Rate*

$$\text{SR} = \frac{\text{Anzahl der erfolgreichen Läufe}}{\text{Anzahl aller Läufe}}$$

Die SR liegt im Intervall $[0, 1]$.

9.2.7. Typische Läufe





- Populationsgröße $\mu = 15$
- Anzahl Nachfahren $\lambda = 100$
- Abbruch nach $maxGen = 200$ Generationen

Stochastischer Algorithmus! Ausreichend Wiederholungen durchführen und mitteln!

Hinweis: Die Parameter müssen problemabhängig gewählt werden. Zu hohe Werte für μ und λ führen dazu, dass man bei kleinen Problemen mit hoher Wahrscheinlichkeit bereits am Anfang eine Lösung "würfelt", also gar kein GA nutzt. Wenn dies allerdings nicht passiert, sorgt eine hohe Populationsgröße dafür, dass jeder Schritt sehr lange dauert. Die Abbruchgrenze ist ebenfalls mit Augenmaß zu wählen: Ein zu kleiner Wert sorgt für zu frühen Abbruch (keine Lösung!), ein zu hoher Wert sorgt beim Festfressen des Algorithmus für eine unnötige weitere "Suche" …

9.2.8. Wrap-Up

Lokale Suchverfahren: Nur das Ergebnis zählt!

- Evolutionäre Algorithmen:
  - Begriffe: Individuum, Population, Kodierung
  - Operationen: Selektion, Rekombination, Mutation
  - Bewertung mit Fitnessfunktion

Zum Nachlesen

- Russell und Norvig (2021): GA: Abschnitt 4.1.4
- Weicker (2015)

Lernziele

- k3: Ich kann GA anwenden, insbesondere für ein Beispiel passende Kodierung, Fitnessfunktion, Operatoren und Auswertung formulieren und den Ablauf erklären

Quizzes

- Selbsttest EA/GA (ILIAS)

Challenges

Sudoku
Ein 9×9-*Sudoku*-Rätsel soll mit einem GA gelöst werden.
Geben Sie für dieses Problem jeweils eine geeignete **Kodierung** der Individuen, passende Operatoren (**Crossover**, **Mutation**) und eine geeignete **Fitnessfunktion** an, damit das Problem mit einem GA gelöst werden kann. Begründen Sie Ihre Wahl!
Was würden Sie noch benötigen, um das Probleme mit Simulated Annealing lösen zu können?
Travelling Salesman Problem
Das *Travelling Salesman Problem* für 10 Städte, d.h. das Finden der kürzesten Route zwischen 10 Städten, soll mit einem GA gelöst werden.
Geben Sie für dieses Problem jeweils eine geeignete **Kodierung** der Individuen, passende Operatoren (**Crossover**, **Mutation**) und eine geeignete **Fitnessfunktion** an, damit das Problem mit einem GA gelöst werden kann. Begründen Sie Ihre Wahl!
Was würden Sie noch benötigen, um das Probleme mit Simulated Annealing lösen zu können?

Teil III.

Praktikum/Übung

Hier finden Sie die Übungsblätter.

Bitte bearbeiten Sie vor der jeweiligen Sprechstunde das bereitgestellte Material (Skripte, Videos, Quizzes, Literatur, …) und versuchen Sie vor der Sprechstunde das entsprechende Übungsblatt zu lösen, damit Sie in der der Sprechstunde Ihre Fragen zum Thema stellen können.

10. Übungsblatt: Problemlösen, Suche

10.1. Bonus: Möglichkeiten und Grenzen sowie Auswirkungen der KI (2P)

Recherchieren Sie, welche Probleme bereits mittels Computer- bzw. Robotereinsatz gelöst werden können und welche aktuell noch ungelöst sind.

Recherchieren und diskutieren Sie Auswirkungen auf die Gesellschaft durch die KI, etwa durch autonomes Fahren oder durch *Large Language Models* (LLM).

Hier ein paar Startpunkte:

- "Empirical evidence of Large Language Model's influence on human spoken communication" (https://arxiv.org/abs/2409.01754) - Gut? Schlecht? Egal?
- "AI and Cognitive Decline" (https://www.reddit.com/r/Professors/s/Otxpcpe65L)
- "Data Center the Size of Manhattan" (https://www.reddit.com/r/artificial/comments/1m1oup8/zuckerberg_says_meta_will_b
- "Mach dir selbst ein Video" (https://www.zeit.de/digital/2024-07/ki-filme-videos-newsletter-kuenstliche-intelligenz)
- "Police pulls over a driverless Waymo car" (https://www.reddit.com/r/Damnthatsinteresting/comments/1dvsod6/phoenix_pc
- "AI is making it up. Your authoritative source is the documentation" (https://github.com/jgm/pandoc/discussions/10994)

Thema: Gefühl für bereits realisierbare Aufgaben, Chancen und Risiken, Ethik

10.2. Search.01: Problemformalisierung, Zustandsraum (3P)

Drei Elben und drei Orks befinden sich an einem Ufer eines Flusses und wollen diesen überqueren. Es steht dazu ein Pferd zur Verfügung, welches maximal zwei Wesen tragen kann. Das Pferd kann den Fluss nicht allein überqueren.

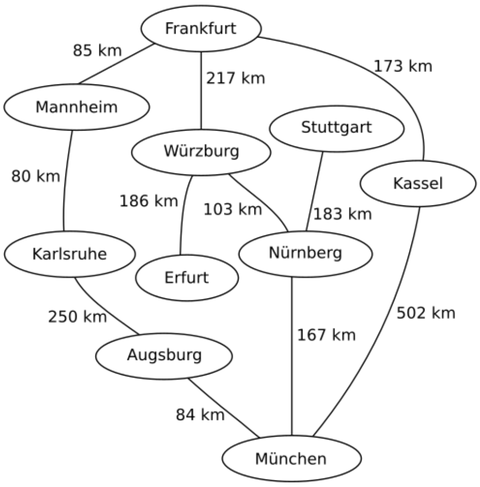
Gesucht ist eine Möglichkeit, alle Elben und Orks über den Fluss zu bringen. Dabei darf zu keiner Zeit an keinem Ufer die Anzahl der sich dort befindlichen Orks größer sein als die der dort wartenden Elben, da es sonst zu Konflikten zwischen beiden Gruppen kommt.

1. Formalisieren Sie das Problem (Zustände, Aktionen, Start- und Endzustand).
2. Skizzieren Sie den Problemgraph.

Thema: Formalisierung von Problemen, Problemgraph

10.3. Search.02: Suchverfahren (5P)

Betrachten Sie folgende Landkarte und Restwegschätzungen:


Frankfurt
85 km
217 km
173 km
Mannheim
Stuttgart
Würzburg
Kassel
80 km
186 km
103 km
183 km
Karlsruhe
Erfurt
Nürnberg
250 km
502 km
167 km
Augsburg
84 km
München

Quelle: MapGermanyGraph.svg by Regnaron and Jahobr on Wikimedia Commons (Public Domain)

Stadt	$h(n)$
Augsburg	0 km
Erfurt	400 km
Frankfurt	100 km
Karlsruhe	10 km
Kassel	460 km
Mannheim	200 km
München	0 km
Nürnberg	537 km
Stuttgart	300 km
Würzburg	170 km

Schätzungen der Restwegkosten für das Ziel *München*.

1. Finden Sie nacheinander mit Tiefensuche (1P) und Breitensuche (1P) (jeweils in der Graph-Search-Variante) sowie A* (2P) (in der Tree-Search-Variante mit der Verbesserung “keine Zyklen”, siehe Vorlesung) jeweils einen Weg von Würzburg nach München.

   Vergleichen Sie die drei Algorithmen: Wie viele Einträge gibt es in der Datenstruktur maximal, wie oft wird die Hauptschleife durchlaufen (also ein Element aus der Datenstruktur entnommen, untersucht und weiterentwickelt)?

   Sie können dafür eine Handsimulation anwenden oder die Algorithmen implementieren. Sie können gern auch die Java-Klassen im Paket `aima.core.search` bzw. die Python-Klassen in `search.py` als Ausgangspunkt nutzen.[1]

2. Dürfen die oben gegebenen Restkostenabschätzungen in A* verwendet werden? (1P)

---

[1] Im Python-Code tauchen immer wieder “TODO”-Marker auf - bitte mit Vorsicht genießen!

- Falls ja, warum?
- Falls nein, warum? Wie müssten die Abschätzungen ggf. korrigiert werden?

Falls Sie der Meinung waren, die Abschätzungen sind nicht korrekt, korrigieren Sie die Abschätzungen nun und führen Sie erneut eine Suche mit A* durch.

Hinweis: Reihenfolge bei gleichen $f(n)$-Kosten: alphabetische Reihenfolge, d.h. Mannheim käme vor München, Karlsruhe vor Kassel etc.

10.4. Search.03: Dominanz (1P)

Was bedeutet *"Eine Heuristik $h_1(n)$ dominiert eine Heuristik $h_2(n)$"*?

Wie wirkt sich die Nutzung einer dominierenden Heuristik $h_1(n)$ in A* aus (im Vergleich zur Nutzung einer Heuristik h_2, die von h_1 dominiert wird)?

Geben Sie selbstgewählte Beispiele an.

Thema: Begriff der dominierenden Heuristik (Selbststudium)

10.5. Search.04: Beweis der Optimalität von A* (1P)

Beweisen Sie, dass A* in der Tree-Search-Variante bei Nutzung einer zulässigen Heuristik optimal ist.

Thema: Bedeutung einer zulässigen Heuristik (Selbststudium)

11. Übungsblatt: Spiele

11.1. Bonus: Verlässlichkeit von ChatGPT (2P)

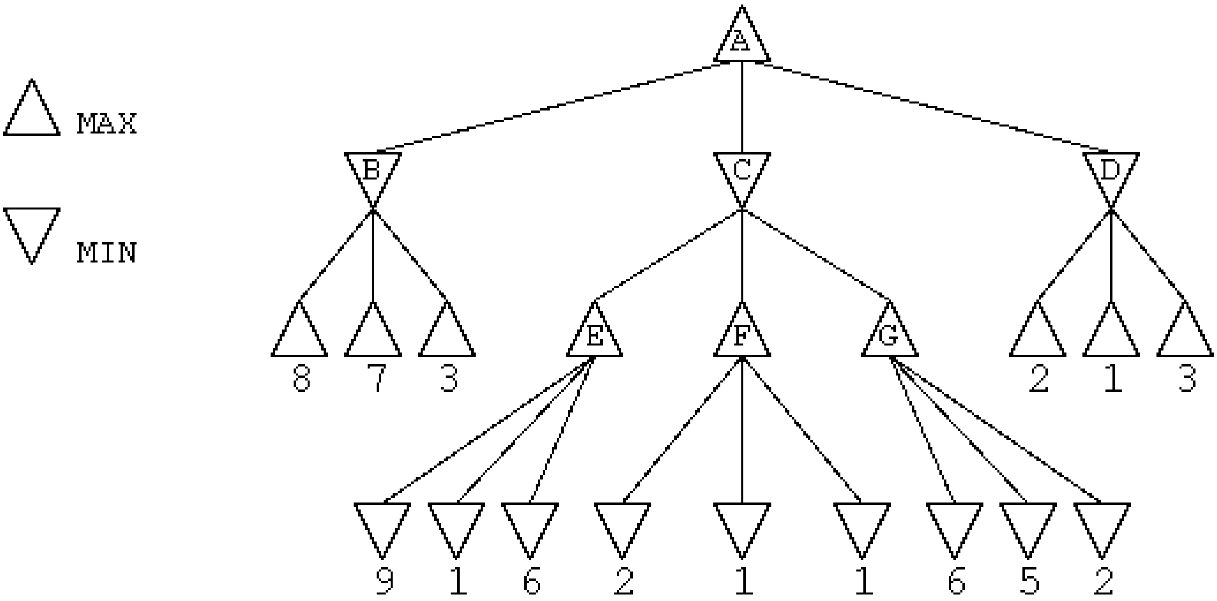
Vergleichen Sie Informationen aus zuverlässigen Quellen mit Informationen aus einem LLM. Beurteilen Sie, ob die vom Model generierte Information korrekt ist oder ob Dinge fehlen oder Halluzinationen vorliegen.

Beispiel I: Erstellung einer Zeitleiste eines historischen Ereignisses.

Beispiel II: Erstellen von Quellcode zu einem Standardproblem aus der Informatik.

Thema: Qualität und Zuverlässigkeit von generierten Inhalten

11.2. Games.01: Handsimulation: Minimax und alpha-beta-Pruning (3P)



1. (1P) Geben Sie für den Spielbaum die Minimax-Bewertungen an.
2. (1P) Markieren Sie die Kanten, die bei alpha-beta-Pruning nicht mehr untersucht werden würden, d.h. wo Pruning stattfinden würde. Geben Sie für jeden Knoten die (sich ändernden) α- und β-Werte an.
3. (1P) Können die Knoten derart geordnet werden, dass alpha-beta-Pruning eine größere Anzahl von Zweigen abschneidet? Wenn ja, geben Sie eine solche Ordnung an. Wenn nein, begründen Sie Ihre Antwort.

Hinweis: Reihenfolge der Abarbeitung der Kindknoten: Wie in der VL von links nach rechts.

Thema: Minimax und alpha-beta-Pruning

11.3. Games.02: Optimale Spiele: Minimax und alpha-beta-Pruning (4P)

1. (2P) Implementieren Sie den Minimax-Algorithmus (wie in der VL besprochen) am Beispiel *Tic Tac Toe* in einer Sprache Ihrer Wahl.
2. (1P) Ergänzen Sie Ihre Implementierung um alpha-beta-Pruning.
3. (1P) Vergleichen Sie die Anzahl der jeweils berechneten Knoten. Überlegen Sie sich dazu ein **sinnvolles** Szenario.

Thema: Anwendung Minimax und alpha-beta-Pruning

11.4. Games.03: Minimax vereinfachen (1P)

Vereinfachen Sie den Minimax-Algorithmus aus der Vorlesung, indem Sie die Eigenschaft *Nullsummenspiel* berücksichtigen und die Funktionen `Min-Value` und `Max-Value` in eine einzige Funktion ohne explizite Unterscheidung der Spieler zusammenfassen.

Überlegen Sie sich einen Beispielbaum und zeigen Sie anhand dessen die Bewertung durch den Minimax-Algorithmus und durch Ihren vereinfachten Algorithmus.

Thema: Nullsummenspiel, Minimax

11.5. Games.04: Suchtiefe begrenzen (1P)

Die Verwendung der Suchtiefenbeschränkung erfordert den Einsatz einer Evaluierungsfunktion.

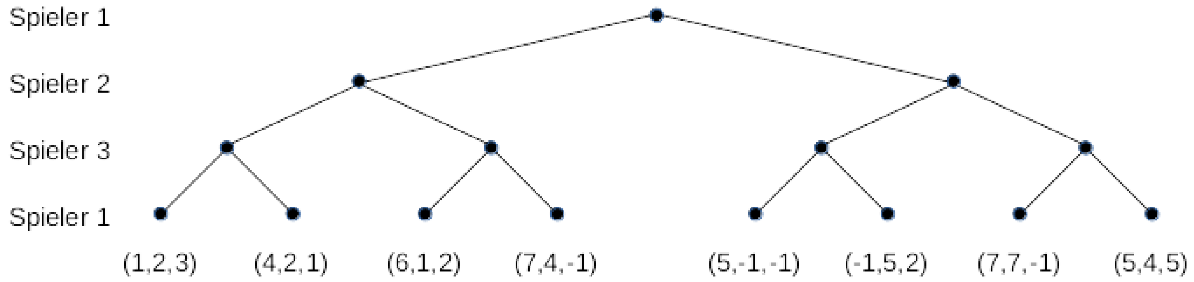
Betrachten Sie die auf https://github.com/aimacode/aima-exercises/blob/master/markdown/5-Adversarial-Search/exercises/ex_9/question.md gegebene Evaluierungsfunktion für *Tic-Tac-Toe*.

Geben Sie die Werte der Evaluierungsfunktion für sechs verschiedene Spielzustände an (3 Endzustände, 3 Zwischenzustände). Begründen Sie, warum diese Evaluierungsfunktion im Zusammenhang mit *Tic-Tac-Toe* sinnvoll sein kann.

Thema: Suchtiefenbegrenzung und Evaluierungsfunktion

11.6. Games.05: Minimax generalisiert (1P)

Betrachten Sie nun das Problem, den Spielbaum eines Drei-Personen-Spiels zu evaluieren, das nicht notwendigerweise die Nullsummenbedingung erfüllt.


Spieler 1
Spieler 2
Spieler 3
Spieler 1
(1,2,3)
(4,2,1)
(6,1,2)
(7,4,-1)
(5,-1,-1)
(-1,5,2)
(7,7,-1)
(5,4,5)

Die Spieler heißen 1, 2 und 3. Im Gegensatz zu Zwei-Personen-Nullsummenspielen liefert die Bewertungsfunktion nun Tripel (x_1, x_2, x_3) zurück, wobei x_i der Wert für Spieler i ist. Allianzen zwischen Spielern sind nicht erlaubt.

Vervollständigen Sie den Spielbaum, indem Sie alle inneren Knoten und den Wurzelknoten mit den entsprechenden Wert-Tripeln annotieren.

Thema: Minimax generalisiert für mehrere Spieler

12. Übungsblatt: Entscheidungsbäume (Decision Tree Learner DTL)

12.1. Starke vs. Schwache KI (1P)

Recherchieren Sie und diskutieren Sie folgende Punkte:

- Ist ein System wie ChatGPT "intelligent"? Was ist der Kern des Systems?
- Gibt es Systeme, die intelligent sind? Was tun diese, wie arbeiten sie?
- Brauchen wir wirklich Intelligenz in Systemen? Reicht auch schwache KI, d.h. reichen intelligent *wirkende* Systeme?
- Absicht vs. Auswirkung: vorteilhafte Anwendungen vs. Unfälle (Robustheit und falsche Korrelationen, Fairness, Sicherheit) vs. Missbrauch (Spam, Betrug, Spear-Phishing, Desinformation) vs. doppelte Verwendung ("*dual use*": Raketen, Kernkraft, Genbearbeitung, …) vs. *Bias* (Voreingenommenheit)

Thema: Schwache vs. starke KI, Auswirkungen und Nutzen

12.2. DTL.01: Entscheidungsbäume mit ID3 (6P)

Es ist wieder Wahlkampf: Zwei Kandidaten O und M bewerben sich um die Kanzlerschaft. Die folgende Tabelle zeigt die Präferenzen von sieben Wählern.

Nr.	Alter	Einkommen	Bildung	Kandidat
1	≥ 35	hoch	Abitur	O
2	< 35	niedrig	Master	O
3	≥ 35	hoch	Bachelor	M
4	≥ 35	niedrig	Abitur	M
5	≥ 35	hoch	Master	O
6	< 35	hoch	Bachelor	O
7	< 35	niedrig	Abitur	M

Trainieren Sie nacheinander mit dem Verfahren ID3 auf der obigen Trainingsmenge je einen Entscheidungsbaum. Nutzen Sie dafür eine Handsimulation.

12.3. DTL.03: Machine Learning mit Weka (3P)

Weka (waikato.github.io/weka-wiki/) ist eine beliebte Sammlung von (in Java implementierten) Algorithmen aus dem Bereich des Maschinellen Lernens. Laden Sie sich das Tool in der aktuellen stabilen Version herunter und machen Sie sich mit der beiliegenden Dokumentation vertraut.

Laden Sie sich die Beispieldatensätze “Zoo” (`zoo.csv`) und “Restaurant” (`restaurant.csv`) aus dem AIMA-Repository (github.com/aimacode/aima-data) herunter.[1] Zum Laden der Beispieldatensätze in Weka müssen die `.csv`-Dateien eine Kopfzeile mit den Namen der Attribute haben. Passen Sie die Dateien entsprechend an und laden Sie diese im Reiter “Pre-Process” mit “Open file …”.

Hinweis: Wenn Sie *Weka 3.6* einsetzen, sind alle für dieses Blatt erforderlichen Algorithmen bereits vorhanden. In neueren Versionen müssen Sie in der Weka-Haupt-GUI den Paketmanager unter “Tools” starten und dort nach einem Paket suchen, welches ID3 enthält, und dieses Paket nachinstallieren.

1. Training mit J48 (1P)

   Wechseln Sie auf den Reiter “Classify” und wählen Sie mit dem Button “Choose” den Entscheidungsbaum-Lerner J48 aus. (Dies ist eine Java-Implementierung von C4.5. Die ID3-Implementierung funktioniert für den `zoo.csv`-Datensatz leider nicht …)

   Lernen Sie für die beiden Datensätze je einen Entscheidungsbaum. Wie sehen die Bäume aus? Wie hoch ist jeweils die Fehlerrate für den Trainingssatz? (Stellen Sie unter “Test options” den Haken auf “Use training set”.) Interpretieren Sie die **Confusion Matrix**.

2. ARFF-Format (1P)

   Lesen Sie in der beiliegenden Doku zum Thema “ARFF” nach. Dabei handelt es sich um ein spezielles Datenformat, womit man Weka mitteilen kann, welche Attribute es gibt und welchen Typ diese haben und welche Werte auftreten dürfen. (Link)

   Erklären Sie die Unterschiede zwischen “nominal”, “ordinal” (bzw. “numeric”) und “string”.

   Konvertieren Sie den Zoo- und Restaurantdatensatz in das ARFF-Format. Beachten Sie, dass die ID3-Implementierung von Weka nicht mit bestimmten Attributtypen umgehen kann.

3. Training mit ID3 und J48 (1P)

   Trainieren Sie für die im letzten Schritt erstellten Datensätze (Zoo und Restaurant) im ARFF-Format erneut Entscheidungsbäume. Nutzen Sie diesmal sowohl ID3 als auch J48.

   Vergleichen Sie wieder die Ergebnisse (Entscheidungsbäume, Fehlerraten, Confusion Matrix) untereinander und mit den Ergebnissen aus dem J48-Lauf mit den `.csv`-Dateien.

Thema: Kennenlernen von Weka

---

[1] Zum Zoo-Datensatz gibt es die Erklärung direkt im Repo, für den Restaurant-Datensatz finden Sie die Erklärung im AIMA (Buch).

13. Übungsblatt: Perzeptron

13.1. NN.Perzeptron.01: Entscheidungsgrenze (2P)

- (1P) Betrachten Sie das durch den Gewichtsvektor $(w_0, w_1, w_2)^T = (2, 1, 1)^T$ gegebene Perzeptron. Zeichnen Sie die Trennebene und markieren Sie den Bereich, der mit $+1$ klassifiziert wird.
- (1P) Welche der folgenden Perzeptrons haben die selbe Trennebene? Welche weisen exakt die gleiche Klassifikation auf?
    - $(w_0, w_1, w_2)^T = (1, 0.5, 0.5)^T$
    - $(w_0, w_1, w_2)^T = (200, 100, 100)^T$
    - $(w_0, w_1, w_2)^T = (\sqrt{2}, \sqrt{1}, \sqrt{1})^T$
    - $(w_0, w_1, w_2)^T = (-2, -1, -1)^T$

Thema: Verständnis Interpretation Perzeptron (Trennebene/Entscheidungsgrenze)

13.2. NN.Perzeptron.02: Logische Funktionen als Perzeptron (2P)

- (1.5P) Das Perzeptron kann zur Ausführung zahlreicher logischer Funktionen verwendet werden. Konstruieren Sie jeweils ein Perzeptron zur Realisierung der binären Logikfunktionen UND, ODER und KOMPLEMENT und demonstrieren Sie die Wirkung der Perzeptrone in der Übung/im Praktikum. (Hinweis: Sie brauchen/sollen hier nichts implementieren oder trainieren!)
- (0.5P) Eine grundlegende Einschränkung des Perzeptrons besteht darin, dass es die EXKLUSIV-ODER-Funktion nicht implementieren kann. Erklären Sie den Grund für diese Einschränkung.

Thema: Verständnis Perzeptron

13.3. NN.Perzeptron.03: Perzeptron Lernalgorithmus (6P)

Ziel dieser Aufgabe ist es, mit Hilfe eines Experiments ein Gefühl für die Laufzeit des Perzeptron-Lernalgorithmus zu bekommen und eine Art empirische Approximation zu bestimmen.

13.3.1. Datensatz (1P)

- Konstruieren Sie Ihren eigenen Datensatz $\mathcal{D}$ mit $m = 10$ gleichförmig verteilten Zufallspunkten aus dem Bereich $\mathcal{X} = [-1, 1] \times [-1, 1]$.

- Wählen Sie auf ähnliche Weise zwei zufällige, gleichmäßig verteilte Punkte aus dem Bereich $[-1, 1] \times [-1, 1]$. Verwenden Sie die Gerade, die durch diese zwei Punkte verläuft, als die Entscheidungsgrenze Ihrer Zielfunktion f. Sie können die positiv beschriftete Seite beliebig festlegen.
- Werten Sie die Zielfunktion für jeden Datenpunkt $\mathbf{x}^{(j)}$ aus, um die entsprechenden Beschriftungen (Ausgangslabel) $y^{(j)}$ zu erhalten.

13.3.2. Training (3P)

Führen Sie nun den Perzeptron-Lernalgorithmus 1000 mal hintereinander aus. Initialisieren Sie jedes Mal die Gewichte mit 0. Wählen Sie in jedem Lernschritt einen Punkt $\mathbf{x}^{(i)}$ *zufällig* aus der Menge der falsch klassifizierten Punkte und aktualisieren Sie die Gewichte entsprechend der folgenden Formel:

$$\mathbf{w} := \mathbf{w} + \alpha(y^{(i)} - h(\mathbf{x}^{(i)}))\mathbf{x}^{(i)}$$

Nehmen Sie $\alpha = 1$ als Lernrate. Halten Sie für jeden Durchlauf fest, wie viele Schritte der Algorithmus benötigt, um zu der endgültigen Hypothese $h^*(\mathbf{x})$ zu konvergieren. Berechnen Sie am Ende die durchschnittliche Anzahl von benötigten Schritten. In welcher Größenordnung liegt sie?

13.3.3. Experimente (2P)

Wiederholen Sie das obige Experiment mit $m = 100$ und $m = 1000$ Datenpunkten, jeweils ein Mal mit den Lernraten $\alpha = 1$ und $\alpha = 0.1$. In welcher Größenordnung liegt die durchschnittliche Anzahl von benötigten Schritten in diesen Fällen?

Um eine zuverlässigere Schätzung zu erhalten, können Sie dasselbe Experiment mehrfach mit anderen zufällig generierten Datensätzen derselben Größe m wiederholen und danach den Durchschnitt über alle Wiederholungen betrachten.

13.3.4. Visualisierung (optional)

- Halten Sie während des Trainings die Anzahl der falsch klassifizierten Punkte fest und veranschaulichen Sie anschließend den Lernprozess mit Hilfe eines zweidimensionalen Plots.
- Visualisieren Sie (auf eine geeignete Weise) Meilenstein 2.1, wie sich die Entscheidungsrenze während des Trainings verändert.

Sie können das folgende **Jupyter Notebook** als Startpunkt benutzen.

Quelle: *Idee nach* Yaser S. Abu-Mostafa, Malik Magdon-Ismail, and Hsuan-Tien Lin. 2012. Learning From Data. AMLBook.

14. Übungsblatt: Lineare / Logistische Regression & Gradientenabstieg

14.1. NN.Regression.01: Lineare Regression & Gradientenabstieg (3P)

Es sind folgende Trainingsdaten gegeben:

$$(x^{(1)}, y^{(1)}) = (1, 1), (x^{(2)}, y^{(2)}) = (2, 1), (x^{(3)}, y^{(3)}) = (3, 2)$$

Es soll das lineare Regressionsmodell $h(x) = w_0 + w_1 x$ mit diesen Daten trainiert werden, wobei die zu minimierende Kostenfunktion (durchschnittliche Summe der Fehlerquadrate) wie folgt gegeben ist:

$$J(\mathbf{w}) = \frac{1}{2m} \sum_{j=1}^{m} (h(x^{(j)}) - y^{(j)})^2$$

- (1P) Geben Sie n und m an und schreiben Sie die Kostenfunktion für die gegebenen Datenpunkte explizit auf. Berechnen Sie den Gradientenvektor ∇J und beschreiben Sie die Bedeutung dieses Vektors.
- (2P) Seien die Gewichte in einem Iterationsschritt $w_0 = 1, w_1 = 1$. Führen Sie für die Lernraten $\alpha = 0.01$, $\alpha = 0.1$ und $\alpha = 1$ jeweils fünf aufeinanderfolgende Iterationen des Gradientenabstieg (Gradient Descent) Algorithmus durch.

  Erstellen Sie eine Tabelle mit den Spalten $w_0, w_1, J(\mathbf{w}), \nabla J(\mathbf{w}), \alpha \cdot \nabla J(\mathbf{w})$ und notieren Sie die zugehörigen Werte für jede Iteration. Erklären Sie, wie die Gewichtsaktualisierungen durchgeführt werden und geben Sie die dafür verwendete Formel an.

  Wie verändern sich die Kosten während des Gradientenabstieges für die unterschiedlichen Lernraten? Begründen Sie dieses Verhalten.

  Sie können das folgende **Geogebra-Arbeitsblatt** zu Hilfe nehmen.

Thema: Verständnis und Ablauf Gradientenabstieg und Lernrate

14.2. NN.Regression.02: Logistische Regression & Gradientenabstieg (7P)

14.2.1. Datensatz (1P)

- Konstruieren Sie Ihren eigenen Datensatz $\mathcal{D}$ mit $m = 100$ gleichförmig verteilten Zufallspunkten aus dem Bereich $\mathcal{X} = [-1,1] \times [-1,1]$.
- Wählen Sie auf ähnliche Weise zwei zufällige, gleichmäßig verteilte Punkte aus dem Bereich $[-1,1] \times [-1,1]$. Verwenden Sie die Gerade, die durch diese zwei Punkte verläuft, als die Entscheidungsgrenze Ihrer Zielfunktion f. Sie können die positiv beschriftete Seite beliebig festlegen.
- Werten Sie die Zielfunktion für jeden Datenpunkt $\mathbf{x}^{(j)}$ aus, um die entsprechenden Beschriftungen (Ausgangslabel) $y^{(j)}$ zu erhalten.

14.2.2. Training (3P)

Trainieren Sie ein logistisches Regressionsmodell auf diesen Daten, um $h^* = \sigma(w^T x)$ zu finden. Verwenden Sie dazu den Gradientenabstieg-Algorithmus. Initialisieren Sie alle Gewichtswerte mit 0 und führen Sie 2000 Iterationen durch. Nehmen Sie $\alpha = 0.1$ als Lernrate. Speichern Sie alle 100 Schritte die berechneten Kosten. Zeichnen Sie am Ende die Kosten als Diagramm über die Anzahl der Iterationen auf.

14.2.3. Experimente (3P)

Wiederholen Sie das obige Experiment mit unterschiedlichen Lernraten, z.B. $\alpha = 0.1$, $\alpha = 0.01$ und $\alpha = 0.001$. Vergleichen Sie die Kosten-Diagramme.

Sie können das folgende **Jupyter Notebook** als Startpunkt benutzen. Sie können alternativ auch eine andere Programmiersprache und/oder einen anderen Datensatz (z.B. zufällig generierter Datensatz mittels Numpy and Scikit-Learn) verwenden.

Thema: Verständnis und Implementierung Logistische Regression

15. Übungsblatt: Overfitting & MLP

15.1. NN.MLP.01: Perzeptron-Netze (2P)

Konstruieren Sie ein Netz mit drei Perzeptrons, welches für zwei Eingabevariablen x_1 und x_2 die in der folgenden Abbildung gestrichelt markierten Bereiche mit +1 klassifiziert. Benutzen Sie die sign-Funktion als Aktivierungsfunktion.

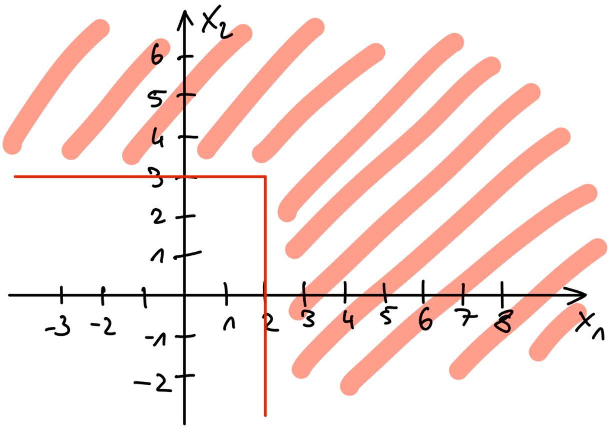


Abbildung 15.1.: Abbildung 1

15.2. NN.MLP.02: Vorwärtslauf im MLP (2P)

Gegeben sei ein MLP mit 25 Zellen in der Eingangsschicht, 64 Zellen in der ersten versteckten Schicht, 32 Zellen in der zweiten versteckten Schicht und 4 Zellen in der Ausgabeschicht (die Bias-Zellen nicht mitgezählt). In allen Zellen wird die ReLU Aktivierungsfunktion verwendet.

- Was sind die Dimensionen der Gewichtsmatrizen $W^{[1]}$, $W^{[2]}$ und $W^{[3]}$ und der Bias-Vektoren $b^{[1]}$, $b^{[2]}$ und $b^{[3]}$?
- Wie wird die Ausgabe berechnet? Schreiben Sie den Vorwärtslauf in Matrix-Notation auf. Wie könnte man die Ausgabe deuten; welches Problem könnte durch dieses Netzwerk möglicherweise gelöst werden?

15.3. NN.MLP.03: Tensorflow Playground (6P)

Benutzen Sie den Neural Network Playground, um die unten gelisteten Experimente durchzuführen. Achten Sie bei allen Experimenten auf das Verhalten der Trainings- und Testkosten. Sie können mit Hilfe der Checkbox unter der Ausgabezelle (ganz rechts, unten) die Testdaten ein- und ausblenden. Der Play-Knopf startet dabei das Training und der Reload-Knopf setzt das Netzwerk zurück.

1. (1P) Trainieren Sie ein **logistisches Regressionsmodell** zunächst auf dem "**Gaussian**" Datensatz (linear separierbarer Datensatz links-unten), danach auf den anderen Datensätzen.
2. (3P) Trainieren Sie ein **MLP** mit

   - einer versteckten Schicht mit 2 Neuronen,
   - einer versteckten Schicht mit 3 Neuronen,
   - einer versteckten Schicht mit 5 Neuronen,
   - zwei versteckten Schichten mit jeweils 5 Neuronen pro Schicht
   - drei versteckten Schichten mit jeweils 7 Neuronen pro Schicht
   - vier versteckten Schichten mit jeweils 7 Neuronen pro Schicht

   auf dem kreisförmigen (**Circle**) und auf dem spiralförmigen (**Spiral**) Datensatz, mehrmals mit jeweils den Aktivierungsfunktionen ReLU, tanh und Sigmoid. Hat die Auswahl der Aktivierungsfunktion einen Einfluss auf die Form der Entscheidungsgrenze oder die Geschwindigkeit der Berechnung?
3. (2P) Setzen Sie nun den **Noise-Level auf 15** und wiederholen Sie die Experimente. Wann kann von einer Überanpassung gesprochen werden?

Sprechen Sie für alle Experimente die folgenden Punkte an:

- Wie verhält sich die Entscheidungsgrenze?
- Was können Sie über Trainings- und Testkosten sagen? Entsteht eine Überanpassung?
- Wie schnell wird die Entscheidungsgrenze berechnet?
- Können alle Datenpunkte jedes mal korrekt klassifiziert werden? Warum?
- Untersuchen und vergleichen Sie die Ausgaben der Zellen in den versteckten Schichten, in dem Sie die Maus über die jeweilige Zelle bewegen. Bemerken Sie einen wesentlichen Unterschied in den Ausgaben der ersten Schicht im Vergleich zu der letzten Schicht?

16. Übungsblatt: Backpropagation

16.1. NN.Backprop.01: Gewichtsupdates für versteckte Schichten (2P)

In der Vorlesung wurde(n) die Gewichtsupdates bei der Backpropagation für die Ausgabeschicht und die davor liegende letzte versteckte Schicht hergeleitet, wobei in der Ausgabeschicht die Sigmoid und in der versteckten Schicht die ReLU Aktivierungsfunktionen eingesetzt wurden. Leiten Sie die Gewichtsupdates für die erste versteckte Schicht (für ein Netz mit zwei echten versteckten Schichten) her. Verwenden Sie dabei die Sigmoid Funktion als Aktivierung in allen Schichten.

Thema: Verständnis Backpropagation

16.2. NN.Backprop.02: Forward- und Backpropagation (2P)

Betrachten Sie das folgende MLP mit zwei Schichten mit insgesamt zwei Zellen. Die Gewichte sind an den Kanten angegeben. Das Netz erhält den skalaren Input x und berechnet daraus die Ausgabe y. Beide Zellen verwenden die Aktivierungsfunktion $\sigma(z) = \frac{1}{1+e^{-z}}$.

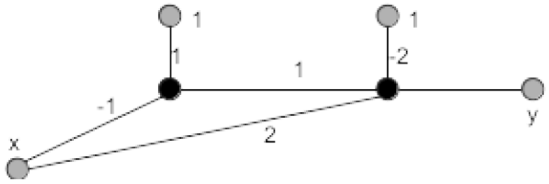


Abbildung 16.1.: Abbildung 1

- (1P) Berechnen Sie die Ausgabe y für die Eingabe $(x, y_T) = (0, 0.5)$. Wie groß ist der Fehler?
- (1P) Berechnen Sie die partiellen Ableitungen für die Gewichte. Wie lauten die Gewichtsupdates für das obige Trainingsbeispiel? Setzen Sie $\alpha = 0.01$.

16.3. NN.Backprop.03: MLP und Backpropagation (6P)

Implementieren Sie ein Feedforward MLP mit mindestens einer versteckten Schicht. Nutzen Sie die Cross-Entropy Verlustfunktion.

- (2P) Implementieren Sie die Forwärtspropagation. Nutzen Sie als Aktivierungsfunktion in der Ausgangsschicht $g(z) = \frac{1}{1+e^{-z}}$ und in der versteckten Schicht $g(z) = ReLU(z)$.
- (2P) Implementieren Sie das Backpropagations-Verfahren zum Aktualisieren der Gewichte. Achten Sie insbesondere darauf, die bereits berechneten partiellen Ableitungen der jeweils hinteren Schicht wieder zu verwenden (und nicht jeweils erneut zu berechnen!), d.h. propagieren Sie die Fehler von hinten nach vorn durch das Netz.
- (2P) Trainieren Sie das Netz für den Iris-Datensatz (iris.csv) aus dem AIMA-Repository und nutzen Sie dabei die Variante des stochastischen Gradientenabstiegs. Messen Sie pro Epoche (also nach jedem Durchlauf durch den kompletten Datensatz) den Trainingsfehler. Zeichnen Sie den Trainingsfehler als Diagramm über den Epochen auf.

Falls der Trainingsfehler nach einigen tausend Epochen nicht gegen einen Wert nahe Null strebt, erweitern Sie Ihr Netz (beispielsweise eine versteckte Schicht mehr oder mehr Zellen in der schon existierenden versteckten Schicht, …) und trainieren Sie es erneut. Nach wievielen Epochen ist der Trainingsfehler fast Null?

Thema: Verständnis MLP und Backpropagation, Gefühl für nötige Größe des Netzes

17. Übungsblatt: Constraints

17.1. CSP.01: Logikrätsel (2P)

Betrachten Sie die Variante des berühmten "Einstein-Rätsels" auf Wikipedia.

Formulieren Sie das Problem als CSP (Variablen, Wertebereiche, Constraints) zunächst auf dem Papier. Machen Sie sich klar, was die Variablen und was deren Wertebereiche sind. Schreiben Sie die Constraints als (unäre bzw. binäre) Relationen zwischen den Variablen auf.

Hinweis: Machen Sie sich zunächst klar, was die Variablen und was deren Wertebereiche sind. Schreiben Sie die Constraints als (unäre bzw. binäre) Relationen auf.

Thema: Formulierung von Problemen als CSP

17.2. CSP.02: Framework für Constraint Satisfaction (4P)

Lösen Sie nun das obige Rätsel (aus CSP.01):

1. Lösen Sie das Rätsel zunächst mit dem Basis-Algorithmus `BT_Search` aus der Vorlesung.
2. Erweitern Sie den Algorithmus um die Heuristiken MRV und Gradheuristik und lösen Sie das Problem erneut. Vergleichen Sie die Ergebnisse und die Laufzeit der beiden Experimente.
3. Wenden Sie vor dem Start von `BT_Search` den AC-3 an. Erhalten Sie damit bereits eine Lösung (bzw. Unlösbarkeit)? Falls nicht, wenden Sie anschließend den ergänzten Algorithmus aus Schritt (2) an. Vergleichen Sie wieder die Ergebnisse und die Laufzeiten.
4. Wenden Sie die *Min-Conflicts* Heuristik zur Lösung des Problems an. Vergleichen Sie wieder die Ergebnisse und die Laufzeiten.

Sie können dafür eine Handsimulation anwenden oder die Algorithmen implementieren. Sie können gern auch die Java-Klassen im Paket `aima.core.search.csp` bzw. die Python-Klassen in `csp.py` als Ausgangspunkt nutzen.[1]

17.3. CSP.03: Kantenkonsistenz mit AC-3 (2P)

Sei $D = \{0, \ldots, 5\}$, und ein Constraintproblem definiert durch

---

[1] Im Python-Code tauchen immer wieder "TODO"-Marker auf - bitte mit Vorsicht genießen!

$$\langle\{v_1, v_2, v_3, v_4\}, \{D_{v_1} = D_{v_2} = D_{v_3} = D_{v_4} = D\}, \{c_1, c_2, c_3, c_4\}\rangle$$

mit

- $c_1 = \big((v_1, v_2), \{(x, y) \in D^2 | x + y = 3\}\big)$,
- $c_2 = \big((v_2, v_3), \{(x, y) \in D^2 | x + y \leq 3\}\big)$,
- $c_3 = \big((v_1, v_3), \{(x, y) \in D^2 | x \leq y\}\big)$ und
- $c_4 = \big((v_3, v_4), \{(x, y) \in D^2 | x \neq y\}\big)$.

1. Zeichnen Sie den Constraint-Graph
2. Wenden Sie den AC-3-Algorithmus auf das CSP an. Geben Sie den Zustand der Queue und das Ergebnis von `ARC_Reduce`, d.h. den Ergebniszustand des aktuellen D_i, für jede Iteration des Algorithmus an.

Thema: Handsimulation des AC-3-Algorithmus

17.4. CSP.04: Forward Checking und Kantenkonsistenz (2P)

Betrachten Sie erneut das CSP aus der vorigen Aufgabe und die Zuweisung $\alpha = \{v_1 \to 2\}$.

1. Erzeugen Sie Kantenkonsistenz in α. Geben Sie hierzu die Wertebereiche der Variablen vor und nach dem Erzeugen der Kantenkonsistenz an.

   Hinweis: Sie dürfen annehmen, dass der Wertebereich von Variablen mit bereits zugewiesenen Werten nur aus dem zugewiesenen Wert besteht, während unbelegte Variablen den vollen Wertebereich haben.

   Hinweis: Sie müssen zur Lösung dieser Teilaufgabe nicht den AC-3 nutze.

2. Führen Sie Forward-Checking in α aus. Vergleichen Sie das Ergebnis mit (1).

Thema: Kantenkonsistenz und Forward Checking verstehen

17.5. Bonus: Planung von Indoor-Spielplätzen (3P)

Sie sind für die Planung von Indoor-Spielplätzen zuständig.

Ein Spielplatz hat eine rechteckige Form, etwa 40x100 m. Zur Vereinfachung wird diese Fläche in ein gleichmäßiges Raster unterteilt, beispielsweise 10x10 cm. Es gibt am Rand mehrere Türen (normaler Eingang, Notausgänge).

Auf dieser Grundfläche sollen verschiedene Spielgeräte angeordnet werden, beispielsweise Go-Kart-Bahnen, Hüpfburgen und Kletterberge. Diese Spielgeräte haben selbst eine rechteckige Grundfläche, wobei die Abmessungen rastergenau sein sollen (also Vielfache der Rastergröße). Weiterhin gibt es eine Bar, die als größerer rechteckiger Bereich modelliert werden kann (keine Unterscheidung in Tresen plus Tische plus Stühle o.ä. notwendig).

Die Aufgabe besteht darin, die Spielgeräte und den Bar-Bereich nach bestimmten Randbedingungen anzuordnen. Zu diesen Vorgaben gehört, dass sich die Spielgeräte nicht überlappen oder berühren, dass bestimmte Sicherheitsabstände eingehalten werden (1m zw. den Spielgeräten), dass Entspannungszonen eingerichtet werden und dass Sichtlinien gewährleistet sind (z.B. sollten die Aktivitäten am Kletterberg vom Barbereich aus beobachtet werden können). Die Bar ist bevorzugt direkt am Eingang zu finden. Notausgänge dürfen nicht verstellt werden.

Abstrahieren Sie das gegebene Problem angemessen und geben Sie eine geeignete Modellierung als CSP an. Definieren Sie sich ein paar Spielgeräte und lösen Sie das Problem mit Hilfe von *MAC* und *Min-Conflicts*.

Thema: Modellierung eines Real-World-Problems

17.6. Bonus: DSL für Constraint Solving: MiniZinc (2P)

MiniZinc ist eine *Domain Specific Language* (DSL) zum Modellieren von Constraint- und Optimierungs-Problemen.

Schauen Sie sich das Tutorial an. Stellen Sie das Einstein-Rätsel oder das Spielplatz-Problem in MiniZinc dar. Nutzen Sie zur Ausführung einen der im Playground angebotenen Solver. Vergleichen Sie die Modellierung und die Laufzeiten mit Ihren Ergebnissen aus den vorigen Aufgaben.

Thema: DSL zur Formulierung von Constraint-Problemen

18. Übungsblatt: Naive Bayes

18.1. NB.01: Wahlkampf mit Naive Bayes (4P)

Betrachten Sie erneut das Szenerio von Aufgabe DTL.01.

(2P) "Trainieren" Sie für den gezeigten Datensatz einen Naive Bayes Klassifikator (manuell).

(2P) Welchen Kandidaten würde der Klassifikator einem Wähler (< 35, niedrig, Bachelor) zuordnen? Erklären Sie die Arbeitsweise des Klassifikators.

18.2. NB.02: Textklassifikation mit Naive Bayes: Spam-Erkennung (6P)

Laden Sie sich den Datensatz "Spam Mails Dataset" (Kaggle) herunter. Dieser besteht aus knapp 5000 vorklassifizierten Einträgen (Mails mit den Klassen `ham` bzw. `spam`).

(2P) Bereiten Sie diesen Datensatz für das Training eines Naive Bayes Klassifikators vor. Überlegen Sie sich, was mögliche Merkmale sein könnten und schreiben Sie sich ein Skript, welches den Datensatz entsprechend bearbeitet/transformiert. (*Tipp*: Ein "Bag-of-Words" ist ein guter Anfang.)

(2P) Implementieren Sie einen Naive Bayes Klassifikator in einer Programmiersprache Ihrer Wahl oder machen Sie sich mit existierenden Implementierungen vertraut, beispielsweise in NLTK oder scikit-learn oder Weka.

(2P) Splitten Sie den vorbereiteten Datensatz in eine Trainings- und eine Testmenge auf und trainieren Sie den Naive Bayes Klassifikator. Wie sieht ihr Klassifikator aus, was sind die wichtigsten Begriffe jeweils für die Klasse `spam` bzw. `ham`? Bewerten Sie das Testergebnis.

19. Übungsblatt: Lokale Suche, GA

19.1. Bonus: ChatGPT Taboo (2P)

Spielen Sie Tabu mit Ihrem Lieblings-Chatbot und bringen Sie den Bot dazu, ein bestimmtes Wort oder einen Satz zu sagen. Sie haben dabei eine Liste mit Wörtern, die Sie nicht verwenden dürfen.

Geben Sie einen Bericht darüber ab, was Sie versucht haben, wie leicht/schwer es war und eine vollständige Historie des "Gesprächs". (Grenze: nicht länger als 30 Minuten, nicht mehr als 5 Seiten)

Thema: Interaktion mit Chatbots

19.2. EA.01: Modellierung von GA (2P)

Betrachten Sie das 8-Queens-Problem sowie das Landkarten-Färbeproblem (aus Vorlesung CSP: Intro). Starten Sie beim Färbeproblem mit fünf verschiedenen Farben, Ziel sollte eine konfliktfreie Einfärbung mit einer minimalen Anzahl an Farben sein.

Geben Sie für beide Probleme je eine geeignete **Kodierung** der Individuen, passende Operatoren (**Crossover**, **Mutation**) und eine geeignete **Fitnessfunktion** an, damit die Probleme mit einem GA gelöst werden können. Begründen Sie Ihre Wahl!

Was würden Sie noch benötigen, um die obigen Probleme jeweils mit Simulated Annealing lösen zu können?

Thema: Modellierung für GA und Gradientensuche

19.3. EA.02: Implementierung (5P)

Implementieren Sie den in der Vorlesung besprochenen GA und wenden Sie den Algorithmus nacheinander auf beide Probleme an. Sie können gern auch die Java-Klassen im Paket `aima.core.search.local` bzw. die Python-Klassen in `search.py` als Ausgangspunkt nutzen.[1]

Untersuchen Sie **systematisch** unterschiedliche Varianten/Einstellungen der in der VL vorgestellten Operatoren. Führen Sie pro Einstellung jeweils mind. 100 Läufe durch und messen Sie die besprochenen Kennzahlen.

Erstellen Sie eine geeignete (systematische!) Auswertung Ihrer Experimente.

---

[1] Im Python-Code tauchen immer wieder "TODO"-Marker auf - bitte mit Vorsicht genießen!

19.4. EA.03: Anwendungen (3P)

1. Analysieren Sie die Implementierung von Randal Olson “Here’s Waldo: Computing the optimal search strategy for finding Waldo” (Direktlink).
2. Schauen Sie sich nun den “Evolution Simulator” an. Wie ist dort die Modellierung erfolgt (Kodierung, Operatoren, Fitnessfunktion)?
3. Wie werden EA/GA konkret im “american fuzzy lop” eingesetzt?

Welche Fitnessfunktion wurden in den drei Beispielen jeweils genutzt, wie die Individuen und die Operatoren codiert?

Recherchieren Sie, in welchen *anderen* Anwendungen Evolutionäre Algorithmen eingesetzt werden. Erklären Sie kurz, wie und wofür die EA/GA jeweils genutzt werden.

Thema: Analyse von GA-Implementierungen

Teil IV.

Organisatorisches

20. Prüfungsvorbereitung (HSBI)

TL;DR

Durchführung: Präsenz
Die Klausur wird dieses Semester elektronisch stattfinden. Dazu werden wir den E-Assessment-ILIAS der HSBI nutzen.
Die Klausur wird in den Räumen der HSBI am Campus Minden unter Aufsicht durchgeführt. Hier werden Ihnen Rechner für den Zugang zum Prüfungs-ILIAS zur Verfügung gestellt, Sie benötigen nur Ihre HSBI-Zugangsdaten (Username, Passwort), einen Studierendenausweis und Personalausweis sowie Ihren DIN-A4-Spickzettel.
Ablauf der Klausur
Die Prüfung (das ILIAS-Objekt) selbst schalte ich zum Start der Prüfung online. Sie loggen sich vor Ort in den ILIAS mit Ihren HSBI-Zugangsdaten ein und können die Prüfung nach dem gemeinsamen Start beginnen. Nach Ablauf der Bearbeitungszeit beenden Sie die Prüfung und loggen sich wieder aus. Parallel zur Prüfung werde ich in einer Zoom-Sitzung zur Verfügung stehen, in der Sie Fragen stellen können.
Hilfsmittel und Themen
Als Hilfsmittel ist ein Spickzettel (DIN A4, beidseitig beschrieben) zugelassen.
Es wird keines der behandelten Themen ausgeschlossen, allerdings eignen sich manche Themen etwas besser für Klausurfragen als andere . . .

Videos

1. Fragetypen-Demo
    - **Video** zur Fragetypen-Demo: [YT], [HSBI]
    - **Fragetypen-Demo** im ILIAS
2. Hinweise zur Prüfung: [YT], [HSBI]

20.1. Elektronische Klausur: Termin, Materialien

20.1.1. Termin

Die schriftliche Prüfung erfolgt durch eine Klausur, die als digitale Prüfung auf einem Prüfungs-ILIAS durchgeführt wird.

Die Klausur wird in Präsenz in den Rechnerpools am Campus Minden durchgeführt.

Es wird in beiden Prüfungszeiträumen ein Termin angeboten. Die Termine werden vom Prüfungsamt bekannt gegeben.

Dauer: siehe Aushang bzw. Mail.

20.1.2. Zugelassene Hilfsmittel: DIN-A4-Spickzettel (beidseitig)

Sie dürfen **einen** Spickzettel im **DIN-A4**-Format benutzen, der beidseitig beschrieben sein kann.

Ich möchte Sie hier noch einmal ermuntern, diesen Zettel tatsächlich manuell zu erstellen (also ganz traditionell zu **schreiben**), da bereits der Schreibvorgang einen gewissen Lerneffekt bewirkt! (Wenn Sie sich bewusst dafür entscheiden, können Sie den Spickzettel natürlich auch ausdrucken.)

20.1.3. Einsicht

Die Prüfungseinsicht findet möglichst zeitnah und online statt. Details werden per Mail bekanntgegeben.

20.2. Technische Vorbereitungen

Sie benötigen am Prüfungstag:

1. **HSBI-Zugangsdaten**: Username, Passwort

   Bei der Durchführung der Prüfung am Campus Minden wird Ihnen ein Rechner zur Verfügung gestellt. Dort läuft voraussichtlich ein Browser im Kiosk-Mode, wo Sie sich am Prüfungs-ILIAS anmelden. Dazu benötigen Sie Ihre HSBI-Zugangsdaten, mit denen Sie sich auch im “normalen” ILIAS anmelden.

2. **Studierendenausweis** und Personalausweis

   An der Prüfung dürfen nur Personen teilnehmen, die dafür im LSF/CAT angemeldet sind. Es findet eine entsprechende Kontrolle statt. Halten Sie dazu Ihren Studierendenausweis und Personalausweis bereit.

20.3. Bearbeitung des E-Assessment

1. Lesen Sie sich die Hinweise auf der Startseite durch

2. Bearbeiten Sie die Aufgaben in **einem einzigen** Browser-Tab

   Öffnen Sie die Aufgaben *NICHT* in parallelen Tabs! Es kann sonst zu Fehlfunktionen von ILIAS kommen.

   Bewegen Sie sich **nicht per Browser-Navigation** (“vor”, “zurück” im Browser) durch die Aufgaben, sondern nutzen Sie dafür die **Buttons “nächste Frage”, “Weiter” oder “Zurück” in ILIAS**!

3. Hinweis zur Anzeige der restlichen Bearbeitungsdauer

   Wenn Sie den Browser bzw. das Tab mit der Prüfung im Laufe der Prüfung verlassen, wird Ihnen bei der Rückkehr unter Umständen eine falsche restliche Bearbeitungsdauer angezeigt. Sie können die Anzeige korrigieren/aktualisieren, indem Sie einfach zu einer vorigen oder nächsten Aufgabe navigieren.

   Hinweis: Die restliche Bearbeitungsdauer wird im Test nur dann angezeigt, wenn diese Funktion von den Prüfenden aktiviert wurde.

4. Parallel zum E-Assessment läuft eine Zoom-Session, dort können Sie Fragen stellen

20.4. Fragetypen-Demo

In Ihrem ILIAS-Kurs finden Sie eine **Fragetypen-Demo** mit den wichtigsten Fragetypen. Machen Sie sich mit der Mechanik der Fragetypen vertraut und schauen Sie sich die Kommentare bei den einzelnen Aufgaben an. Sie können die Demo bei Bedarf beliebig oft wiederholen.

20.5. Hinweise zu den Inhalten

- Klausurrelevant: Vorlesung und Praktikum
- Für Verständnis u.U. hilfreich: Studium der vertiefenden Literaturangaben

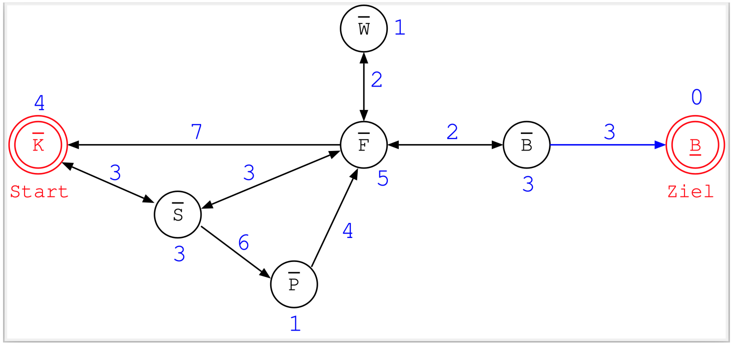
- **Fragen**:
  - Schauen Sie sich die Challenges an …
  - Schauen Sie sich die Praktikumsaufgaben an …
  - Überlegen Sie sich, was zu einem Themengebiet im Rahmen einer Prüfung möglich ist und (wie) gefragt werden könnte …

Können vor Kennen

20.6. Beispiele für mögliche Fragen

20.6.1. Breitensuche

Betrachten Sie den folgenden Graphen:



Mittels Breitensuche (Variante "Tree-Search") soll der kürzeste Weg von "K" nach "B" gefunden werden. Die folgende Queue stellt den aktuellen Zustand der Suche dar:

`[KSF, KSP]`

Führen Sie den nächsten Schritt des Breitensuche-Algorithmus durch und geben Sie die resultierende Queue an. Achten Sie auf die korrekte Sortierung und halten Sie sich an die vorgegebene Notation.

Hinweis: Die vorgegebene Queue kann willkürlich konstruiert sein und muss nicht viel mit Breitensuche zu tun haben.

20.6.2. Suche und Constraints

- Worin liegt der Unterschied zwischen einem Knoten und einem Zustand?
- Was ist ein Suchbaum? Wo liegt der Unterschied zum Zustandsraum?
- Muss der Suchbaum über eine eigene Datenstruktur realisiert werden?
- Was bedeutet Kantenkonsistenz?

Erklären Sie die jeweiligen Begriffe nachvollziehbar. Nutzen Sie dazu ein eigenes kleines Beispiel.

20.6.3. Handsimulation Perzeptron

Gegeben sei folgendes Perzeptron mit den Gewichten $w_0 = 0, w_1 = 0, w_2 = 0$. Betrachten Sie den folgenden Eingabevektor $x_0 = 1, x_1 = 1, x_2 = 0$ mit der Klasse $+1$. Geben Sie für diesen Vektor folgende Werte an:

(a) Skalarprodukt $\mathbf{w}^T\mathbf{x}$,
(b) Aktivierung sgn$(\mathbf{w}^T\mathbf{x})$.
(c) Falls notwendig, führen Sie einen Lernschritt durch. Nutzen Sie dabei $\alpha = 1$.

Geben Sie die resultierenden (neuen) Gewichte an (auch wenn Sie keinen Lernschritt durchführen mussten).

20.6.4. Entscheidungsbäume mit ID3

Gegeben sei folgender Trainingsdatensatz …

Mit Hilfe von ID3 soll für diesen Datensatz ein Entscheidungsbaum konstruiert werden. Der Algorithmus läuft schon und hat das Merkmal x_2 als Wurzelknoten ausgewählt. Beschreiben Sie detailliert, wie ID3 die nächste Ebene aufbaut. Gehen Sie dabei konkret auch auf den Algorithmus ein. Geben Sie die eingesetzten Formeln für Entropien und Gains an und erklären Sie Ihr Vorgehen.

Hinweis: Sie sollen die Formeln nur einsetzen, aber nicht ausrechnen.

20.6.5. CSP

Betrachten Sie das folgende Constraintproblem:

- $\mathrm{C}_1 = \{(a,b) | a \neq b\}$
- $\mathrm{C}_2 = \{(b,c) | b \neq c\}$
- $\mathrm{C}_3 = \{(a,c) | a = c\}$

- $D_a = D_b = D_c = \{1,2,3,4,5\}, D_d = \{1,2,5\}, D_e = \{1,3\}$

Zeigen Sie per Handsimulation, wie der Algorithmus AC-3 schrittweise Kantenkonsistenz herstellt. Hier ist die Queue in einem bestimmten Schritt: … Führen Sie einen Schritt durch und geben Sie die resultierende Queue an. Erklären Sie Ihr Ergebnis.

20.6.6. NB

Betrachten Sie den folgenden Datensatz: … "Trainieren" Sie für diesen Datensatz einen vollständigen Naive-Bayes-Klassifikator. Nutzen Sie die Variante "Multinomial NB". Geben Sie alle Bestandteile an. Erklären Sie Ihr Vorgehen nachvollziehbar.

Hinweis: Sie brauchen die Brüche nicht ausrechnen.

Quellen

Abu-Mostafa, Y. S., M. Magdon-Ismail, und H. Lin. 2012. *Learning From Data*. AMLBook. https://work.caltech.edu/telecourse.

Breiman, L. 2001. „Random Forests". *Machine Learning* 45 (1): 5–32. https://doi.org/10.1023/A:1010933404324.

Ertel, W. 2025. *Grundkurs Künstliche Intelligenz*. 6th edition. Springer Vieweg Wiesbaden. https://doi.org/10.1007/978-3-658-44955-1.

Geron, A. 2023. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3. Auflage. O'Reilly. https://learning.oreilly.com/library/view/hands-on-machine-learning/9781098125967/.

Goldberg, D. E. 1986. „The genetic algorithm approach: why, how, and what next?“ In *Adaptive and learning systems: Theory and applications*. Springer.

Holland, J. H. 1975. „Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence“. Phdthesis, PhD Thesis, University of Michigan Press, Ann Arbor.

Holland, J. H. 1992. *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. MIT press.

Mitchell, T. 2010. *Machine Learning*. McGraw-Hill.

Rechenberg, I. 1978. „Evolutionsstrategien“. *Simulationsmethoden in der Medizin und Biologie: Workshop, Hannover, 29. Sept.–1. Okt. 1977*, 83–114.

Russell, S., und P. Norvig. 2014. *Artificial Intelligence: A Modern Approach*. 3rd revised edition. Pearson. http://aima.cs.berkeley.edu.

Russell, S., und P. Norvig. 2021. *Artificial Intelligence: A Modern Approach*. 4th Edition. Pearson. http://aima.cs.berkeley.edu.

Schwefel, H. P. 1975. „Evolutionsstrategie und numerische Optimierung“. Phdthesis, PhD Thesis, Technical University of Berlin, Department of Process Engineering.

Schwefel, H. P. 1995. *Evolution and Optimum Seeking*. Wiley.

Silver, D., A. Huang, C. Maddison, u. a. 2016. „Mastering the Game of Go with Deep Neural Networks and Tree Search“. *Nature* 529 (7587): 484. https://doi.org/10.1038/nature16961.

Unger, S., und F. Wysotzki. 1981. *Lernfähige Klassifizierungssysteme (Classifier Systems Which Are Able to Learn)*. Akademie-Verlag.

Weicker, K. 2015. *Evolutionäre Algorithmen*. 3rd edition. Springer Vieweg Wiesbaden. https://doi.org/10.1007/978-3-658-09958-9.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Exceptions:

- AvB - RoboCup 2013 - Eindhoven by RoboCup2013 on Flickr.com (CC BY 2.0)
- Photo Evolution by Johannes Plenio on Unsplash.com (Unsplash License)
- “Exp e.svg” by Marcel Marnitz, reworked by Georg-Johann on Wikimedia Commons (Public Domain)
- MapGermanyGraph.svg by Regnaron and Jahobr on Wikimedia Commons (Public Domain)
- “künstliche intelligenz” by Gerd Altmann (geralt) on Pixabay.com (Pixabay License)
- “Intelligenz” by Cumtempore and others on Wikipedia (CC BY-SA 3.0)
- Turing Test version 3.png by Bilby on Wikimedia Commons (Public Domain)
- Backgammon lg.png by Ptkfgs on Wikimedia Commons (Public Domain)
- “Kognition” by Arbraxan and others on Wikipedia (CC BY-SA 3.0)

> **Last modified:** 8007fe4 2026-10-17 problemsolving: replace challenge